

Современный Java Concurrency

Глубокое погружение в Virtual Threads, Structured Concurrency и Scoped Values

Modern Concurrency in Java

A Deep Dive into Virtual Threads, Structured Concurrency, and Scoped Values

Ещё больше полезной литературы по Java в наших каналах:

https://max.ru/bookofgeek

https://t.me/bookofgeek

https://vk.com/javatutorial



Современный Java Concurrency

Глубокое погружение в Virtual Threads, Structured Concurrency и Scoped Values



2026

Современный Java Concurrency. Глубокое погружение в Virtual Threads, Structured Concurrency и Scoped Values

Добро пожаловать в будущее Java! Изучите главное новшество Java 21 — виртуальные потоки. Помните, как приходилось бороться с высокой стоимостью создания потоков, сталкиваться с ограничениями масштабируемости и трудностями при попытке достичь высокой пропускной способности? Те дни прошли! Это практическое руководство проведет вас от Java 1.0 до прорывных достижений Project Loom.

Автор не ограничивается теорией, а предлагает набор инструментов, которые применимы для реальных проектов. Погрузитесь в тонкости виртуальных потоков и такие сложные темы, как ForkJoinPool, продолжение, ограничение скорости, отладка и мониторинг. Вы не только узнаете принципы их работы, но и получите экспертные советы и приемы для освоения этих концепций. Особое внимание уделено структурированной многопоточности и scoped values — ключевым навыкам для создания масштабируемых и эффективных Java-приложений.

Краткое содержание

Предисловие 12

Глава 1. Введение в конкурентность 21

Глава 2. Виртуальные потоки 52

Глава 3. Механизм современной конкурентности в Java 115

Глава 4. Структурированная конкурентность 148

Глава 5. Scoped values 239

Глава 6. Актуальность реактивного программирования на Java в контексте виртуальных потоков 268

Глава 7. Современные фреймворки, использующие виртуальные потоки 318

Глава 8. Заключение и основные выводы 331

https://max.ru/bookofgeek

Оглавление

Предисловие 12
Почему я написал эту книгу 12
Для кого эта книга 13
Что представляет собой эта книга 14
Структура издания 15
Как читать эту книгу 17
Использование примеров кода 17
Условные обозначения 18

Глава 1. Введение в конкурентность 21
Краткая история потоков в Java 21
Язык Java построен на основе потоков 22
Потоки — основа программной платформы Java 24
Зарождение потоков в Java 1.0 27
Запуск потоков 28
Скрытые затраты на потоки 29
Сколько потоков можно создать 31
Эффективность использования ресурсов в высоконагруженных приложениях 32
Стратегия параллельного выполнения 33
Знакомство с фреймворком Executor 38
Оставшиеся проблемы 39
Выход за пределы базового пула потоков 41
Привязка к кэшу и распределение задач 41

https://max.ru/bookofgeek

Алгоритм кражи задач 41
Компоновка с помощью CompletableFuture 43
Другая парадигма асинхронного программирования 45
Отрицательная сторона использования реактивных фреймворков 47
Революция в конкурентности Java 49
Виртуальные потоки: новый подход 49
Бесшовная интеграция с существующими кодовыми базами 49
Виртуальные и платформенные потоки 50
Интеллектуальная обработка блокирующих операций 50
Преимущества использования виртуальных потоков 50
В заключение 51

Глава 2. Виртуальные потоки 52
Что такое виртуальные потоки 52
Два вида потоков в Java 53
Ключевые отличия виртуальных потоков от платформенных 55
Настройка окружения для работы с виртуальными потоками 56
Создание виртуальных потоков в Java 57
Адаптация к виртуальным потокам 58
Пример создания виртуальных потоков в Java 62
Пропускная способность и масштабируемость 63
Фундаментальный принцип масштабируемости виртуальных потоков 64
Практические выводы 67
Виртуальные потоки: внутреннее устройство 68
Стековые фреймы и управление памятью 68
Потоки-носители и взаимодействие с ОС 68
Обработка блокирующих операций 69
Прозрачность и невидимость 69
Упрощение асинхронных операций 69
Потенциал структурированной конкурентности 72
Управление ограничением ресурсов с помощью Rate Limiting 73
Семафоры в Java: принцип работы 76
В каких случаях использовать семафор 81
Ограничения виртуальных потоков 83
Пиннинг 85

https://max.ru/bookofgeek

Решение проблемы пиннинга с помощью ReentrantLock ... 87
Вызов нативных методов и пиннинг ... 91
Дилемма применения переменных ThreadLocal в виртуальных потоках ... 94
Проблемы виртуальных потоков ... 95
Мониторинг ... 98
Мониторинг ThreadLocals ... 98
Мониторинг пиннинга ... 99
Просмотр виртуальных потоков в дампах потоков с помощью jcmd ... 105
Генерация дампов потоков с помощью HotSpotDiagnosticsMXBean ... 110
Практические советы по миграции на виртуальные потоки ... 111
Еще раз о преимуществах виртуальных потоков ... 112
Масштабируемость ... 113
В заключение ... 114

Глава 3. Механизм современной конкурентности в Java ... 115
Пул потоков ... 115
Зачем нужен пул потоков ... 116
Создание простого пула потоков на Java ... 116
Фреймворк Executor ... 120
Интерфейсы Callable и Future: обработка результатов задач ... 125
ForkJoinPool ... 127
Почему ForkJoinPool используется для виртуальных потоков ... 132
Континуация ... 137
Создание виртуальных потоков с нуля ... 140
Виртуальные потоки и опрос ввода-вывода ... 146
В заключение ... 147

Глава 4. Структурированная конкурентность ... 148
Проблемы неструктурированной конкурентности ... 149
Потенциал структурированной конкурентности ... 155
Основы API ... 156
StructuredTaskScope ... 157
Области и задачи: связи и жизненный цикл ... 161
Политики объединения через Joiner ... 162

https://max.ru/bookofgeek

Стандартные политики объединения подзадач в одной области ... 163
Обработка исключений в StructuredTaskScope ... 189
Конфигурация ... 202
Кастомные джоинеры ... 208
Согласованность памяти ... 225
Вложенные области ... 228
Наблюдаемость ... 232
В заключение ... 237

Глава 5. Scoped values ... 239
В чем проблема передачи контекста ... 239
Загрязнение параметрами ... 242
Уязвимость интерфейсов ... 242
Стыковка и тестируемость ... 242
Использование ThreadLocal ... 242
Ограничения переменных ThreadLocal ... 244
Облегченная модель совместного использования данных ... 247
Основные компоненты ScopedValue ... 248
Запуск ScopedValue ... 250
ScopedValue и структурированная конкурентность ... 257
Вопросы производительности ... 258
Удобство использования и проектирование API ... 258
В заключение ... 267

Глава 6. Актуальность реактивного программирования на Java в контексте виртуальных потоков ... 268
Основы реактивного программирования в Java ... 269
Блокирующий и неблокирующий ввод-вывод ... 270
Событийно-ориентированная архитектура ... 293
Асинхронные API ... 298
Реактивное программирование в Java ... 300
Понятие реактивных потоков ... 300
Backpressure ... 308
Преимущества и недостатки реактивного программирования ... 314
В заключение ... 316

https://max.ru/bookofgeek

Глава 7. Современные фреймворки, использующие виртуальные потоки 318
Spring Boot 318
Ручное конфигурирование 321
Quarkus 323
Jakarta EE 327
В заключение 330

Глава 8. Заключение и основные выводы 331

https://max.ru/bookofgeek

Предисловие

Почему я написал эту книгу

Конкурентность (concurrency) долго была одной из самых сложных задач разработки на Java. Чтобы соответствовать требованиям современной разработки ПО, конкурентность непрерывно развивается и сохраняет при этом высокий уровень обратной совместимости. Среди всех улучшений, внесенных в Java за время существования, именно введение виртуальных потоков из Project Loom ознаменовало фундаментальный сдвиг в мире конкурентности.

Конкурентность была сложной изначально, и по мере увеличения требований к производительности трудности только нарастали. Даже для опытных разработчиков эффективное управление ею остается нетривиальной задачей. Современные приложения в основном ориентированы на ввод-вывод, поскольку взаимодействуют со множеством других систем. Особенно это касается микросервисной архитектуры, которая преобладает в разработке, чтобы удовлетворить растущие потребности в масштабируемости.

Выполнение операций ввода-вывода часто занимает много времени. При традиционном подходе поток, инициировавший такой вызов, обычно вынужден ожидать завершения операции. Хотя современные операционные системы (ОС) могут управлять миллионами открытых сокетов, количество доступных потоков по-прежнему ограничено. В результате с помощью традиционных потоков все сложнее обеспечивать более высокую пропускную способность.

Для преодоления этого ограничения были разработаны различные методы и альтернативные модели конкурентности, но с каждой из них приходится идти на определенные компромиссы. Виртуальные потоки — перспективное решение, использующее легковесные потоки на уровне пользователя, которые могут выполняться в большом количестве поверх небольшого числа потоков ядра. Такой подход позволил экономнее расходовать системные ресурсы и повысить масштабируемость. В области конкурентности наметился существенный прогресс.

Я заинтересовался виртуальными потоками сразу же, как о них узнал. Они показались мне долгожданным прорывом, способным кардинально изменить

https://max.ru/bookofgeek

подход к написанию конкурентных программ для JVM. Эти потоки предложили простое и элегантное решение именно там, где его так не хватало в Java.

Я начал экспериментировать с виртуальными потоками, описывал свои достижения в блоге и выступал с докладами на конференциях. Восторженный прием со стороны сообщества разработчиков подтвердил мою уверенность, что виртуальные потоки — не просто очередное улучшение, а принципиальный сдвиг в подходе к конкурентности.

Продолжив изыскания, я обнаружил множество ресурсов, связанных с виртуальными потоками: официальную документацию, содержательные статьи в блогах, репозитории GitHub с практическими примерами и отличные доклады с конференций. Это были ценные сведения, но они охватывали лишь разрозненные разделы темы. Одни фокусировались на технической реализации, в других обсуждались стратегии миграции и конкретные сценарии использования.

Благодаря тому, что я непрерывно писал статьи, тексты для выступлений и применял свои выводы на практике, у меня сформировалось довольно четкое и всестороннее понимание вопроса в целом. Я детально изучил мотивации создания проекта Loom, его интеграцию с текущей моделью конкурентности в Java и влияние на разработку масштабируемых и поддерживаемых систем. Мое уважение к способности Java адаптироваться к изменениям, сохраняя обратную совместимость, еще больше возросло.

В итоге я осознал, что можно объединить все полученные знания в общий практический ресурс. Эта книга представляет собой не только теоретическое исследование, но и практическое руководство, единый комплексный источник, в котором собраны все необходимые концепции, примеры и лучшие практики.

Для кого эта книга

Книга предназначена для Java-разработчиков, уже имеющих базовое представление о конкурентности и многопоточности. Она не является руководством для начинающих, а ориентирована на тех, кто уже писал конкурентные программы с помощью традиционных инструментов: Thread, ExecutorService, синхронизации и утилит коллекций вроде ReentrantLock и Semaphore, и хочет глубже разобраться в современных фичах конкурентности, добавленных в последние версии Java. Например, в виртуальных потоках, структурированной конкурентности и scoped values. Тем, кто хочет изучить основы конкурентности, рекомендую книгу Брайана Гетца (Brian Goetz) *Java Concurrency in Practice*[1].

[1] *Гетц Б., Пайерлс Т., Блох Дж., Боубер Дж.* Java Concurrency на практике. — СПб.: Питер, 2020.

https://max.ru/bookofgeek

Если вы сталкивались с проблемами ограничения числа потоков, блокирующего ввода/вывода, настройки пула потоков или управления сложной логикой жизненного цикла и отмены задач, данная книга поможет вам переосмыслить эти вопросы в контексте развивающейся модели конкурентности Java. Книга будет особенно полезна:

- разработчикам среднего и высокого уровня, которые стремятся модернизировать конкурентный код;
- архитекторам, проектирующим масштабируемые системы;
- инженерам, которые заинтересованы в создании надежных и высокопроизводительных приложений с использованием конкурентности;
- тимлидам, оценивающим новые технологии;
- всем, кто интересуется будущим конкурентности в Java.

Начинающим Java-разработчикам эта книга все же пригодится для общего ознакомления с современной конкурентностью. Однако им для полного понимания рекомендую предварительно или параллельно познакомиться с основополагающими темами: синхронизацией, состоянием гонки (race condition) и публикацией данных (data publishing). Чтобы успешно писать конкурентный код, знать эти основы необходимо. Хотя при работе с современными фреймворками разработчик, как правило, не сталкивается с этими понятиями напрямую, они становятся ключевыми, когда приходится разбираться с серьезными ошибками. Базовые знания необходимы для глубокого понимания любой темы — и конкурентность не исключение.

Что представляет собой эта книга

Это исчерпывающее руководство, где собрано все, что я узнал о Project Loom и виртуальных потоках. Здесь вы найдете:

- рассказ об эволюции конкурентности в Java: от платформенных потоков и фреймворка Executor до CompletableFuture и реактивного программирования;
- подробный разбор механизмов виртуальных потоков, структурированной конкурентности и scoped values;
- практические примеры реального применения, показывающие, как эффективно использовать эти возможности;
- широкий охват тем: знакомство не только с виртуальными потоками, но и со структурированной конкурентностью и scoped values;

https://max.ru/bookofgeek

- сведения по интеграции виртуальных потоков в современных фреймворках Spring Boot и Quarkus, а также Jakarta EE.

Независимо от того, познакомились вы с виртуальными потоками недавно или следите за Project Loom с самого начала, вы найдете здесь ценные идеи и практические знания.

Приведенные в книге примеры и концепции требуют как минимум JDK 21, поскольку именно в этой версии виртуальные потоки стали официально доступны. Однако в некоторых главах рассматриваются предварительные версии функций или недавно завершенные. Поэтому, чтобы извлечь максимальную пользу из всех примеров и обсуждений, предпочтительнее иметь доступ к новой версии JDK (24 или даже 25).

Структура издания

Я выстроил главы так, чтобы провести вас от понимания важности виртуальных потоков до умения применять их в продакшен-приложениях. Вот что вас ждет в каждой главе.

Глава 1. Введение в конкурентность

Мы начнем с изучения эволюции конкурентности в Java, проследим ее развитие от платформенных потоков через фреймворк Executor, Fork/Join и CompletableFuture, а также кратко ознакомимся с реактивным программированием. Экскурс в историю поможет осознать, почему виртуальные потоки стали настоящим прорывом. Вы узнаете о сложностях, которые приходилось преодолевать многие годы, и поймете, почему Project Loom использует принципиально иной подход.

Глава 2. Виртуальные потоки

В этой главе мы активно знакомимся с виртуальными потоками. Вы узнаете, что они собой представляют, чем отличаются от платформенных потоков и как их создавать. Мы рассмотрим, как улучшать пропускную способность, в чем преимущества масштабируемости, а также разберем практические примеры. В числе основных тем: ограничение скорости с помощью семафоров (semaphores) и важные ограничения, связанные с пиннингом (pinning) и использованием `ThreadLocal`.

Глава 3. Механизм современной конкурентности в Java

Здесь мы подробно исследуем базовые компоненты конкурентности. Изучим пулы потоков, фреймворк Executor и планировщик виртуальных потоков `ForkJoinPool`. Ключевой момент — создание собственной реализации

https://max.ru/bookofgeek

виртуальных потоков с нуля с применением континуаций (continuations). На примере нашего класса `NanoThread` вы детально разберетесь, как именно работают виртуальные потоки.

Глава 4. Структурированная конкурентность

В этой главе мы рассмотрим одну из важнейших инноваций Project Loom — `StructuredTaskScope`. Вы узнаете, как этот API преодолевает проблемы неструктурированной конкурентности, научитесь работать с различными стратегиями объединения, корректно обрабатывать исключения и создавать надежные приложения, использующие конкурентность. Глава охватывает все аспекты — от базового использования до настраиваемых джоинеров (custom joiners) и вложенных областей исполнения (nested scopes). Обратите внимание: на момент написания книги структурированная конкурентность находилась на предварительной стадии разработки (preview). Надеюсь, что эта версия не претерпит значительных изменений; если же они будут, учту это в следующих изданиях.

Глава 5. Scoped values

Распространение контекста в высококонкурентных приложениях всегда было сложной задачей. В этой главе показано, что для виртуальных потоков scoped values являются наилучшей альтернативой классу `ThreadLocal`. Вы изучите этот API, увидите практические примеры в контексте структурированной конкурентности и поймете, когда и как переходить от использования `ThreadLocal` к scoped values.

Глава 6. Актуальность реактивного программирования на Java в контексте виртуальных потоков

В этой главе мы исследуем различия в подходе к конкурентности виртуальных потоков и реактивного программирования. Мы рассмотрим разницу между блокирующим и неблокирующим вводом/выводом, архитектуру на основе событий (Event-Driven Architecture, EDA), а также компромиссы, связанные с каждым из подходов. Прочитав эту главу, вы сможете осознанно решать, когда использовать виртуальные потоки, а когда — реактивные фреймворки.

Глава 7. Современные фреймворки, использующие виртуальные потоки

В этой главе вы увидите, как ведущие фреймворки внедряют поддержку виртуальных потоков. В ней рассказывается о Spring Boot, Quarkus и Jakarta EE, а также приводятся практические примеры их настройки и рекомендации по применению. От изучения теории виртуальных потоков мы переходим к их использованию в продакшене.

Глава 8. Заключение и основные выводы

Здесь представлены ключевые идеи и прогнозы о будущем конкурентного программирования в Java.

https://max.ru/bookofgeek

Как читать эту книгу

Книгу желательно читать последовательно, поскольку в каждой новой главе развиваются концепции, изложенные в предыдущих. Однако вы можете пропускать уже знакомые темы и переходить к более интересным для вас главам.

Для тех, кто еще мало знает о виртуальных потоках

Начните с главы 1, чтобы ознакомиться с историческим контекстом. Затем изучите остальные главы по порядку.

Для опытных разработчиков

При желании можно бегло просмотреть главу 1 и сразу перейти к главе 2 для практического знакомства с виртуальными потоками. Не стоит пропускать главу 3 — реализация виртуальных потоков с нуля даст вам ценную информацию.

Для архитекторов и тимлидов

Особое внимание стоит уделить главам 4 и 5, посвященным структурированной конкурентности и scoped values, а также главе 6, где эти концепции сравниваются с реактивным программированием. Это поможет принимать обоснованные архитектурные решения.

Использование примеров кода

Каждая глава содержит множество примеров кода, и я надеюсь, что во время чтения у вас будет открыта IDE, чтобы последовательно их тестировать. Не просто читайте код — запускайте его, изменяйте, ломайте и исправляйте. Не существует лучшего способа усвоить что угодно, чем эксперименты.

Для удобства весь исходный код доступен по адресу https://github.com/Modern-Concurrency-in-Java. Каждая глава сопровождается отдельным пакетом с полностью рабочими примерами. Я настоятельно рекомендую набирать код вручную во время чтения. Однако если вы предпочитаете просмотреть код и поэкспериментировать с ним, можно клонировать репозиторий еще до начала работы с книгой.

https://max.ru/bookofgeek

Условные обозначения

Курсив

Курсивом выделены новые термины.

`Моноширинный шрифт`

Используется для листингов программ, а также внутри абзацев для обозначения таких элементов, как переменные и функции, базы данных, типы данных, переменные среды, операторы и ключевые слова, имена файлов и их расширений.

`Моноширинный жирный шрифт`

Показывает команды или другой текст, который пользователь должен ввести самостоятельно.

`Моноширинный курсив`

Показывает текст, который должен быть заменен значениями, введенными пользователем, или значениями, определяемыми контекстом.

Шрифт без засечек

Используется для обозначения URL, адресов электронной почты, названий кнопок, каталогов.



Этот рисунок обозначает совет или рекомендацию.



Этот рисунок обозначает общий комментарий.



Этот рисунок указывает на предупреждение или предостережение.

https://max.ru/bookofgeek

ГЛАВА 1

Введение в конкурентность

> Конкурентность — это когда удерживаешь в поле зрения множество задач одновременно.
>
> Параллелизм — это когда выполняешь множество задач одновременно.
>
> *Роб Пайк (Rob Pike)*

Чтобы по-настоящему оценить какое-либо явление, важно знать, как оно возникло, тем более если есть возможность отследить пройденные этапы и преодоленные трудности. Такой подход не только выдвигает на первый план текущие достижения, но и помогает осознать их значимость. Конкурентность в Java с момента своего появления прошла долгий путь. Потребовалось немало времени, чтобы она эволюционировала до теперешнего состояния. Прежде чем разбираться в современных достижениях — таких как виртуальные потоки и структурированная конкурентность, — необходимо проследить их появление и развитие. В этой главе я дам общее представление о конкурентности в Java, и мы кратко обсудим ее развитие.

Краткая история потоков в Java

Java изначально разрабатывали с прицелом на конкурентность. Это один из первых языков со встроенной поддержкой многопоточности. Год за годом возможности конкурентности в Java росли и совершенствовались. На этом пути были как успехи, так и ошибки, на которых приходилось учиться.

Развитие конкурентности в Java началось с базовой синхронизации и управления потоками. Затем в Java 5 добавили пакет java.util.concurrent (https://oreil.ly/5e8s1) с новыми возможностями, в том числе фреймворком Executor (https://oreil.ly/lDFVL), механизмом блокировок и конкурентными коллекциями. С появлением в Java 7 фреймворка Fork/Join многоядерные процессоры

https://max.ru/bookofgeek

получили способность конкурентно обрабатывать задачи. И уже совсем недавно, благодаря Project Loom (https://oreil.ly/gcaYC), преодолены сложность и ограничения традиционных потоков.

Проект предложил структурированную конкурентность и легкие потоки, построенные в пользовательском пространстве, без участия ядра ОС (user-mode threads). Все это призвано в конечном счете упростить и ускорить разработку конкурентного кода.

Почему так важны эти изменения? История эволюции конкурентности в Java — это неустанный поиск эффективности и простоты в условиях постоянно растущей сложности. Такая тенденция прослеживается не только в Java: она отражает как неизменные устремления сообщества разработчиков, так и траекторию развития индустрии ПО в целом.

Рассмотрим подробнее этапы развития конкурентности в Java и оценим достигнутый прогресс.

Язык Java построен на основе потоков

При проектировании языка Java конкурентность была явно обозначенной целью. Изначально встроенная поддержка потоков (threads) и концепция многопоточности — ключевые отличительные особенности данного языка. Благодаря этому Java-разработчики могут использовать конкурентность без оглядки на особенности конкретных операционных систем.

Поток в Java — это наименьшая единица выполнения. Он представляет собой независимый путь выполнения задачи внутри программы. Потоки разделяют одно адресное пространство, то есть имеют доступ к одним и тем же переменным, а также структурам данных программы. При этом у каждого потока есть собственный программный счетчик (program counter), стек и локальные переменные, что позволяет ему работать независимо. При необходимости такая архитектура упрощает взаимодействие между потоками.

Однако модель многопоточности в Java зависит от базовой операционной системы, которая планирует и выполняет потоки. Чтобы обеспечить эффективное выполнение, ОС распределяет процессорное время между потоками, управляя переходами между их состояниями. Распределяя потоки между несколькими процессорами (CPU), система достигает истинного параллелизма, повышая производительность конкурентных приложений (рис. 1.1).

Чем больше потоков в нашей Java-программе, тем больше сред выполнения мы фактически создаем. Мы получаем возможность выполнять несколько операций одновременно. Наиболее выгодно это для приложений, требующих высокого уровня параллелизма или конкурентности: для веб-серверов, пайплайнов

https://max.ru/bookofgeek

обработки данных и систем реального времени. Используя множество потоков, такие приложения могут выполнять несколько задач одновременно, повышая пропускную способность и уменьшая время отклика.

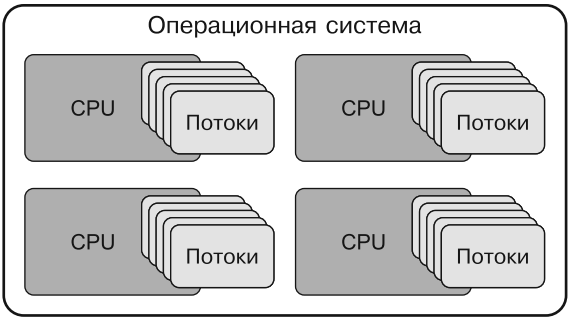


Рис. 1.1. Выполнение потоков разными CPU



Параллелизм и конкурентность

Термины *параллелизм (parallelism)* и *конкурентность (concurrency)* часто используются как синонимы, хотя они обозначают разные концепции. Параллелизм подразумевает одновременное выполнение нескольких задач, для чего требуется несколько единиц обработки (например, два или более ядер CPU). Конкурентность же связана с разработкой программ, отдельные части операций в которых могут перекрываться, даже если они не всегда выполняются одновременно. Параллелизм можно сравнить с работой нескольких строителей, бок о бок укладывающих кирпичи в стену дома, а конкурентность — с работой повара, который ловко управляется с приготовлением нескольких блюд. Теперь добавим еще поваров — суммарная производительность вырастет, но работа не обязательно станет параллельной: один повар может резать лук, а другой — ставить жаркое в духовку. Конечная цель остается неизменной — приготовить еду, возможно, из нескольких блюд.

Понятие потоков лежит в основе всей экосистемы Java и определяет принципы ее функционирования. Это фундамент, на котором построены многие мощные функции и инструменты. Даже функции, которые большинство программистов воспринимает как данность, без потоков были бы невозможны. Будь то система сборки мусора, решающая проблему управления памятью в Java, или выполнение простейшей Java-программы для вывода `Hello, World!` — в фоновом режиме работают потоки.

Приведу пример. Даже те, кто только знакомится с языком Java, используют потоки, сами того не зная. Казалось бы, какие потоки нужны для строки кода,

https://max.ru/bookofgeek

с которой начинается большинство учебников по программированию. Но JVM (Java Virtual Machine) выполняет данный код в потоке, который принято называть *главным потоком программы (main thread)*:

```
public class HelloWorld {
  public static void main(String[] args) {
    System.out.println("Hello, World!");
    // Вывод имени потока, выполняющего метод main
    System.out.println("Executed by thread: "
      + Thread.currentThread().getName());
  }
}
```

Запустим эту программу и получим имя потока, выполняющего метод `main`, как правило — тоже `main`:

```
Hello, World!
Executed by thread: main
```

Данный пример показывает, что даже самые простые Java-программы работают в парадигме потоков. Это влечет за собой серьезные последствия: Java-разработчики — сознавая это или нет — используют мощь потоков практически на каждом этапе своей работы, начиная от запуска элементарных программ и заканчивая применением продвинутых механизмов вроде сборки мусора.

Таким образом, потоки — обязательный элемент Java: именно они делают его языком, способным масштабироваться для работы с крупными базами данных и распределенными системами.

Потоки — основа программной платформы Java

Потоки интегрированы во все уровни программной платформы Java и играют ключевую роль не только в простом выполнении кода. Например, они лежат в основе обработки исключений, отладки и профилирования Java-приложений.

Исключения и потоки

В Java каждый поток имеет собственный стек вызовов, который фиксирует все вызовы методов в течение жизненного цикла потока. При возникновении исключения этот стек становится важной частью диагностической информации: он отображает последовательность вызовов методов, которые привели к исключению, — это помогает разработчикам выявить первопричину проблемы. Рассмотрим тот же пример:

```
import java.sql.SQLException;

public class CallStackDemo {
  public static void main(String[] args) throws InterruptedException {
    Thread thread = new Thread(CallStackDemo::processOrder);
```

https://max.ru/bookofgeek

```
    thread.setName("mcj-thread");
    thread.start(); thread.join();
  }

  static void processOrder() {
    validateOrderDetails();
  }

  static void validateOrderDetails() {
    checkInventory();
  }

  static void checkInventory() {
    updateDatabase();
  }

  static void updateDatabase() {
    try {
      throw new SQLException("Database connection error");
    } catch (SQLException e) {
      throw new InventoryUpdateException("Database Error: " +
        "Unable to update inventory", e);
    }
  }
}

class InventoryUpdateException extends RuntimeException {
  public InventoryUpdateException(String message, Throwable cause) {
    super(message, cause);
  }
}
```

В данном сценарии операция обновления базы данных завершается с ошибкой, которая распространяется вверх по стеку вызовов. Основной поток начинает с вызова метода `processOrder`, вызывающего `validateOrderDetails`, который вызывает `checkInventory`, а тот, в свою очередь, — `updateDatabase`, который выбрасывает исключение. В консоли мы увидим такой вывод:

```
Exception in thread "mcj-thread" ca.bazlur.mcj.chap1.InventoryUpdateException:
Database Error: Unable to update inventory
  at ca.bazlur.mcj.chap1.CallStackDemo.updateDatabase(CallStackDemo.java:33)
  at ca.bazlur.mcj.chap1.CallStackDemo.checkInventory(CallStackDemo.java:26)
  at ca.bazlur.mcj.chap1.CallStackDemo.
  validateOrderDetails(CallStackDemo.java:22)
  at ca.bazlur.mcj.chap1.CallStackDemo.processOrder(CallStackDemo.java:18)
  at java.base/java.lang.Thread.run(Thread.java:1583)
Caused by: java.sql.SQLException: Database connection error
  at ca.bazlur.mcj.chap1.CallStackDemo.updateDatabase(CallStackDemo.java:31)
  ... 4 more
```

Этот стек-трейс позволяет разработчикам изучить стек вызовов в контексте конкретного потока (в данном случае потока `mcj-thread`, где возникло

https://max.ru/bookofgeek

исключение). Такая гранулярность крайне полезна в JVM, особенно при отладке многопоточных приложений, где несколько потоков могут выполняться конкурентно для решения одинаковых или различных задач. Эта информация делает отладку более целенаправленной и эффективной, помогает быстрее выявлять проблемы и в итоге ускоряет их устранение.

Отладчик и потоки

Каким образом отладчик Java определяет, где именно приостановить выполнение работающего приложения? Ответ снова кроется в потоках. Подключаясь к различным потокам приложения, отладчик может выбрать, какой из них проверить или даже какому изменить состояние во время отладки активного приложения. Это критически важно для поиска и исправления ошибок в приложениях, где несколько потоков способны одновременно выполняться на разных этапах, а их взаимодействие требует анализа.

В контексте потоков доступны операции перемещения по коду: шаг со входом (step into), шаг с обходом (stepped over) и шаг с выходом (step out). В одной сессии отладки могут инспектироваться один или несколько независимых стеков вызовов активных потоков. Стек вызовов показывает историю событий, например, какие методы привели поток к текущей точке программы. Когда пытаешься обнаружить причину неожиданных исключений или ошибочных манипуляций с данными, очень важно понять, как поток пришел к определенному состоянию.

Профилировщик и потоки

Потоки равно важны как для понимания механизма управления задачами в Java, так и для профилирования. Их применение способствует высочайшей точности при отладке и обеспечивает гранулярный подход при профилировании. Фактически потоки являются основой анализа производительности в Java. Инструменты профилирования используют информацию о потоках, чтобы показать детальную картину работы приложения: выявляют узкие места, помогают диагностировать сложные проблемы синхронизации в многопоточном коде и находить способы ускорения его работы. Одним словом, потоки — это ключ к пониманию и повышению производительности многопоточных Java-приложений.

Потоки в Java играют важную роль во многих операциях (например, в диагностике, отладке и профилировании), предоставляя детальную аналитическую информацию о выполнении программ на уровне отдельных потоков. Хотя разработчики не используют потоки напрямую, те работают в фоновом режиме, усиливая преимущества Java-приложений. Примеры таких потоков: потоки сборщика мусора для очистки памяти; потоки компиляции в JIT-системе для повышения производительности; диспетчер сигналов, финализаторы

https://max.ru/bookofgeek

и обработчики ссылок для бесперебойной работы JVM, а также потоки виртуальной машины и сервисные потоки для основных задач JVM и диагностики. Можно смело утверждать, что потоки в Java представляют собой один из важнейших уровней программной платформы, и Java-разработчик обязан в них разбираться.

Зарождение потоков в Java 1.0

Выпущенная в 1996 году Java 1.0 изначально поддерживала потоки. Эта ключевая особенность выделяла Java среди многих языков того времени. Поначалу потоки можно было создавать только двумя способами: через наследование класса `Thread` или реализацию интерфейса `Runnable`. Например:

```
public class ThreadCreationDemo {
  public static void main(String[] args) {
    // Расширение класса Thread
    Thread t1 = new ThreadByExtension("Worker-1"); ❶
    t1.start();
    // Реализация интерфейса Runnable (классическая)
    Thread t2 = new Thread(
      new RunnableImplementation(), "Worker-2"); ❷
    t2.start();
    // Анонимный внутренний класс (Java 1.1)
    Thread t3 = new Thread(new Runnable() { ❸
      @Override
      public void run() {
        System.out.println("Anonymous: " +
          Thread.currentThread().getName());
      }
    }, "Worker-3");
    t3.start();
    // Лямбда-выражение (Java 8)
    Thread t4 = new Thread(() -> ❹
      System.out.println("Lambda: " +
        Thread.currentThread().getName()),
        "Worker-4");
    t4.start();
  }
}

class ThreadByExtension extends Thread {
  public ThreadByExtension(String name) {
    super(name);
  }
  @Override
  public void run() {
    System.out.println("Extended Thread: " + getName());
  }
}
```

https://max.ru/bookofgeek

```
class RunnableImplementation implements Runnable {
  @Override
  public void run() {
    System.out.println("Runnable: " +
      Thread.currentThread().getName());
  }
}
```

Рассмотрим ключевые принципы проектирования с самого начала по порядку.

❶ Наследование от класса `Thread` предлагает прямой доступ к методам потока, но это единичное наследование.

❷ Реализация `Runnable` — предпочтительный подход: определение задач отделяется от управления потоком, и сохраняется гибкость наследования.

❸ Анонимные внутренние классы, добавленные в Java 1.1, снизили объем boilerplate-кода для разовых задач.

❹ Лямбда-выражения (Java 8) сделали реализацию интерфейса `Runnable` еще лаконичнее, поскольку этот интерфейс — функциональный.

Этот базовый API оставался неизменным почти три десятилетия, что показывает устойчивость дизайна. Хотя в Java добавили множество решений для улучшения конкурентности, от `java.util.concurrent` до виртуальных потоков, потоки как исходные строительные блоки остаются основой многопоточной модели Java.

Запуск потоков

Независимо от способа создания потока его запуск всегда начинается с вызова метода `start` у объекта `Thread`. Это необходимый шаг, поскольку вызов `start` не только запускает метод `run`, но и выполняет подготовительные операции, такие как выделение системных ресурсов. После этого метод `start` вызывает метод `run` в новом потоке выполнения.



Крайне важно не вызывать метод `run` напрямую, поскольку это приведет к выполнению `run` в уже вызванном потоке, а не в новом.

Фреймворк Executor позволяет создавать потоки в современных Java-приложениях гораздо эффективнее, чем вручную через конструкторы. Исполнители (executors) абстрагируют процесс управления потоками, поддерживая пул потоков — группу заранее созданных потоков, готовых к выполнению задач. Этот подход существенно снижает затраты на создание потоков.

Когда задача передается исполнителю, она помещается в очередь. Затем один из потоков пула забирает задачу из очереди и выполняет ее. Такая организация

https://max.ru/bookofgeek

упрощает конкурентное программирование и оптимизирует использование ресурсов благодаря неоднократному использованию потоков.

Простой пример применения исполнителя для управления пулом потоков:

```
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;

public class ExecutorExample {
  public static void main(String[] args) {
    try (ExecutorService executor = Executors.newFixedThreadPool(5)) {
      for (int i = 0; i < 10; i++) {
        final int taskId = i;
        executor.submit(() -> {
          System.out.println("Executing task " + taskId
          + " in thread " + Thread.currentThread().getName());
        });
      }
    }
  }
}
```

Скрытые затраты на потоки

Многие современные веб-приложения работают по модели thread-per-request, где для каждого запроса предназначен отдельный поток, который управляет всем жизненным циклом запроса/ответа. В качестве примера рассмотрим жизненный цикл запроса в типичном веб-приложении, работающем под управлением контейнера сервлетов (Apache Tomcat, Jetty или любого Java EE веб-сервера). *Контейнер сервлетов (servlet container)* — это приложение, отвечающее за обработку запросов к веб-приложению. Получив запрос, контейнер назначает его для обработки одному из потоков своего пула. Поток активируется, как бы говоря: «Эй, мне пришел запрос от пользователя. Теперь это моя задача, я ею займусь». По сути, поток отвечает за обработку запроса/ответа в течение всего жизненного цикла (рис. 1.2).

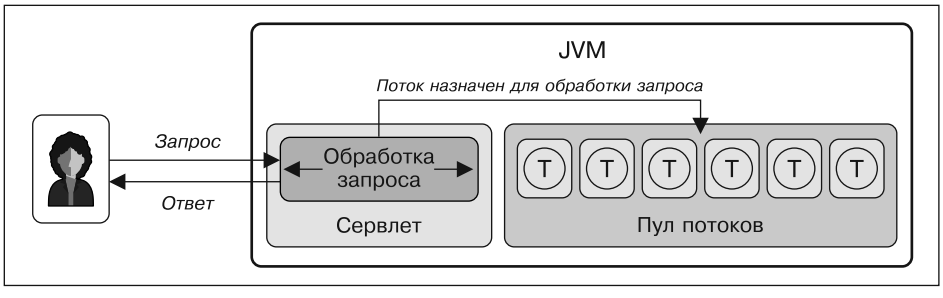


Рис. 1.2. Пул потоков для обработки запросов в сервлете

https://max.ru/bookofgeek

Эта модель особенно полезна при большом числе одновременных подключений, поскольку увеличение конкурентности, в свою очередь, повышает пропускную способность веб-приложения. Эта ключевая особенность модели thread-per-request позволяет современным веб-приложениям масштабироваться, чтобы эффективно справляться с растущим объемом запросов.

Большинство современных операционных систем способны обрабатывать миллионы конкурентных подключений, поэтому кажется разумным создавать еще больше потоков для увеличения пропускной способности. Из закона Литтла (Little's Law) можно сделать вывод: чем больше потоков в пуле, тем выше должна быть пропускная способность[1]. Однако на практике такая логика оказывается неоднозначной и не всегда приводит к ожидаемым результатам.



Пропускная способность (throughput) веб-приложений — это скорость обработки запросов и выдачи сервером ответов; обычно измеряется в запросах в секунду (RPS) или транзакциях в секунду (TPS). Она отражает способность приложения справляться с нагрузкой и служит количественным показателем для оценки производительности, масштабируемости и использования ресурсов. Пропускную способность можно рассчитать по формуле:

Пропускная способность = Общее количество обработанных запросов / Общее время.

Эта формула служит ориентиром при тестировании производительности, масштабируемости и прогнозировании нагрузки, а также помогает оптимально распределять ресурсы для обеспечения качественного пользовательского опыта.

Важно учитывать объем памяти, занимаемый каждым потоком: около 2 МиБ[2] (вне кучи на каждый поток). В крупномасштабных приложениях с тысячами конкурентных подключений можно быстро достичь значительного расхода памяти. Совокупный объем памяти, занятый потоками, существенно влияет на количество соединений, с которыми может работать приложение. Более того, если приложение исчерпает физическую память, начнется подкачка данных на диск — операция, которая выполняется значительно медленнее, чем обращение к RAM. Время чтения с диска может быть в 1000 раз больше, чем из RAM. Уже один этот фактор способен серьезно повлиять на производительность.

Кроме прочего, важно понимать, что потоки в Java представляют собой тонкую обертку над нативными потоками базовой операционной системы.

---

[1] Закон Литтла — это фундаментальный принцип теории очередей, применимый и к многопоточным приложениям. Он связывает три ключевых параметра производительности: задержку (latency), конкурентность (concurrency) и пропускную способность (throughput). В следующей главе мы детально рассмотрим этот закон, и вы увидите, как увеличение конкурентности повышает пропускную способность.

[2] Это стандартный размер для большинства Linux-окружений, однако в зависимости от операционной системы он может меняться.

https://max.ru/bookofgeek

Максимальное количество потоков, которое можно создать для приложения, фактически ограничено лимитом на создание нативных потоков, установленным базовой ОС. Почти все операционные системы имеют верхний предел числа создаваемых потоков — это узкое место для масштабирования приложения.

Помимо прочего, существуют затраты на переключение контекста. Создание потока — это нагрузка не только на память, но и на процессор. При переключении между потоками происходит смена контекста: необходимо сохранить состояние текущего потока и восстановить состояние нового. Все переключения контекста потребляют такты CPU, что при высокой нагрузке существенно влияет на производительность. Это еще один недооцененный источник нагрузки в вашем приложении.

Сколько потоков можно создать

Теперь рассмотрим, как измерить ограничение на создание потоков в вашей среде. Запустив простую тестовую программу, можно определить максимальное количество потоков, которое система способна обработать без возникновения проблем. Вот заготовка кода для начала:

```
import java.util.concurrent.atomic.AtomicInteger;
import java.util.concurrent.locks.LockSupport;

public class ThreadLimitTest {
  public static void main(String[] args) {
    var threadCount = new AtomicInteger(0);
    try {
      while (true) {
        var thread = new Thread(() -> {
          threadCount.incrementAndGet();
          LockSupport.park();
        });
        thread.start();
      }
    } catch (OutOfMemoryError error) {
      System.out.println("Reached thread limit: " + threadCount);
      error.printStackTrace();
    }
  }
}
```

Выполним эту программу и посмотрим, сколько потоков можно создать, не исчерпав память и не достигнув других системных ограничений:

```
Reached thread limit: 16363
java.lang.OutOfMemoryError: unable to create native thread: possibly out of
memory or process/resource limits reached
  at java.base/java.lang.Thread.start0(Native Method)
  at java.base/java.lang.Thread.start(Thread.java:1526)
  at ca.bazlur.chapter0.ThreadLimitTest.main(ThreadLimitTest.java:15)
```

https://max.ru/bookofgeek

На моей машине ошибка `OutOfMemoryError` (https://oreil.ly/vD03r) возникла после создания 16 363 потоков. Этот эксперимент показывает, что существует предел числа создаваемых потоков, ограниченный в основном операционной системой и оборудованием. Соответственно, на вашей машине этот лимит может отличаться.

Как видите, потоки обеспечивают значительные преимущества для масштабирования веб-приложений, но при этом они связаны с определенными затратами. Эти, иногда неочевидные, затраты необходимо тщательно учитывать для достижения оптимальной производительности.

Эффективность использования ресурсов в высоконагруженных приложениях

Современные приложения часто должны обрабатывать большие объемы данных и одновременно обслуживать высокий входящий трафик. Эти условия создают серьезные проблемы для финансовой устойчивости, особенно когда облачные решения стали для многих компаний средой по умолчанию. Даже незначительная неэффективность использования ресурсов может привести к стремительному росту затрат. Поскольку количество доступных потоков ограничено, важно использовать их рационально. Однако на практике потоки часто блокируются, что снижает их эффективность.

Рассмотрим следующий пример, где на основе различных факторов рассчитывается персональная кредитоспособность:

```
public Credit calculateCredit(Long personId) {
  var person = getPerson(personId);         // Вызов БД блокирует поток
  var assets = getAssets(person);           // Вызов API блокирует поток
  var liabilities = getLiabilities(person); // Вызов БД блокирует поток
  importantWork();                          // Ресурсоемкие вычисления
  return calculateCredits(assets, liabilities);
}
```

В приведенном фрагменте кода показана последовательность из пяти вызовов методов, выполняющихся один за другим. Допустим, каждый вызов занимает 200 мс. Тогда время выполнения метода `calculateCredit()` составит около 1 с — примерно в пять раз больше (1000 мс = 200 мс × 5).

Основная проблема здесь заключается не в простое потока, а в его блокировке во время операций ввода-вывода. Когда поток выполняет `getPerson(personId)`, он делает запрос к базе данных и вынужден ждать ответа по сети. В этот период поток нельзя переназначить для обработки других запросов или выполнения иной работы. Аналогичная блокировка происходит при вызове `getAssets()` и `getLiabilities()`. Хотя технически поток «занят» выполнением этих методов,

https://max.ru/bookofgeek

большую часть времени он заблокирован операциями ввода-вывода, а не занимается продуктивными вычислениями.

Такое поведение с блокировками весьма неэффективно: ценные ресурсы потоков простаивают в ожидании ответов внешних систем, хотя могли бы обрабатывать другие запросы. В высоконагруженных приложениях это существенно ограничивает масштабируемость, так как каждый заблокированный поток означает, что для обработки входящих запросов стало одним потоком меньше.

Чтобы достичь высокой эффективности потоков, в частности минимизировать время блокировки при операциях ввода-вывода, в Java был внесен ряд важных изменений. Среди них: поддержка асинхронных вызовов методов (позволяющих методам выполняться независимо во время операций ввода-вывода), пулы потоков (неоднократное использование фиксированного числа потоков для выполнения задач) и, не так давно, современные реактивные модели программирования (основанные на неблокирующем событийно-ориентированном взаимодействии).

Начнем с классических методов работы с потоками, таких как ручное создание и управление отдельными потоками, а затем постепенно перейдем к современным предложениям.

Стратегия параллельного выполнения

В приведенном ранее фрагменте кода мы видим, что вызовы методов `getAssets()`, `getLiabilities()` и `importantWork()` не зависят друг от друга, так что их можно выполнять параллельно. Такой подход сократит время блокировки нашего главного потока, хотя и не устранит ее полностью.

Наша стратегия параллельного выполнения действительно уменьшает время простоя главного потока. Поскольку он меньше простаивает, то после завершения выполнения метода `calculateCredit()` быстрее освобождается для обработки других задач.

Начнем с создания базовых структур данных, которые понадобятся для нашего кредитного калькулятора:

```
// Модели для расчета кредита
record Credit(double score) {}
record Person(Long id, String name) {}
record Asset(String type, double value) {}
record Liability(String type, double amount) {}
```

Реализуем наш подход к параллельному выполнению:

```
import java.util.List;
import java.util.concurrent.Callable;
import java.util.concurrent.atomic.AtomicReference;
```

https://max.ru/bookofgeek

```
Credit calculateCreditWithUnboundedThreads(Long personId)
                                    throws InterruptedException {

  var person = getPerson(personId);

  var assetsRef = new AtomicReference<List<Asset>>(); ❶
  var t1 = new Thread(() -> {
    var assets = getAssets(person);
    assetsRef.set(assets); ❷
  });

  var liabilitiesRef = new AtomicReference<List<Liability>>();
  Thread t2 = new Thread(() -> {
    var liabilities = getLiabilities(person);
    liabilitiesRef.set(liabilities);
  });

  var t3 = new Thread(() -> importantWork()); ❸

  t1.start(); ❹
  t2.start();
  t3.start();

  t1.join(); ❺
  t2.join();

  var credit = calculateCredits(assetsRef.get(), liabilitiesRef.get()); ❻

  t3.join(); ❼

  return credit;
}
```

Рассмотрим, как работает данный код.

❶ Для безопасного обмена данными между потоками используем `AtomicReference` (https://oreil.ly/zmP1k).

❷ Потокобезопасное присваивание результатов, доступных из основного потока.

❸ Независимые задачи могут выполняться параллельно, не влияя на расчет кредита.

❹ Все три потока запускаются конкурентно, сокращая общее время выполнения.

❺ Ожидается завершение потоков для активов и пассивов перед последующим расчетом кредита.

❻ Расчет кредита выполняется с использованием результатов параллельных операций.

❼ Обеспечивает завершение независимой работы перед возвращением методом значений.

https://max.ru/bookofgeek

Мы использовали `AtomicReference` из стандартного пакета конкурентности Java. `AtomicReference` — это потокобезопасная структура данных, содержащая ссылку на единственный объект. Поскольку мы работаем с несколькими потоками, использование этого инструмента крайне важно.



Используйте `AtomicReference`, когда требуется безопасно распределять ссылки на объекты между потоками. Для примитивных значений рекомендуется выбирать `AtomicInteger` (https://oreil.ly/0m6-z), `AtomicLong` (https://oreil.ly/ bwxw6) или другие атомарные типы.

Чтобы завершить создание калькулятора кредитов, необходимо добавить вспомогательные методы. Каждый из них имитирует реальные операции, такие как запросы к базе данных и вызовы API:

```
// Имитационные методы с задержкой 200 мс каждый
private Person getPerson(Long personId) {
  simulateDelay(200);
  return new Person(personId, "John Doe");
}

private List<Asset> getAssets(Person person) {
  simulateDelay(200);
  return List.of(
    new Asset("House", 300000),
    new Asset("Car", 25000)
  );
}

private List<Liability> getLiabilities(Person person) {
  simulateDelay(200);
  return List.of(
    new Liability("Mortgage", 200000),
    new Liability("Credit Card", 5000)
  );
}

private void importantWork() {
  simulateDelay(200);
  System.out.println("Important work completed");
}

private Credit calculateCredits(List<Asset> assets,
                        List<Liability> liabilities) {
  simulateDelay(200);
  double totalAssets = assets.stream().mapToDouble(Asset::value).sum();
  double totalLiabilities = liabilities.stream()
    .mapToDouble(Liability::amount)
    .sum();
    double creditScore = (totalAssets - totalLiabilities) / 1000;
    return new Credit(creditScore);
  }
```

https://max.ru/bookofgeek

```
private void simulateDelay(int milliseconds) {
  try {
    Thread.sleep(milliseconds);
  } catch (InterruptedException e) {
    Thread.currentThread().interrupt();
    throw new RuntimeException(e);
  }
}
```



Корректная обработка InterruptedException

При работе с `InterruptedException` в Java важно сохранять статус прерывания потока. Приведенный шаблон демонстрирует корректный способ обработки этого исключения:

```
catch (InterruptedException e) {
  Thread.currentThread().interrupt();
  throw new RuntimeException(e);
}
```

Когда поток прерывается через `Thread.interrupt()`, устанавливается внутренний флаг, заставляя блокирующие методы (например, `sleep()`, `wait()` или блокирующие операции ввода-вывода) выбросить `InterruptedException`. Однако перехват этого исключения сбрасывает флаг прерывания, что может вызвать проблемы для кода выше по стеку вызовов, которому необходимо знать о прерывании. Вызов `Thread.currentThread().interrupt()` повторно устанавливает флаг прерывания для текущего потока. Это гарантирует, что любой вызывающий код, который проверяет `Thread.interrupted()` или `Thread.currentThread().isInterrupted()`, безошибочно обнаружит факт прерывания.

Можно обернуть `InterruptedException` в `RuntimeException`, что позволит прерыванию распространяться вверх по стеку вызовов даже через методы, не объявляющие `InterruptedException` в своей сигнатуре. Это особенно полезно в случаях, когда нельзя добавлять проверяемые исключения в сигнатуры методов (например, при реализации интерфейсов, которые их не объявляют).

Для сравнения мы также сохраним оригинальную последовательную версию в коде:

```
// Последовательная версия (оригинал)
public Credit calculateCredit(Long personId) {
// Тело метода доступно выше
}
```

Чтобы проанализировать, повысилась ли производительность, полезно измерять время выполнения методов. Вот utility-класс для этой цели:

```
import java.util.concurrent.Callable;

public class ExecutionTimer {
  public static <T> T measure(Callable<T> task) throws Exception {
    long startTime = System.nanoTime();
```

https://max.ru/bookofgeek

```
    try {
      return task.call();
    } finally {
      long endTime = System.nanoTime();
      // Преобразование в миллисекунды
      long duration = (endTime - startTime) / 1_000_000;
      System.out.println("Execution time: " + duration +
                         " milliseconds");
    }
  }
}
```



Такой подход призван упростить понимание кода. Для надежного бенчмаркинга рекомендуется использовать специализированные инструменты, например JMH (https://oreil.ly/w6Nsy), которые учитывают такие факторы, как прогрев JVM, устранение мертвого кода и другие оптимизации.

Мы предположили, что время выполнения каждого метода составляет 200 мс, поэтому можем определить, сколько потребуется для выполнения приведенного выше кода.

Для симуляции времени выполнения можно использовать метод `Thread.sleep(200)`. Теперь сравним оба подхода:

```
public class ParallelExecutionDemo {

  public static void main(String[] args) throws Exception {
    CreditCalculatorService service = new CreditCalculatorService();

    System.out.println("=== Sequential Execution ===");
    ExecutionTimer.measure(() -> service.calculateCredit(1L));

    System.out.println("\n=== Parallel Execution ===");
    ExecutionTimer.measure(() -> {
      try {
        return service.calculateCreditWithUnboundedThreads(1L);
      } catch (InterruptedException e) {
        Thread.currentThread().interrupt();
        throw new RuntimeException(e);
      }
    });
  }
}
```

При выполнении предыдущего кода будет получен примерно такой результат:

```
=== Sequential Execution ===
Important work completed
Execution time: 1026 milliseconds
=== Parallel Execution ===
Important work completed
Execution time: 616 milliseconds
```

https://max.ru/bookofgeek

Благодаря рефакторингу кредитных расчетов и использованию многопоточности мы добились значительного роста производительности. Новая версия работает в 1,66 раза быстрее предыдущей (616 мс против 1026 мс) — пользователи, несомненно, заметят ускорение.

Хотя параллельное выполнение действительно повышает производительность, уменьшая время блокировки инициирующего потока, оно создает собственные проблемы, особенно с управлением потоками и использованием системных ресурсов. Создание потоков напрямую, как мы видели, не предусматривает подконтрольного механизма управления их жизненным циклом и распределения ресурсов. Число потоков может вырасти настолько, что они исчерпают системные ресурсы. Каждый поток — ресурсоемкая сущность, потребляющая память и вычислительную мощность. В результате, поскольку потоки создаются на лету (ad hoc) и бесконтрольно, их избыточное число может привести к исключениям `java.lang.OutOfMemoryError` (https://oreil.ly/vD03r), аварийному завершению и другим узким местам во время выполнения.

Знакомство с фреймворком Executor

Чтобы избежать рисков, связанных с созданием потоков напрямую, лучше использовать один из фреймворков конкурентности Java, например `ExecutorService`, который предлагает более структурированный подход к работе с потоками.

Более того, применение `ExecutorService` не только позволяет контролировать число конкурентно выполняемых потоков, но и обеспечивает их эффективное неоднократное использование благодаря управлению жизненным циклом. Это устраняет проблему избыточного потребления ресурсов, связанных с жизненным циклом потока.

Сделаем рефакторинг предыдущего кода и используем `ExecutorService`:

```
public Credit calculateCreditWithExecutor(Long personId)
  throws ExecutionException, InterruptedException {

  try (ExecutorService executor = Executors.newFixedThreadPool(5)) { ❶
    var person = getPerson(personId);

    var assetsFuture = executor.submit(() -> getAssets(person)); ❷
    var liabilitiesFuture = executor.submit(() -> getLiabilities(person)); ❷
    executor.submit(() -> importantWork()); ❸

    return calculateCredits(assetsFuture.get(), liabilitiesFuture.get()); ❹
  }
}
```

Разберем пошагово, что мы сделали.

❶ Создали пул потоков, который автоматически завершит работу при выходе из блока `try`.

https://max.ru/bookofgeek

❷ Приняли на выполнение обе критические задачи, результаты которых нам необходимы.

❸ Приняли дополнительную работу, не требующую блокировки основного потока.

❹ Дождались завершения критических задач и обработали их результаты.

Теперь измерим время выполнения кода с помощью `ExecutionTimer.measure()` и сравним результаты с предыдущей реализацией:

```
public static void main(String[] args) throws Exception {
  System.out.println("\n=== Parallel Execution With Executors ===");
  ExecutionTimer.measure(() -> {
    try {
      return service.calculateCreditWithExecutor(1L);
    } catch (InterruptedException e) {
      Thread.currentThread().interrupt();
      throw new RuntimeException(e);
    }
  });
}
```

При запуске кода мы получим следующий вывод:

```
=== Parallel Execution With Executors ===
Important work completed
Execution time: 630 milliseconds
```

Примечательно, что время выполнения осталось практически таким же, как в предыдущей версии. Пакет Executors предоставляет полезные функции для управления конкурентными задачами в Java-приложениях: они упрощают создание потоков и управление ими, могут ускорять рабочие нагрузки, а также делают код более организованным и чистым. Теперь разберемся с проблемами, которые не решило использование пакета Executors.

Оставшиеся проблемы

Фреймворк Executor значительно улучшает управление ресурсами и асинхронное выполнение в Java-приложениях. Однако, чтобы получать от использования этого подхода максимум преимуществ, следует знать и о его ограничениях.

Блокировка при вызове Future.get()

Несмотря на асинхронность, которую обеспечивают объекты `Future`, вызов метода `get()` по-прежнему блокирует выполнение. То есть мы переместили блокировку из одного места в другое, но полностью устранить ее не удалось.

Избыточные затраты на поддержание когерентности кэшей. Ложное разделение данных

Иногда главный поток работает на одном из ядер CPU, а задачи, которые отправляются в пул потоков, выполняются потоками, работающими на

https://max.ru/bookofgeek

других ядрах. В такой ситуации состояния кэшей двух ядер могут стать неконсистентными, и возникает риск снижения производительности, поскольку протоколы когерентности кэшей (cache coherence protocols) будут непрерывно инвалидировать и синхронизировать кэш-линии между ядрами. Основная идея заключается в ускорении доступа к данным, поэтому все современные CPU оснащены L-кэшами[1]. Они хранят небольшие блоки данных (размер которых зависит от архитектуры оборудования), чтобы при следующем обращении CPU к этим данным их можно было получить из кэша, а не из более медленной оперативной памяти. Такой блок данных называется *кэш-линией (cache line)*, и его использование существенно повышает общую производительность.

Однако при конкурентном выполнении нескольких задач есть вероятность, что данные из двух последовательных задач окажутся в одной кэш-линии. Если обе задачи выполняются на одном CPU, повторного обращения к основной памяти не требуется, так как данные уже находятся в кэше. В этом случае вторая операция завершится немного быстрее первой. Но если задачи выполняются на разных CPU, второй CPU должен инвалидировать свою кэш-линию при изменении любой ее части первым CPU, а затем перезагрузить всю кэш-линию из основной памяти или кэша другого ядра.

Это явление, известное как ложное разделение данных (false sharing), возникает, когда разные потоки изменяют разные переменные, которые случайно оказались в одной кэш-линии. Частые инвалидации и перезагрузки кэш-линий могут негативно сказаться на общей производительности для такой функциональности, как Executor, где задачи распределяются между несколькими CPU.

Недостаток компонуемости

Фреймворк Executor по своей природе остается императивным. В императивном стиле программирования нет ничего плохого, но многие разработчики считают, что функциональный и декларативный стили удобнее читать и поддерживать.



Протоколы когерентности кэшей

Для обеспечения согласованного представления памяти во всех ядрах современные CPU используют протоколы когерентности кэшей (например, MESI, MOESI). Когда одно ядро записывает данные в кэш-линию, протокол инвалидирует или обновляет эту линию в кэшах других ядер. Это гарантирует корректность работы, но может привести к снижению производительности из-за частых инвалидаций и передачи данных между ядрами.

[1] L-кэши представляют собой многоуровневую иерархию сверхбыстрой памяти (L1, L2, L3) внутри CPU, которая хранит часто используемые данные ради быстрого доступа к ним.

https://max.ru/bookofgeek

Мы вернулись к исходному вопросу: «Что же делать?» Как решить эти проблемы и как с помощью исполнителей написать более эффективный и поддерживаемый код?

Выход за пределы базового пула потоков

Фреймворк Executor весьма полезен. Однако, как уже было сказано, такие сценарии, как интенсивное соперничество за кэш (cache contention) или сложные зависимости между задачами, могут создавать проблемы. Их решает функциональность `ForkJoinPool` благодаря своей ориентированной на производительность архитектуре и специализированным алгоритмам. Эта реализация `ExecutorService` позволяет повысить производительность и эффективнее использовать ресурсы благодаря усовершенствованному механизму пула потоков и интеллектуальному планированию задач. Поговорим об этом подробнее.

Привязка к кэшу и распределение задач

Привязка к кэшу (cache affinity) позволяет выполнять задачи с учетом местоположения кэша CPU. Если задача запускается на определенном CPU, ее данные загружаются в кэш ядра этого процессора. Связанные с ней задачи, выполняющиеся там же, могут использовать уже сохраненные в кэше данные, что сокращает время доступа к памяти и повышает производительность.

`ForkJoinPool` способствует привязке к кэшу, так как потоки-воркеры обычно выполняют задачи из своих локальных очередей. Это особенно выгодно, когда задачи часто обращаются к одним и тем же общим данным. Поддерживая привязку, `ForkJoinPool` сводит к минимуму промахи кэша (cache misses) — ситуации, когда необходимых задаче данных в кэше нет и их требуется получать из оперативной памяти, что существенно замедляет процесс.

Рассмотрим, к примеру, набор задач, параллельно обрабатывающих большой массив. Поскольку они обрабатывают различные фрагменты одного массива, есть смысл выполнять их на одном ядре: данные фрагментов хранятся в кэше, и следующая задача получает к ним быстрый доступ. В противном случае данные придется постоянно перезагружать в кэш. Привязка к кэшу снижает затраты, связанные с повторной загрузкой данных в него.

Алгоритм кражи задач

Для эффективного распределения нагрузки между потоками пула `ForkJoinPool` также использует динамический алгоритм кражи задач (work-stealing algorithm). Вместо единой общей очереди задач каждый поток в пуле имеет собственную очередь. Когда у потока заканчиваются задачи, он «крадет» задачи из хвоста очереди другого потока. Этот динамический алгоритм позволяет максимально

https://max.ru/bookofgeek

задействовать процессор: ни один поток не простаивает, пока есть задачи для выполнения. На рис. 1.3 показано, как работает алгоритм кражи задач в пуле `ForkJoinPool`. Динамическое перераспределение обеспечивает постоянную продуктивность всех потоков и тем самым повышает общую загруженность CPU и минимизирует время простоя.

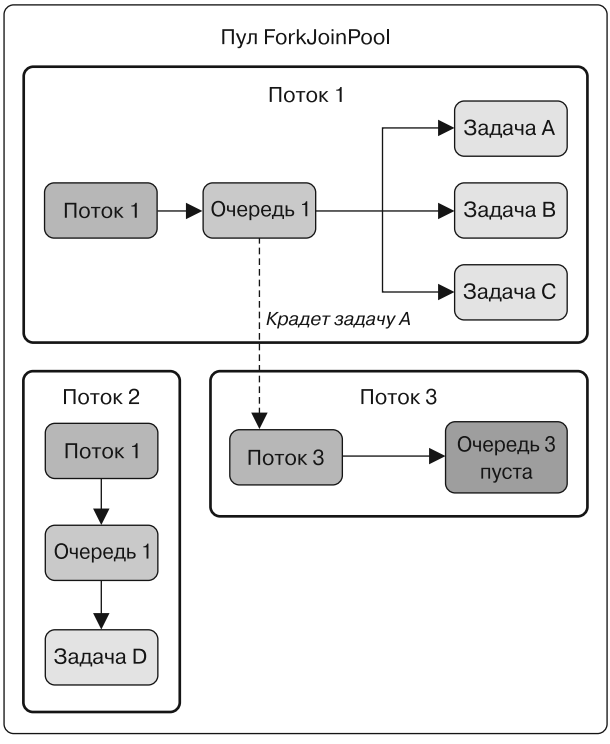


Рис. 1.3. Алгоритм кражи задач в пуле ForkJoinPool

Пример использования ForkJoinPool

Пул `ForkJoinPool` упрощает процесс отправки задач и весьма эффективно автоматически распределяет их. Дополнительного ручного управления не требуется. Базовый пример:

```
ForkJoinPool forkJoinPool = new ForkJoinPool();
forkJoinPool.submit(() -> {
  // здесь ваши параллельные задачи
}).join();
```

Обратите внимание, насколько просто отправлять задачи в пул ForkJoinPool. Благодаря этой функциональности работа эффективно распределяется автоматически, что избавляет разработчика от сложного управления потоками.

https://max.ru/bookofgeek

Обычно `ForkJoinPool` ассоциируется с алгоритмом «Разделяй и властвуй», который разбивает задачи на подзадачи с помощью классов `RecursiveAction` (https://oreil.ly/JNpA4) и `RecursiveTask` (https://oreil.ly/4kYNM). Однако сейчас мы фокусируемся на самом `ForkJoinPool`, а не на этой стратегии.

Более подробно о `ForkJoinPool` мы поговорим в следующей главе.

Компоновка с помощью CompletableFuture

Итак, `ForkJoinPool` и фреймворк Executor предлагают механизмы повышения производительности, но компоновать сложные асинхронные процессы по-прежнему обременительно. В Java 8 появился класс `CompletableFuture` (https://oreil.ly/5VTeu), способный справиться с этой проблемой. Он упрощает асинхронное программирование, фокусируясь на компоновке и удобстве использования.

Рассмотрим, как компонуемый fluent API с `CompletableFuture` справится с предыдущим примером.

Компонуемый fluent API

`CompletableFuture` позволяет писать асинхронный код в иной манере: перейти от стиля с массой колбэков к декларативному и функциональному. Богатый API дает возможность легко объединять несколько асинхронных операций в цепочку, улучшая читабельность кода и упрощая его поддержку. Вернемся к предыдущему примеру:

```
import java.util.concurrent.CompletableFuture;
import java.util.concurrent.ExecutionException;
import static java.util.concurrent.CompletableFuture.*;

public Credit calculateCreditWithCompletableFuture(Long personId)
  throws InterruptedException, ExecutionException {
    return runAsync(() -> importantWork()) ❶
      .thenCompose(aVoid -> supplyAsync(() -> getPerson(personId))) ❷
      .thenCombineAsync(supplyAsync(() -> getAssets(getPerson(personId))), ❸
              (person, assets)
                 -> calculateCredits(assets, getLiabilities(person))) ❹
      .get(); ❺
}
```

Рассмотрим подход с использованием `CompletableFuture`.

❶ Начинается асинхронное выполнение независимой задачи `importantWork`.

❷ После завершения `importantWork` — асинхронное получение `personId`.

❸ Объединение данных `person` с полученными параллельно `assets`.

❹ Расчет итогового кредитного рейтинга на основе скомбинированных данных.

❺ Блокировка и ожидание завершения финального результата.

https://max.ru/bookofgeek

Из этого примера видно, как `CompletableFuture` позволяет структурировать асинхронные рабочие процессы понятно и лаконично. Однако и у этого подхода есть недостатки.

Рассмотрим преимущества и недостатки данного API.

Преимущества CompletableFuture

`CompletableFuture` кардинально меняет подход к структурированию асинхронного кода благодаря своему fluent API. Вместо того чтобы использовать запутанные вложенные колбэки или разрастающуюся иерархию классов объектов задач, вы определяете рабочие процессы как серию преобразований и промежуточных вычислений. Получившийся код проще читать и поддерживать, а вероятность ошибок в приложениях, где асинхронность играет ключевую роль, снижается.

В основе `CompletableFuture` лежит фреймворк Fork/Join, применяющий оптимизированный алгоритм кражи задач для эффективного распределения нагрузки и улучшенной масштабируемости. Встроенные методы, такие как `exceptionally()`, `handle()` и `whenComplete()`, позволяют управлять обработкой исключений. Вы также можете использовать кастомные исполнители для тонкой настройки управления потоками под конкретные рабочие нагрузки. И наконец, хотя блокирующий `get()` иногда необходим на границах пайплайна, основная часть пайплайна остается неблокирующей, что позволяет создавать высокоотзывчивый и эффективный с точки зрения ресурсов код.

Недостатки и ограничения

Хотя `CompletableFuture` открывает большие возможности, разработчикам, привыкшим к императивному стилю программирования, придется изменить образ мышления. Скорее всего, нужно будет долго изучать этот богатый API, чтобы использовать его на все сто. Необходимо разобраться с оптимальными точками его применения и механизмом передачи значений для предоставляемых им методов. Потребуется немало времени на изучение и практику, при этом не исключен риск неправильного использования.

Например, необходимо очень осторожно применять блокирующий метод `get()`, иначе он может привести к блокировке в потоке выполнения, что сведет к нулю преимущества асинхронного программирования. Могут также возникнуть проблемы с цепочками `CompletableFuture`, когда имеется несколько зависимостей и для восстановления требуется выбрасывать и ловить исключения. При неправильном проектировании отладка может превратиться в кошмар. Наконец, отладка кода, написанного с цепочками методов в стиле `CompletableFuture`, создает дополнительные сложности с управлением потоком выполнения. Если вы привыкли к стандартным инструментам отладки

https://max.ru/bookofgeek

в различных IDE — пошаговому выполнению, точкам останова в определенных местах и просмотру контекста выражений с возвратом назад по коду, то придется столкнуться с немалыми трудностями. Это связано с инвертированным и неочевидным потоком выполнения асинхронного кода, который не выполняется построчно.



Хотя преимущества асинхронного программирования очевидны, а `CompletableFuture` наряду с альтернативными решениями (например, реактивными фреймворками) предлагают мощные инструменты, возникает судьбоносный вопрос: готовы ли мы к преодолению организационной сложности?

Асинхронное программирование кардинально меняет подходы к проектированию, отладке и поддержке приложений. Прирост производительности сопровождается существенными затратами: повышенной когнитивной нагрузкой, сложностью отладки и архитектурной усложненностью. Прежде чем внедрять эти паттерны, оцените, обладает ли ваша команда разработчиков достаточной квалификацией для соблюдения принципов чистоты кода, обработки сложных сценариев ошибок и эффективной отладки асинхронных цепочек вызовов.

Это не просто техническое решение. Это архитектурное обязательство, которое повлияет на все аспекты жизненного цикла приложения. Делайте осознанный выбор с учетом компетенций команды и реальных требований к производительности вашего приложения.

Другая парадигма асинхронного программирования

Класс `CompletableFuture` предоставляет широкие возможности, однако, чтобы преодолеть оставшиеся ограничения (или достичь более высоких уровней производительности в некоторых случаях), требуется сменить не только образ мышления. Еще одна парадигма, ориентированная на потоки данных, асинхронную обработку событий и неблокирующие операции, — реактивное программирование. Такие фреймворки, как RxJava, Akka, Eclipse Vert.x, Spring WebFlux и др., реализуют эту парадигму в Java, предлагая богатый набор инструментов.

В качестве примера переделаем с помощью Spring WebFlux наш предыдущий калькулятор `calculateCredit()`. Сначала добавим необходимую зависимость и импорты:

```
<dependency>
  <groupId>org.springframework</groupId>
  <artifactId>spring-webflux</artifactId>
  <version>6.0.0</version>
</dependency>
```

https://max.ru/bookofgeek



Для тестирования реактивного примера создайте Maven-проект и добавьте зависимость `reactor-core`, как показано в предыдущем файле `pom.xml`.

Добавим в наш класс `CreditCalculatorService.java` следующий метод:

```
public Mono<Credit> calculateCreditReactive(Long personId) {
  Mono<Void> importantWorkMono = Mono.fromRunnable(() -> importantWork()); ❶
  Mono<Person> personMono = Mono.fromSupplier(() -> getPerson(personId)); ❷
  Mono<List<Asset>> assetsMono = personMono
    .map(person -> getAssets(person)); ❸
  Mono<List<Liability>> liabilitiesMono = personMono
    .map(person -> getLiabilities(person)); ❹

  return importantWorkMono.then( ❺
    Mono.zip(assetsMono, liabilitiesMono) ❻
      .map(tuple -> {
        List<Asset> assets = tuple.getT1();
        List<Liability> liabilities = tuple.getT2();
        return calculateCredits(assets, liabilities); ❼
      })
  );
}
```

Рассмотрим реактивный поток выполнения.

❶ Создаем Mono, который асинхронно выполняет `importantWork` при подписке.

❷ Обертываем поиск `person` в реактивный поток (reactive stream).

❸ Преобразуем поток `person` для асинхронного получения активов.

❹ Преобразуем поток `person` для асинхронного получения обязательств.

❺ Ожидаем завершения `importantWork`, затем переходим к расчету кредита.

❻ Объединяем реактивные потоки `assets` и `liabilities` в общий поток, содержащий оба результата.

❼ Вычисляем итоговый кредитный рейтинг на основе объединенных данных.

Просматривая код, вы заметите концептуальное сходство с подходом на основе `CompletableFuture`, но уже с использованием терминологии и операторов реактивных потоков. Тех, кто не знаком с реактивным программированием, подобный стиль может озадачить и потребует некоторых усилий для освоения таких концепций, как Publisher (издатель), Subscriber (подписчик), backpressure (обратное давление) и реактивные операторы.

В мои планы не входило объяснять реактивное программирование в этой книге, но вы можете поискать другие издания, посвященные реактивному стеку.

https://max.ru/bookofgeek

Мы рассмотрим некоторые ограничения реактивного подхода, чтобы понять, почему нам нужно что-то иное — а именно виртуальные потоки, которые и *являются* основной темой этой книги.

Отрицательная сторона использования реактивных фреймворков

Фраза «бесплатный сыр бывает только в мышеловке» вполне подходит реактивному программированию. Перечислим ключевые проблемы, которые оно создает.

Сложности при изучении

Освоение основ реактивного программирования требует серьезной перестройки мышления для разработчиков, привыкших к императивному стилю. Изучение таких фундаментальных для реактивных фреймворков концепций, как Observables (https://oreil.ly/vboPE), Observable-операторы (https://oreil.ly/PEMon), Schedulers (https://oreil.ly/5vIfH) и backpressure (https://oreil.ly/4koKP), занимает значительное время и может оказаться сложнее по сравнению с другими моделями конкурентности.

Повышенная когнитивная нагрузка

Функциональный стиль программирования, принятый в реактивных фреймворках, непривычен для разработчиков с многолетним опытом работы в императивном или объектно-ориентированном стиле. Активное применение лямбда-выражений, функций высшего порядка, композиции функций, цепочки операторов и преобразования усложняет первоначальное понимание кода. Его труднее поддерживать (по крайней мере, на первых порах) в крупных проектах или командах, где не все участники имеют достаточный опыт работы с реактивным программированием.

Сложности отладки

Традиционные методы и инструменты отладки не всегда успешно работают с реактивными системами на лету. Например, цепочки операторов приводят к асинхронному выполнению, при котором трассировка стека в случае ошибки может не дать достаточно контекста для выявления первопричины проблемы. Событие, перешедшее между потоками и операторами, легко упустить. Для отладки могут потребоваться специализированные инструменты, которые, вероятно, поставляются вместе с реактивным фреймворком. Это дополнительно усложняет процесс обучения. Например, представим себе реактивный пайплайн, который получает данные из четырех разных источников, преобразует четыре реактивных потока и затем объединяет результаты. Ошибка на этапе объединения способна привести к стек-трейсу,

https://max.ru/bookofgeek

указывающему на оператор объединения, и будет трудно определить, что вызвало проблему: конкретный предшествующий источник или преобразование. Очень сложно отслеживать поток данных по всему реактивному пайплайну с десятками операторов и множеством асинхронных операций и т. д. В традиционной модели потоков отладка относительно проста и требует меньших затрат.

Риск избыточной сложности

Хотя компоновка с помощью реактивных фреймворков на основе операторов чрезвычайно мощна, она может привести к излишне сложным решениям, не соответствующим бизнес-требованиям или элементам интерфейса. Подобно тому как кто-то не устоит перед искушением приготовить курицу, используя предметы из фамильного серебряного сервиза, так кому-то может показаться заманчивым решать каждую проблему чуть более сложным способом, чем требуется. Реактивные фреймворки несут в себе риск, позволяя неопытным разработчикам создавать усложненные решения, — проблема, характерная для любых мощных абстракций в руках того, кто не знает меры.

Неверный выбор парадигмы

Реактивные фреймворки оптимальны для сценариев с интенсивными асинхронными операциями, событийно-ориентированными потоками данных или высокой пропускной способностью. Часть возможностей таких систем может оказаться избыточной — в случаях, когда асинхронность минимальна, потоки данных не ориентированы на события, а затраты на изучение и внедрение фреймворка превышают преимущества более простого синхронного подхода или модели «запрос — ответ».

Зависимость от вендора

Основные идеи реактивного программирования применимы на разных платформах, однако каждый из серьезных реактивных фреймворков обладает собственным набором API. Выбор конкретного фреймворка означает привязку к нему, что усложнит последующую замену библиотек и может ограничить гибкость на протяжении всего жизненного цикла проекта. Такая зависимость чревата сложностями с поиском разработчиков, знакомых с выбранным фреймворком, риском его стагнации или прекращения поддержки, а также трудностями миграции на другой фреймворк при необходимости.

Невзирая на все упомянутые проблемы, нельзя отрицать, что реактивные фреймворки обладают огромным потенциалом для упрощения управления сложными асинхронными сценариями. Необходимо детально проанализировать их преимущества и недостатки, чтобы понять, насколько оправданно применение реактивного программирования в конкретном проекте.

https://max.ru/bookofgeek

Революция в конкурентности Java

Традиционные механизмы конкурентности в Java долгое время устраивали разработчиков, однако по мере усложнения сценариев их применения и появления новых вариантов использования стали накапливаться проблемы. Именно поэтому было критически важно найти новые решения.

Виртуальные потоки: новый подход

Разобравшись с ограничениями традиционных методов, можно искать лучший способ решить возникающие проблемы. Здесь на сцену выходит Project Loom — первоклассный проект для внедрения современной конкурентности в Java. Он представил новый тип потоков — *виртуальные потоки (virtual threads)*, открыв новую эру конкурентности в Java. Они также известны как легкие потоки, которые создаются по требованию и потребляют значительно меньше ресурсов по сравнению с традиционными. Виртуальные потоки можно создавать миллионами, что открывает новые возможности для высококонкурентных и масштабируемых приложений.

Рассмотрим ключевые характеристики виртуальных потоков.

Бесшовная интеграция с существующими кодовыми базами

Одно из ключевых преимуществ Project Loom — полная совместимость с существующими кодовыми базами на Java. Например, если ваше приложение уже использует фреймворк Executor, для работы с виртуальными потоками достаточно передать `Executors.newVirtualThreadPerTaskExecutor()` (https://oreil.ly/K6hJ6) в имеющийся у вас `ExecutorService`:

```
Credit calculateCreditWithVirtualThread
    (Long personId) throws ExecutionException, InterruptedException {
  try (var executor = Executors.newVirtualThreadPerTaskExecutor()) { ❶
    var person = getPerson(personId);
    var assetsFuture = executor.submit(() -> getAssets(person));
    var liabilitiesFuture = executor.submit(() -> getLiabilities(person));
    executor.submit(this::importantWork);
    return calculateCredits(assetsFuture.get(), liabilitiesFuture.get());
  }
}
```

❶ Заменим любой традиционный исполнитель на `Executors.newVirtualThreadPerTaskExecutor()` и мгновенно получим все преимущества виртуальных потоков.

Такая простота интеграции обеспечивает плавный переход к новой модели конкурентности.

https://max.ru/bookofgeek

Виртуальные и платформенные потоки

Виртуальные потоки работают поверх *платформенных потоков* (*platform threads*, также называемых *classical threads*). Те, в свою очередь, используют `ForkJoinPool`, наследуя преимущества этого более совершенного пула потоков. Такое сочетание обеспечивает оптимальное использование ресурсов и масштабируемость.

Интеллектуальная обработка блокирующих операций

Интеллектуальная обработка блокирующих операций — одна из самых впечатляющих возможностей виртуальных потоков. Как только виртуальный поток встречает блокирующую операцию (например, `sleep` или сетевой ввод-вывод), он автоматически возвращает управление базовому платформенному потоку. Это позволяет ему продолжить выполнение других виртуальных потоков, обеспечивая оптимальное использование доступных ресурсов. После завершения блокирующей операции виртуальный поток может просто возобновить выполнение с точки останова, избегая видимых узких мест.

Преимущества использования виртуальных потоков

Прежде чем углубиться в следующей главе в детали работы виртуальных потоков, кратко рассмотрим, как они меняют подход к обработке конкурентности в Java. Вот некоторые ключевые преимущества.

Эффективное использование ресурсов

Виртуальные потоки легкие, их можно создавать миллионами без риска исчерпать ресурсы, что позволяет строить высокомасштабируемые приложения с высокой степенью конкурентности.

Простота кода

Возможность писать блокирующий код без потери производительности позволяет Java-разработчикам использовать более простой и привычный для них императивный стиль программирования. Им не требуется осваивать новые концепции, а поддержка кода остается простой.

Оптимальное использование ресурсов

При выполнении блокирующих операций виртуальные потоки передают управление, что гарантирует загрузку платформенных потоков и повышает производительность CPU и приложения.

Главная сложность для разработчиков, использующих конкурентность в Java, — необходимость изучения специфичной для нее информации. Project Loom совершает значительный прорыв: он позволяет создавать высокомасштабируемые и эффективные приложения, сохраняя привычные паттерны программирования.

https://max.ru/bookofgeek

В заключение

В следующей главе мы продолжим знакомство с виртуальными потоками и подробнее исследуем эту революционную технологию.

По мере углубления в концепцию виртуальных потоков задумайтесь, как они могут изменить ваш подход к проектированию конкурентных приложений. Виртуальные потоки предлагают не просто новый способ обработки конкурентности, но и более эффективный, масштабируемый и интуитивно понятный метод, соответствующий современным требованиям к вычислениям.

https://max.ru/bookofgeek

ГЛАВА 2

Виртуальные потоки

> Цена надежности — стремление к предельной простоте. Это та цена, которую самым богатым труднее всего заплатить.
>
> *Тони Хоар (Tony Hoare)*

Виртуальные потоки — функциональность, которая открывает новую страницу конкурентности в Java, кардинально изменяя подход к созданию многопоточных программ. Теперь стало целесообразно использовать их как основную единицу конкурентности в масштабных приложениях. В этой главе мы подробно рассмотрим, как и чем виртуальные потоки отличаются от платформенных, или классических, потоков. В то время как платформенными потоками управляет операционная система или библиотека потоков (например, POSIX), виртуальные потоки легковесны, ими управляет непосредственно виртуальная машина Java (JVM). Такой переход от управления операционной системой к управлению JVM позволяет приложениям создавать миллионы потоков без присущих традиционным потокам проблем с памятью и производительностью, из-за которых подобные решения были невыгодными. Мы детально рассмотрим архитектуру виртуальных потоков, их отличия от платформенных, причины появления в современных Java-приложениях, а также то, как они упрощают конкурентное программирование, обеспечивая беспрецедентную масштабируемость.

Что такое виртуальные потоки

Виртуальные потоки управляются JVM, поэтому работают эффективнее по сравнению с традиционными, которые планируются и управляются операционной системой. Виртуальные потоки выполняются поверх потоков-носителей (carrier threads), которые, по сути, являются потоками из пула `ForkJoinPool`. Такая архитектура позволяет виртуальным потокам

https://max.ru/bookofgeek

использовать преимущества продвинутых механизмов пула потоков и алгоритмов «кражи задач» (или «кражи работ»).

Стоит отметить, что планировщик виртуальных потоков в JVM основан на крадущем задачи `ForkJoinPool`, который работает по принципу FIFO (First-In, First-Out, «первым пришел — первым вышел»). Пул `ForkJoinPool`, который служит основой для планирования виртуальных потоков, следует отличать от общего пула: тот применяется для реализации параллельных потоков, работающих по принципу LIFO (Last-In, First-Out — «последним пришел — первым вышел»).

Параллелизм планировщика виртуальных потоков связан с числом платформенных потоков, доступных для потоков виртуальных. Данный параметр можно настраивать. По умолчанию он равен количеству доступных процессоров в системе, что позволяет оптимально использовать аппаратные ресурсы. Однако разработчики могут точно настроить этот параметр с помощью системного свойства `jdk.virtualThreadScheduler.parallelism`, адаптируя уровень параллелизма под конкретные требования приложения и характеристики рабочей нагрузки.

В Java системные свойства можно задавать с помощью опции командной строки `-D` при запуске приложения, например:

```
java -Djdk.virtualThreadScheduler.parallelism=4 -jar ваше_приложение.jar
```

В данном примере системное свойство `jdk.virtualThreadScheduler.parallelism` установлено равным `4`. Замените *`ваше_приложение.jar`* фактическим именем своего Java-приложения.

В коде можно использовать метод `System.setProperty(`*`ключ`*`, `*`значение`*`)` (https://oreil.ly/-goJu):

```
System.setProperty("jdk.virtualThreadScheduler.parallelism", "4");
```

Важно помнить, что эти настройки необходимо задавать до их первого использования. Обычно это делается в начале метода `main` или даже до инициализации контекста приложения Spring/Jakarta EE/Quarkus.

Два вида потоков в Java

В Java теперь существуют два вида потоков: платформенные и виртуальные (рис. 2.1).

Платформенные потоки

Существуют в языке Java с момента его создания, поэтому их иногда называют *традиционными* или *классическими*. Можно также встретить термины *нативные потоки (native threads)* или *потоки ОС (OS threads)*.

https://max.ru/bookofgeek

В JDK (Java Development Kit) они официально именуются *платформенными потоками (platform threads)*. Платформенный поток — это ресурсоемкий поток, который выполняется операционной системой и зависит от ее механизмов планирования и управления. Между платформенными потоками Java и потоками ядра существует взаимно-однозначное соответствие (1:1), причем управление последними осуществляется операционной системой. Такие потоки используют встроенные механизмы ОС для планирования и переключения контекста. С момента создания языка платформенные потоки являлись основой модели конкурентного программирования в Java.

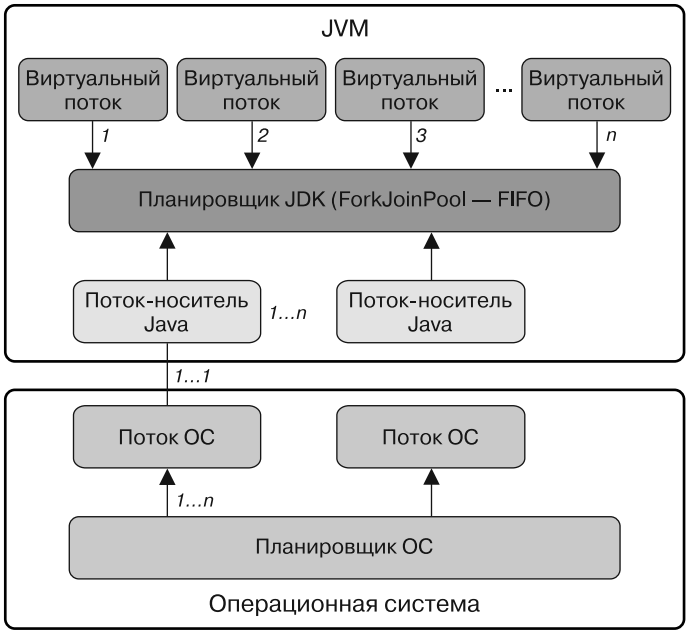


Рис. 2.1. Внутренняя организация планирования потоков: виртуальные потоки и потоки-носители

Виртуальные потоки

Это нововведение в модели конкурентности Java, появившееся в JDK 21. Иногда они называются *user-mode threads* или *lightweight threads*. В отличие от платформенных, виртуальные потоки полностью управляются JVM. Можно создавать миллионы таких потоков без риска исчерпать системные ресурсы. Виртуальные потоки не сопоставляются напрямую с потоками ядра. Вместо этого они используют общий пул, состоящий из меньшего

https://max.ru/bookofgeek

числа потоков-носителей (платформенных потоков), что позволяет JVM мультиплексировать множество виртуальных потоков на относительно небольшом количестве ресурсов ОС.

Ключевые отличия виртуальных потоков от платформенных

Итак, мы выяснили, что потоки бывают двух видов: платформенные и виртуальные. Рассмотрим основные различия между ними.

Малая ресурсоемкость/легковесность

Виртуальные потоки потребляют значительно меньше памяти и ресурсов по сравнению с платформенными, поэтому даже миллионы виртуальных потоков не исчерпают ресурсы системы.

Планирование

Виртуальные потоки планируются JVM, а не ОС, поэтому лучше используют циклы CPU и не требуют дополнительных затрат, связанных с планировщиком потоков ОС.

Устойчивость к блокировкам

Виртуальные потоки не снижают производительность при выполнении блокирующих операций, таких как чтение из консоли, файла или через сетевое соединение, запись в файл или через сетевое соединение, а также при выполнении метода `sleep()`. Это происходит потому, что при возникновении блокирующей операции виртуальный поток возвращает управление потоку-носителю, позволяя другим виртуальным потокам продолжать выполнение.

Бесшовная интеграция

Виртуальные потоки разработаны для бесшовной интеграции в существующие кодовые базы. Разработчики могут продолжать использовать привычные абстракции и паттерны программирования, не прибегая к адаптации рабочих процессов.

В то время как ограничения платформенных потоков нельзя устранить, а возможности их масштабирования ограничены, виртуальные потоки предлагают более масштабируемую и производительную модель конкурентности. Уже только поэтому отношение Java-разработчиков к программированию с использованием конкурентности обязательно изменится. В этой и последующих главах книги мы подробно разберем принципы работы виртуальных потоков, рассмотрим их применение в реальных проектах и поработаем с примерами, которые помогут вам приступить к разработкам в этой перспективной области.

https://max.ru/bookofgeek

Настройка окружения для работы с виртуальными потоками

Виртуальные потоки проекта Loom стали стабильной фичей начиная с JDK 21. Если версия JDK 21 у вас еще не установлена, установите ее. Для управления несколькими версиями JDK я рекомендую SDKMAN (https://sdkman.io). Этот универсальный инструмент упрощает установку и позволяет легко переключаться между версиями. Подробные инструкции доступны на официальном сайте SDKMAN. Установка JDK с помощью SDKMAN упрощает управление версиями Java и зависимостями в системе, что облегчает жизнь разработчику.

После установки SDKMAN выполните следующую команду, чтобы вывести список доступных версий JDK (результат показан на рис. 2.2):

```
sdk list java
```

```
================================================================================
Available Java Versions for macOS ARM 64bit
================================================================================
 Vendor        | Use | Version      | Dist    | Status     | Identifier
--------------------------------------------------------------------------------
 Corretto      |     | 21.0.2       | amzn    |            | 21.0.2-amzn
               |     | 21.0.1       | amzn    |            | 21.0.1-amzn
               |     | 17.0.10      | amzn    |            | 17.0.10-amzn
               |     | 17.0.9       | amzn    |            | 17.0.9-amzn
               |     | 11.0.22      | amzn    |            | 11.0.22-amzn
               |     | 11.0.21      | amzn    |            | 11.0.21-amzn
               |     | 8.0.402      | amzn    |            | 8.0.402-amzn
               |     | 8.0.392      | amzn    |            | 8.0.392-amzn
 Gluon         |     | 22.1.0.1.r17 | gln     |            | 22.1.0.1.r17-gln
               |     | 22.1.0.1.r11 | gln     |            | 22.1.0.1.r11-gln
 GraalVM CE    |     | 21.0.2       | graalce |            | 21.0.2-graalce
               |     | 21.0.1       | graalce |            | 21.0.1-graalce
               |     | 17.0.9       | graalce |            | 17.0.9-graalce
 GraalVM Oracle|     | 21.0.2       | graal   |            | 21.0.2-graal
               |     | 21.0.1       | graal   |            | 21.0.1-graal
               |     | 17.0.10      | graal   |            | 17.0.10-graal
               |     | 17.0.9       | graal   |            | 17.0.9-graal
 Java.net      |     | 23.ea.12     | open    |            | 23.ea.12-open
               |     | 23.ea.11     | open    |            | 23.ea.11-open
               |     | 23.ea.10     | open    |            | 23.ea.10-open
               |     | 23.ea.8      | open    | installed  | 23.ea.8-open
               |     | 23.ea.7      | open    |            | 23.ea.7-open
               |     | 23.ea.6      | open    |            | 23.ea.6-open
               |     | 23.ea.5      | open    |            | 23.ea.5-open
               |     | 23.ea.4      | open    |            | 23.ea.4-open
               |     | 23.ea.3      | open    |            | 23.ea.3-open
               |     | 23.ea.2      | open    |            | 23.ea.2-open
               |     | 23.ea.1      | open    |            | 23.ea.1-open
               |     | 22.ea.36     | open    |            | 22.ea.36-open
               |     | 22.ea.35     | open    |            | 22.ea.35-open
               |     | 22.ea.34     | open    |            | 22.ea.34-open
```

Рис. 2.2. Результат выполнения команды list в SDKMAN: список доступных версий Java

https://max.ru/bookofgeek

Для установки JDK 21 (если эта версия у вас еще не установлена) выполните команду:

```
sdk install java <нужный_JDK_дистрибутив>
```

Например, для OpenJDK используйте:

```
sdk install java 21.0.2-open
```



В этой главе используется JDK 21. На момент написания книги это самая популярная версия для работы с виртуальными потоками в продакшене. В следующих главах мы переключимся на более новые версии JDK, чтобы изучить такие возможности, как:

- scoped values;
- структурированная конкурентность.

Создание виртуальных потоков в Java

Итак, JDK 21 установили, теперь рассмотрим способы создания виртуальных потоков в Java.

Для простых сценариев можно использовать метод `Thread.startVirtualThread()` (https://oreil.ly/FIiZ3). Он принимает `Runnable`, так что мы можем передать лямбда-выражение — и любой переданный код будет выполнен. Например:

```
Thread.startVirtualThread(() -> {
  System.out.println("Unleash massive parallelism with virtual
  threads! Here's a taste.");
});
```

Возможно, после запуска этого кода вы ожидали увидеть сообщение в консоли. Однако ничего не выведется! Почему? Виртуальные потоки в Java по умолчанию являются демон-потоками, то есть выполняются в фоновом режиме. Когда главный поток, который создал виртуальный поток, завершается, JVM останавливает все оставшиеся демон-потоки.

Чтобы гарантировать завершение задачи виртуального потока, необходимо дождаться его окончания:

```
public static void main(String[] args) throws InterruptedException {
  Thread vThread = Thread.startVirtualThread(() -> {
    System.out.println("Virtual threads make concurrency effortless!" +
      "See for yourself.");
  });
  vThread.join();
}
```

https://max.ru/bookofgeek

Для более гибкого управления созданием потоков можно использовать Thread Builder API:

```
var startedThread = Thread.ofVirtual()
  .start(() -> System.out.println("Hello world!"));
startedThread.join();
```

Чтобы создать поток без немедленного запуска, используйте такой код:

```
var unstartedThread = Thread.ofVirtual()
  .unstarted(() -> System.out.println("Hello world!"));
// Запустить поток позже, когда потребуется
unstartedThread.start();
```

Если в приложении уже применяются исполнители (Executors), можно плавно перевести его на виртуальные потоки:

```
try (var virtualExecutor = Executors.newVirtualThreadPerTaskExecutor()) {
  Future<String> future = virtualExecutor.submit(this::callService);
  // Обработать результат future
}
```

Это особенно полезно в крупных проектах, где немедленный рефакторинг под виртуальные потоки может быть нецелесообразным.

Такое разнообразие подходов позволяет бесшовно интегрировать виртуальные потоки в существующую кодовую базу. Не отказываясь от привычных паттернов и абстракций, вы получите все преимущества новой модели конкурентности — повышенную производительность, масштабируемость и эффективное использование ресурсов.

Адаптация к виртуальным потокам

При введении виртуальных потоков потребовалось внести изменения в Java Thread API, но при этом учли привычный для разработчиков подход. Виртуальный поток в Java — это, по сути, экземпляр класса `Thread` (https://oreil.ly/miPlt), а его отмена работает так же, как и для платформенных потоков: через вызов метода `interrupt()` (https://oreil.ly/hcXDN). Код, выполняемый в потоке, должен или проверять флаг прерывания (`interrupted`), или вызывать методы, которые автоматически обрабатывают прерывание, как делает большинство блокирующих методов.

Рассмотрим примеры прерывания платформенного и виртуального потоков. Прерывание платформенного потока:

```
public class PlatformThreadInterruption {
  public static void main(String[] args) {
    Thread platformThread = Thread.ofPlatform().start(() -> { ❶
      try {
        System.out.println("Platform thread started...");
```

https://max.ru/bookofgeek

```
        for (int i = 0; i < 5; i++) {
          System.out.println("Platform thread working: " + i);
          Thread.sleep(1000); // Симуляция работы
        }
        System.out.println("Platform thread finished.");
      } catch (InterruptedException e) { ❷
        System.out.println("Platform thread interrupted!");
        // При необходимости выполнить очистку
      }
    });
    try {
      Thread.sleep(2500); // Даем потоку немного поработать ❸
    } catch (InterruptedException e) {}
    platformThread.interrupt(); ❹
  }
}
```

Прерывание виртуального потока:

```
public class VirtualThreadInterruption {
  public static void main(String[] args) {
    Thread virtualThread = Thread.ofVirtual().start(() -> { ❶
      try {
        System.out.println("Virtual thread started...");
        for (int i = 0; i < 5; i++) {
          System.out.println("Virtual thread working: " + i);
          Thread.sleep(1000); // Автоматически уступает
                              // другим виртуальным потокам
        }
        System.out.println("Virtual thread finished.");
      } catch (InterruptedException e) { ❷
        System.out.println("Virtual thread interrupted!");
        // При необходимости выполнить очистку
      }
    });
    try {
      Thread.sleep(2500); // Даем потоку немного поработать ❸
    } catch (InterruptedException e) {}
    virtualThread.interrupt(); ❹
  }
}
```

Разберем логику прерывания в обоих примерах.

❶ Создаем платформенный или виртуальный поток с помощью паттерна Строитель (Builder).

❷ Перехватываем `InterruptedException` при прерывании потока во время блокирующей операции.

❸ Главный поток ожидает 2,5 с, позволяя рабочему потоку выполнить две-три итерации.

❹ Прерываем рабочий поток, что вызывает исключение и очистку.

https://max.ru/bookofgeek

При выполнении этого кода вывод будет следующим:

```
java PlatformThreadInterruption.java
Platform thread started...
Platform thread working: 0
Platform thread working: 1
Platform thread working: 2
Platform thread interrupted!
```

и

```
java VirtualThreadInterruption.java
Virtual thread started...
Virtual thread working: 0
Virtual thread working: 1
Virtual thread working: 2
Virtual thread interrupted!
```

Оба примера выводят идентичные результаты, что подтверждает одинаковую семантику прерывания у виртуальных и платформенных потоков. Основная разница заключается в том, что операции блокировки виртуальных потоков (например, `Thread.sleep()`) автоматически освобождают базовый поток-носитель для других виртуальных потоков.

Хотя виртуальные потоки сохраняют привычные API, у них есть некоторые уникальные особенности, отражающие их легковесную, управляемую природу (например, группы потоков).

Все виртуальные потоки принадлежат к одной группе потоков. API для создания виртуального потока в другой группе не существует. Если мы создадим виртуальный поток и вызовем метод `getThreadGroup()`, мы всегда получим экземпляры одной и той же группы `ThreadGroup`. Например:

```
public class VirtualThreadGroupExample {
  public static void main(String[] args) throws InterruptedException {
    Set<ThreadGroup> threadGroups = new HashSet<>();

    for (int i = 0; i < 100; i++) { ❶
      Thread vThread = Thread.ofVirtual().start(() -> {
        try {
          Thread.sleep(10);
        } catch (InterruptedException e) {
          Thread.currentThread().interrupt();
        }
      });
      threadGroups.add(vThread.getThreadGroup()); ❷
    }
    Thread.sleep(1000);    // Ожидаем завершения потоков
    System.out.println("Unique thread groups: " + threadGroups.size());
    System.out.println("Thread group: " + threadGroups.iterator().next());
  }
}
```

https://max.ru/bookofgeek

В приведенном коде мы выполнили следующее.

❶ Создали 100 виртуальных потоков для проверки назначения в группу потоков.

❷ Собрали группы потоков из всех виртуальных потоков во множество.

При выполнении этого кода мы получим следующий вывод:

```
Unique thread groups: 1
Thread group: java.lang.ThreadGroup[name=VirtualThreads,maxpri=10]
```

Из вывода видно, что виртуальные потоки имеют несколько неизменяемых характеристик, упрощающих управление.

Приоритет

Уровень приоритета виртуальных потоков устанавливается в `NORM_PRIORITY` (https://oreil.ly/g1mpf).

Статус демона

Все виртуальные потоки по умолчанию являются демон-потоками. Попытка изменить этот статус с помощью `setDaemon` (https://oreil.ly/9VuRC) не увенчается успехом.

Приоритет потока

Вызов `setPriority` (https://oreil.ly/IRu_B) для виртуального потока не изменит его приоритет.

Thread API также был обновлен: добавлены новые методы, а некоторые больше не используются.

Метод isVirtual

Метод экземпляра класса `Thread::isVirtual` (https://oreil.ly/OeLMn) позволяет выяснить, является ли поток виртуальным.

Методы на основе класса Duration

В Java 19 появились методы экземпляра класса `join(Duration)` (https://oreil.ly/TgkJ9) и `sleep(Duration)` (https://oreil.ly/sIYV6), которые не связаны с виртуальными потоками, но повышают удобство работы.

ID потока

Нефинальный метод `getId()` (https://oreil.ly/klDiG) объявлен устаревшим, вместо него рекомендуется использовать метод `threadId()` (https://oreil.ly/vq0IK).

Что касается устаревших методов, начиная с Java 20 методы `stop()` (https://oreil.ly/-V_Ao), `suspend()` (https://oreil.ly/hflbW), `resume()` (https://oreil.ly/SsQv5) и `countStackFrames()` (https://oreil.ly/fFdg8) будут выбрасывать исключение

https://max.ru/bookofgeek

`UnsupportedOperationException` (https://oreil.ly/UZ7zY) как для платформенных, так и для виртуальных потоков, поскольку эти методы являются небезопасными по своей природе, как указано в документации Java Documentation (https://oreil.ly/ISBHU). Эти методы считаются устаревшими с версии 1.2 и запланированы к удалению.

Виртуальные потоки все-таки имеют некоторые незначительные ограничения. Следует отметить, что статический метод `Thread::getAllStackTraces` (https://oreil.ly/ZCVGV) возвращает стек-трейсы только платформенных потоков, но не виртуальных. Кроме того, в настоящее время нет способа определить, поверх какого именно платформенного потока выполняется данный виртуальный поток.

Хотя виртуальные потоки вносят некоторые изменения и ограничения в Thread API, разработчики практически не замечают перехода на них благодаря их высокой совместимости с существующим API и концепциями многопоточности. Используя новые возможности в соответствии с обновленными рекомендациями, разработчики могут продолжать работу с существующей кодовой базой и при этом получают преимущества от высокой производительности и масштабируемости виртуальных потоков.

Пример создания виртуальных потоков в Java

Виртуальные потоки способны полностью раскрыть потенциал масштабируемости Java-приложений. Благодаря им вы можете значительно увеличить число параллельно выполняемых задач. Для современных серверных приложений критически важно обрабатывать тысячи конкурентных запросов. Рассмотрим виртуальные потоки в действии.

В этом примере мы отправляем 10 000 задач с помощью исполнителя, который для каждой из них создает новый виртуальный поток:

```
try (var executor = Executors.newVirtualThreadPerTaskExecutor()) {
  IntStream.range(0, 10_000).forEach(i -> {
    executor.submit(() -> {
      Thread.sleep(Duration.ofSeconds(1));
      return i;
    });
  });
}
```

Задачи предельно просты: каждая просто «спит» одну секунду. На современном оборудовании это легко реализуется даже для 10 000 виртуальных потоков, работающих конкурентно. Впечатляет то, что JDK справляется с этим, используя минимальное число потоков ОС — возможно, всего один.

https://max.ru/bookofgeek

Для сравнения: при попытке создать 10 000 платформенных потоков с помощью `Executors.newCachedThreadPool()` (https://oreil.ly/mzIbV) программа может аварийно завершиться из-за перерасхода ресурсов. А при использовании фиксированного пула, например `Executors.newFixedThreadPool(200)` (https://oreil.ly/dttrO), конкурентность будет ограничена. При наличии всего лишь 200 платформенных потоков многие задачи станут выполняться последовательно, а время работы программы существенно увеличится.

Пропускная способность и масштабируемость

После достаточного прогрева виртуальные потоки могут обеспечивать пропускную способность около 10 000 задач в секунду. При увеличении показателя до 1 000 000 программа продолжит работать стабильно.

Важно отметить, что виртуальные потоки предназначены не для ускорения выполнения задач, а для обеспечения большей масштабируемости. В соответствии с законом Литтла они повышают пропускную способность благодаря увеличению конкурентности, а не за счет ускоренного выполнения отдельных задач.

В идеальных условиях виртуальные потоки могут значительно повысить пропускную способность приложений со следующими характеристиками.

Большое количество конкурентных задач

Виртуальные потоки могут кардинально улучшить ситуацию, если приложение выполняет больше, чем две-три тысячи конкурентных задач. Они идеально подходят для высоконагруженных веб-серверов, обрабатывающих тысячи запросов одновременно, или для приложений, параллельно выполняющих множество операций, связанных с вводом-выводом.

Рабочая нагрузка, не нагружающая процессор

Виртуальные потоки особенно эффективны, когда задачи больше времени проводят в ожидании (например, во время операций ввода-вывода), чем выполняют операции с интенсивной обработкой данных процессором (CPU-bound operations).

Важное предостережение. Хотя виртуальные потоки обеспечивают новый уровень масштабируемости Java-приложений, они не являются универсальным решением для всех проблем конкурентности. Наибольшая эффективность достигается при большом количестве конкурентных задач с легковесными рабочими нагрузками, не связанными с процессором. По сравнению с традиционной многопоточностью виртуальные потоки существенно расширяют возможности масштабирования. Это позволяет создавать приложения с высокой степенью конкурентности и масштабируемости, способные обрабатывать огромное количество запросов без чрезмерной нагрузки на CPU или память.

https://max.ru/bookofgeek

Фундаментальный принцип масштабируемости виртуальных потоков

Закон Литтла — это фундаментальный принцип, который помогает разобраться в работе систем очередей, и в том числе в многопоточных приложениях. Он устанавливает математическую зависимость между задержкой, конкурентностью и пропускной способностью — тремя ключевыми показателями, напрямую влияющими на производительность любой вычислительной системы.

Закон формулируется предельно просто: для стабильной системы пропускная способность (λ) вычисляется как:

$$\lambda = \frac{N}{d}.$$

Рассмотрим компоненты закона Литтла и их значение.

Пропускная способность (λ)

Среднее число элементов (например, задач или запросов), выполненных за единицу времени.

Конкурентность (N)

Среднее число элементов, обрабатываемых в одно и то же время.

Время отклика (d)

Среднее время обработки одного элемента от начала до завершения.

Закон Литтла универсален и не зависит от природы описываемой системы. Он не различает время, затраченное на «выполнение работы» и «ожидание», а также не учитывает, что именно является единицей конкурентности: поток, ядро процессора, банкомат или даже банковский служащий. Данный закон просто предлагает формулу для увеличения пропускной способности.

Виртуальные потоки и закон Литтла

Традиционные модели многопоточности часто упираются в ограничения ОС на количество создаваемых потоков. В таких случаях пропускная способность системы начинает определяться законом Литтла. Он гласит: если нельзя увеличить N (конкурентность), единственная переменная, которую можно изменить, — это d (задержка). Однако разработчик не всегда может уменьшить задержку, особенно в задачах, связанных с вводом-выводом.

В этом случае только виртуальные потоки позволяют преодолеть ограничение, увеличивая значение N, то есть повышают конкурентность без изменения модели программирования. Это важное преимущество, поскольку, согласно закону Литтла, оно напрямую ведет к росту пропускной способности. Виртуальные потоки позволяют увеличивать N без пропорционального уменьшения d.

https://max.ru/bookofgeek

Проиллюстрируем этот принцип: создадим пример кода, который подсчитывает, как, согласно закону Литтла, пропускная способность связана с виртуальными потоками и их повышенной масштабируемостью:

```
import java.time.Duration;
import java.time.Instant;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
import java.util.concurrent.atomic.AtomicLong;
import java.util.stream.IntStream;

public class LittleLawExample {
  public static void main(String[] args) {
    int numTasks = 10000; ❶
    int avgResponseTimeMillis = 500; // Среднее время выполнения задачи ❷
    //  Имитация настраиваемой работы, связанной с вводом/выводом
    Runnable ioBoundTask = () -> {
      try {
        Thread.sleep(Duration.ofMillis(avgResponseTimeMillis)); ❸
      } catch (InterruptedException e) {
        Thread.currentThread().interrupt();
      }
    };

    System.out.println("=== Little's Law Throughput Comparison ===");
    System.out.println("Testing " + numTasks + " tasks with "
                       + avgResponseTimeMillis + "ms latency each\n");
    benchmark("Virtual Threads",
      Executors.newVirtualThreadPerTaskExecutor(), ioBoundTask, numTasks);
    benchmark("Fixed ThreadPool (100)",
      Executors.newFixedThreadPool(100), ioBoundTask, numTasks);
    benchmark("Fixed ThreadPool (500)",
      Executors.newFixedThreadPool(500), ioBoundTask, numTasks);
    benchmark("Fixed ThreadPool (1000)",
      Executors.newFixedThreadPool(1000), ioBoundTask, numTasks);
  }

  static void benchmark(String type, ExecutorService executor,
                        Runnable task, int numTasks) {
    Instant start = Instant.now(); ❹
    AtomicLong completedTasks = new AtomicLong();
    try (executor) { ❺
      IntStream.range(0, numTasks)
        .forEach(i -> executor.submit(() -> {
          task.run();
          completedTasks.incrementAndGet(); ❻
        }));
      } ❼
      Instant end = Instant.now();
      long duration = Duration.between(start, end).toMillis();
      // Задач в секунду
      double throughput = (double) completedTasks.get() / duration * 1000; ❽
```

https://max.ru/bookofgeek

```
      System.out.printf("%-25s - Time: %5dms, Throughput: %8.2f tasks/s%n",
        type, duration, throughput);
    }
  }
}
```

Посмотрим, что мы реализовали в этой программе.

❶ Десять тысяч задач создают достаточную нагрузку для демонстрации разницы в пропускной способности, сохраняя при этом приемлемое время выполнения.

❷ Время отклика 500 мс имитирует реалистичные операции ввода-вывода, такие как запросы к базе данных или вызовы API, что соответствует компоненту *d* (задержка) в законе Литтла.

❸ `Thread.sleep()` имитирует блокирующие операции ввода-вывода. В случае с виртуальными потоками при этом автоматически освобождается поток-носитель для других виртуальных потоков.

❹ Измеряется общее время выполнения от начала отправки задач в пул и до завершения всех задач.

❺ `try-with-resources` гарантирует корректное завершение работы исполнителя и ожидает завершения всех поставленных задач.

❻ `AtomicLong` безопасно отслеживает завершенные задачи во всех конкурентных потоках.

❼ При выходе из блока `try` все отправленные в пул задачи завершают свое выполнение.

❽ Чтобы было проще сравнивать, общее число задач, завершенных за все время, пересчитывается в число задач в секунду.

В этом примере мы симулируем задачу, связанную с вводом-выводом, со средним временем отклика 500 мс. Затем сравниваем пропускную способность виртуальных потоков с фиксированными пулами потоков, свойственными традиционной модели многопоточности.

После выполнения кода мы получаем результаты, показывающие значительное преимущество виртуальных потоков:

```
=== Little's Law Throughput Comparison ===
Testing 10000 tasks with 500ms latency each
Virtual Threads         - Time:   552ms, Throughput: 18115.94 tasks/s
Fixed ThreadPool (100)  - Time: 50381ms, Throughput:   198.49 tasks/s
Fixed ThreadPool (500)  - Time: 10106ms, Throughput:   989.51 tasks/s
Fixed ThreadPool (1000) - Time: 5080ms, Throughput:   1968.50 tasks/s
```

Как видно из результатов, виртуальные потоки значительно превосходят традиционные пулы потоков по пропускной способности в сценарии с операциями ввода-вывода. Виртуальные потоки обеспечивают пропускную способность

https://max.ru/bookofgeek

около 18 115,94 задачи в секунду, тогда как даже крупный пул из 1000 потоков — всего лишь около 1968,50 задачи в секунду.



Предыдущий пример — базовая демонстрация закона Литтла для виртуальных потоков. Для более тщательного анализа производительности в продакшен-средах можно применить следующее:

- прогоны для разогрева JVM для получения преимуществ JIT-компиляции;
- многократные итерации бенчмарка со статистическим анализом (среднее значение, стандартное отклонение);
- мониторинг использования памяти для анализа распределения ресурсов;
- реалистичную симуляцию ввода-вывода с вариативностью задержки (±10–20 %) для приближения к реальным условиям;
- тестирование масштабируемости при различных объемах задач для выявления предельных значений;
- защиту от тайм-аутов и корректную обработку ошибок для повышения надежности измерений.

Эти усовершенствования помогают отличить реальное увеличение производительности от артефактов измерений и дают более достоверную аналитическую информацию для принятия решений в продакшене. Если оценка производительности критически важна, рекомендуется использовать проверенные фреймворки для микробенчмарка, например Java Microbenchmark Harness (JMH).

При использовании платформенных потоков пропускная способность может расти с увеличением размера пула, но лишь до определенного предела. В итоге ограничения ОС и излишний расход ресурсов на управление потоками становятся узким местом.

Данный пример наглядно показывает, что виртуальные потоки следуют закону Литтла: они увеличивают N (конкурентность) без пропорционального снижения d (задержки). Виртуальные потоки могут обеспечить высочайшую пропускную способность благодаря огромному росту числа конкурентных задач, особенно в сценариях, связанных с вводом-выводом, где уменьшить задержку невозможно.

Практические выводы

Виртуальные потоки позволяют повысить пропускную способность, не требуя от разработчиков тонкой настройки или даже кардинального пересмотра моделей программирования. Если приложение содержит множество задач, в основном связанных с вводом-выводом, где значительное время тратится на ожидание, можно существенно увеличить пропускную способность, не изменяя реализацию этих задач.

Используя виртуальные потоки, мы фактически увеличиваем N в законе Литтла, то есть повышаем конкурентность и, как следствие, пропускную

https://max.ru/bookofgeek

способность. Это особенно полезно, когда задержку *d* сложно уменьшить из-за характера задачи (например, запись/чтение диска или при сетевых операциях).

Однако важно понимать, что виртуальные потоки не являются универсальным решением всех проблем производительности. В ситуациях, где узким местом оказывается ввод-вывод, они значительно повышают пропускную способность. Однако, если ограничение связано не с числом активных конкурентных задач, а с доступной вычислительной мощностью, виртуальные потоки не помогут.

Усвоив применение закона Литтла к виртуальным потокам, разработчик сможет точнее определять, когда и каким образом использовать их для повышения масштабируемости и производительности приложений.

Виртуальные потоки: внутреннее устройство

Виртуальные потоки — громадный скачок в развитии модели конкурентности в Java. Чтобы в полной мере оценить их мощь, ознакомимся с основными механизмами их работы.

Стековые фреймы и управление памятью

Традиционные потоки хранят стековые фреймы в совместно используемых блоках памяти, которые выделяет ОС.

В отличие от них, виртуальные потоки в своей основе — альтернативная реализация `java.lang.Thread`, которая хранит стековые фреймы в куче (heap), управляемой сборщиком мусора. Такой подход устраняет необходимость предварительно оценивать размер стека: по умолчанию виртуальный поток занимает в памяти всего пару сотен байт и автоматически изменяется при росте или сокращении стека вызовов. Динамическое управление памятью существенно повышает эффективность использования ресурсов.

Потоки-носители и взаимодействие с ОС

Операционная система «не знает» о виртуальных потоках: она работает только с платформенными потоками, которые являются единицей планирования на уровне ОС. Чтобы выполнить код в виртуальном потоке, среда выполнения для Java (JRE) монтирует его на платформенный поток, так называемый *поток-носитель (carrier thread)*, входящий в специализированный `ForkJoinPool`. В процессе работы необходимые стековые фреймы временно копируются из кучи в стек потока-носителя.

По сути, поток-носитель «дают взаймы» для выполнения кода виртуального потока.

https://max.ru/bookofgeek

Обработка блокирующих операций

Одно из самых значительных усовершенствований — решение проблемы блокирующих операций с помощью виртуальных потоков. Когда виртуальный поток достигает точки блокировки (например, ожидание ввода-вывода), он отсоединяется от потока-носителя. Измененные стековые фреймы копируются обратно в кучу, а поток-носитель освобождается для выполнения других задач. Эта функциональность была добавлена практически во все точки блокировки в JDK. Именно поэтому виртуальные потоки используют ресурсы чрезвычайно эффективно.

Прозрачность и невидимость

Процесс монтирования и отсоединения виртуального потока от физического является прозрачным для Java-кода. Невозможно определить с помощью Java-кода, какой именно поток-носитель выполняет виртуальный поток в данный момент. Даже значения `ThreadLocal` (https://oreil.ly/0TLqb) потока-носителя остаются невидимыми для виртуального потока. Такой уровень абстракции позволяет использовать виртуальные потоки без каких-либо изменений в существующем коде.

В общих чертах идея виртуальных потоков напоминает систему виртуальной памяти. В ней приложения работают с иллюзией неограниченного адресного пространства. Это возможно благодаря сопряжению на аппаратном уровне между частью виртуальной памяти, которая хранится в оперативной (физической) памяти, и частью на диске. Подобным образом и виртуальные потоки создают иллюзию неограниченной многопоточности, сообща используя ограниченное число платформенных потоков, затратных с точки зрения ресурсов. Еще одна аналогия: неактивные стековые фреймы виртуальных потоков выгружаются в кучу, а неиспользуемые блоки виртуальной памяти (страницы) — на диск.

Подробнее о внутреннем устройстве и работе виртуальных потоков вы узнаете из следующих глав.

Упрощение асинхронных операций

С помощью виртуальных потоков гораздо легче выполнять асинхронные операции и агрегацию задач. В Java практически снимается проблема блокировки при ожидании завершения асинхронной задачи. Теперь разработчики могут свободно вызывать блокирующий метод `get` (https://oreil.ly/2JFZL) у объекта `Future` (https://oreil.ly/CWUYq) без потери производительности. Однако главное преимущество виртуальных потоков проявляется при агрегировании результатов нескольких асинхронных задач.

https://max.ru/bookofgeek

Рассмотрим случай, когда нужно получить различные части предложения через разные API и объединить их в связную фразу. Например, можно получить прилагательное и существительное для создания простой, но случайно сгенерированной фразы. С виртуальными потоками эта задача становится гораздо проще:

```
public String generatePhrase() throws ExecutionException, InterruptedException
{
  try (var executor = Executors.newVirtualThreadPerTaskExecutor()) {
    Future<String> adjectiveFuture = executor.submit(this::fetchAdjective);
    Future<String> nounFuture = executor.submit(this::fetchNoun);
    String adjective = adjectiveFuture.get();
    String noun = nounFuture.get();
    return adjective + " " + noun;
  }
}
private String fetchAdjective() {
  // Получить прилагательное через API
  return "beautiful";
}
private String fetchNoun() {
  // Получить существительное через API
  return "sunset";
}
```

В этом примере мы используем `VirtualThreadPerTaskExecutor` (https://oreil.ly/K6hJ6) — удобный инструмент для создания виртуальных потоков под отдельные задачи. С помощью метода `submit` из интерфейса `ExecutorService` мы отправляем две задачи: одну для получения прилагательного, другую — существительного.

Истинную красоту виртуальных потоков можно оценить при вызове `get` у объектов `Future`, которые возвращают отправленные задачи. Благодаря их малой ресурсоемкости блокировка в ожидании результата не влечет лишних затрат и не снижает производительность. Такой подход упрощает работу с асинхронными операциями: избавляет от необходимости использовать сложные колбэк-структуры или замысловатые механизмы синхронизации.

Получив прилагательное и существительное, мы можем легко объединить их в словосочетание, в данном случае — beautiful sunset.

Метод `invokeAll` удобен для однотипных задач, возвращающих результаты одного типа, особенно когда их нужно собрать в список. Например, вам нужно обработать разные изображения схожим образом:

```
package ca.bazlur.modern.concurrency.c02;

import javax.imageio.ImageIO;
import java.awt.image.BufferedImage;
```

https://max.ru/bookofgeek

```
import java.io.File;
import java.io.IOException;
import java.util.List;
import java.util.concurrent.*;

public class ImageProcessingExample {

  public static void main(String[] args)
      throws InterruptedException, ExecutionException {
    try (var service = Executors.newVirtualThreadPerTaskExecutor()) { ❶

      List<Callable<BufferedImage>> tasks = List.of(
        () -> resize("https://example.com/img1.jpg", 200, 200),
        () -> grayscale("https://example.com/img2.jpg"),
        () -> rotate("https://example.com/img3.jpg", 90)
      ); ❷

      List<Future<BufferedImage>> results = service.invokeAll(tasks); ❸
      // Обработка и сохранение преобразованных изображений
      int i = 1;
      for (Future<BufferedImage> future : results) { ❹
        BufferedImage image = future.get(); ❺
        ImageIO.write(image, "jpg",
          new File("output_image" + i + ".jpg"));
        i++;
      }
    } catch (IOException e) {
      throw new RuntimeException(e);
    }
 }

  static BufferedImage resize(String url, int width, int height) {
    // Логика загрузки и изменения размера изображения
    return null;
  }

  static BufferedImage grayscale(String url) {
    // Логика загрузки и преобразования формата изображения
    // в оттенки серого
    return null;
  }

  static BufferedImage rotate(String url, double angle) {
    // Логика загрузки и поворота изображения
    return null;
  }
}
```

Посмотрим, что здесь происходит.

❶ Используем исполнитель виртуальных потоков, но теперь для множества конкурентных задач обработки изображений.

https://max.ru/bookofgeek

❷ Создаем список вызываемых (`Callable`) задач, каждая из которых представляет собой отдельную операцию преобразования изображения.

❸ Одновременно отправляем все задачи с помощью `invokeAll()`. Они выполняются конкурентно в отдельных виртуальных потоках.

❹ Итерация результатов выполняется в том же порядке, что и исходный список задач.

❺ Каждый результат извлекаем с помощью блокирующих вызовов `get()`, которые полностью освобождают виртуальные потоки во время ожидания.

Современным приложениям часто требуется возможность обрабатывать множество задач одновременно, агрегируя их результаты. Виртуальные потоки предлагают красивое решение, обеспечивая параллельное выполнение с беспрепятственным сбором результатов. В современных вычислительных средах, где производительность приложений определяют операции ввода-вывода, это дает значительные преимущества.

Потенциал структурированной конкурентности

Все предыдущие примеры валидны и предлагают относительно простые способы работы с конкурентными задачами, но их возможности несколько ограничены — например, исключения и тайм-ауты обрабатываются недостаточно хорошо. Метод `Future::get` может выбрасывать исключения, но без механизма тайм-аута он теоретически будет блокировать поток бесконечно.

Пятая предварительная версия JEP 505 вводит в Java структурированную конкурентность, цель которой — упростить написание надежного кода с использованием конкурентности. API предоставляет структурированную область (structured scope), в которой выполняются задачи. Если одна из них завершается с ошибкой или не укладывается в лимит времени, остальные задачи в этой области автоматически отменяются. Вот краткий пример:

```
public static void main(String[] args) {
  try (StructuredTaskScope scope = StructuredTaskScope.open()) {
    StructuredTaskScope.Subtask<String> subtask1
      = scope.fork(() -> fetchData("https://api1.example.com"));
    StructuredTaskScope.Subtask<String> subtask2 =
      scope.fork(() -> fetchData("https://api2.example.com"));
    scope.join();
    var result = subtask2.get() + subtask1.get();
    System.out.println(result);
  } catch (InterruptedException e) {
    throw new RuntimeException(e);
  }
}
```

https://max.ru/bookofgeek



Приведенный код требует JDK 25 с включенными предварительными функциями (`--enable-preview`).

Такие fail-fast-области (fail-fast scopes) также позволяют задавать тайм-ауты и упрощают обработку исключений. Внутри области `subtask1` и `subtask2` выполняются как подзадачи. Если любая из них завершится ошибкой (выбросит исключение) или исчерпает тайм-аут, все задачи в этой области отменяются, а исключение передается обратно. Это способствует созданию более чистого и надежного кода, инкапсулирующего распространение исключений и тайм-ауты внутри встроенной конструкции.

Введение структурированной конкурентности в Java — важнейший шаг к облегчению разработки приложений. Предоставляя структурированную область выполнения задач, API обеспечивает автоматическую отмену и обработку исключений. В результате снижается нагрузка на разработчиков: им больше не требуется управлять этим вручную.

Кроме прочего, структурированная конкурентность способствует более декларативному стилю программирования, позволяя разработчикам четко и лаконично выражать свои намерения. Вместо того чтобы разбираться с низкоуровневыми деталями, такими как создание потоков, управление ими или реализация сложной логики отмены, разработчики могут сосредоточиться на высокоуровневых задачах.

Это лишь беглый взгляд на возможности структурированной конкурентности. В последующих главах мы подробнее исследуем эту захватывающую тему, рассмотрим современные паттерны, различные типы областей (scopes) и то, как структурированная конкурентность дополняет виртуальные потоки для создания по-настоящему устойчивых конкурентных приложений.

Управление ограничением ресурсов с помощью Rate Limiting

Виртуальные потоки открывают дверь к неограниченной конкурентности, позволяя одновременно обрабатывать недостижимое ранее число задач. До этого момента мы оценивали такую возможность только с положительной стороны, но в некоторых случаях могут возникать сложности. Дело в том, что не все компоненты программного обеспечения способны выдерживать одинаковую нагрузку. Например, ваше веб-приложение может принимать миллион запросов, создавая миллион виртуальных потоков, однако база данных может не справиться с таким количеством запросов.

https://max.ru/bookofgeek

До появления виртуальных потоков ограничивать число запросов помогал пул потоков, дополнительных механизмов для этого не требовалось. Если пул содержит 1000 потоков, то базовый ресурс мог обрабатывать не более 1000 нагрузок одновременно. Однако теперь, когда число виртуальных потоков практически не ограничено, возникает головоломный вопрос: как предотвратить перегрузку сервисов при их использовании?

Решение заключается в реализации Rate Limiting — механизмов ограничения скорости, специфичных для каждого ресурса. Они могут быть как простыми, так и сложными, в зависимости от характера ресурса и соглашений об уровне сервиса (SLA), которых необходимо придерживаться.

Рассмотрим пример использования семафора, контролирующего число конкурентных запросов к веб-сервису. Важно отметить, что основное внимание здесь уделяется принципу ограничения скорости:

```
import java.net.URI;
import java.net.http.*;
import java.time.Duration;
import java.time.Instant;
import java.util.List;
import java.util.concurrent.*;
import java.util.stream.IntStream;

public class ResourceAwareRateLimitExample {
  private static final HttpClient CLIENT = HttpClient.newBuilder()
     .connectTimeout(Duration.ofSeconds(10)) ❶
     .build();
  private static final int MAX_PARALLEL = 10; ❷
  private static final Semaphore gate = new Semaphore(MAX_PARALLEL); ❸
  private static final String API_URL =
     "https://api.chucknorris.io/jokes/random";

  public static void main(String[] args) throws Exception {
    Instant start = Instant.now();
    List<String> jokes = fetchJokes(50); ❹
    long ms = Duration.between(start, Instant.now()).toMillis();
    System.out.printf("Fetched %d jokes in %d ms (avg %d ms)%n",
      jokes.size(), ms, ms / jokes.size());
    jokes.stream().limit(3).forEach(j -> System.out.println("• " + j));
  }

  private static List<String> fetchJokes(int n) throws Exception {
    try (ExecutorService pool = Executors.newVirtualThreadPerTaskExecutor()) { ❺
      List<Future<String>> futures = IntStream.range(0, n)
        .mapToObj(i -> pool.submit(ResourceAwareRateLimitExample::fetchJoke))
        .toList();
      return futures.stream()
        .map(ResourceAwareRateLimitExample::join) ❻
        .toList();
    }
  }
```

https://max.ru/bookofgeek

```
  private static String fetchJoke() throws Exception {
    HttpRequest req = HttpRequest.newBuilder(URI.create(API_URL))
      .GET()
      .timeout(Duration.ofSeconds(30)) ❼
      .build();
    try {
      gate.acquire(); ❽
      HttpResponse<String> res
        = CLIENT.send(req, HttpResponse.BodyHandlers.ofString());
      if (res.statusCode() != 200)
        throw new RuntimeException("API error " + res.statusCode());
      return parseJoke(res.body());
    } finally {
      gate.release(); ❾
    }
  }

  private static String parseJoke(String json) { ❿
    int s = json.indexOf("\"value\":\"") + 9;
    int e = json.indexOf('"', s);
    return json.substring(s, e).replace("\\\"", "\"");
  }

  private static <T> T join(Future<T> f) {
    try {
      return f.get();
    } catch (InterruptedException e) { ⓫
      Thread.currentThread().interrupt();
      throw new CompletionException(e);
    } catch (ExecutionException e) {
      throw new CompletionException(e.getCause());
    }
  }
}
```

Разберем реализованное решение по шагам.

❶ Мы сконфигурировали общий `HttpClient` с десятисекундным тайм-аутом подключения, чтобы предотвратить проблемы зависания в сети.

❷ Определили максимальное количество конкурентных API-запросов.

❸ Создали в качестве ограничителя скорости счетный семафор (counting semaphore), инициализированный максимальным числом параллельных запросов.

❹ Пытаемся извлечь 50 joke (превышая лимит), чтобы показать, как семафор контролирует конкурентность.

❺ Создаем исполнитель, порождающий новый виртуальный поток для каждой задачи, — это идеально для операций, связанных с вводом-выводом.

❻ Блокируем до завершения всех `Future`, собирая результаты в порядке их выполнения.

https://max.ru/bookofgeek

❼ Устанавливаем 30-секундный тайм-аут для каждого запроса, предотвращая бесконечное блокирование.

❽ Получаем разрешение от семафора. При исчерпании лимита разрешений происходит блокирование — так ограничивается скорость.

❾ Освобождаем разрешение в блоке `finally`, гарантируя его возврат даже при возникновении исключения.

❿ Выполняем упрощенный парсинг JSON для демонстрации. В продакшене следует использовать специализированные библиотеки, например Jackson или Gson.

⓫ Выполняем улучшенную обработку ошибок с сохранением статуса прерывания и разворачиваем исключение из `ExecutionException`, чтобы узнать его первопричину.

В данном примере мы ограничили количество конкурентных задач до десяти, установив счетчик семафора на это значение. Каждая новая задача блокируется, если лимит уже достигнут. Важно помнить, что блокировка виртуальных потоков обходится дешево — именно поэтому намеренно созданное узкое место почти не сказывается на общей производительности приложения.

Семафоры в Java: принцип работы

Семафор (semaphore) — это механизм синхронизации, который действует как привратник, разрешая или запрещая доступ к общему ресурсу. В Java класс `Semaphore` (https://oreil.ly/8ACAn) из пакета java.util.concurrent предоставляет удобный способ управления доступом.

Принцип работы семафора простой: он отслеживает фиксированное число разрешений (permits). Прежде чем получить доступ к определенному участку кода, поток должен запросить разрешение, а по завершении работы вернуть его. Если свободных разрешений нет, поток ожидает, пока одно из них не освободится.

Рассмотрим основные методы семафора.

`acquire()` (https://oreil.ly/SEwgu)
: Метод запрашивает разрешение. Если свободных разрешений нет, поток будет ожидать.

`release()` (https://oreil.ly/_mtvT)
: Возвращает разрешение семафору, что позволит другому ожидающему потоку продолжить работу.

`availablePermits()` (https://oreil.ly/pCyco)
: Метод для проверки количества свободных разрешений.

https://max.ru/bookofgeek

Представьте себе пул из пяти соединений с базой данных. Семафор гарантирует, что эти соединения одновременно смогут использовать не более пяти потоков. Например:

```
import java.util.Optional;
import java.util.concurrent.Semaphore;

public class ResourcePool {
  private final Semaphore semaphore; ❶
  public ResourcePool(int resourceCount) {
    this.semaphore = new Semaphore(resourceCount); ❷
  }

  public Optional<String> useResource(String query){
    try {
      semaphore.acquire(); ❸
      try {
        // Имитация получения и использования соединения с БД
        return queryDatabase(query); ❹
      } finally {
        semaphore.release(); ❺
      }
    } catch (InterruptedException e) {
      Thread.currentThread().interrupt(); ❻
      return Optional.empty();
    }
  }

  private Optional<String> queryDatabase(String query) {
    // Имитация запроса к БД с небольшой задержкой
    try {
      Thread.sleep(100);
    } catch (InterruptedException e) {
      Thread.currentThread().interrupt();
      return Optional.empty();
    }
    return Optional.of("Result for: " + query);
  }
}
```

Рассмотрим, как данная реализация управляет доступом к ограниченным ресурсам.

❶ Поле `semaphore` хранит примитив синхронизации. Этот семафор выступает в роли потокобезопасного счетчика доступных разрешений, контролируя конкурентный доступ к соединениям с базой данных.

❷ При создании объекта мы устанавливаем лимит ресурсов. Далее инициализируем семафор максимальным количеством разрешенных конкурентных обращений, чем фактически задаем размер пула соединений.

https://max.ru/bookofgeek

❸ Здесь происходит ограничение скорости. Метод `acquire()` блокирует текущий поток до тех пор, пока не станет доступно разрешение, и таким образом обеспечивает ограничение ресурсов.

❹ После получения разрешения мы попадаем в критическую секцию. Здесь происходит фактическое использование ограниченного ресурса (соединения с базой данных).

❺ Освобождение ресурсов критически важно в конкурентном программировании. Мы всегда освобождаем разрешение в блоке `finally`, чтобы предотвратить утечку ресурсов даже при возникновении исключения.

❻ Очень важно корректно обработать прерывание потока. Мы сохраняем статус прерывания для правильной обработки, позволяя вызывающему коду реагировать соответствующим образом.

В предыдущем классе метод `useResource()` разрешает доступ только для `resourceCount` количества потоков. Если число обращений превышает этот лимит, остальные потоки будут ждать, пока занятые освободят ресурс.

В продакшене зачастую полезно выполнять мониторинг использования ресурсов и использовать тайм-ауты для предотвращения бесконечной блокировки. Чтобы повысить надежность работы семафора, модифицируем наш класс `ResourcePool`.

Обновленный код:

```
import java.util.Optional;
import java.util.Random;
import java.util.concurrent.Semaphore;
import java.util.concurrent.TimeUnit;
import java.util.concurrent.atomic.AtomicInteger;

public class MonitoredResourcePool {
  private final Semaphore semaphore;
  private final AtomicInteger activeConnections; ❶
  private final AtomicInteger peakConnections; ❷

  public MonitoredResourcePool(int resourceCount) {
    this.semaphore = new Semaphore(resourceCount, true); ❸
    this.activeConnections = new AtomicInteger(0);
    this.peakConnections = new AtomicInteger(0);
  }

  public Optional<String> useResource(String query) {
    boolean acquired = false;
    try {
      acquired = semaphore.tryAcquire(5, TimeUnit.SECONDS); ❹
      if (!acquired) {
        return Optional.empty(); ❺
      }
```

https://max.ru/bookofgeek

```
      int current = activeConnections.incrementAndGet();
      peakConnections.updateAndGet(peak -> Math.max(peak, current)); ❻
      return queryDatabase(query);
    } catch (InterruptedException e) {
      Thread.currentThread().interrupt();
      return Optional.empty();
    } finally {
      if (acquired) {
        activeConnections.decrementAndGet();
        semaphore.release();
      }
    }
  }

  public int getCurrentActiveConnections() {
    return activeConnections.get();
  }

  public int getPeakConnections() {
    return peakConnections.get(); ❼
  }

  private Optional<String> queryDatabase(String query) {
    try {
      Thread.sleep(new Random().nextInt(500) + 500);
    } catch (InterruptedException e) {
      Thread.currentThread().interrupt();
      return Optional.empty();
    }
    return Optional.of("Result for: " + query);
  }
}
```

Улучшенная реализация включает несколько важных для продакшен-среды возможностей.

❶ Мониторинг активных подключений помогает при отладке и планировании мощности ресурсов. Счетчик `activeConnections` отслеживает текущие активные подключения только для мониторинга, а не для ограничения доступа.

❷ Информация о периодах пиковой нагрузки критически важна для планирования ресурсов. Этот счетчик фиксирует максимальное количество конкурентных подключений за время работы приложения.

❸ Справедливое распределение предотвращает голодание (starvation) потоков в сценариях высокой конкурентности. Передав `true` вторым параметром, мы создаем «честный» семафор, гарантирующий получение разрешений в порядке FIFO.

❹ Тайм-ауты исключают бесконечную блокировку и повышают отказоустойчивость. Метод `tryAcquire` с тайм-аутом прерывает ожидание по истечении заданного времени, возвращая управление вызывающему коду.

https://max.ru/bookofgeek

❺ Некоторое снижение производительности предпочтительнее бесконечного зависания. Если невозможно получить ресурс в течение тайм-аута, для индикации ошибки мы возвращаем пустой `Optional`.

❻ Атомарные операции обеспечивают потокобезопасный сбор статистических данных. Обновление счетчика пиковой нагрузки на их основе исключает состояние гонки в отслеживаемом коде.

❼ Получаем метрики для обеспечения наблюдаемости. Этот метод позволяет отслеживать закономерности использования пула ресурсов, что крайне важно для мониторинга и оповещения о проблемах.

Удостоверимся, что наш семафор действительно ограничивает конкурентный доступ, — создадим тест, который попытается излишне нагрузить пул ресурсов.

```
package ca.bazlur.modern.concurrency.c02;

import java.util.ArrayList;
import java.util.List;
import java.util.Optional;
import java.util.concurrent.*;

public class ResourcePoolTest {

  public static void main(String[] args) throws Exception {
    int maxConcurrentThreads = 5;
    int totalRequests = 50;
    var pool = new MonitoredResourcePool(maxConcurrentThreads);

    var futures = new ArrayList<Future<Optional<String>>>();

    try (var executor = Executors.newVirtualThreadPerTaskExecutor()) {
      for (int i = 0; i < totalRequests; i++) {
        final int taskId = i;
        futures.add(executor.submit(() -> pool.useResource("Query " + taskId)));
      }

      int successCount = 0;
      int timeoutCount = 0;

      for (Future<Optional<String>> future : futures) {
        Optional<String> result = future.get();
        if (result.isPresent()) {
          successCount++;
        } else {
          timeoutCount++; ❶
        }
      }

      System.out.printf("""
              requests : %d
              successful: %d
```

https://max.ru/bookofgeek

```
                timed-out : %d
                peak usage: %d%n""",
          totalRequests, successCount, timeoutCount, pool.getPeakConnections()); ❷

        assert pool.getPeakConnections() <= maxConcurrentThreads
            : "Peak connections exceeded limit!";
      }
    }
}
```

Наш тест показал, что Rate limiter на основе семафора действует эффективно.

❶ Отслеживание тайм-аутов помогает выяснить, как ведет себя система под нагрузкой. Этот счетчик отслеживает запросы, которые не смогли получить разрешение в течение заданного времени, что указывает на соперничество за ресурсы.

❷ Пиковое количество подключений подтверждает корректность работы ограничения. Эта проверка показывает, что семафор успешно ограничивал конкурентный доступ в течение всего теста.

При запуске этого теста вы увидите, что даже при 50 одновременных запросах пиковое число подключений никогда не превышает установленный лимит 5. Некоторые запросы могут завершиться тайм-аутом, если не получат разрешения в течение 5 с, то есть наш семафор успешно защищает от перерасхода ресурсов.

На моей машине вывод выглядит так:

```
Total requests: 50
Successful: 32
Timed out: 18
Peak concurrent connections: 5
```

В каких случаях использовать семафор

Мы посмотрели, что семафор эффективно регулирует конкурентность. Теперь выясним, в каких распространенных сценариях этот механизм синхронизации оказывается незаменимым.

Управление ресурсами

Семафоры идеально подходят для ограничения доступа к общим ресурсам: сетевым сокетам, подключениям к БД или другим ресурсам с ограниченной пропускной способностью. Допустим, ваше приложение имеет лицензию только на десять конкурентных подключений к базе данных. Семафор с начальным значением счетчика 10 гарантирует, что вы никогда не превысите указанный лимит, предотвращая нарушение лицензии и ошибки подключения.

https://max.ru/bookofgeek

Ограничение скорости

Используйте семафоры для контроля частоты обращений к веб-сервису или API, чтобы предотвратить перегрузку и обеспечить удобное для пользователя взаимодействие с сайтом. Например, если внешний API разрешает только 100 запросов в минуту, можно использовать семафор вместе с планировщиком, который работает с числом разрешений, чтобы поддержать ограничение скорости запросов.

Общий контроль конкурентности

Семафоры предоставляют способ точного управления числом потоков, одновременно выполняющих определенный участок кода. Это критически важно для поддержания стабильности в высококонкурентных приложениях, особенно при работе с ресурсами, которые способны обрабатывать только определенный уровень параллелизма.

При работе с семафорами важно учитывать следующее.

Справедливость

Класс `Semaphore` в Java позволяет настраивать справедливость (fairness). В справедливом семафоре разрешения предоставляются в порядке поступления запросов. Однако такие семафоры могут быть менее производительными по сравнению с несправедливыми из-за затрат на поддержание порядка запросов. Выбирайте fairness, когда предотвратить голодание потоков важнее, чем добиться максимальной производительности.

Блокировка

Метод `acquire()` может заблокировать поток, если нет доступных разрешений. К счастью, появление в Java виртуальных потоков значительно снижает затраты ресурсов, связанные с такой блокировкой. В отличие от платформенных, заблокированные виртуальные потоки не потребляют ресурсы ОС, поэтому синхронизация на основе семафоров гораздо лучше масштабируется.

Обработка ошибок

Всегда освобождайте разрешения внутри блока `finally`. Это необходимо, чтобы предотвратить утечку ресурсов и правильно управлять ими даже при возникновении исключений.

Учет разрешений

Важно помнить, что семафоры не связывают разрешения с конкретными потоками. Любой поток способен освободить разрешение, даже если он его не получал. Такая гибкость может быть полезной, но требует аккуратного проектирования, чтобы избежать случайного увеличения общего числа доступных разрешений.

https://max.ru/bookofgeek

ОГРАНИЧЕНИЯ СЕМАФОРОВ

Семафоры необходимы для защиты ограниченных ресурсов, но у них есть опасная особенность: они не отслеживают, какой именно поток получил какое разрешение. Любой поток может освободить разрешение, даже если не получал его.

Вот как легко может возникнуть проблема:

```
// Начинаем с двух разрешений
Semaphore semaphore = new Semaphore(2);
// Поток 1: забывает освободить
Thread.ofVirtual().start(() -> {
  semaphore.acquire();
  // Не освобождает!
});
// Поток 2: освобождает, не получив разрешения
Thread.ofVirtual().start(() -> {
  semaphore.release(); // ОШИБКА!
  semaphore.release(); // ДВОЙНАЯ ОШИБКА!
});
// Результат: теперь у нас больше трех разрешений вместо двух!
```

В продакшене это может привести к превышению лимита подключений к базе данных, перегрузке API запросами или исчерпанию памяти.

Вот отказоустойчивый шаблон:

```
public <T> T useResource(Callable<T> task)
  throws Exception {
    semaphore.acquire(); // Получаем разрешение до try
    try {
      return task.call();
    } finally {
      semaphore.release(); // ВСЕГДА освобождает
    }
  }
```

В заключение: разобравшись, как работают семафоры, вы получаете мощный инструмент для управления общими ресурсами в Java-приложениях. Это особенно ценно при работе с виртуальными потоками, так как помогает обеспечить стабильность и оптимальную производительность.

Ограничения виртуальных потоков

До этого момента мы знакомились с преимуществами виртуальных потоков. Они созданы как легковесные сущности, которые среда выполнения Java легко планирует, чтобы как можно эффективнее управлять конкурентностью. Однако и у них есть недостаток — так называемый *пиннинг (pinning)*. Он может негативно повлиять на масштабируемость и производительность приложения.

https://max.ru/bookofgeek

Пиннинг — ситуация, когда виртуальный поток временно прикрепляется к своему потоку-носителю (базовому потоку платформы, на котором выполняется) и не может от него отсоединиться даже при блокирующих операциях. Фактически виртуальный поток монополизирует поток-носитель на время пиннинга.

Пиннинг выполняется в двух основных случаях.

Внутри синхронизированных блоков или методов

Когда виртуальный поток входит в `synchronized` блок или метод, он закрепляется за своим потоком-носителем. Во время выполнения такого блока или метода поток-носитель не может быть использован для других задач.

При выполнении нативных методов или внешних функций

При выполнении нативного метода или внешней функции виртуальный поток также находится в состоянии пиннинга.

Ключевое преимущество виртуальных потоков — возможность отсоединяться от потоков-носителей при выполнении блокирующих операций, освобождая их для других задач. Однако после пиннинга это становится невозможным. Появляется проблема, ведь количество потоков-носителей ограничено. Если множество виртуальных потоков остаются закрепленными в течение длительного времени, они блокируют свои потоки-носители. В результате другие виртуальные потоки не могут выполняться, что серьезно снижает преимущества от их использования для конкурентности.

Проблема пиннинга сводит на нет эти преимущества следующими способами.

Снижение пропускной способности

Поскольку виртуальный поток в состоянии пиннинга оккупирует поток-носитель, другим виртуальным потокам приходится ожидать освобождения потоков-носителей, в результате общая пропускная способность системы снижается.

Неэффективное использование ресурсов

Число потоков-носителей ограничено возможностями системы. Блокировка из-за пиннинга виртуальных потоков приводит к нерациональному использованию потоков-носителей.

Проблемы масштабируемости

Если значительная часть виртуальных потоков оказывается в состоянии пиннинга из-за использования множества синхронизированных блоков или нативных методов, возникают проблемы с масштабированием.

Чтобы смягчить последствия пиннинга, используйте такие стратегии.

- Вместо блоков или методов `synchronized` применяйте `ReentrantLock` (https://oreil.ly/DkvNO) из пакета java.util.concurrent.locks (https://oreil.ly/dg6lJ), так как

https://max.ru/bookofgeek

он позволяет виртуальному потоку отсоединяться от потока-носителя при блокировке.

- Регулярно анализируйте свой код, чтобы выявлять и сокращать использование синхронизированных методов и блоков, а также нативных методов в контексте виртуальных потоков.

Понимая, какие ограничения связаны с пиннингом виртуальных потоков, вы сможете принимать более обоснованные архитектурные решения при разработке Java-приложений.

Пиннинг

Рассмотрим пример на Java, демонстрирующий концепцию пиннинга виртуальных потоков:

```
import java.util.List;
import java.util.stream.IntStream;

public class ThreadPinnedExample {
  private static final Object lock = new Object();

  public static void main(String[] args) {
    List<Thread> threadList = IntStream.range(0, 10)
      .mapToObj(i -> Thread.ofVirtual().unstarted(() -> {
        if (i == 0) {
          System.out.println(Thread.currentThread()); ❶
        }
        synchronized (lock) { ❷
          try {
            Thread.sleep(25); ❸
          } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
          }
        }
        if(i == 0) {
          System.out.println(Thread.currentThread()); ❹
        }
      })).toList();

    threadList.forEach(Thread::start);
    threadList.forEach(t -> {
      try {
        t.join();
      } catch (InterruptedException e) {
        Thread.currentThread().interrupt();
      }
    });
  }
}
```

https://max.ru/bookofgeek

Проследим, что происходит при выполнении этого кода.

❶ Для первого виртуального потока (при `i == 0`) мы выводим информацию о текущем потоке перед входом в блок `synchronized`. Так мы узнаем, на каком потоке-носителе выполняется виртуальный поток.

❷ Когда виртуальный поток входит в этот синхронизированный блок, он прикрепляется (pinned) к своему потоку-носителю. Это происходит потому, что `synchronized` блоки в текущей реализации не позволяют виртуальным потокам отсоединяться от своих потоков-носителей.

❸ Операция `sleep` имитирует блокирующую I/O-операцию. В обычных условиях спящий виртуальный поток отсоединился бы от потока-носителя, освобождая его для выполнения других виртуальных потоков. Однако внутри `synchronized`-блока виртуальный поток остается прикрепленным, монополизируя поток-носитель на все 25 мс.

❹ После выхода из `synchronized`-блока мы снова выводим информацию о потоке, чтобы проверить, остался ли виртуальный поток на том же потоке-носителе.

При выполнении этого кода вывод для потока 0 демонстрирует эффект пиннига:

```
VirtualThread[#21]/runnable@ForkJoinPool-1-worker-1
VirtualThread[#21]/runnable@ForkJoinPool-1-worker-1
```

Обратите внимание, что идентификатор потока-носителя (`ForkJoinPool-1-worker-1`) один и тот же до и после синхронизированного блока. Мы убедились: виртуальный поток не переключился на другой поток-носитель, что указывает на пиннинг во время блокирующей операции.



Начиная с JDK 24 виртуальные потоки больше не будут находиться в состоянии пиннинга в синхронизированных блоках. JVM теперь поддерживает их отсоединение от потока-носителя внутри синхронизированных секций. Проблема пиннинга полностью устраняется. Однако в JDK 23 или более ранних версиях при проектировании конкурентных приложений пиннинг по-прежнему необходимо учитывать.

Представьте, что произойдет, если масштабировать проблему пиннинга до 1000 или 10 000 виртуальных потоков, каждый из которых пытается войти в синхронизированные блоки с блокирующими операциями. Большинство виртуальных потоков будет стоять в очереди, ожидая освобождения одного из немногих потоков-носителей. В такой ситуации нет смысла использовать виртуальные потоки для повышения конкурентности.

https://max.ru/bookofgeek

Решение проблемы пиннинга с помощью ReentrantLock

`ReentrantLock` — это механизм синхронизации в Java, который дает больше гибкости по сравнению с традиционными блоками с ключевым словом `synchronized`. Он поддерживает более сложное взаимодействие потоков и предлагает дополнительные функции, такие как справедливость (fairness), попытка захвата блокировки (try-lock) и возможность прерывать поток (interruptibility). Одно из основных преимуществ `ReentrantLock` — способность обходить проблему пиннинга виртуальных потоков.

Используем эту его способность в следующем примере. Вместо блока `synchronized`, в котором виртуальный поток находится в состоянии пиннинга, используем для синхронизации `ReentrantLock`. Виртуальный поток теперь может отсоединиться от потока-носителя при блокировке, освободив его для выполнения других задач. Например:

```
import java.util.concurrent.locks.ReentrantLock;
import java.util.stream.IntStream;

public class PreventPinningExample {
  private static final ReentrantLock lock = new ReentrantLock(); ❶

  public static void main(String[] args) {
    var threadList = IntStream.range(0, 10)
      .mapToObj(i -> Thread.ofVirtual().unstarted(() -> {
        if (i == 0) {
          System.out.println(Thread.currentThread()); ❷
        }
        lock.lock(); ❸
        try {
          Thread.sleep(25); ❹
        } catch (InterruptedException e) {
          Thread.currentThread().interrupt();
        } finally {
          lock.unlock(); ❺
        }
        if(i == 0) {
          System.out.println(Thread.currentThread()); ❻
        }
    })).toList();

    threadList.forEach(Thread::start);
    threadList.forEach(thread -> {
      try {
        thread.join();
```

https://max.ru/bookofgeek

```
      } catch (InterruptedException e) {
        Thread.currentThread().interrupt();
      }
    });
  }
}
```

Рассмотрим ключевые отличия от примера с `synchronized`.

❶ Вместо того чтобы использовать монитор объекта, создаем экземпляр класса `ReentrantLock`. Этот механизм обеспечивает то же взаимное исключение (mutual exclusion), что и `synchronized`, но с более широкими возможностями для виртуальных потоков.

❷ Для первого виртуального потока фиксируем начальную информацию о потоке-носителе.

❸ Метод `lock()` захватывает блокировку аналогично входу в `synchronized`-блок, но, в отличие от него, не делает пиннинг виртуального потока.

❹ Пока внутри блокировки выполняется операция sleep, виртуальный поток имеет возможность отсоединиться от потока-носителя. Носитель становится доступным для других виртуальных потоков, пока первый блокирован.

❺ Операцию `unlock` всегда выполняем в блоке `finally`. Это гарантирует освобождение от блокировки даже при исключении, предотвращая взаимные блокировки.

❻ После снятия блокировки повторно выводим информацию о потоке, чтобы отследить смену потока-носителя.

После выполнения кода этого примера мы видим:

```
VirtualThread[#20]/runnable@ForkJoinPool-1-worker-1
VirtualThread[#20]/runnable@ForkJoinPool-1-worker-3
```

Вывод можно интерпретировать следующим образом.

`VirtualThread[#20]/runnable@ForkJoinPool-1-worker-1`

Указывает, что виртуальный поток с ID `#20` находится в состоянии `runnable` и в данный момент прикреплен к потоку-носителю с именем `ForkJoinPool-1-worker-1`.

`VirtualThread[#20]/runnable@ForkJoinPool-1-worker-3`

Показывает, что тот же виртуальный поток с ID `#20` остается в состоянии `runnable`, но теперь прикреплен к другому потоку-носителю `ForkJoinPool-1-worker-3`.

Из примера видно, что виртуальный поток изначально выполнялся на одном потоке-носителе, а затем был перемещен на другой. Мы достигли этого благодаря использованию класса `ReentrantLock` вместо блока с ключевым словом

https://max.ru/bookofgeek

`synchronized`. Такой подход дает возможность избежать проблемы пиннинга и позволяет среде выполнения Java оптимально планировать выполнение виртуального потока на доступных потоках-носителях.

Главное различие заключается в реализации. Если `synchronized`-блоки используют мониторы объекта, которые в настоящее время требуют пиннинга, то `ReentrantLock` задействует механизм park/unpark, учитывающий особенности виртуальных потоков.

МЕХАНИЗМ PARK/UNPARK

Механизм park/unpark — это низкоуровневый примитив для координации потоков в Java. Он лежит в основе многих высокоуровневых инструментов конкурентности.

Когда поток вызывает метод `LockSupport.park()` (https://oreil.ly/rvEjY), он «припарковывается» (parked), то есть блокируется до тех пор, пока не произойдет что-то из перечисленного ниже:

- другой поток вызовет `unpark()` для первого потока;
- поток будет прерван;
- вызов завершается ложным пробуждением потока (редко, но случается).

Вызов `LockSupport.unpark(thread)` (https://oreil.ly/Hl-pC) переводит целевой поток в состояние готовности к продолжению выполнения:

```
// Поток A
LockSupport.park(); // Блокируется до вызова unpark
// Поток B
LockSupport.unpark(threadA); // Пробуждает поток A
```

Почему же механизм park/unpark предпочтительнее для виртуальных потоков? Дело в том, что JVM может определить момент «парковки» виртуального потока и отсоединить его от потока-носителя. В этом отличие данного механизма от монитора объекта (в синхронизированном блоке), который требует, чтобы виртуальный поток оставался прикрепленным к потоку-носителю.

Когда виртуальный поток блокируется с помощью `ReentrantLock`, среда выполнения Java может:

- сохранить состояние виртуального потока;
- отсоединить его от потока-носителя;
- использовать данный носитель для других виртуальных потоков;
- позже смонтировать этот виртуальный поток на любом доступном потоке-носителе, когда можно будет снова захватить блокировку.

https://max.ru/bookofgeek

СИНХРОНИЗИРОВАННЫЕ БЛОКИ И ПИННИНГ ВИРТУАЛЬНЫХ ПОТОКОВ

Добавление ключевого слова `synchronized` приводит к пиннингу, но степень его воздействия варьируется в зависимости от конкретного кода внутри синхронизированного блока. Рассмотрим несколько примеров.

Пример 1. Минимальный риск пиннинга

```
synchronized (this) {
  return this.a + this.b;
}
```

В данном случае операция внутри синхронизированного блока выполняется очень быстро. Даже если виртуальному потоку придется ожидать блокировку, он, скорее всего, будет очень быстро откреплен и негативное влияние окажется минимальным.

Пример 2. Потенциальная проблема из-за пиннинга

```
synchronized (this) {
  var response = httpClient.sendBlockingRequest(request);
  return response;
}
```

В данном случае блокирующий сетевой вызов (`httpClient.sendBlocking-Request`) внутри синхронизированного блока создает риск негативного влияния пиннинга. Виртуальный поток может оставаться прикрепленным к потоку-носителю на протяжении всего времени выполнения сетевого запроса, что снижает преимущества конкурентности.

Пример 3. Аналогичная проблема из-за пиннинга

```
synchronized (this) {
  this.wait();
}
```

Вызовы `Object.wait()` также приводят к пиннингу внутри `synchronized`-блоков, поскольку виртуальный поток останется в состоянии пиннинга вплоть до получения уведомления.

Основной вывод

Степень воздействия пиннинга, вызванного ключевым словом `synchronized`, зависит от характера операций внутри блока. Короткие неблокирующие операции обычно не вызывают проблем, тогда как блокирующие вызовы требуют особого внимания.

Дальнейшее развитие

В будущих обновлениях JDK (https://oreil.ly/QaB-6) могут появиться операции ввода-вывода, «осведомленные о пиннинге» (unpinning-aware), что

https://max.ru/bookofgeek

смягчит влияние пиннинга в некоторых сценариях. Пока же для достижения максимальной масштабируемости с помощью виртуальных потоков предпочтительнее использовать `ReentrantLock` как более гибкий механизм синхронизации.

Отдельная благодарность Симоне Борде (Simone Bordet, https://oreil.ly/7vHP-) за то, что доходчиво объяснила эту тему.

Вызов нативных методов и пиннинг

Аналогично экспериментам с `synchronized`-блоками проведем небольшое исследование, как вызов нативных методов может привести к пиннингу виртуального потока. Сделаем это на примере простой функции на языке C и Foreign Function & Memory API, доступного начиная с JDK 22.

Сначала создадим простую нативную функцию, имитирующую некоторую работу:

```
#include <unistd.h>
// Определение функции
int addNumbers(int number1, int number2) {
  // Приостанавливаем выполнение на 200 000 мкс (200 мс)
  usleep(200000);

  return number1 + number2;
}
```

Скомпилируем эту функцию в соответствии с вашей операционной системой:

- Linux: `gcc -shared -fPIC -o libaddNumbers.so addNumbers.c`;
- macOS: `gcc -shared -fPIC -o libaddNumbers.dylib addNumbers.c`;
- Windows: в среде GCC (например, MinGW) выполните `gcc -shared -o addNum bers.dll addNumbers.c`.

После компиляции на macOS я получил файл `libaddNumbers.dylib`. Теперь вызовем этот нативный метод с использованием виртуальных потоков:

```
import java.lang.foreign.*;
import java.lang.invoke.MethodHandle;
import java.nio.file.Path;
import java.util.List;
import java.util.stream.IntStream;

public class ThreadPinnedNativeMethodExample {
  public static void main(String[] args) {
```

https://max.ru/bookofgeek

```
    List<Thread> threadList = IntStream.range(0, 10)
      .mapToObj(i -> Thread.ofVirtual().unstarted(() -> {
        if (i == 0) {
          System.out.println(Thread.currentThread()); ❶
        }
        int sum = invokeNativeAddNumbers(56, 11); ❷
        if (i == 0) {
          System.out.println(Thread.currentThread()); ❸
        }
    })).toList();
    threadList.forEach(Thread::start);
    threadList.forEach(t -> {
      try {
        t.join();
      } catch (InterruptedException e) {
        Thread.currentThread().interrupt();
      }
    });
  }

  public static int invokeNativeAddNumbers(int a, int b) {
    try (Arena arena = Arena.ofConfined()) { ❹
      SymbolLookup lookup = SymbolLookup.libraryLookup(
        Path.of("libaddNumbers.dylib"), arena); ❺
      MemorySegment memorySegment = lookup.find("addNumbers")
        .orElseThrow(() ->
           new RuntimeException("addNumbers function not found"));
      Linker linker = Linker.nativeLinker();
      FunctionDescriptor addNumbersDescriptor = FunctionDescriptor.of(
        ValueLayout.JAVA_INT, // тип возвращаемого значения
        ValueLayout.JAVA_INT, // параметр 1
        ValueLayout.JAVA_INT); // параметр 2
      MethodHandle addNumbersHandle = linker.downcallHandle(
        memorySegment, addNumbersDescriptor); ❻
      try {
        return (int) addNumbersHandle.invokeExact(a, b); ❼
      } catch (Throwable e) {
        throw new RuntimeException(e.getMessage());
      }
    }
  }
}
```

Попробуем понять, что происходит во время выполнения.

❶ Перед вызовом нативного метода фиксируем информацию о потоке-носителе — это будет тот уровень, с которым можно сравнивать.

❷ Здесь происходит вызов нативного метода, в ходе которого виртуальный поток прикрепляется к потоку-носителю.

❸ После завершения нативного вызова снова проверяем поток-носитель, чтобы узнать, не изменился ли он.

https://max.ru/bookofgeek

❹ Использование API Foreign Function & Memory (FFM) (https://oreil.ly/TF4DO) — современный подход в Java к взаимодействию с нативным кодом. Этот API пришел на смену устаревшему JNI (Java Native Interface). Впервые представленный как предварительная функция в JDK 19 и окончательно реализованный в JDK 22, FFM API обеспечивает более безопасный и производительный способ вызова нативного кода и управления памятью вне кучи. Компонент `Arena` (https://oreil.ly/vcHHG) управляет жизненным циклом сегментов нативной памяти, гарантируя их правильную очистку при выходе из блока `try`. Для запуска этого примера с разрешенным доступом к нативным функциям используйте: `Java --enable-preview --source 21 --enable-native-access=ALL-UNNAMED ThreadPinned NativeMethodExample.java`.

❺ Настройте путь к библиотеке в соответствии с вашей ОС: используйте расширение `.so` для Linux, `.dylib` для macOS и `.dll` для Windows.

❻ Дескриптор downcall создает мост между Java и нативной функцией, позволяя вызывать код на C из Java.

❼ На протяжении этого вызова нативной функции виртуальный поток не может отсоединиться от своего потока-носителя, что приводит к его пиннингу на все 200 мс выполнения.

Вывод будет примерно таким:

```
VirtualThread[#20]/runnable@ForkJoinPool-1-worker-1
VirtualThread[#20]/runnable@ForkJoinPool-1-worker-1
```

Идентификаторы потоков до и после вызова `invokeNativeAddNumbers` идентичны, что подтверждает наличие пиннинга виртуального потока во время выполнения нативного метода. Возникает вопрос: почему так происходит при вызове нативных методов?

Пиннинг возникает потому, что JVM не способен анализировать или контролировать выполнение нативного кода. Из-за этого действует ряд ограничений: нативный код может содержать локальное состояние потока (thread-local state), которое невозможно передать между потоками; нативный стек вызовов нельзя сохранить и восстановить, как стековые фреймы Java; нативный код может напрямую взаимодействовать с примитивами потоков на уровне ОС. Эти ограничения вынуждают виртуальный поток оставаться смонтированным на своего носителя в течение выполнения нативного вызова.

Такое поведение пиннинга ведет к тому, что приложения с частыми вызовами нативных функций не получат заметного преимущества от масштабируемости виртуальных потоков. Для смягчения этой проблемы можно группировать операции, чтобы сократить число таких вызовов, или использовать асинхронные нативные API, или переписать нативную логику на чистой Java. Всегда оценивайте фактическое влияние нативных вызовов на конкурентность вашего приложения, чтобы определить, создают ли они узкое место.

https://max.ru/bookofgeek

JEP 491: СИНХРОНИЗАЦИЯ ВИРТУАЛЬНЫХ ПОТОКОВ БЕЗ ПИННИНГА

JEP 491: Synchronize Virtual Threads Without Pinning (https://oreil.ly/zGYVq) — это нововведение JDK 24. В нем сделан значительный шаг вперед благодаря переработке ключевого слова `synchronized` для лучшей совместимости с виртуальными потоками. В текущей версии при входе виртуального потока в синхронизированный блок он прикрепляется к платформенному потоку без возможности отсоединиться. JEP 491 позволяет виртуальным потокам захватывать, удерживать и освобождать мониторы независимо от своих потоков-носителей. Планировщик JVM теперь позволит заблокированным виртуальным потокам отключаться от платформенных, освобождая ресурсы для эффективного выполнения других задач. Однако пиннинг не устраняется полностью. В уникальных ситуациях: при блокировке внутри инициализаторов классов, ожидании инициализации класса другим потоком или разрешении символьных ссылок во время загрузки классов, а также вызовах нативного кода пиннинг по-прежнему будет возникать. Хотя такие ситуации бывают относительно редко, они могут создавать проблемы в высококонкурентных приложениях. JEP 491 предлагает отслеживать эти случаи и при необходимости дорабатывать подход в будущих обновлениях. Однако есть нюанс: для использования этих улучшений потребуется переход на JDK 24+. Поскольку JDK 21 остается версией с длительной поддержкой (Long-Term Support, LTS), на которую опирается большинство приложений, важно учитывать проблемы пиннинга и проектировать приложения соответствующим образом. Например, если запустить следующий код с помощью JDK 25, проблема пиннинга исчезнет:

```
java --enable-native-access=ALL-UNNAMED
  ThreadPinnedNativeMethodExample.java
```

Вывод будет таким:

```
VirtualThread[#26]/runnable@ForkJoinPool-1-worker-1
VirtualThread[#26]/runnable@ForkJoinPool-1-worker-6
```

Дилемма применения переменных ThreadLocal в виртуальных потоках

С помощью переменных `ThreadLocal` (https://oreil.ly/0TLqb) в Java можно ограничить данные конкретным потоком. Класс `ThreadLocal` позволяет создавать переменные, доступные для чтения и записи только тем потоком, который их создал. Таким образом, отпадает необходимость синхронизации в сценариях, где несколько потоков одновременно пытаются получить доступ к одним и тем же данным.

https://max.ru/bookofgeek

Вот типичные варианты использования.

Изоляция ресурсов

Переменные, созданные классом `ThreadLocal`, идеально подходят для хранения ресурсов, не являющихся потокобезопасными. Классический пример — `SimpleDateFormat`[1], где каждый поток может иметь собственный экземпляр, чтобы предотвратить конфликты.

Неявный контекст

Эти переменные также часто используются для хранения контекстной информации конкретной задачи потока, например: подключение к базе данных, данные пользовательской сессии или идентификаторы транзакций.

Проблемы виртуальных потоков

Виртуальные потоки в Java созданы легковесными, что в идеале позволяет запускать миллионы таких потоков в одном приложении. Однако при чрезмерном использовании переменных `ThreadLocal` такое их количество создает следующие проблемы.

Потребление памяти

Каждый виртуальный поток имеет собственную копию переменной `ThreadLocal`, из-за них может стремительно увеличиться потребление памяти, особенно при большом объеме хранимых данных.

Перерасход ресурсов

Инициализация и очистка `ThreadLocal`-переменных чревата излишним потреблением ресурсов. При работе с огромным количеством виртуальных потоков эти операции могут значительно снизить производительность.

Наследование

Как и традиционные потоки, виртуальные наследуют значения `ThreadLocal` от родительского потока. Такое наследование может приводить к неявным ошибкам, которые сложно выявить и исправить.

Рассмотрим пример кода, демонстрирующий потенциальные недостатки активного использования переменных `ThreadLocal` с виртуальными потоками:

```
import java.time.Duration;
import java.util.stream.IntStream;

public class ThreadLocalExample {
```

[1] В Java теперь доступен потокобезопасный класс DateTimeFormatter (https://oreil.ly/2WT6n), который устраняет необходимость использования ThreadLocal при обработке даты и времени.

https://max.ru/bookofgeek

```
  public static void main(String[] args) {
    ThreadLocal<LargeObject> threadLocal
           = ThreadLocal.withInitial(LargeObject::new);

    var threadList = IntStream.range(0, 1000)
      .mapToObj(i -> Thread.ofVirtual().unstarted(() -> {
         LargeObject largeObject = threadLocal.get();
         useIt(largeObject);
         sleep();
      })).toList();

    threadList.forEach(Thread::start);
    threadList.forEach(thread -> {
      try {
        thread.join();
      } catch (InterruptedException e) {
        Thread.currentThread().interrupt();
      }
    });
  }

  private static void useIt(LargeObject largeObject) {
    System.out.println(largeObject.data.length);
  }

  private static void sleep()
    try {
      Thread.sleep(Duration.ofMinutes(5));
    } catch (InterruptedException e) {
      throw new RuntimeException(e);
    }
  }

  static class LargeObject {
    private byte[] data = new byte[1024 * 500]; // 500 KB
  }
}
```

В этом примере создается объект типа LargeObject объемом 500 Кбайт. Каждый из 1000 виртуальных потоков хранит свою копию `LargeObject` в `ThreadLocal`. Открыв JConsole[1] после выполнения программы, мы можем посмотреть, как используется память (рис. 2.3).

Очевидно, что использование памяти возросло. Однако, если выполнить ту же программу без применения `ThreadLocal`, использование памяти сразу снижается, несмотря на те же 1000 виртуальных потоков (рис. 2.4).

[1] JConsole — это графический инструмент мониторинга, входящий в Java Development Kit (JDK).

https://max.ru/bookofgeek

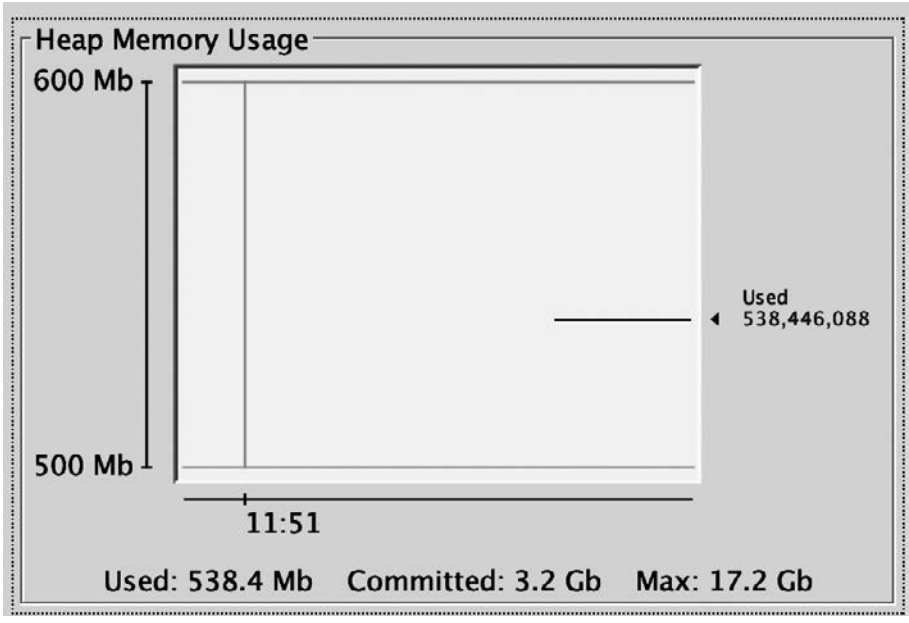


Рис. 2.3. JConsole: использование кучи при работе с ThreadLocal

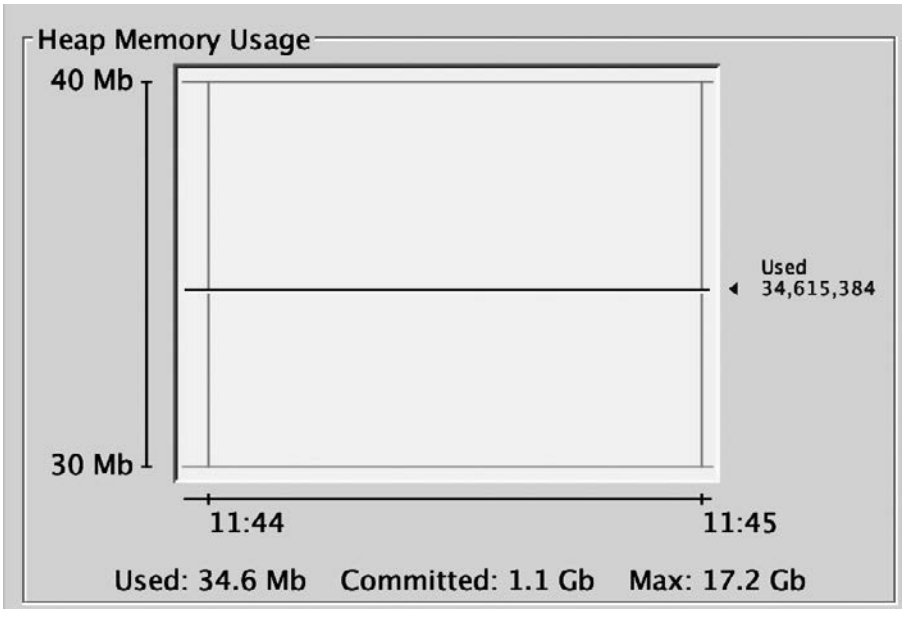


Рис. 2.4. JConsole: использование кучи без ThreadLocal

https://max.ru/bookofgeek

Перерасход памяти и ресурсов из-за применения `ThreadLocal` в среде виртуальных потоков побуждает искать альтернативные решения. Вот две основные стратегии.

Scoped values

Scoped values разрабатывались специально для виртуальных потоков. Неизменяемость и ограниченное время жизни делают scoped values идеальными для безопасной и эффективной передачи данных между потоками. Мы подробно рассмотрим эту тему в главе 5.

Переосмысление модели совместного доступа

Появление виртуальных потоков заставляет пересмотреть общий подход к разделению данных. Стоит подумать о реструктуризации приложения так, чтобы свести к минимуму потребность в `ThreadLocal`. Это, как правило, ведет к более масштабируемым решениям.

Мониторинг

Очевидно, что ограничения виртуальных потоков идут в основном из двух источников: пиннинга и использования `ThreadLocal`. Новое приложение можно спроектировать с их учетом, но такая возможность есть не всегда. Часто приходится работать с унаследованным кодом, который порой содержит миллионы строк. Миграция на виртуальные потоки в таких случаях требует внимательнейшего анализа проблем совместимости. К счастью, существуют специальные утилиты для выявления потенциальных проблем.

Мониторинг ThreadLocals

Чтобы отследить использование `ThreadLocal`, можно запустить JVM с флагом `-Djdk.traceVirtualThreadLocals`. При этом будут выводиться трассировки стека при каждом обращении к переменной `ThreadLocal` в виртуальном потоке. Это помогает выявить чрезмерное их применение или неправильное управление.

Для мониторинга активности `ThreadLocal` в нашем примере `ThreadLocalExample` используется команда:

```
java -Djdk.traceVirtualThreadLocals ThreadLocalExample.java
```

При ее выполнении мы получим результат следующего вида:

```
VirtualThread[#23]/runnable@ForkJoinPool-1-worker-4
...
VirtualThread[#25]/runnable@ForkJoinPool-1-worker-6
   java.base/java.lang.ThreadLocal.setInitialValue(ThreadLocal.java:236)
```

https://max.ru/bookofgeek

```
java.base/java.lang.ThreadLocal.get(ThreadLocal.java:194)
java.base/java.lang.ThreadLocal.get(ThreadLocal.java:172)
com.example.myapp.ThreadLocalExample.
            lambda$main$0(ThreadLocalExample.java:13)
java.base/java.lang.VirtualThread.run(VirtualThread.java:329)
```

Каждая запись в выводе дает ценную информацию о том, где и как используется `ThreadLocal`, что позволяет нам оптимизировать подход к обеспечению совместимости с виртуальными потоками.

При анализе вывода можно заметить несколько закономерностей. Сначала мы видим вызов `ThreadLocal.setInitialValue()`, что ожидаемо, поскольку каждый виртуальный поток должен инициализировать свою локальную переменную. Трассировки стеков `java.base/java.lang.ThreadLocal.setInitialValue(ThreadLocal.java:236)` подтверждают это.

Далее, когда потоку требуется получить доступ к своей локальной переменной, он вызывает `ThreadLocal.get()`. Выходные данные, содержащие экземпляры `java.base/java.lang.ThreadLocal.get(ThreadLocal.java:194)`, явно указывают на моменты такого доступа.

Что особенно важно, стек вызовов позволяет точно определить места в вашем коде, где используется `ThreadLocal`. Строки вида `com.example.myapp.ThreadLocalExample.lambda$main$0(ThreadLocalExample.java:13)` прямо указывают на эти критические точки в приложении, предоставляя точные ссылки для дальнейшего анализа или рефакторинга.

Становится очевидным, что каждый виртуальный поток инициализирует собственный экземпляр `ThreadLocal`, что подтверждает наши опасения: при большом количестве виртуальных потоков возрастает риск увеличенного потребления памяти. Этот момент требует тщательного анализа при миграции на виртуальные потоки.

В то же время трассировки стеков предоставляют бесценную информацию для отладки, точно показывая места использования `ThreadLocal`. Эти данные позволяют оптимизировать производительность приложения: контролировать целесообразность применения `ThreadLocal` и реструктурировать его использование, если он создает узкие места.

Мониторинг пиннинга

Наряду с `ThreadLocal` мы можем вести мониторинг на наличие пиннинга в существующем исходном коде. Для этого есть несколько инструментов, начиная от флагов JVM до Java Flight Recorder (JFR) и `jcmd` (https://oreil.ly/nnqx2) для дампов потоков. Начнем с изучения возможностей флага JVM.

https://max.ru/bookofgeek

Использование флага JVM

Системное свойство `jdk.tracePinnedThreads` предназначено для активации трассировки стека при возникновении блокирующей операции в потоке с пиннингом. При запуске JVM с параметром `-Djdk.tracePinnedThreads=full` выводится полная трассировка стека, охватывающая не только Java-фреймы, но и нативные, а также фреймы, удерживающие мониторы (они выделяются особо). Для более краткого вывода, сфокусированного на основной проблеме, `-Djdk.tracePinnedThreads=short` ограничивает трассировку только проблемными фреймами.

Выполним `ThreadPinnedExample.java` с флагом JVM:

```
java -Djdk.tracePinnedThreads=full ThreadPinnedExample.java
```

Вот фрагмент возможного вывода:

```
VirtualThread[#20]/runnable@ForkJoinPool-1-worker-1
VirtualThread[#21]/runnable@ForkJoinPool-1-worker-2 reason:MONITOR
  java.base/java.lang.VirtualThread$VThreadContinuation.onPinned(VirtualThread
  .java:199)
  java.base/jdk.internal.vm.Continuation.onPinned0(Continuation.java:393)
  java.base/java.lang.VirtualThread.parkNanos(VirtualThread.java:635)
  java.base/java.lang.VirtualThread.sleepNanos(VirtualThread.java:812)
  java.base/java.lang.Thread.sleepNanos(Thread.java:489)
  java.base/java.lang.Thread.sleep(Thread.java:522)
  ca.example.myapp.
.ThreadPinnedExample.lambda$main$0(ThreadPinnedExample.java:20) <== monitors:1
  java.base/java.lang.VirtualThread.run(VirtualThread.java:329)
VirtualThread[#20]/runnable@ForkJoinPool-1-worker-1
```

Данный вывод документирует каждый случай пиннинга потока. В частности, тег `reason:MONITOR` указывает, что поток ожидает захвата монитора — это ценный индикатор потенциальных узких мест. Трассировка стека показывает не только *место*, но и *причину* — в данном случае строка, заканчивающаяся на `<== monitors:1`, обозначает поток, удерживающий монитор.

Анализ таких выходных данных позволяет принимать обоснованные решения по оптимизации использования потоков и минимизации случаев пиннинга, выявляя проблемные участки кода и при необходимости заменяя их.

Использование Java Flight Recorder

Java Flight Recorder (JFR) — чрезвычайно ценный инструмент, выходящий за рамки традиционного мониторинга. Он предоставляет специализированные средства для диагностики и детальной отладки виртуальных потоков. JFR предлагает мощный набор возможностей для досконального и эффективного процесса отладки, что крайне важно при решении сложных задач конкурентного программирования.

https://max.ru/bookofgeek

JFR предлагает действенные инструменты для мониторинга поведения виртуальных потоков. Рассмотрим наиболее важные события.

- `jdk.VirtualThreadStart` и `jdk.VirtualThreadEnd` отмечают создание и завершение виртуального потока. По умолчанию они отключены, но их активация позволяет детально отслеживать жизненный цикл потоков, что полезно для отладки или гранулированного анализа.
- `jdk.VirtualThreadPinned` — это критически важное событие, включенное по умолчанию, сигнализирует, когда виртуальный поток долго не освобождает поток-носитель (порог настраиваемый, по умолчанию составляет 20 мс). События пиннинга указывают на потенциальные узкие места производительности, в которых теряются преимущества виртуальных потоков.
- `jdk.VirtualThreadSubmitFailed` — еще одно событие, включенное по умолчанию. Оно сигнализирует об ошибках при запуске или восстановлении (unpark) приостановленного виртуального потока. Оно может свидетельствовать об исчерпании ресурсов или неожиданных узких местах в JVM-управлении потоками.

Для мониторинга таких событий, как `jdk.VirtualThreadStart` и `jdk.VirtualThreadEnd`, существуют два основных подхода: использование JDK Mission Control (https://oreil.ly/4p_Ek) для работы через GUI или создание кастомного конфигурационного файла JFR для непосредственного управления.

Создание конфигурационного файла JFR (с расширением `.jfc`) — это простой способ отслеживать интересующие нас события. Такой адаптированный файл позволяет проводить выборочный мониторинг, концентрировать ресурсы и анализировать релевантные точки данных.

Вот базовый шаблон файла `.jfc`, который запишет ключевые события виртуальных потоков:

```
<?xml version="1.0" encoding="UTF-8"?>
<configuration version="2.0" label="Virtual Thread Events"
    description="JFR configuration to record virtual thread events">
  <event name="jdk.VirtualThreadStart">
  <setting name="enabled">true</setting>
  </event>
  <event name="jdk.VirtualThreadEnd">
  <setting name="enabled">true</setting>
  </event>
  <event name="jdk.VirtualThreadPinned">
  <setting name="enabled">true</setting>
  </event>
  <event name="jdk.VirtualThreadSubmitFailed">
  <setting name="enabled">true</setting>
  </event>
</configuration>
```

https://max.ru/bookofgeek

Чтобы использовать этот шаблон, сохраните XML-контент в файл с именем `VThreadEvents.jfc`.

Разберемся, как эффективно отслеживать виртуальные потоки, на примере кода, демонстрирующего JFR в действии. Вот программа на Java, которую мы разберем далее:

```
import java.time.Duration;
import java.util.concurrent.locks.Lock;
import java.util.concurrent.locks.ReentrantLock;

public class JFRVirtualThreadDemo {
  private static final Object syncLock = new Object();
  private static final Lock reentrantLock = new ReentrantLock();
  public static void main(String[] args) {
    // Запуск событий жизненного цикла виртуальных потоков
    Thread vThreadStartEnd = Thread.ofVirtual().unstarted(() -> {
      System.out.println("Virtual thread started and will end soon.");
    }); ❶
    vThreadStartEnd.start(); joinThread(vThreadStartEnd);
    // Пиннинг в синхронизированном блоке
    Thread vThreadPinnedSync = Thread.ofVirtual().unstarted(() -> {
      synchronized (syncLock) { ❷
        sleepUninterruptibly(Duration.ofMillis(500));
      }
    });
    vThreadPinnedSync.start();
    joinThread(vThreadPinnedSync);
    // При использовании ReentrantLock нет пиннинга
    Thread vThreadWithLock = Thread.ofVirtual().unstarted(() -> {
      reentrantLock.lock();
      try {
        sleepUninterruptibly(Duration.ofMillis(500)); ❸
      } finally {
        reentrantLock.unlock();
      }
    });
    vThreadWithLock.start();
    joinThread(vThreadWithLock);
  }

  private static void joinThread(Thread thread) {
    try {
      thread.join();
    } catch (InterruptedException e) {
      Thread.currentThread().interrupt();
    }
  }

  private static void sleepUninterruptibly(Duration duration) { ❹
    boolean interrupted = false;
```

https://max.ru/bookofgeek

```
    try {
      long remainingNanos = duration.toNanos();
      long end = System.nanoTime() + remainingNanos;
      while (true) {
        try {
          Thread.sleep(remainingNanos / 1_000_000,
              (int) (remainingNanos % 1_000_000));
          return;
        } catch (InterruptedException e) {
          interrupted = true;
          remainingNanos = end - System.nanoTime();
        }
      }
    } finally {
      if(interrupted) {
        Thread.currentThread().interrupt();
      }
    }
  }
}
```

Рассмотрим фрагменты кода.

❶ Простой виртуальный поток демонстрирует базовые события жизненного цикла. JFR фиксирует как события `VirtualThreadStart`, так и `VirtualThreadEnd`, позволяя увидеть, насколько быстро виртуальные потоки выполняют короткие задачи.

❷ В блоке с ключевым словом `synchronized` происходит пиннинг потока. Когда виртуальный поток находится в состоянии сна внутри этого блока, он прикреплен к своему потоку-носителю в течение всех 500 мс. JFR зафиксирует это как событие `VirtualThreadPinned`.

❸ Несмотря на использование блокировки, `ReentrantLock` не служит причиной пиннинга. Виртуальный поток может отключаться от своего потока-носителя во время сна, что показывает преимущество использования `ReentrantLock` для виртуальных потоков.

❹ Этот вспомогательный метод гарантирует, что поток будет находиться в состоянии сна полное время, даже если его прервут, то есть измерение времени для демонстрационных целей становится более предсказуемым.

Мы все подготовили, теперь можно запускать запись и выполнять наш пример. Эта команда инициализирует запись событий с помощью JFR:

```
java -XX:StartFlightRecording=filename=recording.jfr,settings=VThreadEvents.jfc \
JFRVirtualThreadDemo.java
```

По этой команде будут записываться только те события, которые мы указали в нашем пользовательском файле `.jfc`.

https://max.ru/bookofgeek

После выполнения приложения мы можем проанализировать файл `recording.jfr` с помощью таких инструментов, как JDK Mission Control, или вывести события в консоль командой:

```
jfr print --events jdk.VirtualThreadStart,jdk.VirtualThreadEnd,\
jdk.VirtualThreadPinned,jdk.VirtualThreadSubmitFailed recording.jfr
```

Таким образом, мы получим аналитическую информацию о поведении виртуальных потоков в нашем Java-приложении:

```
jdk.VirtualThreadStart {
  startTime = 10:23:14.936 (2024-03-23)
  javaThreadId = 23
  eventThread = "" (javaThreadId = 23, virtual)
}
jdk.VirtualThreadEnd {
  startTime = 10:23:14.939 (2024-03-23)
  javaThreadId = 23
  eventThread = "" (javaThreadId = 23, virtual)
}
jdk.VirtualThreadStart {
  startTime = 10:23:14.940 (2024-03-23)
  javaThreadId = 35
  eventThread = "" (javaThreadId = 35, virtual)
}
jdk.VirtualThreadPinned {
  startTime = 10:23:14.941 (2024-03-23)
  duration = 504 ms
  eventThread = "" (javaThreadId = 35, virtual)
}
jdk.VirtualThreadEnd {
  startTime = 10:23:15.445 (2024-03-23)
  javaThreadId = 35
  eventThread = "" (javaThreadId = 35, virtual)
}
jdk.VirtualThreadStart {
  startTime = 10:23:15.446 (2024-03-23)
  javaThreadId = 37
  eventThread = "" (javaThreadId = 37, virtual)
}
jdk.VirtualThreadEnd {
  startTime = 10:23:15.948 (2024-03-23)
  javaThreadId = 37
  eventThread = "" (javaThreadId = 37, virtual)
}
```

Эти логи показывают, насколько быстро создаются и завершаются виртуальные потоки, что подчеркивает: они предназначены для кратковременных задач. Кроме того, логи показывают моменты пиннинга виртуальных потоков. Время пиннинга указывает, как долго виртуальный поток остается присоединен к потоку-носителю, что помогает выявить замедление работы.

https://max.ru/bookofgeek

Дополнительно мы можем загрузить файл `recording.jfr` в JDK Mission Control или в Azul Mission Control (https://oreil.ly/qKQZ9) и проанализировать его (рис. 2.5).

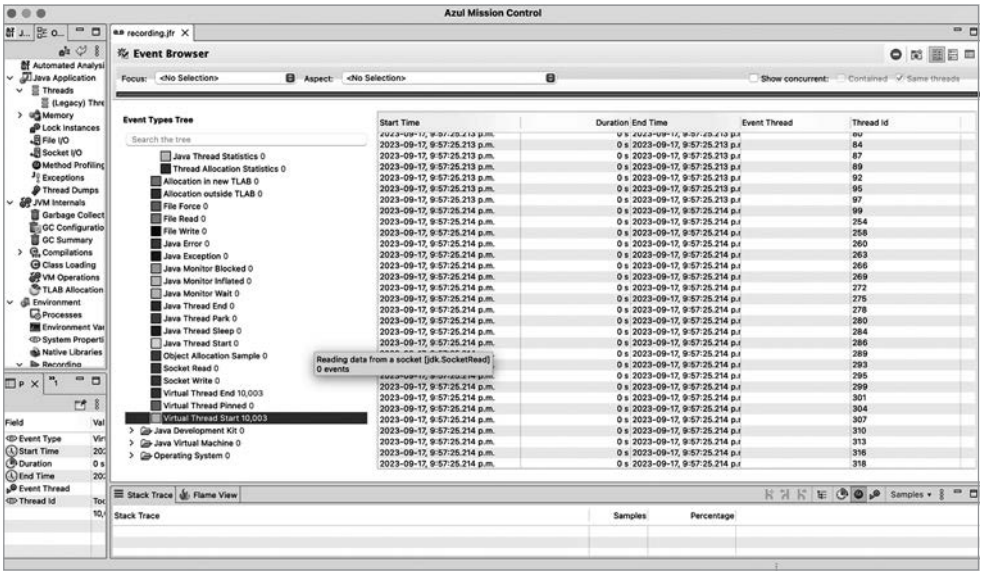


Рис. 2.5. Анализ Java Flight Recorder с помощью Azul Mission Control

Просмотр виртуальных потоков в дампах потоков с помощью jcmd

Такие инструменты, как JDK Mission Control, предоставляют расширенный анализ, но иногда для быстрой диагностики полезно использовать дамп потоков. Утилита `jcmd` (https://oreil.ly/nnqx2) дает такую функциональность и, что особенно важно, позволяет получать информацию о виртуальных потоках.

Для генерации дампа потоков сначала получите идентификатор процесса (PID, process ID) вашего Java-приложения с помощью инструментов `ps` или `jps`[1]. Затем выберите желательный формат дампа.

Простой текст

```
jcmd <PID> Thread.dump_to_file -format=text <файл>
```

JSON

```
jcmd <PID> Thread.dump_to_file -format=json <файл>
```

[1] С помощью команды ps можно определить PID для любых процессов, а утилита jps предназначена для поиска процессов Java-приложений. Чтобы вывести все процессы с фильтрацией по имени, используйте команду ps -ef | grep *<имя_процесса>* (замените *<имя_процесса>* на фактическое имя). В первом столбце отображается PID. Для Java-процессов используйте jps, а для детализированного списка — jps -l.

https://max.ru/bookofgeek

Замените <*PID*> фактическим идентификатором процесса, <*файл*> — нужным именем файла.

Например, для создания JSON-дампа потоков можно выполнить следующую команду:

```
jcmd 12345 Thread.dump_to_file -format=json threaddump.json
```

Формат JSON отлично подходит для автоматизированных инструментов отладки и анализа, которые могут потреблять данные в этом формате. Этот машиночитаемый формат позволяет программно анализировать состояние потоков, что особенно полезно в сложных системах с большим их количеством.

Дамп потоков, снятый с помощью `jcmd`, охватывает виртуальные потоки, заблокированные операциями сетевого ввода-вывода, а также потоки, созданные через интерфейс `ExecutorService`. Однако у дампа есть ограничения — в него не включаются:

- адреса объектов;
- блокировки;
- статистика Java Native Interface (JNI);
- статистика кучи;
- другая информация, часто встречающаяся в традиционных дампах потоков.

Акцент на ключевой информации упрощает выявление проблем, связанных именно с выполнением и блокировкой потоков, без лишних, часто нерелевантных, деталей.

Используя `jcmd` вместе с JFR, мы получаем полную картину работы виртуальных потоков в приложении, что помогает при отладке и оптимизации.

Программные дампы потоков в Java

Помимо использования утилиты командной строки, можно программно генерировать дампы потоков непосредственно в Java-приложении. Следующий фрагмент кода на Java демонстрирует эту технику:

```
import java.io.IOException;
import java.lang.ProcessHandle;
import java.time.Duration;
import java.util.List;
import java.util.concurrent.Executors;
import java.util.concurrent.TimeUnit;
import java.util.stream.IntStream;

public class ThreadDumpDemo {
  private static final int THREAD_COUNT = 1_000;
```

https://max.ru/bookofgeek

```
  private static final Duration WORK_DURATION = Duration.ofSeconds(5);
  private static final Duration DELAY_BEFORE_DUMP = Duration.ofSeconds(2);

  public static void main(String[] args) {
    long pid = ProcessHandle.current().pid(); ❶
    String outputFile = "dump.json";
    try (var executor = Executors.newVirtualThreadPerTaskExecutor()) {
      IntStream.range(0, THREAD_COUNT).forEach(i ->
          executor.submit(() -> sleep(WORK_DURATION))); ❷
      executor.submit(() -> {
        sleep(DELAY_BEFORE_DUMP); ❸
        runJcmdDump(pid, outputFile);
      });
    }
  }

  private static void sleep(Duration d) {
    try {
      TimeUnit.NANOSECONDS.sleep(d.toNanos());
    } catch (InterruptedException e) {
      Thread.currentThread().interrupt();
    }
  }

  private static void runJcmdDump(long pid, String file) {
    ProcessBuilder pb = new ProcessBuilder(List.of(
      "/bin/sh", "-c",
    String.format("jcmd %d Thread.dump_to_file -format=json %s",
        pid, file))); ❹
    try {
      Process p = pb.start();
      int exit = p.waitFor();
      if (exit != 0) {
        System.err.printf("jcmd exited %d%n", exit);
        p.getInputStream().transferTo(System.err);
        p.getErrorStream().transferTo(System.err);
      }
    } catch (IOException | InterruptedException e) {
      Thread.currentThread().interrupt();
      System.err.println("Failed to run jcmd: " + e.getMessage());
    }
  }
}
```

Разберемся, как эта программа захватывает состояние потоков.

❶ Мы получаем текущий PID с помощью API `ProcessHandle` (https://oreil.ly/DmzRe). Этот PID подсказывает инструменту `jcmd`, для какого именно процесса JVM нужен дамп.

❷ Создаем 1000 виртуальных потоков, каждый из которых «засыпает» на 5 с. Это моделирует высоконагруженную конкурентную среду, гарантируя наличие множества активных потоков при создании дампа.

https://max.ru/bookofgeek

❸ Дамп потоков выполняется с двухсекундной задержкой. Такого интервала достаточно, чтобы большинство виртуальных потоков были активными (или в состоянии сна) при фиксации состояния системы.

❹ Для выполнения команды `jcmd` с выводом данных в формате JSON используем класс Java `ProcessBuilder` (https://oreil.ly/gHhAl). JSON-формат предоставляет структурированные данные, которые проще программно анализировать, чем традиционные текстовые дампы.

На примере этой программы мы видим, как инициировать и захватывать дампы потоков с использованием `ProcessBuilder`. Программа планирует большое количество виртуальных потоков, а непосредственно перед запуском последнего формирует дамп. Он сохраняется в файл, а PID выводится в консоль для справки.

Сгенерированный файл `dump.json` содержит ценную информацию о состоянии системы:

```
{
  "threadDump": { "processId": "76586",
    "time": "2024-03-23T15:01:41.901030Z",
    "runtimeVersion": "21+35-2513", "threadContainers": [
      {
        "container": "<root>",
        "parent": null,
        "owner": null,
        "threads": [
          {
            "tid": "1",
            "name": "main",
            "stack": [
              "java.base\/java.io.FileInputStream.readBytes(Native Method)",
              "java.base\/java.io.FileInputStream.read(FileInputStream.java:287)",
              "java.base\/java.io.BufferedInputStream.read1
                                        (BufferedInputStream.java:345)",
              "java.base\/java.io.BufferedInputStream.implRead
                                        (BufferedInputStream.java:420)",
              ... (additional stack trace elements)"
            ]
          },
          ....
          "... (additional threads)"
        ],
        "threadCount": "7"
      },
      {
        "container": "java.util.concurrent.ThreadPerTaskExecutor@768b771c",
        "parent": "<root>",
```

https://max.ru/bookofgeek

```
        "owner": null,
        "threads": [
          {
            "tid": "20",
            "name": "",
            "stack": [
              "java.base\/java.lang.VirtualThread.parkNanos
                                        (VirtualThread.java:631)",
              "java.base\/java.lang.VirtualThread.sleepNanos
                                        (VirtualThread.java:803)",
              "java.base\/java.lang.Thread.sleep(Thread.java:507)",
              "ThreadDumpDemo.sleepOfSeconds(ThreadDumpDemo.java:38)",
              "ThreadDumpDemo.lambda$main$0(ThreadDumpDemo.java:29)",
              "java.base\/java.util.concurrent.ThreadPerTaskExecutor$TaskRunner
                   .run(ThreadPerTaskExecutor.java:314)",
              "java.base\/java.lang.VirtualThread.run(VirtualThread.java:311)"
            ]
          }
        "... (additional threads)"
      "... (additional thread containers)"
    ]
  }
}
```

Посмотрим, что мы получили.

Главный поток

Поток с идентификатором `tid: "1"` и именем `main` — это главный поток приложения, с которого начинается выполнение программы. В данный момент он выполняет операцию чтения файла.

Системные потоки

Системные потоки `Reference Handler`, `Finalizer`, `Signal Dispatcher` и `Notification Thread` отвечают за сборку мусора, ссылочные объекты и другие задачи на уровне JVM.

Поток Common-cleaner

Этот поток отвечает за очистку ссылочных объектов после их обработки сборщиком мусора.

Множество виртуальных потоков

Мы видим много потоков, скорее всего, виртуальных. Это многочисленные безымянные потоки с идентификаторами (`tid`), например `20`, `22`, `23` и т. д. Их принадлежность к виртуальным потокам подтверждается наличием в стек-трейсах вызовов `java.lang.VirtualThread.parkNanos` и `java.lang.VirtualThread.sleepNanos`, что, вероятно, связано с нашими вызовами метода `sleepOfSeconds`.

https://max.ru/bookofgeek

В реальных условиях необходимо обращать внимание на следующие ключевые моменты в дампе потоков.

Заблокированные потоки

Потоки в состояниях `BLOCKED` и `WAITING` часто указывают на проблемы, такие как взаимные блокировки или конкуренция за ресурсы.

Использование ресурсов

Анализ дампа потоков следует сочетать с данными о загруженности CPU и использовании памяти, чтобы выявить возможную перегрузку системы, из-за чего также могут возникать узкие места.

Генерация дампов потоков с помощью HotSpotDiagnosticsMXBean

Технология Java Management Extensions (JMX) предлагает мощные инструменты для мониторинга приложений. Интерфейс `HotSpotDiagnosticMXBean` (https://oreil.ly/H4dl2) из пакета com.sun.management (https://oreil.ly/bVf_E) был усилен новым методом для указания нужного формата дампов потоков. В число доступных вариантов входят JSON (для структурированного анализа) и традиционный текстовый формат.

Так можно программно сгенерировать дамп потоков в формате JSON:

```
public static void takeThreadDump(String outputFile) {
  var hotSpotDiagnosticMXBean
    = ManagementFactory.getPlatformMXBean(HotSpotDiagnosticMXBean.class);
  try {
  // Убедимся, что путь к выходному файлу абсолютный
  if (!new File(outputFile).isAbsolute()) {
    throw new IllegalArgumentException("Output path must be absolute.");
  }
  hotSpotDiagnosticMXBean.dumpThreads(outputFile,
    HotSpotDiagnosticMXBean.ThreadDumpFormat.JSON);
  } catch (IOException e) {
    throw new RuntimeException("An error occurred while taking thread dump", e);
  }
}
```

Метод `dumpThreads` (https://oreil.ly/uekb2) — это нововведение, он принимает аргумент из перечисления `ThreadDumpFormat` (https://oreil.ly/Npslp). Метод позволяет выбирать между форматами `TEXT_PLAIN` (https://oreil.ly/_dQ4n) и `JSON` (https://oreil.ly/s5RUt). Чтобы избежать ошибок, важно всегда указывать абсолютный путь для параметра `outputFile`. Этот метод можно использовать с инструментами JMX как для локальной, так и для удаленной диагностики.

https://max.ru/bookofgeek

Практические советы по миграции на виртуальные потоки

Итак, мы рассмотрели преимущества и ограничения, а также методы отладки виртуальных потоков. Теперь я дам несколько практических рекомендаций по миграции на них.

Обновление библиотек — ключ к успеху

Наилучший способ избежать проблем с пиннингом — использовать библиотеки, специально адаптированные к виртуальным потокам. Они поддерживают современные механизмы синхронизации, которые не связывают потоки-носители.

Если обновления недоступны

Если невозможно обновить библиотеки, следует вынести устаревшие операции ввода-вывода и другие блокирующие операции в традиционные пулы потоков (`Executors.newFixedThreadPool()`). Это позволяет ограничить влияние блокирующего кода.

Дилемма семафоров

Семафоры могут ограничивать число виртуальных потоков, которые попадают в участки кода, склонные к пиннингу. Но используйте их крайне осторожно. Слишком низкий лимит подавит конкурентность приложения и сведет на нет преимущества виртуальных потоков.

Появление виртуальных потоков принесло значительную выгоду, но важно учитывать несколько подводных камней, чтобы полностью раскрыть потенциал этих потоков и избежать типичных проблем.

Эволюция экосистемы

Многие библиотеки еще не адаптировались к виртуальным потокам. Нужно регулярно проверять обновления.

Компромиссы между различными факторами

Виртуальные потоки позволяют выполнять больше задач конкурентно, но могут привести к нехватке ресурсов. До сих пор нужны способы для ограничения использования этих потоков.

Знайте свой фреймворк

Необходимо изучить, как работают с виртуальными потоками выбранные вами фреймворки и языки. Возможно, имеются конкретные паттерны, которых следует избегать.

https://max.ru/bookofgeek

При внедрении виртуальных потоков необходимо усилить мониторинг. Стоит сосредоточиться на следующих ключевых метриках.

CPU

Важно увидеть прирост масштабируемости! Отслеживайте профиль использования CPU, убедитесь, что есть эффект от использования виртуальных потоков, и минимизируйте число неожиданных событий пиннинга.

Память и сборка мусора

Виртуальные потоки изменяют схему использования памяти. Возможно, потребуется настройка размера кучи или оптимизация сборки.

Задержка и пропускная способность

Это основной показатель. Уменьшилось ли время отклика для пользователей? Способна ли система обрабатывать больше запросов под нагрузкой? Эти метрики покажут реальное влияние миграции.

Еще раз о преимуществах виртуальных потоков

Мы уже рассмотрели механику работы виртуальных потоков, методы мониторинга и потенциальные зоны риска. Теперь вспомним основные преимущества, которые делают их ценным дополнением к инструментарию конкурентности Java.

Проще использовать конкурентность

Виртуальные потоки кардинально упрощают написание конкурентного кода. Их легковесная природа позволяет разработчику более естественно реализовывать задумки, снижает когнитивную нагрузку и число потенциальных ошибок по сравнению с управлением традиционными потоками.

Улучшенная масштабируемость

Приложения, используя виртуальные потоки, часто могут достичь значительно более высокого уровня конкурентности, не страдая от ограничения ресурсов, характерного для потоков ОС. Высокий уровень масштабируемости особенно важен для современных приложений, обрабатывающих множество одновременных подключений или запросов.

Эффективное использование ресурсов

Виртуальные потоки потребляют минимум памяти, а JVM эффективно их планирует. Это напрямую ведет к лучшему использованию аппаратных ресурсов, снижает эксплуатационные затраты или позволяет делать больше на том же оборудовании.

Совместимость

Важное преимущество виртуальных потоков — интеграция с существующими парадигмами Java. Их можно без значительного рефакторинга кода внедрить во многие приложения, чтобы повысить производительность.

https://max.ru/bookofgeek

Снижение нагрузки на CPU

Поскольку виртуальные потоки не потребляют время процессора в режиме простоя (например, при ожидании ввода-вывода), приложение может выделять больше вычислительных ресурсов активным задачам, повышая общую отзывчивость.

Оптимизация программирования на стороне сервера

Обеспечивая значительно более высокую масштабируемость по сравнению с традиционными потоками, виртуальные потоки позволяют возродить классическую модель «один запрос — один поток» в серверных приложениях.

Масштабируемость

Понятие масштабируемости в Java связано со способностью приложения обрабатывать возрастающую нагрузку. Исторически главным ограничивающим фактором здесь были операции ввода-вывода. Платформенные потоки можно сравнить с кассирами в банке: клиенты стоят в очереди, но сами кассиры бездействуют, ожидая обработки данных или ответа от сети.

Виртуальные потоки кардинально меняют ситуацию. Они настолько легковесны, что их можно параллельно запускать тысячами или даже миллионами без ощутимой нагрузки на систему. Более того, во время ожидания ввода-вывода виртуальные потоки могут легко «демонтироваться» со своих потоков-носителей, освобождая при этом память. В результате система обрабатывает значительно больше задач за то же время.

Виртуальные потоки раскрывают всю мощь современных CPU. Их способность поддерживать огромное количество параллельных задач позволяет максимально эффективно использовать аппаратные ресурсы. Увеличение числа потоков приводит к повышению пропускной способности и отзывчивости приложения даже при значительной нагрузке.

Рассмотрим преимущества для вашего масштабируемого «банка» (приложения).

Обслуживание большего числа клиентов (конкурентных запросов)

Теперь ваш «банк», то есть приложение, может обрабатывать значительно больший объем запросов без потери производительности.

Сокращение времени ожидания (увеличенная пропускная способность)

Эффективное использование ресурсов обеспечивает ускоренное обслуживание, позволяя обрабатывать больше запросов за то же время.

Оптимизированное использование ресурсов (меньший объем памяти)

Виртуальные потоки экономны, поэтому вы можете обслуживать больше клиентов, используя то же количество кассиров (базовых ресурсов).

https://max.ru/bookofgeek

Виртуальные потоки — огромный скачок в развитии масштабируемых Java-приложений. Освобождаясь от ограничений традиционных потоков, они открывают путь к созданию более эффективных, отзывчивых систем, способных справляться с постоянно растущими нагрузками.

В заключение

Внедрение виртуальных потоков в Java ознаменовало собой важный этап в развитии модели конкурентности языка, значительно повысив масштабируемость и производительность сервисных приложений. Осваивая эту технологию, задействуйте ее ключевое преимущество — возможность обрабатывать огромное количество конкурентных задач с минимальными затратами ресурсов. Это позволит создавать более производительные, отзывчивые и масштабируемые приложения.

Начните с внедрения виртуальных потоков в те сценарии использования приложения, где они принесут максимальную пользу, например, в операциях, связанных с вводом-выводом. Постепенно наращивайте их использование, тщательно отслеживая влияние на производительность и потребление ресурсов.

Контролируйте активность потоков, выявляйте возможные узкие места и убедитесь, что вы полностью используете преимущества виртуальных потоков. Не забывайте об ограничениях, связанных с пиннингом и `ThreadLocal`. Для предотвращения нежелательного пиннинга используйте современные инструменты синхронизации, например `ReentrantLock`. Следите за новейшими библиотеками и фреймворками, чтобы использовать преимущества оптимизации и новых функций.

Пересмотрите свои подходы к конкурентности и перейдите на более простые и естественные модели работы с виртуальными потоками. Упрощение логики конкурентности не только повышает производительность, но и делает код чище, а также удобнее в сопровождении.

Постоянное отслеживание нововведений в Java и адаптация к ним позволят по максимуму использовать новые мощные возможности, чтобы раскрыть весь потенциал ваших приложений.

https://max.ru/bookofgeek

ГЛАВА 3

Механизм современной конкурентности в Java

> Все на свете имеет свои причины. Пока не познаешь причины явления, не поймешь его и не углубишь знания о нем.
>
> *Ибн Сина (Авиценна, Ibn Sina/Avicenna), ок. 1025 г. н. э.*

Важно не только уметь использовать что-либо, но и понимать, как оно действует. Знание внутренних механизмов позволяет глубже оценить возможности и оперативно устранять возникающие проблемы. Поэтому грамотному Java-разработчику необходимо знать, как реализованы виртуальные потоки и каково их внутреннее устройство. В данной главе мы исследуем эти вопросы.

Чтобы понять, как реализованы виртуальные потоки, необходимо усвоить две ключевые концепции: `ForkJoinPool` и *континуация* (continuation — «продолжение»). `ForkJoinPool` играет роль планировщика виртуальных потоков, а континуации отвечают за их приостановку и последующее возобновление выполнения. Мы начнем с общего знакомства с пулом потоков, затем постепенно углубимся в особенности `ForkJoinPool` и его реализацию, слегка модифицированную для поддержки виртуальных потоков.

Пул потоков

В первой главе мы говорили о Java-фреймворке Executor, который, по сути, является реализацией пула потоков. Но мы еще не выяснили, как именно пул потоков работает и, что еще важнее, зачем он вообще нужен.

Пул потоков — это группа потоков, которые создаются при запуске приложения и работают на протяжении всего его жизненного цикла. Стратегия создания пула зависит от его типа, но основная идея заключается в том, чтобы иметь

https://max.ru/bookofgeek

набор готовых потоков и поддерживать их работу. В пуле потоков есть очередь для хранения задач. Он выбирает задачи из очереди и выполняет их, как только появляются свободные потоки. Завершив работу, поток сразу переключается на следующую задачу, если таковая имеется. При отсутствии задач пул переходит в режим ожидания. Если все потоки заняты, новые задачи помещаются в очередь и ждут, пока потоки освободятся.

Зачем нужен пул потоков

Хотя создание потока — простая операция (как мы видели в главе 1), спонтанное создание множества потоков может вызвать проблемы. Если увеличивать число потоков по мере роста нагрузки, можно достичь системного лимита, установленного операционной системой, что приведет к нежелательному сценарию — аварийному завершению приложения. Пул потоков позволяет заранее задать и контролировать их максимальное количество в процессе работы. К тому же он естественным образом ограничивает нагрузку на ресурсы, предотвращая перегрузку при массовой отправке конкурентных задач. Определив оптимальное для приложения число потоков и установив этот лимит заранее, мы предотвращаем исчерпание ресурсов, особенно при резких всплесках активности. Например, в веб-приложении создание нового потока для каждого входящего запроса во время пикового трафика может привести к сбою.

Помимо контроля, пул потоков обеспечивает мгновенное выполнение задач без ожидания создания нового потока, что сокращает время отклика. Он также управляет жизненным циклом потоков. Ручная обработка потоков сводится к минимуму, и снижается риск ошибок, например утечки потоков. Кроме того, пул потоков позволяет рассматривать задачи как самостоятельные единицы работы, не вникать детально в управление потоками, а сосредоточиваться на бизнес-логике, которую нужно выполнять конкурентно.

Итак, мы узнали, что такое пул потоков и зачем он нужен. Теперь реализуем его базовую версию с нуля, чтобы разобраться во внутренней работе.

Создание простого пула потоков на Java

На примере нашего класса `SimpleThreadPool` рассмотрим основные концепции пула потоков. Разберемся в его реализации:

```
class Worker extends Thread {
  public Worker(ThreadGroup threadGroup, String name) {
    super(threadGroup, name);
  }
  @Override
  public void run() {
    while (running) {
      try {
```

https://max.ru/bookofgeek

```
        Runnable task = queue.take();
        task.run();
      } catch (InterruptedException e) {
        Thread.currentThread().interrupt();
      }
    }
  }
}
```

Здесь `Worker` — это внутренний класс обрамляющего класса `SimpleThreadPool`, поэтому он имеет прямой доступ к очереди `Queue`, которая будет хранить задачи в форме `Runnable`. Каждый экземпляр класса `Worker` представляет собой независимый поток, который непрерывно извлекает задачи из общей очереди и выполняет их. Флаг `running` управляет жизненным циклом рабочего потока. Он является полем нашего пула и позволяет при необходимости корректно остановить его работу. Основная идея заключается в том, что, пока в очереди есть задачи и пул выполняется, поток будет продолжать извлекать задачи из очереди. Мы используем блокирующую очередь, поэтому при отсутствии задач поток будет ожидать их появления.

Все потоки `Worker` сгруппированы в `ThreadGroup` (https://oreil.ly/jHIrv), поэтому ими проще управлять. При необходимости можно прервать сразу всю группу, а не обращаться к каждому потоку отдельно — хотя технически прерывание все равно применяется к каждому потоку внутри группы.

Рассмотрим, как реализован наш класс `SimpleThreadPool`:

```
import java.util.concurrent.BlockingQueue;
import java.util.concurrent.LinkedBlockingDeque;

public class SimpleThreadPool implements AutoCloseable {

  private final BlockingQueue<Runnable> queue;
  private final ThreadGroup threadGroup;
  private volatile boolean running = true; ❶

  public SimpleThreadPool(int poolSize, int queueSize) {
    Worker[] threads = new Worker[poolSize];
    this.queue = new LinkedBlockingDeque<>(queueSize); ❷
    this.threadGroup = new ThreadGroup("SimpleThreadPool");

    for (int i = 0; i < poolSize; i++) {
      threads[i] = new Worker(threadGroup, "Worker-" + i);
      threads[i].start(); ❸
    }
  }

  public void submit(Runnable task) {
    try {
      queue.put(task); ❹
    } catch (InterruptedException e) {
```

https://max.ru/bookofgeek

```
      Thread.currentThread().interrupt();
    }
  }

  public void shutdown() {
    this.running = false;
    threadGroup.interrupt(); ❺
  }

  @Override
  public void close() {
    while (!queue.isEmpty()) { ❻
      try {
        Thread.sleep(100);
      } catch (InterruptedException e) {
        Thread.currentThread().interrupt();
        return; // штатный контролируемый выход
      }
    }
    shutdown();
  }

  class Worker extends Thread {
    public Worker(ThreadGroup threadGroup, String name) {
      super(threadGroup, name);
    }

    @Override
    public void run() {
      while (running) {
        try {
          Runnable task = queue.take();
          task.run();
        } catch (InterruptedException e) {
          // пул не использует прерывания для завершения работы;
          // если даже прерывается, просто выходит
          Thread.currentThread().interrupt();
        }
      }
    }
  }
}
```

Важные проектные решения в реализации.

❶ Наличие ключевого слова `volatile` гарантирует, что сигнал к остановке не будет пропущен, поскольку все потоки немедленно видят изменение флага `running`.

❷ `LinkedBlockingDeque` предоставляет потокобезопасные операции для отправки в пул и для извлечения задач. Ограниченная емкость пула предотвращает исчерпание памяти из-за неконтролируемого накопления задач.

https://max.ru/bookofgeek

❸ Рабочие потоки запускаются сразу после создания и готовы обрабатывать задачи по мере их поступления.

❹ Метод `put()` блокируется при заполненной очереди, препятствуя перегрузке пула.

❺ Сигнал о прерывании группы потоков одновременно поступает всем рабочим потокам, запуская согласованное завершение.

❻ Для плавного завершения ожидаем, пока выполнятся незаконченные задачи, что обеспечивает отсутствие потерь при закрытии пула.

Класс содержит два основных метода: `submit()` и `shutdown()`. Он также реализует интерфейс `AutoCloseable`, что позволяет использовать блок `try-with-resources`, поэтому у нас также есть метод `close()`. Конструктор инициализирует пул с заданным количеством рабочих потоков (`poolSize`) и с очередью определенного размера (`queueSize`). Он создает и запускает рабочие потоки. Для хранения задач мы используем `LinkedBlockingDeque` (https://oreil.ly/Xcv3-). Он предоставляет потокобезопасные операции для добавления и извлечения задач, гарантируя, что рабочие потоки могут конкурентно получать доступ к очереди без проблем.

Метод `submit()` добавляет задачу в очередь на выполнение потоком из пула. Метод `shutdown()` устанавливает значение флага running в false и прерывает все потоки пула, сигнализируя им об остановке. С помощью метода `close()` мы плавно завершаем работу пула потоков. Перед запуском процедуры остановки он некоторое время ожидает опустошения очереди. Таким образом гарантируется завершение всех задач.

Рассмотрим `SimpleThreadPool` в действии. Вот пример того, как его можно использовать:

```
package ca.bazlur.modern.concurrency.c03;

public class SimpleThreadPoolDemo {

  public static void main(String[] args) throws InterruptedException {
    try (var threadPool = new SimpleThreadPool(4, 100)) {
      for (int i = 0; i < 100; i++) {
        int finalI = i;
        threadPool.submit(() -> runTask(finalI));
      }
    }

    Thread.sleep(10_000);
    System.out.println("Main thread finished");
  }
```

https://max.ru/bookofgeek

```
  private static void runTask(int id) {
    System.out.printf("Task %d on %s%n", id,
      Thread.currentThread().getName());
    try {
      Thread.sleep(100); // имитация выполняемой работы
    } catch (InterruptedException e) {
      Thread.currentThread().interrupt();
    }
  }
}
```

В этом примере мы создаем `SimpleThreadPool` с четырьмя рабочими потоками и очередью вместимостью 100 задач. Каждая из сотни отправленных задач выводит свой идентификатор и имя потока, который ее выполняет. Блок `try-with-resources` обеспечивает вызов метода `close()` класса `SimpleThreadPool` при выходе из блока, чтобы корректно завершить работу пула.

После запуска этого кода вы увидите примерно такой вывод:

```
Task 1 is being executed by Worker-2
Task 0 is being executed by Worker-0
Task 2 is being executed by Worker-3
Task 3 is being executed by Worker-1
```

Мы видим, что `SimpleThreadPool` успешно управляет выполнением множества задач, используя ограниченное число потоков. Экспериментируя с разными размерами пула и емкостью очереди, можно оценить их влияние на производительность и использование ресурсов.

Этот пример, конечно, очень упрощен, чтобы облегчить понимание базовой концепции. В реальных приложениях обычно используют интерфейс `ExecutorService`, который предлагает более надежную и полнофункциональную реализацию пула потоков. Например, некоторые пулы поддерживают минимальное и максимальное количество потоков: в штатном режиме активен лишь минимум, но при необходимости их число может увеличиться до максимума.

Фреймворк Executor

Фреймворк Executor создан для увеличения производительности до максимума, и при этом его просто использовать. Он предоставляет несколько реализаций интерфейса `ExecutorService` (https://oreil.ly/JQWme). Для получения экземпляров различных реализаций пула потоков используется фабричный класс Executors (https://oreil.ly/lDFVL), который предлагает методы для создания исполнителей с различными характеристиками.

Например, с помощью следующего кода мы получим пул фиксированного размера в десять потоков:

```
ExecutorService service = Executors.newFixedThreadPool(10);
```

https://max.ru/bookofgeek

«Под капотом» метод `newFixedThreadPool` (https://oreil.ly/dttrO) создает экземпляр `ThreadPoolExecutor` (https://oreil.ly/tRWwo), используя указанное количество потоков с дополнительными параметрами. Большинство методов фабрики `Executors` тоже используют `ThreadPoolExecutor`, конфигурируя его в зависимости от аргументов, чтобы достичь различного поведения.

Чтобы поглубже вникнуть в детали, исследуем конструктор `ThreadPoolExecutor`:

```
public ThreadPoolExecutor(int corePoolSize,
                          int maximumPoolSize,
                          long keepAliveTime,
                          TimeUnit unit,
                          BlockingQueue<Runnable> workQueue,
                          ThreadFactory threadFactory,
                          RejectedExecutionHandler handler) {}
```

Вот параметры, которые он принимает.

`corePoolSize`
: Число потоков, которые сохраняются в пуле даже в состоянии простоя, если не установлен параметр `allowCoreThreadTimeOut` (https://oreil.ly/cnxif).

`maximumPoolSize`
: Максимально допустимое количество потоков в пуле.

`keepAliveTime`
: Максимальное время, в течение которого освободившийся поток может получить новую задачу, иначе будет уничтожен, если количество потоков превышает `corePoolSize`.

`unit`
: Единица времени для аргумента `keepAliveTime`: наносекунды, миллисекунды, секунды и т. д.

`workQueue`
: Очередь, в которой хранятся задачи перед выполнением. Здесь содержатся только задачи в состоянии `Runnable`, отправленные с помощью метода `execute`.

`threadFactory`
: Фабрика, которая используется, когда исполнитель создает новый поток.

`handler`
: Этот обработчик используется при блокировании выполнения из-за достижения границы количества потоков или емкости очереди.

Если знать назначение этих параметров, сразу становится очевидным, как различные конфигурации пулов потоков реализуются через фабричные методы класса `Executors`. Например, `newFixedThreadPool()` устанавливает

https://max.ru/bookofgeek

одинаковые значения для `corePoolSize` и `maximumPoolSize`, создавая пул с фиксированным числом потоков, которые остаются активными до явного завершения.

Фреймворки часто используют этот класс для лучшего управления выполнением задач. Например, Spring оборачивает `ThreadPool` в `ThreadPoolTaskExecutor` (https://oreil.ly/Btpa_), чтобы облегчить обработку асинхронных операций. Правильный выбор размера пула критически важен для достижения оптимальной производительности.

При конфигурировании пула потоков необходимо учитывать два ключевых параметра: его базовый размер (`corePoolSize`) и максимальный размер (`maximumPoolSize`). *Базовый размер* — это минимальное количество потоков, которые всегда остаются в пуле, даже в режиме простоя. Поддерживать их крайне важно, поскольку создание новых потоков — ресурсоемкая операция. Благодаря готовым потокам система может мгновенно назначать задачи без излишних затрат. Они остаются в режиме ожидания, когда задач нет, и немедленно приступают к работе при их поступлении.

Однако избыток простаивающих потоков приводит к нерациональному расходованию ресурсов: CPU и памяти. Поэтому так важно задать оптимальное число потоков. *Максимальный размер* пула определяет, сколько потоков могут быть одновременно активными. Это позволяет динамически масштабировать пул соответственно увеличению нагрузки.

Чтобы повысить производительность, необходимо наилучшим образом определить базовый и максимальный размеры пула. Эти значения могут варьироваться в широких пределах в зависимости от характера рабочей нагрузки и аппаратного обеспечения. К сожалению, универсального решения не существует. Разработчикам часто приходится экспериментировать и анализировать метрики производительности, подбирая оптимальный баланс на основе результатов, полученных опытным путем. Основной принцип: для задач, производительность которых зависит от процессора (например, сложных вычислений), количество потоков должно соответствовать числу доступных ядер CPU. Если потоков больше, производительность падает из-за роста затрат на переключение контекста и синхронизацию. С другой стороны, для операций, связанных с вводом-выводом (например, запросы к БД), целесообразно создавать больше потоков, чем ядер, так как, пока одни ждут завершения операций, другие продолжают работу.

Избежать сложностей управления пулами потоков для задач, связанных с вводом-выводом, позволяют виртуальные потоки, о которых говорится в главе 2. Однако, несмотря на все их достоинства, традиционные пулы потоков не утратят актуальности полностью. Они могут пригодиться для применения в устаревших системах, для обеспечения совместимости или при специфичных требованиях к производительности. Например, недавнее исследование,

https://max.ru/bookofgeek

проведенное специалистами OpenLiberty[1] (https://oreil.ly/NNN5x), показало, что их реализация `ThreadPool` превзошла виртуальные потоки по производительности. Так что крайне важно понимать принципы эффективного конфигурирования и использования классических пулов потоков.

Кратко рассмотрим типы пулов потоков, доступные в JDK в рамках фреймворка Executor.

FixedThreadPool

`FixedThreadPool` (https://oreil.ly/4NKEP) создает пул с фиксированным числом потоков. Если все они заняты, новые задачи будут ожидать в очереди до освобождения потока. Такой подход поддерживает постоянное число активных потоков, что полезно в случае предсказуемой рабочей нагрузки.

Рассмотрим пример:

```
try (ExecutorService fixedPool = Executors.newFixedThreadPool(4)) {
  for (int i = 0; i < 10; i++) {
    fixedPool.submit(() -> {
      System.out.println(Thread.currentThread().getName()
        + " is executing a task");
    });
  }
}
```

Когда использовать: идеально для приложений с постоянным числом конкурентных потоков, например серверных приложений, которые должны поддерживать предсказуемый уровень конкурентности. При ограниченном количестве ресурсов или подключений к базе данных пул с фиксированным числом потоков предотвратит перегрузку системы.

CachedThreadPool

`CachedThreadPool` (https://oreil.ly/MUcqT) динамически по мере необходимости создает новые потоки и повторно использует уже существующие, если они простаивают. Это особенно полезно при изменяющейся нагрузке, так как позволяет пулу гибко адаптироваться к текущим требованиям:

```
try(ExecutorService cachedPool = Executors.newCachedThreadPool()) {
  for (int i = 0; i < 10; i++) {
    cachedPool.submit(() -> {
      System.out.println(Thread.currentThread().getName()
        + " is executing a task");
    });
  }
}
```

[1] OpenLiberty — легковесная облачная среда выполнения Java, оптимизированная для создания микросервисов и облачных приложений, которая поддерживает Jakarta EE, MicroProfile и другие открытые стандарты.

https://max.ru/bookofgeek

Когда использовать: отлично подходит для большого количества кратковременных задач или при непредсказуемых пиках нагрузки. Не годится для задач с длительным временем выполнения, так как может привести к созданию избыточного количества активных потоков.

SingleThreadExecutor

`SingleThreadExecutor` (https://oreil.ly/xdp-9) обеспечивает последовательное выполнение задач в одном потоке. Это эквивалентно наличию единственного исполнителя, который обрабатывает задачи в порядке их поступления:

```
try (var singleThreadPool = Executors.newSingleThreadExecutor()) {
  for (int i = 0; i < 5; i++) {
    singleThreadPool.submit(() -> {
      System.out.println(Thread.currentThread().getName()
        + " is executing a task");
    });
  }
}
```

Когда использовать: если требуется выполнять задачи строго последовательно или же не конкурентно. Часто используется при работе с общим ресурсом, который должен обрабатываться только одной задачей за раз, чтобы исключить состояние гонки.

ScheduledThreadPoolExecutor

`ScheduledThreadPoolExecutor` (https://oreil.ly/Utb7K) планирует выполнение задач с задержкой или через фиксированные интервалы:

```
try (ScheduledExecutorService scheduledPool
              = Executors.newScheduledThreadPool(2)) {
  scheduledPool.scheduleAtFixedRate(() -> {
    System.out.println(Thread.currentThread().getName()
      + " is running a scheduled task");
  }, 0, 5, TimeUnit.SECONDS);
}
```

Когда использовать: для периодических задач, таких как отправка уведомлений, проверка состояния системы или ее обслуживание через заданные интервалы. Этот тип пула потоков обеспечивает автоматическое выполнение задач с требуемой частотой.

WorkStealingPool

`WorkStealingPool` создает пул потоков, который динамически распределяет нагрузку, используя алгоритм кражи потоков. Такой пул оптимизирует использование доступных процессоров:

https://max.ru/bookofgeek

```
try(ExecutorService workStealingPool = Executors.newWorkStealingPool()) {
  for (int i = 0; i < 10; i++) {
    workStealingPool.submit(() -> {
      System.out.println(Thread.currentThread().getName()
        + " is executing a task");
    });
  }
}
```

Когда использовать: подходит для случаев с большим количеством небольших независимых задач и когда требуется максимально эффективно задействовать ядра процессора.

В Java существует особый тип пула потоков — `ForkJoinPool` (https://oreil.ly/rsb95). Он предназначен для задач типа «разделяй и властвуй», когда крупные проблемы можно разбить на мелкие независимые подзадачи для параллельной обработки. На самом деле `ForkJoinPool` внутренне также использует алгоритм кражи потоков. В этой главе мы еще поговорим подробнее о `ForkJoinPool`.

Интерфейсы Callable и Future: обработка результатов задач

До сих пор мы отправляли задачи в пул с помощью `Runnable` (https://oreil.ly/XI_fh), при этом результаты не возвращались. Однако иногда результат выполнения необходим. В этом случае используются `Callable` (https://oreil.ly/TvnqA) и `Future` (https://oreil.ly/ y6fnZ).

Callable

Интерфейс выглядит так:

```
public interface Callable {
  V call() throws Exception;
}
```

Как и `Runnable`, это функциональный интерфейс, но с одним серьезным отличием: он может возвращать результат и бросать исключения.

Рассмотрим пример:

```
import java.util.Map;
import java.util.concurrent.*;

public class CallableExample {
  static final Map<Integer, Long> cache = new ConcurrentHashMap<>(
    Map.of(0, 0L, 1, 1L) ❶
  );
  public static void main(String[] args) throws Exception {
    try (ExecutorService threadPool = Executors.newCachedThreadPool()) {
      Future<Long> fibonacciNumber = threadPool.submit(new Callable<Long>() { ❷
        @Override
```

https://max.ru/bookofgeek

```
        public Long call() throws Exception {
          return fibonacci(50); ❸
        }
      });
    }
  }

  private static Long fibonacci(int n) {
    if (cache.containsKey(n)) {
      return cache.get(n);
    } else {
      long result = fibonacci(n - 1) + fibonacci(n - 2);
      cache.put(n, result); ❹
      return result;
    }
  }
}
```

Ознакомимся с ключевыми компонентами.

❶ Используем `ConcurrentHashMap` для кэширования результатов вычисления чисел Фибоначчи. Это делает нашу рекурсивную реализацию эффективной и потокобезопасной при конкурентном доступе.

❷ Метод `submit` принимает `Callable<Long>` и возвращает `Future<Long>`. Этот объект `Future` представляет будущий результат асинхронного вычисления.

❸ В отличие от `Runnable.run()`, метод `call()` возвращает значение, в данном случае — 50-е число Фибоначчи.

❹ Кэш предотвращает повторные вычисления, что критично для рекурсивных алгоритмов, где одни и те же значения вычисляются многократно.

В данном коде мы отправили задачу на вычисление 50-го числа Фибоначчи. Для этого использовали интерфейс `Callable`. Поскольку этот интерфейс функциональный, мы можем применить лямбда-выражение:

```
Future fibonacciNumber = threadPool.submit(() -> fibonacci(50));
```

Обратите внимание, что результат возвращается обернутым в другой интерфейс — `Future`. Посмотрим, почему так происходит.

Future

У `Future` есть несколько методов:

```
public interface Future<V> {
  boolean cancel(boolean mayInterruptIfRunning);
  boolean isCancelled();
  boolean isDone();
  V get() throws InterruptedException, ExecutionException;
  V get(long timeout, TimeUnit unit)
    throws InterruptedException, ExecutionException, TimeoutException;
}
```

https://max.ru/bookofgeek

Но в большинстве сценариев мы используем методы `get()` и `isDone()`.

Когда вы отправляете задачу с помощью `Callable`, объект `Future` возвращается немедленно, даже если задача еще не завершена. Метод `isDone()` проверяет, завершена ли задача, а метод `get()` возвращает результат.



Метод `get()` является блокирующим. Он блокирует вызывающий поток до тех пор, пока результат не станет доступен.

Рассмотрим пример, где вычисляются несколько чисел Фибоначчи:

```
public static void main(String[] args) {
  List<Future<Long>> futures = new ArrayList<>();
  List<Integer> fibonacciIndices = List.of(10, 20, 30, 40, 50);

  try (ExecutorService threadPool = Executors.newCachedThreadPool()) {
    for (int index : fibonacciIndices) {
      futures.add(threadPool.submit(() -> fibonacci(index)));
  }
    for (Future<Long> future : futures) {
      System.out.println("Fibonacci number: " + future.get());
    }
  } catch (ExecutionException | InterruptedException e) {
    throw new RuntimeException(e);
  }
}
```

В данном примере несколько задач `Callable` отправляются и выполняются параллельно. Но если вызвать `future.get()` внутри цикла, главный поток блокируется до завершения конкретной задачи. Преимущество заключается в том, что в это время другие потоки продолжают свою работу.

ForkJoinPool

Вдобавок к стандартному `ThreadPoolExecutor` в Java 7 появился специальный пул потоков `ForkJoinPool`. Внешне он похож на любой другой пул, но у него особое назначение. Несмотря на то что `ForkJoinPool` является одной из реализаций интерфейса `ExecutorService`, как и `ThreadPoolExecutor`, он кардинально отличается от последнего дизайном и принципом работы. В его основу положен механизм кражи задач, где каждый поток имеет собственную очередь задач. Простаивающие потоки могут «воровать» задачи из хвоста очереди других потоков. Подробнее алгоритм кражи задач мы рассмотрим немного позднее. В традиционном пуле используется общая очередь, откуда задачи распределяются между свободными потоками, при этом могут возникать конфликты и перерасход ресурсов из-за конкуренции потоков за задачи.

https://max.ru/bookofgeek

Традиционный пул потоков обычно поддерживает фиксированное или ограниченное количество потоков, при этом их создание и завершение менее гибкое. В то же время `ForkJoinPool` может адаптивно регулировать количество активных потоков в зависимости от нагрузки и доступности процессоров. Кроме того, он динамически добавляет, приостанавливает или возобновляет потоки для поддержания эффективности.

Для управления доступом к задачам и синхронизации потоков традиционный пул часто полагается на явные механизмы синхронизации, такие как блокировка и условные переменные. `ForkJoinPool`, напротив, благодаря алгоритму кражи задач сводит к минимуму применение явной синхронизации. Это позволяет потокам действовать более автономно, что снижает соперничество за ресурсы и уровень их потребления, обеспечивая высокую производительность.

Традиционные пулы потоков не имеют механизма для распознавания зависимости одних задач от других. Так что часто возникает ситуация, когда поток вынужден ждать завершения дочерней задачи и не может продолжить работу. Это создает проблему. В то же время `ForkJoinPool` специально разработан для рекурсивного разделения задач на более мелкие подзадачи и формирования из них древовидной структуры. Если задача зависит от своих дочерних задач, она может приостановиться, и в это время будут выполняться другие ожидающие завершения задачи. Рассмотрим пример.

Представим, что нам нужно написать программу для вычисления 20-го числа Фибоначчи. Для наглядности сделаем ее многопоточной, предполагая, что вычисление ускорится. Начнем с фиксированного количества потоков, например со 100:

```
import java.util.Map;
import java.util.concurrent.*;

public class FibonacciNumberWithTraditionalThreadPool {
  private static final Map<Integer, Long> cache = new ConcurrentHashMap<>(
     Map.of(0, 0L, 1, 1L)
  );
  private static long getFibonacci(int i, ExecutorService pool) {
    if (cache.containsKey(i)) {
      return cache.get(i);
    }
    Future<Long> future1 = pool.submit(() -> getFibonacci(i - 1, pool)); ❶
    Future<Long> future2 = pool.submit(() -> getFibonacci(i - 2, pool)); ❷
    try {
      long l1 = future1.get(); ❸
      long l2 = future2.get();
      long result = l1 + l2;
```

https://max.ru/bookofgeek

```
      cache.put(i, result);
      return result;
    } catch (InterruptedException | ExecutionException e) {
      throw new RuntimeException(e);
    }
  }

  public static void main(String[] args) {
    try (var pool = Executors.newFixedThreadPool(100)) { ❹
      Future<Long> future = pool.submit(() -> getFibonacci(20, pool));
      Long l = future.get();
      System.out.println("Fibonacci number is: " + l);
    } catch (ExecutionException | InterruptedException e) {
      throw new RuntimeException(e);
    }
  }
}
```

Проанализируем то, что сделали.

❶ Каждая задача подразделяется на две подзадачи для вычисления чисел Фибоначчи. Первая — (n – 1).

❷ Вторая — (n – 2). При этом формируется бинарное дерево задач.

❸ Родительская задача блокируется в ожидании завершения дочерних задач, удерживая поток.

❹ Несмотря на наличие 100 потоков, пул быстро исчерпывается, поскольку каждый из них ожидает выполнения своих подзадач.

Приведенный выше Java-код вычисляет 20-е число Фибоначчи, используя пул потоков и кэширование для повышения эффективности. В нем применяется рекурсивная функция `getFibonacci()`, которая разбивает задачу на более мелкие подзадачи и отправляет каждую в пул потоков. `ConcurrentHashMap` (https://oreil.ly/v5zUs) выступает в роли кэша для хранения ранее полученных результатов, предотвращая избыточные вычисления.

Однако при запуске этого кода мы быстро обнаружим, что он не выдает результат, а создает взаимную блокировку. В данном случае мы ограничены 100 потоками. При вызове метод `getFibonacci()` создает две дочерние задачи и отправляет их в пул. При этом поток, выполняющий родительскую задачу, ожидает завершения дочерних задач. Каждая дочерняя задача порождает новые — и так рекурсивно. В результате пул исчерпывает доступные потоки — все они остаются в состоянии ожидания. Поскольку дочерние задачи находятся в очереди, но не выполняются, а остальные потоки блокируются, и возникает взаимная блокировка. Эту проблему можно было бы решить, предоставив больше потоков, но их количество ограничено.

https://max.ru/bookofgeek

Если сгенерировать дамп потоков с помощью команды:

`jcmd 36427 Thread.dump_to_file -format=json threaddump.json`,

мы увидим результат как на рис. 3.1.

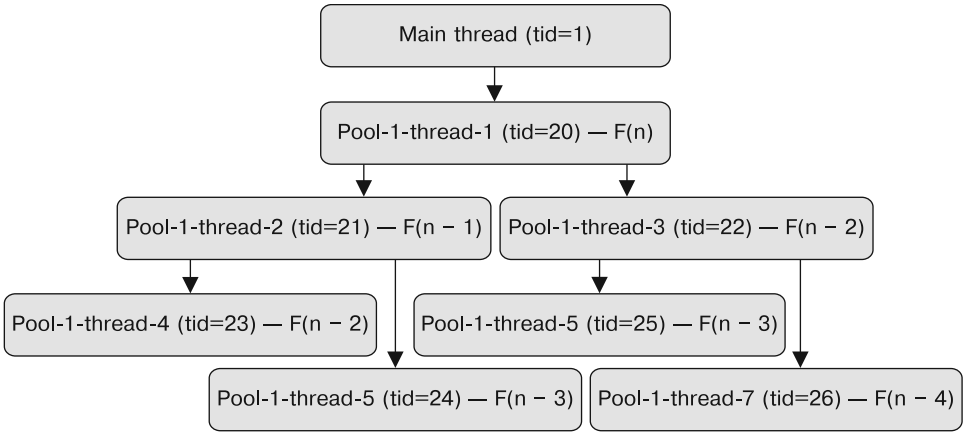


Рис. 3.1. Вычисление чисел Фибоначчи в FixedThreadPoo

Если мы для той же цели используем `ForkJoinPool`, подобной проблемы не возникнет. `ForkJoinPool` специально разработан для декомпозиции задач на подзадачи и умеет приостанавливать текущую задачу, переключаясь на ожидающие, что исключает взаимную блокировку.

Например:

```
import java.util.Map;
import java.util.concurrent.*;

public class FibonacciNumberWithForkJoinPool {
  private static final Map<Integer, Long> cache = new ConcurrentHashMap<>(
    Map.of(0, 0L, 1, 1L)
  );

  static class FibonacciTask extends RecursiveTask<Long> { ❶
    private final int n;
    public FibonacciTask(int n) {
      this.n = n;
    }
    @Override
    protected Long compute() {
      if(cache.containsKey(n)) {
        return cache.get(n);
      }
      FibonacciTask f1 = new FibonacciTask(n - 1);
      f1.fork(); ❷
      FibonacciTask f2 = new FibonacciTask(n - 2);
      long result = f2.compute() + f1.join(); ❸
```

https://max.ru/bookofgeek

```
      cache.put(n, result);
      return result;
    }
  }

  public static void main(String[] args) {
    try (var pool = new ForkJoinPool()) { ❹
      Long result = pool.invoke(new FibonacciTask(20));
      System.out.println("Fibonacci number is: " + result);
    }
  }
}
```

Посмотрим, как `ForkJoinPool` избежал взаимной блокировки.

❶ Класс `RecursiveTask` (https://oreil.ly/4kYNM) предназначен для разделения комплексных задач на подзадачи с помощью fork/join.

❷ Метод `fork()` (https://oreil.ly/Pcvmt) асинхронно отправляет подзадачи в пул, помещая их в двустороннюю очередь текущего потока.

❸ Ключевое отличие: `compute()` (https://oreil.ly/8TBFk) выполняет `f2` непосредственно в текущем потоке (без затрат на новый поток), а `join()` (https://oreil.ly/JkAAe) ожидает завершения подзадачи `f1`. Если `f1` еще не готова, поток может переключиться на другие задачи, а не блокируется.

❹ `ForkJoinPool` использует конфигурацию по умолчанию, оптимизированную под количество доступных процессоров.

Данный код на Java вычисляет 20-е число Фибоначчи с помощью `ForkJoinPool`, который специально разработан для эффективного параллельного выполнения рекурсивных задач. Мы также использовали `ConcurrentHashMap` для хранения ранее рассчитанных чисел Фибоначчи, чтобы избежать избыточных вычислений. В коде используется класс `FibonacciTask`, расширяющий `RecursiveTask`, что позволяет разделять задачи и выполнять их параллельно.

При запуске этого кода мы мгновенно получаем результат.



Выяснив, что `ForkJoinPool` особенный, обсудим, почему он используется для реализации виртуальных потоков.

В Java соперничество за ресурсы возникает, когда нескольким потокам требуется эксклюзивный доступ к общим ресурсам, таким как объекты, переменные или потоки ввода-вывода (I/O streams). Для их защиты применяются механизмы синхронизации, блокировки и т. п. Они обеспечивают корректность программы, но при большом соперничестве за ресурсы производительность приложения может снижаться. Например, если потоки тратят больше времени на ожидание получения блокировок, время выполнения увеличивается. Уменьшение соперничества, как правило, приводит к росту производительности. Фактически `ForkJoinPool` разработан для минимизации конкуренции за ресурсы, поэтому способен обеспечить высокую эффективность.

https://max.ru/bookofgeek

Почему ForkJoinPool используется для виртуальных потоков

Как и обычный пул потоков, `ForkJoinPool` содержит заранее заданное количество рабочих потоков. Если мы не укажем степень параллелизма (по сути, число потоков) в аргументах конструктора, пул запустится с количеством потоков, равным числу доступных процессоров в системе.

`ForkJoinPool` можно использовать как для рабочих нагрузок с интенсивной обработкой данных процессором, так и для операций, связанных с вводом-выводом. Однако важно понимать: если потоков больше, чем доступных ядер процессоров, это не дает преимуществ для задач, зависящих от мощности CPU. В таких случаях каждый поток конкурирует за процессорное время, и поскольку у нас фиксированное количество ядер, избыточные потоки создают лишь дополнительные затраты на переключение контекста. Одновременно может выполняться только количество потоков, равное числу ядер CPU, поэтому лишние просто ожидают, не принося пользы. С другой стороны, задачи, связанные со вводом-выводом, выигрывают от увеличения числа потоков, так как некоторые из них могут блокироваться, пока другие продолжают работу. В предыдущей главе мы обсуждали, почему виртуальные потоки эффективны в таких сценариях. Далее мы рассмотрим связь `ForkJoinPool` с виртуальными потоками, но сначала разберем внутреннее устройство этого пула.

Каждый рабочий поток поддерживает собственную двустороннюю очередь задач, а именно `WorkQueue`. Как правило, рабочий поток добавляет и извлекает задачи с одного конца очереди (по принципу LIFO — «последним пришел, первым ушел»). Другой поток может украсть задачу из чужой очереди с противоположного конца (по принципу FIFO — «первым пришел, первым ушел»). При разбиении задач в рамках алгоритма «разделяй и властвуй» новые подзадачи помещаются в локальную очередь рабочего потока (рис. 3.2).

Внешние потоки (не являющиеся рабочими) отправляют задачи, которые попадают в систему через очереди отправки задач. Они схожи с `WorkQueue`, но управляются самим `ForkJoinPool`. Вместо явного использования механизмов синхронизации, таких как блокировки, для эффективного управления конкурентным доступом применяются атомарные операции типа CAS (Compare-and-Swap).

Рабочие потоки можно метафорически сравнить с «трудолюбивыми пчелами»: они постоянно ищут работу, чтобы все время быть занятыми. Поскольку их количество ограничено, важно обеспечить постоянную загрузку. Когда у потока заканчиваются задачи в собственной очереди, он пытается украсть задачу из очереди другого рабочего потока. В этом и заключается суть механизма «кражи работы»!

На первый взгляд может показаться, что кража задач приведет к соперничеству за ресурсы, когда и рабочий поток, и поток-вор одновременно пытаются получить доступ к одной и той же очереди. Чтобы снизить вероятность такого соперничества, потоки берут задачи с разных концов очереди.

https://max.ru/bookofgeek

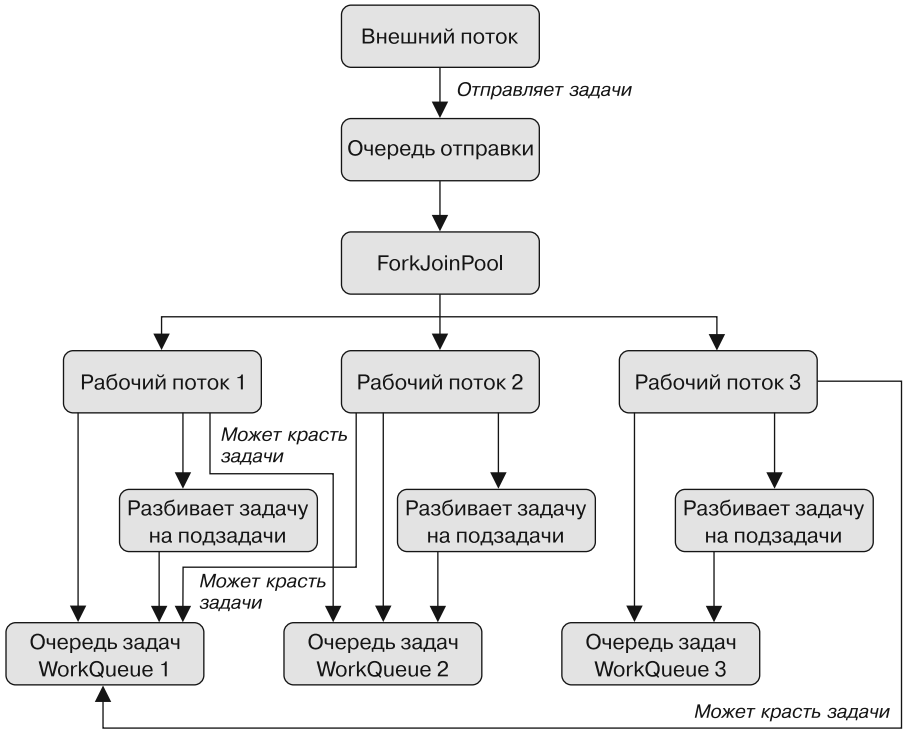


Рис. 3.2. Поток задач в ForkJoinPool: внешние потоки отправляют задачи, которые распределяются между рабочими потоками, управляющими собственными очередями и способными красть задачи друг друга

Как правило, владелец очереди извлекает задачи из головы очереди (LIFO), тогда как потоки-воры — из хвоста (FIFO).

Метод `push` добавляет новые задачи в голову очереди. Когда рабочий поток извлекает задачу, он берет добавленную самой последней. В этом случае очередь работает как стек. Здесь может возникнуть вопрос: не является ли это несправедливым по отношению к более старым задачам, ожидающим в очереди?

Ответ такой: процессоры имеют кэши, которые хранят недавно использованные данные. Высокий приоритет новейших задач увеличивает вероятность того, что необходимые для них данные уже находятся в кэше. Это уменьшает количество промахов кэша (когда CPU приходится загружать данные из основной памяти), тем самым повышается производительность. Но что же происходит со старыми задачами? Их забирают потоки-воры, извлекая из хвоста очереди.

Помимо этих стратегий, `ForkJoinPool` использует для управления очередью умные CAS-операции. Они предназначены для суперэффективной работы даже при наличии множества потоков.

https://max.ru/bookofgeek

РЕАЛИЗАЦИЯ СЧЕТЧИКА БЕЗ БЛОКИРОВКИ С ИСПОЛЬЗОВАНИЕМ CAS (COMPARE-AND-SET)

Техника CAS применяется при разработке конкурентных алгоритмов. Она позволяет безопасно и конкурентно изменять общее значение без использования традиционных механизмов блокировки. Вместо блокирования поток сравнивает текущее значение переменной с ожидаемым, и только если проверка прошла успешно — меняет его на новое.

Рассмотрим пример атомарного счетчика на основе CAS с использованием Java-класса `VarHandle` (https://oreil.ly/3lC84). Счетчик обновляется атомарно посредством CAS-операции, обеспечивая потокобезопасность без явных блокировок:

```
import java.lang.invoke.MethodHandles;
import java.lang.invoke.VarHandle;

public class AtomicCounter {
  private volatile int counter = 0;
  private static final VarHandle COUNTER_HANDLE;
  static {
    try {
      COUNTER_HANDLE = MethodHandles.lookup().findVarHandle(
        AtomicCounter.class, "counter", int.class); ❶
    } catch (ReflectiveOperationException e) {
      throw new Error(e);
    }
  }

  public void increment() {
    int current;
    int next;
    do {
      current = counter; ❷
      next = current + 1;
    } while (!COUNTER_HANDLE.compareAndSet(this, current, next)); ❸
  }

  public int get() {
    return counter;
  }
  public static void main(String[] args) throws InterruptedException {
    AtomicCounter atomicCounter = new AtomicCounter();
    Thread.ofPlatform().start(() -> {
      for (int i = 0; i < 100; i++) {
        atomicCounter.increment();
      }
    });
    Thread.ofPlatform().start(() -> {
      for (int i = 0; i < 100; i++) {
```

https://max.ru/bookofgeek

```
            atomicCounter.increment();
        }
      });
      Thread.sleep(100);
      System.out.println("Final Counter Value: " + atomicCounter.get());❹
    }
}
```

Рассмотрим, как CAS обеспечивает потокобезопасность.

❶ `VarHandle` предоставляет атомарным операциям низкоуровневый доступ к переменным. Это более производительно, чем подходы на основе рефлексии, и дает лучшие гарантии, чем `volatile`.

❷ Мы читаем текущее значение и вычисляем следующее. Это наше «ожидаемое» состояние.

❸ Операция CAS атомарно проверяет, соответствует ли текущее значение счетчика ожидаемому. Если да — обновляет его и возвращает `true`. Если же другой поток модифицировал значение, операция возвращает `false`, после чего попытка повторяется.

❹ Итоговое значение должно быть точно 200, это подтвердит, что ни один инкремент не был потерян, несмотря на конкурентный доступ.

В данном коде мы создали два платформенных потока, каждый из которых 100 раз повышает значение счетчика на единицу. Благодаря операции CAS обновление происходит безопасно без каких-либо проблем. Потоки не блокируют друг друга: каждый из них атомарно пытается обновить общую переменную. Если операция завершается неудачей, поскольку другой поток уже изменил значение, поток повторяет операцию, пока она не увенчается успехом. Такой неблокирующий подход обычно эффективнее при низком уровне конкуренции, поскольку устраняет издержки захвата и освобождения блокировок. Именно так `ForkJoinPool` применяет операции для очередей с кражей задач, при этом потоки больше работают, чем тратят время на ожидание.

Изначально `ForkJoinPool` был создан, чтобы ускорять параллельную обработку, задействовав все доступные ядра процессора. Для этого применяется алгоритм «разделяй и властвуй». Основная идея — большую задачу можно делить на подзадачи до тех пор, пока они не станут достаточно простыми для независимого выполнения. Однако важно понимать, что `ForkJoinPool` лишь управляет пулом потоков, но не занимается разделением задач — эта ответственность лежит на разработчике. Для этого существуют классы `RecursiveTask` (https://oreil.ly/4kYNM), `RecursiveAction` (https://oreil.ly/JNpA4) и др. Разработчик обычно расширяет их, чтобы создать свой класс задач и добавить в него логику. Когда задача разделяется на несколько дочерних задач, родительская ожидает их

https://max.ru/bookofgeek

выполнения, после чего объединяет (сливает) все результаты в один. Пока родительская задача ожидает завершения дочерних, рабочий поток может либо обрабатывать другие задачи из своей очереди, либо украсть их у других рабочих потоков, если собственная очередь пуста. Это принципиальное отличие от других пулов потоков.

Однако со временем область применения `ForkJoinPool` расширилась на множество других сценариев, особенно для событийно-ориентированных задач, которые обычно выполняются независимо и не требуют объединения результатов (то есть ожидания результата выполнения задачи). В таких случаях можно создать `ForkJoinPool`, указав соответствующий аргумент конструктора для асинхронного режима. В этом режиме `ForkJoinPool` переключается на обработку задач по принципу FIFO, который лучше подходит для нерекурсивных сценариев «разделяй и властвуй». Задачи будут обрабатываться в порядке их поступления, что критически важно для обработки событий и внешних триггеров.

Рассмотрим следующий пример:

```
import java.util.concurrent.ForkJoinPool;

public class AsyncModeExample {

  public static void main(String[] args) {
    try (ForkJoinPool forkJoinPool = new ForkJoinPool(4,
      ForkJoinPool.defaultForkJoinWorkerThreadFactory, null, true)) {
      for (int i = 0; i < 10; i++) {
        forkJoinPool.submit(new EventTask("Event " + i));
      }
    }
  }

  record EventTask(String eventName) implements Runnable {
    public void run() {
      System.out.println("Processing " + eventName
        + " in thread: " + Thread.currentThread().getName());
      try {
        Thread.sleep(1000);
      } catch (InterruptedException e) {
        Thread.currentThread().interrupt();
      }
      System.out.println("Completed " + eventName
        + " in thread: " + Thread.currentThread().getName());
    }
  }
}
```

Здесь мы сначала создали экземпляр `ForkJoinPool`, передав в конструктор `true`, чем подтвердили асинхронный режим. Того же эффекта можно достичь с помощью `Executors.newWorkStealingPool()`.

https://max.ru/bookofgeek

Но `ForkJoinPool` значительно эффективнее и производительнее, поэтому и был выбран для планирования виртуальных потоков. При работе с ними используется асинхронный режим. Потоки в `ForkJoinPool` выполняют все виртуальные потоки и выступают в роли планировщика.

Континуация

Континуация, или *продолжение (continuation)*, — это абстрактное понятие, которое имеет различные значения в математике, информатике и разговорном языке. В программировании континуация означает способность программы сохранять текущее состояние выполнения и возобновлять работу с того места, где она остановилась.

Это аналогично тому, как поток при выполнении сохраняет снимок своего состояния во время переключения контекста, чтобы позже возобновить работу. Однако в случае континуаций это происходит на более детальном уровне — вплоть до отдельных методов.

Например, представьте себе метод, содержащий несколько строк кода. Обычно при вызове он выполняется последовательно от начала до конца. Если метод завершается досрочно и затем вызывается снова, то выполняется с самого начала. Но с использованием континуации метод может «сделать паузу» в определенной точке и при повторном вызове продолжить работу с места остановки, сохраняя при этом состояние выполнения.

Континуации широко применяются в программировании. Они позволяют реализовать обработку исключений, неблокирующие операции ввода-вывода, управление конкурентностью, генераторы и многое другое[1].

Рассмотрим, как работают континуации. Начиная с Java 21 континуации включены во внутренний пакет jdk.internal.vm. Хотя в продакшен-коде не рекомендуется напрямую использовать эти внутренние API, полезно понимать, как они работают. Исследуем пример реализации, чтобы поэкспериментировать и больше узнать о континуациях. Рассмотрим следующий код:

```
import jdk.internal.vm.Continuation;
import jdk.internal.vm.ContinuationScope;
// --add-exports java.base/jdk.internal.vm=ALL-UNNAMED

public class ContinuationExample {
  public static void main(String[] args) {
```

[1] Генератор (generator) — это функция, которая может возвращать несколько значений за какое-то время, приостанавливая выполнение между возвратами. При повторном вызове генератор возобновляет выполнение с точки, на которой остановился, подобно тому как работают континуации.

https://max.ru/bookofgeek

```
    ContinuationScope scope = new ContinuationScope("main");
    Continuation continuation = new Continuation(scope, () -> {
      System.out.println("Hello from continuation");
      Continuation.yield(scope);
      System.out.println("Hello again from continuation");
      Continuation.yield(scope);
      System.out.println("Done from continuation");
    });
    System.out.println("Before starting continuation");
    continuation.run();
    System.out.println("After starting continuation");
    continuation.run();
    System.out.println("After starting continuation again");
    continuation.run();
  }
}
```

Здесь мы использовали классы `Continuation` (https://oreil.ly/vfTNY) и `ContinuationScope` (https://oreil.ly/CRbK7) из пакета jdk.internal.vm. Они не являются частью публичного Java API, для доступа к ним требуется добавить специальные аргументы JVM (`--add-exports java.base/jdk.internal.vm=ALL-UNNAMED`).



Континуации — часть внутреннего API, не используйте их в продакшене; приведенный здесь код предназначен только для демонстрации. Внутренние API могут быть изменены или удалены без предупреждения, что повлечет за собой нежелательные последствия.

`ContinuationScope` используется при создании уникальной области для континуации, а `Continuation`, по сути, представляет собой фрагмент кода, выполнение которого можно приостановить и возобновить.

В результате выполнения кода мы получим следующий вывод:

```
Before starting continuation
Hello from continuation
After starting continuation
Hello again from continuation
After starting continuation again
Done from continuation
```

Первая строка в методе `main` выводит `Before starting continuation`. При вызове `continuation.run()` начинается выполнение континуации. Оно печатает `Hello from continuation`, затем при вызове `Continuation.yield(scope)` приостанавливается. В этот момент континуация делает паузу и управление возвращается в `main`, который выводит `After starting continuation`. При повторном вызове `continuation.run()` выполнение возобновляется с точки приостановки: выводится `Hello again from continuation`. Обеспечивается готовность уступить выполнение еще раз. Управление снова возвращается в `main`, который печатает `After starting continuation again`. Наконец, последний вызов `continuation.run()`

https://max.ru/bookofgeek

возобновляет работу континуации и выводит `Done from continuation`. Поскольку больше нет приостановки выполнения, континуация завершается.

По сути, так работают континуации внутри виртуальных потоков. Когда мы начинаем выполнение кода в виртуальном потоке и запускается операция ввода-вывода, континуация вызывает метод `Continuation.yield(scope)`. Он приостанавливает виртуальный поток и снимает его с выполнения потоконосителем. Позже, когда операция ввода-вывода завершится, виртуальный поток будет запущен снова для завершения выполнения.

Возникает закономерный вопрос: как именно `Continuation` делает это?

Любой код выполняется в платформенном потоке, у которого есть стек и набор стековых фреймов. По мере вызова методов стек растет и его указатель перемещается вниз. Рассмотрим рис. 3.3.

У нас есть работающий платформенный поток, который в определенный момент начинает выполнение `Continuation c`, вызвав `c.run()`. Метод `run()` содержит метод `a`, который вызывает `b`, а `b` вызывает `c`, и на `c` мы получаем `c.yield()`. Это означает, что в данной точке `Continuation` приостанавливает выполнение и возвращает управление. `Continuation c` будет выполнен другим платформенным потоком позже, но для текущего потока он должен вернуться. Поток завершит свою текущую работу после возврата `c.run()`. Но на самом деле `c.run()` не завершает свою работу — лишь приостанавливается. Вместо этого он принимает все стековые фреймы от `a` до `c.yield()` и помещает их в объект `Continuation` (рис. 3.4).

При следующем монтировании `Continuation c` идет обратный процесс. Все фреймы потока копируются обратно в стек потока, `c.yield()` возвращается, а выполнение продолжается с точки приостановки. Стек теперь выглядит так, как показано на рис. 3.5.

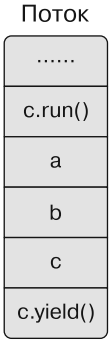


Рис. 3.3. Стек и стековый фрейм Continuation.run()

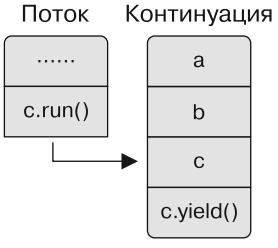


Рис. 3.4. Перенос стека из выполняющегося потока после c.yield

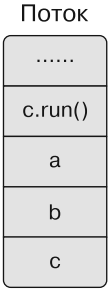


Рис. 3.5. Обратное копирование стековых фреймов при возобновлении континуации

https://max.ru/bookofgeek

На высоком уровне процесс выглядит так: при демонтировании виртуального потока его стековые фреймы копируются в объект континуации, а при повторном монтировании на поток-носитель — возвращаются обратно в стек. Однако это ресурсоемкая операция, особенно при глубокой цепочке вызовов. При каждой приостановке континуации выполняется полное копирование всего стека. Чтобы оптимизировать этот процесс, в JVM реализован механизм ленивого копирования (lazy copying).

Ленивое копирование позволяет избежать лишнего переноса данных. При первой приостановке континуации происходит полное копирование всех стековых фреймов в объект `Continuation`. При последующем возобновлении работы данного объекта `Continuation` вместо полного копирования всего стекового фрейма обратно в поток переносятся только один-два фрейма (рис. 3.6).

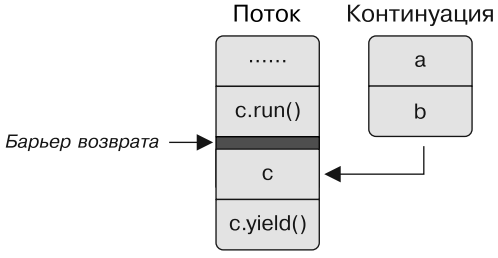


Рис. 3.6. Обратное копирование стека — ленивое: копируются не все фреймы

Здесь `c.yield()` возвращается к `c`, а `c` должен вернуться к `b`; однако `b` отсутствует в стеке. Вместо этого используется механизм под названием *барьеры возврата (return barriers)*. Это небольшие фрагменты кода, которые внедряются в точки возврата функций. При возврате барьер проверяет, нужно ли сохранить текущий стековый фрейм в объекте континуации. Итак, `c` возвращает не в `b`, а в некий код виртуальной машины, который сохранит все фреймы. Теперь он копирует `b` и может продолжить вызов другого метода, например `d`. Затем, если снова появится `c.yield()`, только `b` и `d` будут скопированы в объект `Continuation`.



Такого объяснения, на мой взгляд, достаточно для понимания работы континуаций в Java. Если же вы хотите узнать больше, рекомендую изучить исходный код Project Loom (https://oreil.ly/556J8). Кроме того, в отличном выступлении (https://oreil.ly/Dly1t) Рона Пресслера (Ron Pressler) о континуациях рассказано практически все.

Создание виртуальных потоков с нуля

Чтобы проиллюстрировать ключевую концепцию использования континуаций в виртуальных потоках, создадим упрощенную версию нашей потокоподобной абстракции. Назовем ее `NanoThread`. Используя Java `Continuation` API, мы

https://max.ru/bookofgeek

смоделируем поведение виртуальных потоков. В этом разделе мы поэтапно разберем код: начнем с `NanoThreadScheduler`, затем изучим класс `NanoThread` и в завершение смоделируем операции передачи файлов.

Класс NanoThread

Класс `NanoThread` — наша собственная реализация виртуального потока, построенная с использованием `Continuation` API. `NanoThread` создается путем передачи в `Runnable`, который затем инкапсулируется внутри `Continuation`. Рассмотрим следующий код:

```
import jdk.internal.vm.Continuation;
import jdk.internal.vm.ContinuationScope;
import java.util.concurrent.atomic.AtomicInteger;

public class NanoThread {
  public static final NanoThreadScheduler NANO_THREAD_SCHEDULER
                      = new NanoThreadScheduler();
  private static final AtomicInteger COUNTER
                      = new AtomicInteger(1);
  public static final ContinuationScope SCOPE
                      = new ContinuationScope("nanoThreadScope");
  private final Continuation continuation;
  private final int nid;

  private NanoThread(Runnable runnable) {
    this.nid = COUNTER.getAndIncrement();
    this.continuation = new Continuation(SCOPE, runnable);
  }

  public static void start(Runnable runnable) {
    var nanoThread = new NanoThread(runnable);
    NANO_THREAD_SCHEDULER.schedule(nanoThread);
  }

  public void run() {
    continuation.run();
  }

  public static NanoThread currentVThread() {
    return NanoThreadScheduler.CURRENT_NANO_THREAD.get();
  }

  @Override
  public String toString() {
    return "NanoThread-" + nid + "-" + Thread.currentThread().getName();
  }
}
```

В этом классе мы реализуем `NanoThread`, которому с помощью `COUNTER` присваивается уникальный идентификатор и который управляет `Continuation`.

https://max.ru/bookofgeek

Каждый экземпляр `NanoThread` инициализируется благодаря `Runnable`, инкапсулирующему исполняемую логику.

Основное делает метод `run()` — он просто вызывает `Continuation.run()`, позволяя потоку выполнить код внутри `Runnable`. Если в какой-то момент `NanoThread` готовится приостановить выполнение из-за `Continuation.yield()`, он делает паузу и позже возобновляет работу с точки останова, моделируя поведение виртуального потока.

Метод `start()` запускает `NanoThread` и передает его планировщику для выполнения.

Планировщик NanoThread

Класс `NanoThreadScheduler` играет главную роль в управлении и выполнении экземпляров `NanoThread`. Он использует Fork/JoinPool для планирования задач и `ScheduledExecutorService` (https://oreil.ly/ERUCr) для имитации операций, связанных с вводом-выводом, которые чаще всего приводят к уведомлению планировщика о готовности потока уступить выполнение и его приостановке.

Теперь рассмотрим планировщик:

```
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
import java.util.concurrent.ScheduledExecutorService;

public class NanoThreadScheduler {
   public static final ThreadLocal<NanoThread> CURRENT_NANO_THREAD
                                          = new ThreadLocal<>();

   public static final ScheduledExecutorService IO_EVENT_SCHEDULER =
      Executors.newSingleThreadScheduledExecutor();
   private final ExecutorService workStealingPool =
      Executors.newWorkStealingPool(2);
   public void schedule(NanoThread nanoThread) {
      workStealingPool.submit(() -> {
         CURRENT_NANO_THREAD.set(nanoThread);
         nanoThread.run();
         CURRENT_NANO_THREAD.remove();
      });
   }
}
```

Класс `NanoThreadScheduler` использует `CURRENT_NANO_THREAD` — локальную переменную потока, — чтобы отслеживать, какой именно `NanoThread` в текущий момент выполняется в рабочем потоке. Мы также определили `IO_EVENT_SCHEDULER`, чтобы моделировать операции ввода-вывода с задержками, — это важно для понимания поведения виртуальных потоков.

https://max.ru/bookofgeek

Метод `schedule()` отправляет экземпляры `NanoThread` в ForkJoinPool, где происходит их фактическое выполнение. Этот пул работает с двумя рабочими потоками, постоянно проверяя очередь задач. Можно, конечно, добавить больше потоков, но для нашей демонстрации двух вполне достаточно.

Моделирование операций ввода-вывода при передаче файлов

В нашем примере мы моделируем задачу, связанную с вводом-выводом, — передачу файлов. В такой ситуации очень важен планировщик `IO_EVENT_SCHEDULER`, который планирует выполнение задач со случайными задержками, имитируя реальное поведение операций ввода-вывода:

```
import jdk.internal.vm.Continuation;
import java.util.Random;
import java.util.concurrent.TimeUnit;
import static ca.bazlur.mcj.chap3.custom.NanoThread.NANO_THREAD_SCHEDULER;
import static ca.bazlur.mcj.chap3.custom.NanoThread.SCOPE;
import static ca.bazlur.mcj.chap3.custom.NanoThreadScheduler.CURRENT_NANO_THREAD;
import static ca.bazlur.mcj.chap3.custom.NanoThreadScheduler.IO_EVENT_SCHEDULER;

public class FileOperation {
  private final Random random = new Random();

  public void transfer(String filePath) {
    System.out.println("Start transferring file: " + filePath);
    NanoThread nanoThread = NanoThread.currentVThread();
    IO_EVENT_SCHEDULER.schedule(() ->
            NANO_THREAD_SCHEDULER.schedule(nanoThread),
            random.nextInt(1000), TimeUnit.MILLISECONDS);
    CURRENT_NANO_THREAD.remove();
    Continuation.yield(SCOPE);
    System.out.println("Transfer completed for file: " + filePath);
  }
}
```

Класс `FileOperation` реализует метод `transfer()`, который имитирует передачу файла. С помощью `IO_EVENT_SCHEDULER` он планирует возобновление выполнения `NanoThread` после случайной задержки (имитируя время ожидания ввода-вывода).

В этом процессе текущий `NanoThread` оповещает о своей готовности в передаче выполнения и приостанавливается, что позволяет запуститься другим потокам. При выполнении `Continuation.yield(SCOPE)` в методе передачи рабочий поток в `ForkJoinPool` считает задачу завершенной, если метод `run` возвращает управление и переходит к выполнению других `NanoThread`. После истечения задержки `IO_EVENT_SCHEDULER` повторно ставит в очередь `NanoThread`, позволяя ему продолжить выполнение с точки приостановки.

https://max.ru/bookofgeek

Собираем все вместе

В завершение мы создаем класс NanoThreadDemo для моделирования конкурентной передачи нескольких файлов с использованием нашей системы NanoThread:

```
import java.time.Duration;

  public class NanoThreadDemo {

    public static void main(String[] args) throws Exception {
      FileOperation fileOperation = new FileOperation();
      for (int i = 0; i < 4; i++) {
        int finalI = i;
        NanoThread.start(() -> {
          System.out.println("Transfer: "
            + "File_" + finalI + " Running in VThread: "
            + NanoThread.currentVThread());

        fileOperation.transfer("File_" + finalI);

        System.out.println("Transfer: " + "File_" + finalI
          + " Completed in VThread: " + NanoThread.currentVThread());
        });
      }

      // Ждем минуту, чтобы планировщики
      // могли выполнить все наши нанопотоки
      Thread.sleep(Duration.ofMinutes(1));
   }
}
```

В этом демо-классе мы создаем четыре экземпляра NanoThread, каждый из которых имитирует передачу одного файла. Инициируем передачи конкурентно, позволяя потокам приостанавливаться и возобновлять работу на основе случайных задержек.

Запустим наш код с помощью следующей команды:

```
java --add-exports java.base/jdk.internal.vm=ALL-UNNAMED NanoThreadDemo.java
```

Полученный результат:

```
Transfer: File_1 Running in VThread: NanoThread-2-ForkJoinPool-1-worker-2
Transfer: File_0 Running in VThread: NanoThread-1-ForkJoinPool-1-worker-1
Start transferring file: File_1
Start transferring file: File_0
Transfer: File_2 Running in VThread: NanoThread-3-ForkJoinPool-1-worker-2
Start transferring file: File_2
Transfer: File_3 Running in VThread: NanoThread-4-ForkJoinPool-1-worker-2
Start transferring file: File_3
```

https://max.ru/bookofgeek

```
Transfer completed for file: File_3
Transfer: File_3 Completed in VThread: NanoThread-4-ForkJoinPool-1-worker-1
Transfer completed for file: File_0
Transfer: File_0 Completed in VThread: NanoThread-1-ForkJoinPool-1-worker-1
Transfer completed for file: File_1
Transfer: File_1 Completed in VThread: NanoThread-2-ForkJoinPool-1-worker-1
Transfer completed for file: File_2
Transfer: File_2 Completed in VThread: NanoThread-3-ForkJoinPool-1-worker-1
```

В этом выводе можно заметить, что `File_1` начал выполняться в потоке `NanoThread-2-ForkJoinPool-1-worker-2`, но завершился в потоке `NanoThread-2-ForkJoinPool-1-worker-1`. `NanoThread` остался тем же, но базовый рабочий поток изменился. Именно так ведут себя виртуальные потоки: запустившись в одном рабочем потоке, при приостановке из-за операций ввода-вывода виртуальный поток может возобновить выполнение на совершенно другом (рис. 3.7).

Имейте в виду, что данная реализация предназначена исключительно для демонстрации концепции.

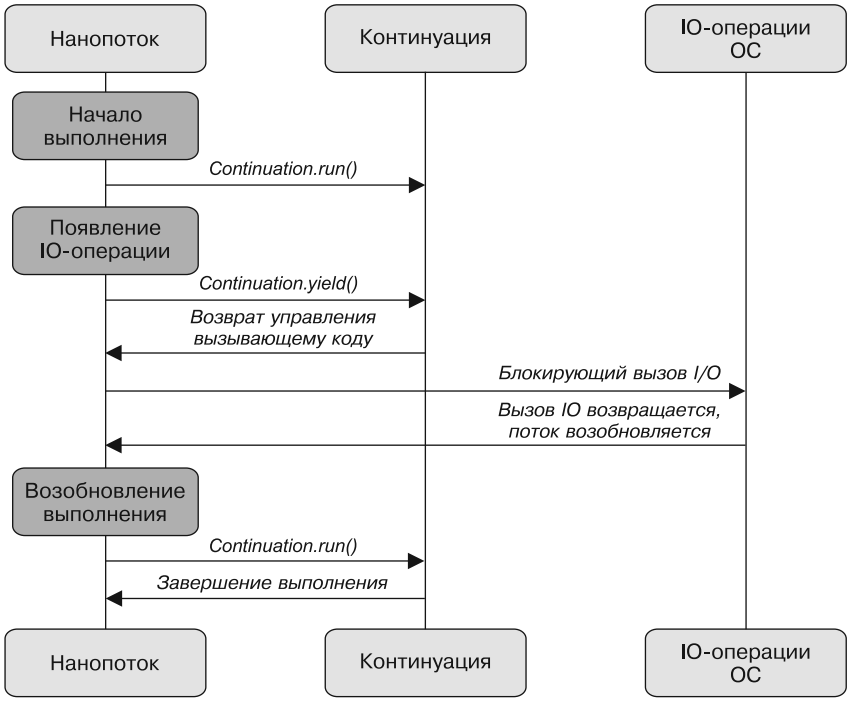


Рис. 3.7. Высокоуровневая архитектура новой реализации NanoThread для иллюстрации концепции

https://max.ru/bookofgeek

Виртуальные потоки и опрос ввода-вывода

Когда виртуальные потоки начинают операцию ввода-вывода, `Continuation` приостанавливается. Как это происходит? Поток вызывает метод `LockSupport.park()` (https:// oreil.ly/rvEjY). Это выглядит следующим образом:

```
public static void park() {
  if (Thread.currentThread().isVirtual()) {
    VirtualThreads.park();
  } else {
    U.park(false, 0L);
  }
}
```

В `VirtualThreads.park()` после нескольких шагов вызывается метод `yieldContinuation()`. Это фактически удаляет виртуальный поток из потоков-носителей — реальных платформенных потоков, выполняющих потоки виртуальные.

Ключевой вопрос: что же вновь запускает виртуальный поток при поступлении данных в сокет?

Для этого используется глобальный для всей JVM механизм, опрашивающий операции чтения (read poller). По сути, это базовый цикл уведомления о событиях, который отслеживает синхронные сетевые операции, например `read()`, `connect()` и `accept()`, когда они вызываются в виртуальном потоке, но не могут быть выполнены немедленно. Когда операция ввода-вывода готова к выполнению (например, при поступлении данных в сокет), опрашивающий механизм получает уведомление и вновь активирует соответствующий приостановленный виртуальный поток. Аналогичный механизм реализован и для операций записи.

На macOS для опроса используется `kqueue` (https://oreil.ly/cB2Fe), на Linux — `epoll` (https://oreil.ly/ksG8z), а на Windows — `wepoll` (https://oreil.ly/JYXmb), реализующий API, подобный epoll, поверх Ancillary Function Driver для Winsock.

Опрашивающий механизм поддерживает карту соответствия между файловыми дескрипторами и виртуальными потоками. Когда файловый дескриптор (например, сокет) регистрируется механизмом опроса, в эту карту добавляется запись, связывающая его с виртуальным потоком, ожидающим завершения операции ввода-вывода. Получив уведомление о событии, цикл находит нужный поток по дескриптору и возобновляет его выполнение, позволяя продолжить работу.

Тем, кто заинтересован в более глубоком знакомстве с этим механизмом, рекомендую изучить его исходный код (https://oreil.ly/MEBWI) в репозитории OpenJDK. Там вы увидите, как JVM обрабатывает операции ввода-вывода для виртуальных потоков с помощью платформенных механизмов вроде `epoll` и `kqueue`, благодаря чему виртуальные потоки не расходуют зря ресурсы во время ожидания ввода-вывода.

https://max.ru/bookofgeek

В заключение

Почти все точки блокировки в библиотеках JDK были переработаны для поддержки виртуальных потоков. Теперь большинство блокирующих операций работают таким образом, что при встрече с блокирующим вызовом виртуальный поток освобождает свой поток-носитель (базовый поток ОС) для выполнения новой работы. Такой подход гарантирует, что система сможет обрабатывать большое количество виртуальных потоков без исчерпания системных ресурсов.

Разобравшись в этих концепциях, вы будете увереннее работать с виртуальными потоками и расширите спектр навыков.

https://max.ru/bookofgeek

ГЛАВА 4

Структурированная конкурентность

Простота — залог надежности.

Эдсгер Дейкстра (Edsger W. Dijkstra)

В предыдущих главах мы рассмотрели, как виртуальные потоки открывают новые возможности для обработки блокирующих задач в Java. Тысячи легковесных виртуальных потоков могут создаваться без значительных затрат ресурсов, что устраняет проблемы с блокирующими вызовами. Однако остается нерешенным один ключевой момент: как управлять взаимосвязями и зависимостями между задачами, когда они делятся на части, а те распределяются между потоками.

Представьте такой сценарий: у родительской задачи несколько зависимых подзадач, которые выполняются в разных потоках. Если родительская задача неожиданно завершается, ее подзадачи продолжают выполняться независимо. Их результаты будут бесполезными, но ресурсы по-прежнему расходуются.

Более того, если родительская задача должна завершаться только после выполнения всех подзадач, то любая ошибка в подзадачах должна в идеале приводить к отмене всей операции. Отсутствие координации может повлечь появление множества «осиротевших» потоков. Они будут расходовать ценную память и время процессора на бесполезную работу. Зачастую это приводит к значительным затратам.

В традиционных моделях конкурентности Java, таких как `ExecutorService` и `Future`, иерархия задач изначально не поддерживается, что затрудняет структурированное распространение отмены операций и обработку ошибок. Поэтому складывается фрагментированный подход к конкурентному программированию, зачастую способствующий появлению ошибок.

Структурированная конкурентность (structured concurrency) — парадигма, которая решает перечисленные проблемы, поскольку рассматривает группы связанных задач как единую рабочую единицу. Она гарантирует, что жизненные циклы подзадач привязаны к родительской задаче, обеспечивая прозрачную

https://max.ru/bookofgeek

и управляемую структуру для конкурентного выполнения. В результате упрощаются обработка ошибок и отмена операций, а также повышается надежность и наблюдаемость конкурентного кода.

В этой главе мы рассмотрим структурированную конкурентность в Java, изучим ее преимущества и узнаем, как она устраняет ограничения традиционных моделей. Мы познакомимся со `StructuredTaskScope` API (https://oreil.ly/eOySX), обсудим, как он помогает управлять иерархиями задач, и приведем практические примеры его использования. К концу главы у вас будет четкое понимание того, как реализовать структурированную конкурентность в Java-приложениях и писать более надежный и поддерживаемый код.

Проблемы неструктурированной конкурентности

В Java конкурентность традиционно реализуется с помощью таких абстракций, как `ExecutorService` и `Future`. Они позволяют выполнять задачи конкурентно, но в то же время создают определенные сложности, особенно при координировании задач.

Рассмотрим типичный сценарий: веб-приложению необходимо получить подробные данные о продукте и связанные с ним отзывы для отображения. Выполним эти операции параллельно с помощью `ExecutorService`, чтобы сократить время отклика.

Для наглядности напишем код на Java для этого сценария поэтапно. Сначала определим структуры данных для хранения информации и отзывов о продукте. Для таких несложных объектов хорошо подходит класс `record`, что достигается благодаря лаконичности и простоте. Создадим также кастомное исключение `ProductServiceException` для более эффективной обработки ошибок, специфичных для нашей продуктово-сервисной логики.

```
record Product(Long id, String name, String description) {}
record Review(Long id, String comment, int rating, Long productId) {}
record ProductInfo(Product product, List<Review> reviews) {}

class ProductServiceException extends RuntimeException {
  public ProductServiceException(String message, Throwable cause) {
    super(message, cause);
  }
  public ProductServiceException(String message) {
    super(message);
  }
}
```

После определения моделей данных и нашего исключения переходим к моделированию реальных операций получения данных. Поскольку в реальных условиях этот процесс часто сопровождается задержками при передаче по сети,

https://max.ru/bookofgeek

мы имитируем их с помощью вспомогательных методов. Класс `ProductService`, который мы вскоре определим, будет включать в себя следующие приватные вспомогательные методы: `sleepForAWhile()` — искусственная задержка, `log()` — для логирования сообщений с идентификатором потока для лучшей читабельности, а также `fetchProduct()`и `fetchReviews()` — для имитации получения данных. Все они выводят сообщения о ходе выполнения, включая имя текущего потока, что помогает визуализировать конкурентность.

Теперь реализуем основной класс `ProductService`. Ключевым методом здесь является `fetchProductInfo()`. Для конкурентного выполнения задач получения продукта и отзывов используется `ExecutorService`. Если конкретнее, то `Executors.newVirtualThreadPerTaskExecutor()`, который подходит для операций ввода-вывода, так как создает легковесные виртуальные потоки для каждой задачи. Затем он использует `Future.get()`для ожидания их завершения и извлечения результатов. Вспомогательные методы инкапсулированы внутри класса как приватные методы экземпляра:

```
public class ProductService {
  public ProductInfo fetchProductInfo(Long productId) {
    log("Fetching product & reviews for id: " + productId);

    try (var ExecutorService
         = Executors.newVirtualThreadPerTaskExecutor()) { ❶

        Future<Product> productTask =
           ExecutorService.submit(() -> fetchProduct(productId)); ❷
        Future<List<Review>> reviewsTask =
           ExecutorService.submit(() -> fetchReviews(productId)); ❸
    Product product = productTask.get(); ❹
        log("Product retrieved for id: " + productId);
        List<Review> reviews = reviewsTask.get(); ❺
        log("Reviews retrieved for id: " + productId);
        log("All info fetched for id: " + productId);
        return new ProductInfo(product, reviews); ❻

    } catch (ExecutionException | InterruptedException e) { ❼
        Throwable cause = e.getCause() != null ? e.getCause() : e;
        log("Error processing product info for id: " +
           productId + ": " + cause.getMessage());

        if (e instanceof InterruptedException) {
           Thread.currentThread().interrupt();
        }

        throw new ProductServiceException(
           "Fetch failed for id: " + productId, cause);
    }
  }
}
```

https://max.ru/bookofgeek

Разберемся, что мы сделали.

❶ Создали исполнитель (executor) виртуальных потоков, который запускает новый виртуальный поток для каждой отправленной задачи.

❷ Отправили операцию получения продукта на конкурентное выполнение.

❸ Отправили операцию получения отзывов на конкурентное выполнение.

❹ Заблокировались, пока не завершится задача получения продукта, и извлекли результат.

❺ Заблокировались до завершения задачи получения отзывов и извлекли результат.

❻ Объединили оба результата в финальный объект `ProductInfo`.

❼ Перехватили исключения, возникающие во время выполнения и прерывания программы, после чего реализовали обработку соответствующим образом.

Для завершения нашего примера конкурентного сервиса продуктов потребуются вспомогательные методы, имитирующие реальные операции:

```
private Product fetchProduct(Long productId) {
  log("Fetching product id: " + productId);
  sleepForAWhile(Duration.ofSeconds(1)); // Simulate network call
  return new Product(productId, "Sample Product",
    "A great product description.");
}
private List<Review> fetchReviews(Long productId) {
  log("Fetching reviews for id: " + productId);
  sleepForAWhile(Duration.ofSeconds(2)); // Simulate network call
  return List.of(
    new Review(1L, "Excellent!", 5, productId),
    new Review(2L, "Good value.", 4, productId)
  );
}

private void sleepForAWhile(Duration duration) {
  try {
    Thread.sleep(duration);
  } catch (InterruptedException e) {
    Thread.currentThread().interrupt();
    throw new RuntimeException("Thread interrupted during sleep", e);
  }
}

private static void log(String message) {
  Thread currentThread = Thread.currentThread();
  String threadName = currentThread.isVirtual()
    ? "VThread[#" + currentThread.threadId() + "]"
    : currentThread.getName(); ❶
```

https://max.ru/bookofgeek

```
  String currentTime = LocalTime.now()
    .format(DateTimeFormatter.ofPattern("HH:mm:ss.SSS"));
  System.out.printf("%s %-15s: %s%n", currentTime, threadName, message);
}

void main() { ❷
  ProductService productService = new ProductService();
  long testProductId = 1L;
  log("Attempting to fetch product info for ID: " + testProductId);
  try {
    ProductInfo productInfo = productService.fetchProductInfo(testProductId);
    log("Successfully retrieved: " + productInfo);
  } catch (ProductServiceException e) {
    log("Service Error: " + e.getMessage() +
      (e.getCause() != null ? " | Caused by: " +
      e.getCause().getMessage() : ""));
  }
}
```

Давайте разберем ключевые детали реализации в этом коде.

❶ Кастомный формат логирования помогает отслеживать, какие потоки выполняют конкурентные операции. Виртуальные потоки обычно не имеют имен, но их метод `toString()` выдает подробные названия вида `VirtualThread[#34]/runnable@ForkJoinPool-1-worker-1`. Мы сокращаем это до `VThread[#34]` для ясности, сохраняя возможность отличать виртуальные потоки от платформенных.

❷ Запуск полного примера показывает как поток конкурентного выполнения, так и обработку ошибок, выводя результат для последующего анализа.

Подготовив все компоненты, запустим код, чтобы наблюдать за конкурентным поведением. Выполните следующую команду на своем терминале, чтобы увидеть, как виртуальные потоки обрабатывают параллельные операции:

```
java ProductService.java
```

Вывод выглядит так:

```
00:50:27.625 main        : Attempting to fetch product info for ID: 1
00:50:27.627 main        : Fetching product & reviews for id: 1
00:50:27.631 VThread[#34]: Fetching product id: 1
00:50:27.631 VThread[#36]: Fetching reviews for id: 1
00:50:28.638 main        : Product retrieved for id: 1
00:50:29.652 VThread[#36]: Fetched reviews for id: 1
00:50:29.653 main        : Reviews retrieved for id: 1
00:50:29.653 main        : All info fetched for id: 1
00:50:29.660 main        : Successfully retrieved Product Info:
ProductInfo[product=Product[id=1, name=Sample Product, description=A great
product description.], reviews=[Review[id=1, comment=Excellent!, rating=5,
productId=1], Review[id=2, comment=Good value., rating=4, productId=1]]]
```

На первый взгляд такой подход кажется вполне разумным. Мы разделили работу на две конкурентные задачи: получение информации о продукте

https://max.ru/bookofgeek

и получение отзывов о нем. Однако, если присмотреться, вырисовываются несколько проблем. Поскольку оба метода выполняются конкурентно, каждый из них может завершиться как успешно, так и с ошибкой независимо от другого.

Представим, что база данных продуктов временно недоступна. Задача `fetchProduct()` может выбросить исключение, что приведет к остановке главного потока запроса. Однако, несмотря на эту ошибку, задача `fetchReviews()` продолжит выполняться в своем потоке. Это приведет к утечке потока, которая свяжет ресурсы и создаст ненужную нагрузку на сервер (рис. 4.1).

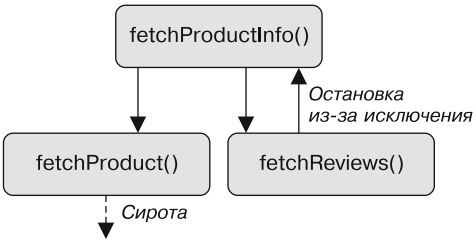


Рис. 4.1. Родительская задача порождает две дочерние задачи. Исключение в одной из них останавливает родительскую задачу, оставляя вторую дочернюю задачу «осиротевшей»

Если пользователь решит отменить запрос, сервер может попытаться остановить поток запроса через прерывание. Тем не менее это не отменит подзадачи: `fetchProduct()` и `fetchReviews()` продолжат выполняться даже после сбоя метода `fetchProductInfo()`. В результате мы получаем утечку двух потоков и бесполезную трату ценных циклов CPU на запрос, который уже неактуален.

Метод `fetchProductInfo()` возвращает результат лишь при успешном завершении обоих дочерних методов. Что же произойдет, если `fetchProduct()` выполняется значительно дольше, чем `fetchReviews()`, и за это время `fetchReviews()` завершается с ошибкой? В таком случае метод `fetchProductInfo()` будет напрасно ожидать завершения `fetchProduct()`, результат которого в итоге окажется невостребованным.

Чтобы проиллюстрировать эти проблемы, модифицируем наши методы для имитации реального сценария сбоя. Мы создадим ситуацию, где одна задача выполняется значительно дольше второй, а вторая быстро завершается с ошибкой:

```
private Product fetchProduct(Long productId) {
  log("Fetching product id: " + productId);
  if(productId == 1L) { log("Product id: " + productId
    + " - simulating long network call (5 seconds).");
  sleepForAWhile(Duration.ofSeconds(5));
  log("Product id: " + productId + " fetch complete.");
  return new Product(productId,
    "Long-Fetched Product", "This product takes time to fetch.");
  }
```

https://max.ru/bookofgeek

```
  log("Product id: " + productId +
    " - simulating standard network call (1 second).");
  sleepForAWhile(Duration.ofSeconds(1));
  log("Product id: " + productId + " fetch complete.");
  return new Product(productId, "Sample Product",
    "A great product description.");
}

private List<Review> fetchReviews(Long productId) {
  log("Fetching reviews for id: " + productId);
  if (productId == 1L) {
    log("Reviews for id: " + productId
      + " - simulating quick failure after 1 second.");
    sleepForAWhile(Duration.ofSeconds(1));
    throw new ProductServiceException("Simulated failure " +
      "fetching reviews for product " + productId);
  }

  log("Reviews for id: " + productId
    + " - simulating network call (2 seconds).");
  sleepForAWhile(Duration.ofSeconds(2));
  List<Review> reviews = List.of(
    new Review(1L, "Excellent!", 5, productId),
    new Review(2L, "Good value.", 4, productId)
  );
  log("Fetched reviews for id: " + productId);
  return reviews;
}
```

В приведенный выше метод мы добавили условие `product Id == 1L`, чтобы создать контролируемый сценарий: получение продукта займет 5 с (имитация медленной базы данных), а запрос отзывов завершится ошибкой за секунду. Это расхождение во времени идеально показывает проблему утечки ресурсов.

При повторном запуске кода вывод будет следующим:

```
00:58:02.885 main         : Attempting to fetch product info for ID: 1
00:58:02.887 main         : Fetching product & reviews for id: 1
00:58:02.890 VThread[#36] : Fetching reviews for id: 1
00:58:02.890 VThread[#34] : Fetching product id: 1
00:58:02.890 VThread[#36] : Reviews for id: 1 - simulating quick failure
                            after 1 second.
00:58:02.891 VThread[#34] : Product id: 1 - simulating long network call
                            (5 seconds).
00:58:07.899 VThread[#34] : Product id: 1 fetch complete.
00:58:07.900 main         : Product retrieved for id: 1
00:58:07.905 main         : Error processing product info for id:
1: Simulated failure fetching reviews for product 1
00:58:07.906 main : Service Error: Fetch failed for id:
1 | Caused by: Simulated failure fetching reviews for product 1
```

https://max.ru/bookofgeek

Вывод демонстрирует именно ту проблему, которую мы описали. Несмотря на исключение, выброшенное методом `fetchReviews()`, метод `productFetch()` продолжает выполнение, тратя полные 5 с на получение продукта. Ожидание оказалось ненужным, так как результат уже не будет использоваться.

В этом примере раскрыта ключевая проблема неструктурированной конкурентности: она рассматривает задачи как независимые сущности, игнорируя их взаимосвязи и зависимости. В результате появляется благодатная почва для ошибок, утечек ресурсов и узких мест производительности. В дальнейшем это усложняет диагностику. Инструменты наблюдаемости (например, дампы потоков) отображают вызовы методов как стеки несвязанных потоков — без всякого намека на отношения «задача — подзадача».

Проведем аналогию с оператором `goto` (https://oreil.ly/i40L7) из старых языков программирования. Его использование приводило к переходу в произвольные точки в коде, затрудняя отслеживание логики программы. Аналогично в неструктурированной конкурентности задачи и потоки выполняются в непредсказуемом порядке, в связи с чем сложно сказать, каково состояние программы на данный момент.

Структурированная конкурентность предлагает смену парадигмы, решающую эти проблемы. Она делает конкурентное программирование интуитивно более понятным и предсказуемым.

Потенциал структурированной конкурентности

Структурированная конкурентность знаменует собой новый подход к конкурентности в Java. В отличие от традиционных методов, где конкретные задачи считаются независимыми сущностями, она организует их в соответствии с логической структурой программ.

Основной принцип прост: если задача порождает несколько конкурентных подзадач, все они должны вернуться к той же точке кода родительской задачи. Это похоже на работу менеджера, который распределяет задачи между членами команды. Проект считается завершенным только после отчета каждого из участников о завершении своей задачи.

Родительская задача выступает в роли руководителя: она отслеживает выполнение подзадач, ожидает их завершения или вмешивается при возникновении проблем. Это ограничение кажется простым, но оно значительно повышает надежность и возможности поддержки конкурентного кода. Неважно, получен успешный результат или возникла ошибка, — каждая подзадача должна вернуть управление родителю перед завершением операции.

https://max.ru/bookofgeek

Теперь рассмотрим некоторые преимущества структурированной конкурентности.

Во-первых, она предоставляет унифицированный механизм для обработки ошибок и отмены группы задач-потомков одного родителя. Например, если одна подзадача завершается с ошибкой или отменяется, то ее задачи-сиблинги автоматически прерываются, а ошибка корректно распространяется вверх по иерархии.

Во-вторых, благодаря строгому родительско-дочернему жизненному циклу задач предотвращается утечка потоков. Дочерние задачи не могут существовать дольше, чем отведенная им область исполнения (scope), так как родительская задача гарантирует корректное удаление всех подзадач после завершения. Это обеспечивает более предсказуемое поведение системы.

Кроме того, структурированная конкурентность улучшает наблюдаемость, предоставляя четкое и интуитивно понятное представление зависимостей задач. Такие инструменты, как дампы потоков и специализированные профилировщики, могут легко отслеживать эти взаимосвязи. Это упрощает диагностику проблем и анализ первопричин при возникновении ошибок.

Данная парадигма также способствует более декларативному стилю программирования, позволяя разработчикам сосредоточиться на бизнес-логике задач, а не разбираться в деталях управления потоками. Это приводит к созданию более чистого и поддерживаемого кода, который проще понимать и модифицировать, что в итоге сокращает время и затраты на разработку.

Наконец, поскольку структурированная конкурентность часто работает поверх виртуальных потоков, их сочетание позволяет создавать огромное количество конкурентных задач с минимальными затратами ресурсов. Предоставляется необходимый каркас для оркестрации виртуальных потоков, что гарантирует их безопасное и эффективное выполнение даже в крупных масштабах.

Структурированная конкурентность готова стать фундаментальным компонентом конкурентного программирования в Java. Она избавляет от многих традиционных проблем, а также открывает путь к созданию более надежных и поддерживаемых систем.

Познакомимся со Structured Concurrency API и посмотрим, как он решает перечисленные проблемы.

Основы API

В основе API структурированной конкурентности лежит `StructuredTaskScope` (https://oreil.ly/k9kPw) из пакета java.util.concurrent (https://oreil.ly/2tqNo). В этом разделе мы рассмотрим его основные возможности, принцип работы и способы применения для упрощения и улучшения конкурентного программирования на Java.

https://max.ru/bookofgeek

StructuredTaskScope

Интерфейс `StructuredTaskScope` (https://oreil.ly/k9kPw) является центральным элементом модели структурированной конкурентности в Java. Он позволяет разработчикам управлять группами связанных задач, гарантируя, что их жизненные циклы связаны между собой. Обработка ошибок, отмена задач и агрегация результатов становятся более простыми и надежными.

`StructuredTaskScope` — это sealed-интерфейс с параметрами `T` и `R`, представляющими типы задач, выполняемых в области:

```
public sealed interface StructuredTaskScope<T,R>
extends AutoCloseable
```

Рассмотрим его ключевые методы.

`<U extends T> StructuredTaskScope.Subtask<U> fork(Callable<? extends U> task)`

Создает и запускает новую подзадачу внутри области (scope). Она выполняется конкурентно в отдельном потоке, обычно виртуальном. Метод возвращает объект `Subtask` (https://oreil.ly/eB1YZ), представляющий выполняемую подзадачу.

`<U extends T> StructuredTaskScope.Subtask<U> fork(Runnable task)`

Создает и запускает новую подзадачу внутри области для выполнения предоставленного действия. Оно выполняется конкурентно в отдельном потоке, обычно виртуальном. Поскольку задача — `Runnable`, она не возвращает результат, поэтому `Subtask.get()` после успешного завершения вернет `null`. Метод возвращает объект `Subtask`, представляющий выполняемую подзадачу.

`join()`

Возвращает результат после ожидания завершения всех подзадач или отмены области. Метод блокирует вызывающий поток (который должен быть владельцем области) вплоть до завершения всех подзадач. Если установлен тайм-аут и он истекает, то область отменяется и выбрасывается `TimeoutException`. После завершения ожидания вызывается метод `result()` объекта Joiner, чтобы получить результат или выбросить исключение. Если `result()` выбрасывает исключение, то, как следствие, выбрасывается `FailedException`. Этот метод может быть вызван владельцем области только один раз.

`close()`

Закрывает область, обеспечивая завершение или отмену всех подзадач. Метод обычно вызывается неявно при использовании блока `try-with-resources` (https://oreil.ly/RkRYx).

https://max.ru/bookofgeek

Поскольку `StructuredTaskScope` является интерфейсом, нельзя создать его экземпляр напрямую. Вместо этого используется ключевая фича данного API — статический фабричный метод `open()` со следующей сигнатурой:

```
static <T> StructuredTaskScope<T,Void> open()
```

Этот метод создает новый `StructuredTaskScope`, который можно использовать для создания подзадач (fork subtasks), возвращающих результаты любого типа. Метод `join()` этой области ожидает либо успешного завершения всех подзадач, либо ошибки в любой из них. У метода `open()` есть перегруженные версии, но мы рассмотрим их в следующем разделе. Поскольку `StructuredTaskScope` расширяет `AutoCloseable`, будем использовать его в блоке `try-with-resources`, что гарантирует автоматическое закрытие. Важно отметить, что в таких случаях метод `close()` не следует вызывать вручную.



`StructuredTaskScope` предназначен для строго структурированного подхода. Когда его метод `close()` вызывается до закрытия всех вложенных областей задач (если таковые имеются), происходит следующее.

1. Попытка закрыть базовую конструкцию каждой вложенной области в порядке, обратном их созданию. 2. Закрытие имплементации интерфейса. 3. Выбрасывается исключение `StructureViolationException`, указывающее на нарушение последовательности закрытия. Вместо вызова `close()` вручную рекомендуется использовать `try-with-resource`.

После того как мы познакомились со структурированной конкурентностью и ее API, реализуем метод `fetchProductInfo` с использованием `StructuredTaskScope`. Он будет конкурентно собирать информацию, получая сведения о продукте и отзывы о нем:

```
public ProductInfo fetchProductInfo(Long productId) {
  log("Fetching product & reviews for id: " + productId);
  try (var scope = StructuredTaskScope.open()) { ❶
    StructuredTaskScope.Subtask<Product> productTask =
      scope.fork(() -> fetchProduct(productId)); ❷
    StructuredTaskScope.Subtask<List<Review>> reviewsTask =
      scope.fork(() -> fetchReviews(productId)); ❸
    scope.join(); ❹
    return new ProductInfo(productTask.get(), reviewsTask.get()); ❺
  } catch (InterruptedException e) {
    Thread.currentThread().interrupt();
    throw new ProductServiceException(
      "Fetch failed for id: " + productId); ❻
  }
}
```

Отследим, что происходит при выполнении структурированного подхода.

❶ Метод `fetchProductInfo()` инициализирует `StructuredTaskScope` в блоке `try-with-resources` для автоматической очистки.

https://max.ru/bookofgeek

❷ Создаем первую подзадачу для получения `Product` и запускаем ее выполнение конкурентно.

❸ Создаем вторую подзадачу для получения `List<Review>` и запускаем ее бок о бок с первой.

❹ Вызываем `scope.join()`, который блокирует выполнение до завершения обеих подзадач, возникновения ошибки или отмены области.

❺ Если обе подзадачи завершаются успешно, `join()` завершается нормально и метод создает экземпляр `ProductInfo`, используя результаты `productTask.get()` и `reviewsTask.get()`.

❻ Если любая из подзадач завершается с ошибкой, `join()` прерывает другую выполняющуюся подзадачу, ожидает ее завершения и выбрасывает `StructuredTaskScope.FailedException` с исходным исключением в качестве причины.



Структурированная конкурентность, о которой говорится в этой главе, в настоящее время доступна в JDK 25 как предварительная функция.

Для ее активации необходимо при компиляции и запуске указать флаг `--enable-preview`. Рассмотрим команды для компиляции и выполнения кода.

- *Для javac:* `javac --release 25 --enable-preview Main.java`
- *Для java:* `java --enable-preview Main`
- *С помощью загрузчика исходного кода:* `java --enable-preview Main.java`
- *С помощью JShell:* `jshell --enable-preview`

Ожидается, что эта возможность будет интегрирована в будущие релизы JDK без необходимости как-то модифицировать код.

Чтобы посмотреть, как структурированная конкурентность обрабатывает ошибки, изменим метод `fetchProduct()`, имитируя ситуацию, когда продукт не найден:

```
private Product fetchProduct(Long productId) {
  log("Fetching product id: " + productId);
  if (productId == 1L) {
    throw new ProductServiceException("Product not found"); ❶
  }
  sleepForAWhile(Duration.ofSeconds(1)); // Имитируем сетевой запрос
  return new Product(productId, "Sample Product",
    "A great product description.");
}
```

❶ Немедленно выбрасывает исключение для продукта с ID 1, имитируя сценарий «не найден».

https://max.ru/bookofgeek

При запуске программы с такой модификацией вывод демонстрирует поведение структурированной конкурентности:

```
04:46:34.321 main : Attempting to fetch product info for ID: 1
04:46:34.321 main : Fetching product & reviews for id: 1
04:46:34.325 VThread[#34]: Fetching product id: 1
04:46:34.325 VThread[#36]: Fetching reviews for id: 1
Exception in thread "main"
    java.util.concurrent.StructuredTaskScope$FailedException:
    ca.bazlur.mcj.chap4.ProductServiceException:
    at java.base/java.util.concurrent.StructuredTaskScopeImpl
        .join(StructuredTaskScopeImpl.java:257)
    at ca.bazlur.mcj.chap4.FailingProductServiceWithStructuredConcurrency
        .fetchProductInfo(FailingProductServiceWithStructuredConcurrency)
    at ca.bazlur.mcj.chap4.FailingProductServiceWithStructuredConcurrency
        .main(FailingProductServiceWithStructuredConcurrency)

Caused by: ca.bazlur.mcj.chap4.ProductServiceException:
    Product not found
    at ca.bazlur.mcj.chap4.FailingProductServiceWithStructuredConcurrency
        .fetchProduct(FailingProductServiceWithStructuredConcurrency)
    at ca.bazlur.mcj.chap4.FailingProductServiceWithStructuredConcurrency

.lambda$fetchProductInfo$0(FailingProductServiceWithStructuredConcurrency)
    at java.base/java.util.concurrent.StructuredTaskScopeImpl$SubtaskImpl
        .run(StructuredTaskScopeImpl.java)
    at java.base/java.lang.VirtualThread.run(VirtualThread.java:456)
```

Стек-трейс показывает именно то, что мы ожидали увидеть. Поскольку подзадача `fetchProduct()` выбросила исключение, `StructuredTaskScope` немедленно отменил конкурентно выполняющуюся подзадачу `fetchReviews()`. Затем метод `scope.join()` выбросил `StructuredTaskScope.FailedException`, обернув исходное исключение `ProductServiceException`, выброшенное в `fetchProduct()`, в качестве своей причины, чтобы родительский поток мог корректно обработать ошибку (рис. 4.2).

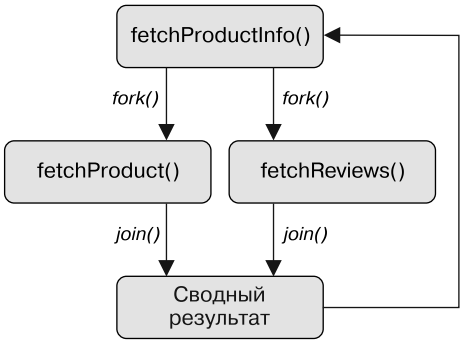


Рис. 4.2. Рабочий процесс StructuredTaskScope, в котором fetchProductInfo() создает подзадачи fetchProduct() и fetchReviews(), объединяет их результаты и отменяет выполнение при ошибке

https://max.ru/bookofgeek

Эта структурированная версия не только устраняет ограничения нашего предыдущего неструктурированного подхода, но и обеспечивает значительные преимущества. Если одна из задач завершается с ошибкой, другая автоматически отменяется, предотвращая напрасные траты ресурсов, которые мы наблюдали ранее. Если родительская задача отменяется, то область закрывается и автоматически завершает обе подзадачи. Метод `join()` гарантирует, что они завершатся до получения результатов, исключая проблемы с неполными данными, которые возникли в нашей исходной реализации.

Помимо технических улучшений, сам структурированный код более лаконичен, понятен и менее подвержен ошибкам. Он четко отражает суть операции: получить информацию о продукте и отзывы, продолжив выполнение лишь при наличии обоих. Поведение при ошибках не допускает двойного толкования.

Применяя структурированную конкурентность, мы можем создавать более эффективный и надежный конкурентный код, который проще поддерживать.

Области и задачи: связи и жизненный цикл

Мы ознакомились с принципами структурированной конкурентности и ее API, теперь рассмотрим взаимосвязь между областями и их подзадачами.

Структурированная конкурентность использует `StructuredTaskScope` как родительский контейнер для управления дочерними подзадачами, контролируя их жизненный цикл и выполнение. Каждая подзадача представляет собой единицу конкурентной работы, выполняемую на виртуальных потоках внутри области. Такое родительско-дочернее взаимодействие гарантирует скоординированное и предсказуемое выполнение.

`StructuredTaskScope` определяет ограниченный контекст для конкурентных операций, управляя всем жизненным циклом подзадач, чтобы обеспечить их упорядоченное выполнение и завершение.

Разветвление задач на подзадачи (Forking subtasks)

Для инициирования конкурентных задач владелец области использует метод `fork()`. API предоставляет две его различные перегруженные версии.

`fork(Callable<? extends U> task)`

: Предназначен для подзадач, которые должны возвращать значение. Он принимает `Callable`, и при выполнении в новом виртуальном потоке будет вызван его метод `call()`.

`fork(Runnable task)`

: Используется для подзадач, которые выполняют действие, но не возвращают результата. Он работает аналогично `Callable`-версии, но если подзадача завершается успешно, соответствующий метод `Subtask.get()` просто вернет `null`.

https://max.ru/bookofgeek

Процесс выполнения fork()

Независимо от того, какая версия метода `fork()` вызывается, процесс идет одинаково.

- Создается объект `Subtask` для представления асинхронной задачи. Этот `Subtask` выступает в роли дескриптора.
- Происходит обращение к объекту `Joiner`, задающему политику, через его метод `onFork()`. Если политика определяет, что подзадача не должна выполняться (например, если область уже отменена), `fork()` возвращает дескриптор, а новый поток не запускается.
- В противном случае запускается новый виртуальный поток для выполнения задачи.

Метод `fork()` немедленно возвращает дескриптор `Subtask`. Владелец области может использовать его только для получения результата от `Subtask.get()`или исключения от `Subtask.exception()` после того, как область будет объединена с помощью метода `join()`.

Завершение подзадачи

Когда подзадача завершает свою работу, выполняющий ее поток уведомляет джоинер (joiner), вызывая его метод `onComplete()` и передавая дескриптор `Subtask`, который теперь содержит конечное состояние (например, `SUCCESS`, `UNAVAILABLE` или `FAILED`). О `Joiner` мы поговорим чуть позднее.

Владелец области использует `join()` для ожидания выполнения политики подзадач. Для области по умолчанию, созданной с помощью `StructuredTaskScope.open()`, это означает либо ожидание успешного завершения всех подзадач, либо возникновение ошибки в одной из них. Если подзадача терпит неудачу, область автоматически отменяет все остальные выполняющиеся подзадачи. Затем метод `join()` выбрасывает исключение `FailedException`, содержащее изначальное исключение из подзадачи, завершившейся с ошибкой.

Наконец, при закрытии области (явном или через блок `try-with-resources`) выполняется корректное завершение: ожидается остановка всех потоков подзадач перед тем, как позволить родительскому потоку продолжить работу. Таким образом предотвращается утечка ресурсов и сохраняется структурированная природа конкурентных операций.

Политики объединения через Joiner

Ключевая особенность `StructuredTaskScope` — возможность устанавливать гибкие политики объединения с помощью объекта `Joiner`. `Joiner` представляет собой мощный механизм управления жизненным циклом подзадач, который определяет условия завершения метода `join()`и задает тип возвращаемого результата.

https://max.ru/bookofgeek

Вместо нескольких фиксированных политик Java предоставляет интерфейс `Joiner`. Его методы можно поделить на методы получения результата и методы, позволяющие выполнять действия на различных этапах жизненного цикла (хуки жизненного цикла):

```
public interface Joiner<T, R> {
  R result(); ❶
  Throwable exception(); ❷
  default boolean onFork(Subtask<? extends T> subtask) { ... } ❸
  default boolean onComplete(Subtask<? extends T> subtask) { ... } ❹
}
```

Рассмотрим, что делают эти методы.

❶ Метод получения результата, который определяет успешный результат для возврата из `scope.join()`.

❷ Метод получения результата, который определяет исключение для выбрасывания из `scope.join()`.

❸ Хук жизненного цикла, вызываемый при каждом вызове `fork()` для отслеживания новых подзадач.

❹ Хук жизненного цикла, вызываемый при завершении любой подзадачи и содержащий основную логику политики.

`result()` и `exception()` — это методы результата, которые определяют контракт для финального результата `scope.join()`. Имплементация должна предоставить логику для одного из этих методов, чтобы либо вернуть успешное значение, либо выбросить исключение.

Метод `onFork()` — это хук жизненного цикла, позволяющий `Joiner` узнавать о новых подзадачах по мере их создания. В методе `onComplete()` обычно содержится основная логика политики, которая проверяет, удовлетворяет ли завершение конкретной подзадачи общему условию (например, «является ли это первым успехом?» или «все ли задачи завершились с ошибкой?»).

Стандартные политики объединения подзадач в одной области

`Joiner` предоставляет статические фабричные методы для создания стандартных политик.

`Joiner.awaitAllSuccessfulOrThrow()`

Политика по умолчанию. Используется при вызове `StructuredTaskScope.open()`. Ее внутренняя логика в `onComplete()` отслеживает количество завершенных подзадач. При сбое любой подзадачи она отменяет область. Успешное завершение `join()` возможно только тогда, когда все подзадачи выполнены без ошибок.

https://max.ru/bookofgeek

`Joiner.anySuccessfulResultOrThrow()`

Реализует политику «гонка за победой» (race to win). Логика `onComplete()` проверяет, успешно ли завершилась хотя бы одна подзадача. Если это так, она немедленно отменяет область, а ее метод `result()` возвращает успешный результат.

`Joiner.allSuccessfulOrThrow()`

В этом варианте хук политики `onComplete()` собирает результаты всех успешно завершившихся подзадач. Он ожидает завершения всех подзадач, после чего предоставляет собранные результаты в `Stream`.

`Joiner.awaitAll()`

Ожидает завершения всех подзадач независимо от их успешности. Она полезна, когда нужно знать результат каждой задачи и обработать его индивидуально. Метод `scope.join()` возвращает поток `Stream<Subtask>`, который можно использовать для проверки состояния, результата или исключения каждой подзадачи.

`Joiner.allUntil(Predicate<Subtask> isDone)`

Наиболее гибкий фабричный метод, позволяющий реализовать собственную логику отмены. Он принимает `Predicate`, который тестируется для каждой завершенной подзадачи. Если `Predicate` возвращает `true`, область отменяется. Метод позволяет реализовать сложные политики (например, «отменить после двух подзадач, потерпевших неудачу»). Если `true` никогда не возвращается, метод ведет себя как `awaitAll()`, ожидая завершения всех подзадач. В любом случае `scope.join()` возвращает поток `Stream<Subtask>` всех созданных подзадач.

Такой подход на основе `Joiner` позволяет гранулярно контролировать критерии завершения области и возвращаемое ей значение: от ожидания успешного выполнения всех подзадач до гонки за первым доступным результатом или сбора нескольких результатов в потоке подзадач.

Рассмотрим каждую из этих политик на практических примерах.

Ожидание успеха всех задач или первого сбоя

При создании `StructuredTaskScope` с помощью метода `open()` по умолчанию применяется политика, требующая успешного выполнения всех подзадач. Этот подход all-or-nothing («все или ничего») внутренне управляется методом `Joiner.awaitAllSuccessfulOrThrow()`. Это означает, что при сбое даже одной подзадачи вся область немедленно завершается. Все выполняющиеся подзадачи отменяются, а ошибка из неудавшейся подзадачи передается в родительскую задачу. Таким образом, результат будет получен только при успешном завершении всех частей операции, что предотвращает использование неполных или частичных результатов.

https://max.ru/bookofgeek

Проиллюстрируем это практическим примером. Сначала создадим записи для данных, аналогичные тем, что мы уже делали в этой главе:

```
record Product(long productId, String name) {}
record Review(String user, int rating) {}
record ProductInfo(Product product, List<Review> reviews) {}
```

Затем нам понадобятся методы, которые будут выполняться как конкурентные подзадачи. Чтобы сделать конкурентность видимой, мы добавим `log()` и используем `Thread.sleep()` для имитации сетевой задержки. Также создадим специальный метод `fetchProductThatFails()` для демонстрации обработки ошибок:

```
// Подзадача успешно выполняется после задержки 1 с
private Product fetchProduct(long productId)
    throws InterruptedException {
  log(" -> Fetching product details... (will take 1s)");
  Thread.sleep(Duration.ofSeconds(1)); ❶
  log(" <- Product details fetched.");
  return new Product(productId, "Sample Product");
}
// Эта подзадача всегда завершается с ошибкой
private Product fetchProductThatFails(long productId) {
  log(" -> Fetching product details... (will fail)");
  throw new ProductServiceException("Product ID " + productId + " not found"); ❷
}
// Подзадача успешно выполняется после задержки 2 с
private List<Review> fetchReviews(long productId)
    throws InterruptedException {
  log(" -> Fetching product reviews... (will take 2s)");
  Thread.sleep(Duration.ofSeconds(2)); ❸
  log(" <- Product reviews fetched.");
  return List.of(new Review("Inaya", 5), new Review("Rushda", 4));
}
```

Каждый метод в нашей демонстрации служит определенной цели.

❶ Имитирует сетевой запрос данных о продукте с задержкой 1 с.

❷ Немедленно выбрасывает исключение для моделирования сценария «продукт не найден».

❸ Имитирует сетевой запрос данных об отзывах с задержкой 2 с.

Мы уже определяли метод `log()` в предыдущем разделе. Теперь преобразуем его в служебный метод для повторного использования в других контекстах:

```
import java.time.LocalTime;
import java.time.format.DateTimeFormatter;

public class Utils {
  public static void log(String message) {
    Thread currentThread = Thread.currentThread();
```

https://max.ru/bookofgeek

```
    String threadName = currentThread.isVirtual()
      ? "VThread[#" + currentThread.threadId() + "]"
      : currentThread.getName();
    String currentTime = LocalTime.now()
      .format(DateTimeFormatter.ofPattern("HH:mm:ss.SSS"));
    System.out.printf("%s %-12s: %s%n", currentTime, threadName, message);
  }
}
```

Теперь определим основную логику, используя политику по умолчанию:

```
public ProductInfo fetchProductInfo(long productId, boolean shouldFail)
    throws InterruptedException {
  Instant start = Instant.now();
  // Использование open() обеспечивает политику fail-fast по умолчанию
  try (var scope = StructuredTaskScope.open()) { ❶
    StructuredTaskScope.Subtask<Product> productTask = shouldFail
      ? scope.fork(() -> fetchProductThatFails(productId)) ❷
      : scope.fork(() -> fetchProduct(productId)); ❸
    StructuredTaskScope.Subtask<List<Review>> reviewsTask
      = scope.fork(() -> fetchReviews(productId)); ❹
    // Ожидает успешного завершения обеих задач
    // или выбрасывает FailedException при первой ошибке
    log("... Scope joining. Waiting for subtasks...");
    scope.join(); ❺
    log("... Scope joined successfully.");
    // Доступно только при успешном join()
    return new ProductInfo(productTask.get(), reviewsTask.get()); ❻
  } catch (StructuredTaskScope.FailedException ex) {
    // Этот блок выполняется только в случае ошибки
    log("... Scope join failed. A subtask threw an exception.");
    throw new RuntimeException("Failed to fetch product info", ex.getCause()); ❼
  } finally {
    Instant end = Instant.now();
    log("Total time taken: " + Duration.between(start, end).toMillis() + "ms");
  }
}
```

Рассмотрим последовательность выполнения.

❶ Создается область с политикой `awaitAllSuccessfulOrThrow()` по умолчанию.

❷ Условно создается подзадача, которая завершится ошибкой (для демонстрации).

❸ Создается подзадача, которая успешно завершится через 1 с.

❹ Создается подзадача, которая успешно завершится через 2 с.

❺ Ожидает успешного завершения всех подзадач или быстрого отказа при первой ошибке.

❻ Получает результаты, только если все подзадачи завершились успешно.

❼ Обрабатывает случай ошибки, оборачивая исходное исключение.

https://max.ru/bookofgeek

Это суть нашего примера. Метод `fetchProductInfo()` использует `StructuredTaskScope` для оркестрации подзадач. Булев флаг `shouldFail` позволяет легко переключаться между успешным и ошибочным путями.

Демонстрационный класс полностью:

```
public class DefaultPolicyDemo {
  // ... (реализации методов из предыдущего кода)
  void main() {
    var demo = new DefaultPolicyDemo();
    log("--- Running Success Scenario ---");
    log("... Expecting to take ~2 seconds (the time of the slowest task)...\n");
    try {
      ProductInfo result = demo.fetchProductInfo(123L, false); ❶
      log("\nSuccess! Result: " + result);
    } catch (Exception e) {
      log("\nCaught unexpected exception in success scenario: " +
        "" + e.getMessage());
    }
  }
}
```

Метод `main` демонстрирует успешный сценарий.

❶ Запускает успешный сценарий, передавая `false` для параметра `shouldFail`.

Запустим код:

```
java --enable-preview DefaultPolicyDemo.java
```



Обратите внимание: мы используем JDK 25, где структурированная конкурентность доступна как предварительная функция.

Вывод программы будет следующим:

```
20:35:18.357 main         : --- Running Success Scenario ---
20:35:18.358 main         : ... Expecting to take ~2 seconds (the time of
                            the slowest task)...
20:35:18.362 main         : ... Scope joining. Waiting for subtasks...
20:35:18.363 VThread[#36] : -> Fetching product reviews... (will take 2s)
20:35:18.363 VThread[#34] : -> Fetching product details... (will take 1s)
20:35:19.367 VThread[#34] : <- Product details fetched.
20:35:20.369 VThread[#36] : <- Product reviews fetched.
20:35:20.370 main         : ... Scope joined successfully.
20:35:20.371 main         : Total time taken: 2012ms
20:35:20.378 main         :
Success! Result: ProductInfo[product=Product[productId=123, name=Sample Product],
reviews=[Review[user=Inaya, rating=5], Review[user=Rushda, rating=4]]]
```

Главный поток создает область и сразу запускает две подзадачи: `fetchProduct()` и `fetchReviews()`. Их подхватывают виртуальные потоки `VThread[#34]` и `Vthread[#36]`. Подзадачи начинают выполняться практически мгновенно.

https://max.ru/bookofgeek

Пока они работают параллельно, основной поток блокируется на `scope.join()`, ожидая результата. Через 1 с `fetchProduct()` завершается, но основной поток продолжает ждать, так как более медленная подзадача `fetchReviews()` все еще выполняется. Когда через 2 с `fetchReviews()` завершится и все подзадачи будут успешно выполнены, метод `join()` наконец возвращает управление. Общее время выполнения составляет ~2000 мс, что подтверждает: задачи выполнялись параллельно, так как продолжительность всей операции и самой длительной подзадачи равны.

Теперь обратим внимание на сценарий с ошибкой. Для этого мы модифицируем метод `main()`, чтобы вызвать `fetchProductInfo()` с флагом `shouldFail`, установленным в `true`. Это приведет к использованию метода `fetchProductThatFails()`, который мы специально разработали для немедленного выбрасывания исключения.

Далее модифицируем метод `main()` для выполнения следующего сценария:

```
void main() {
  var demo = new DefaultPolicyDemo();
  log("--- Running Failure Scenario ---");
  log("... Expecting to fail almost instantly...\n");
  try {
    demo.fetchProductInfo(456L, true); ❶
  } catch (Exception e) {
    // Ожидаем RuntimeException из нашего блока catch
    log("\nCaught expected exception in failure scenario: " + e.getMessage());
    log("Cause: " + e.getCause()); ❷
  }
}
```

Данный сценарий демонстрирует мгновенную обработку ошибок.

❶ Запускает сценарий с ошибкой, передав `true` для параметра `shouldFail`.

❷ Показывает первоначальную причину сбоя, обернутую в `RuntimeException`.

Вывод программы будет следующим:

```
22:20:11.405 main         : --- Running Failure Scenario ---
22:20:11.406 main         : ... Expecting to fail almost instantly...
22:20:11.411 main         : ... Scope joining. Waiting for subtasks...
2:20:11.412 VThread[#34]  : -> Fetching product details... (will fail)
22:20:11.412 VThread[#36] : -> Fetching product reviews... (will take 2s)
22:20:11.413 main         : ... Scope join failed. A subtask threw an exception.
22:20:11.414 main         : Total time taken: 7ms
22:20:11.414 main :
Caught expected exception in failure scenario: Failed to fetch product info
22:20:11.414 main : Cause: ca.bazlur.mcj.chap4.
ProductServiceException: Product ID 456 not found
```

Этот вывод наглядно демонстрирует преимущества политики fail-fast (быстрого отказа). Несмотря на то что и задача `fetchProductThatFails()`, которая завершается ошибкой, и задача `fetchReviews()`, занимающая 2 с, запускаются

https://max.ru/bookofgeek

в виртуальных потоках, вся операция завершается всего за 7 мс. Как только первая подзадача терпит неудачу, политика `Joiner` отменяет остальные выполняющиеся подзадачи и вызывает `scope.join()`, чтобы немедленно выбросить исключение `FailedException`. Система не тратит время на ожидание завершения подзадачи `fetchReviews()`, так как ее результат становится нерелевантным. Такое стремительное распространение ошибок и предотвращение бесполезной работы — ключевое преимущество структурированной конкурентности для операций по принципу «всё или ничего» (all-or-nothing).

Чтобы в полной мере оценить преимущества политики fail-fast, для сравнения выполним ту же задачу, но с применением традиционного подхода на основе `ExecutorService`:

```
ProductInfo fetchProductInfoWithExecutor(Long productId)
    throws ExecutionException, InterruptedException {
  Instant start = Instant.now();
  try (ExecutorService service
      = Executors.newVirtualThreadPerTaskExecutor()) { ❶
    Future<Product> productFuture
      = service.submit(() -> fetchProductThatFails(productId)); ❷
    Future<List<Review>> reviewFuture
      = service.submit(() -> fetchReviews(productId)); ❸
    Product product = productFuture.get(); ❹
    List<Review> reviews = reviewFuture.get(); ❺
    return new ProductInfo(product, reviews);
  } finally {
    Instant end = Instant.now();
    System.out.printf("Time taken: %dms%n",
     end.toEpochMilli() - start.toEpochMilli());
  }
}
```

Мы сразу видим ограничения традиционного подхода.

❶ Создаем `ExecutorService` с виртуальными потоками.

❷ Отправляем задачу получения продукта (которая завершится ошибкой) и получаем `Future`.

❸ Отправляем задачу получения отзывов и получаем `Future` еще раз.

❹ Блокирующий вызов метода `get` ожидает данных о продукте (которых не будет).

❺ Ожидание результатов отзывов должно было бы блокироваться, но эта точка не будет достигнута из-за исключения.

Вызовем этот метод из метода main. Вывод покажет неэффективность подхода:

```
22:46:10.211 main         : --- Running Failure Scenario with Executor ---
22:46:10.212 main         : ... Expecting to fail almost instantly...
22:46:10.215 VThread[#36] : -> Fetching product reviews... (will take 2s)
22:46:10.215 VThread[#34] : -> Fetching product details... (will fail)
```

https://max.ru/bookofgeek

```
22:46:12.221 VThread[#36] : <- Product reviews fetched.
Time taken: 2010ms
22:46:12.227 main         :
Caught expected exception in failure scenario: ca.bazlur.mcj.chap4.
ProductService
Exception: Product ID 456 not found
22:46:12.227 main        : Cause: ca.bazlur.mcj.chap4.ProductServiceException:
Product ID 456 not found
```

При анализе лога консоли неэффективность этого неструктурированного подхода становится очевидной. Мы видим, что, хотя задача `fetchProductThatFails()` в виртуальном потоке `VThread[#34]` завершается с ошибкой почти мгновенно, задача `fetchReviews()` в `VThread[#36]` продолжает выполняться как ни в чем не бывало. Лог подтверждает этот факт, выводя сообщение `Product reviews fetched` («Отзывы на продукт получены») после задержки 2 с. Дело в том, что `ExecutorService` не имеет концепции общей области (shared scope) — для него эти два объекта `Future` независимы, и неудача одного не влияет на другой. Как следствие, наша программа блокируется, пока задача получения отзывов выполняется до завершения, бесполезно расходуя системные ресурсы на результат, который в итоге будет отброшен.

Гонка за первым успешным результатом

С помощью политики `Joiner.anySuccessfulResultOrThrow()` мы можем оркестрировать гонку среди конкурентных подзадач, где победитель объявляется сразу после успешного завершения любой из них. Это можно сравнить с подходом first-past-the-post («первым пересечь финишную прямую»). Как только определяется победитель, `Joiner` отменяет все остальные выполняющиеся подзадачи.

Чтобы проиллюстрировать эту политику, обыграем сценарий получения данных о продукте из трех источников с разным временем отклика. Мы будем использовать первый полученный результат.

Сначала определим три метода получения данных. Каждому зададим разную продолжительность `Thread.sleep()` и добавим логирование. Это позволит следить за гонкой и последующей отменой при выполнении кода:

```
private Product fetchProductFromCache(long productId) throws
    InterruptedException {
  log(" -> Checking cache... (will take 500ms)");
  Thread.sleep(Duration.ofMillis(500)); ❶
  log(" <- Cache has the result!");
  return new Product(productId, "Product from Cache");
}
// Более медленный источник (2000 мс)
private Product fetchProductFromDatabase(long productId)
    throws InterruptedException {
  try {
    log(" -> Querying database... (will take 2s)");
```

https://max.ru/bookofgeek

```
    Thread.sleep(Duration.ofSeconds(2)); ❷
    log(" <- Database has the result!");
    return new Product(productId, "Product from DB");
  } catch (InterruptedException e) {
    // Этот блок выполняется, когда область отменяет задачу
    log(" <- Database query was canceled."); ❸
    Thread.currentThread().interrupt();
    throw e;
  }
}
// Самый медленный источник (3000 мс)
private Product fetchProductFromAPI(long productId)
    throws InterruptedException {
  try {
    log(" -> Calling external API... (will take 3s)");
    Thread.sleep(Duration.ofSeconds(3)); ❹
    log(" <- API has the result!");
    return new Product(productId, "Product from API");
  } catch (InterruptedException e) {
    // Этот блок также выполняется при отмене
    log(" <- API call was canceled."); ❺
    Thread.currentThread().interrupt();
    throw e;
  }
}
```

Каждый метод моделирует источник данных с разным временем отклика.

❶ Кэш отвечает быстрее всех, за 500 мс — это вероятный победитель.

❷ Обращение к базе данных занимает 2 с и включает обработку отмены.

❸ Логирует прерывание запроса к БД при отмене области.

❹ API — самый медленный (3 с) и также обрабатывает отмену.

❺ Логирует прерывание вызова API к БД при отмене области.

Теперь напишем метод-оркестратор. На этот раз при открытии области мы передадим джоинер `Joiner.anySuccessfulResultOrThrow()`, чтобы организовать гонку:

```
public Product fetchProduct(long productId) {
  Instant start = Instant.now();
  // Joiner определяет политику "гонка за победой"
  try (var scope
           = StructuredTaskScope.open( StructuredTaskScope
                .Joiner.<Product>anySuccessfulResultOrThrow())){ ❶
    scope.fork(() -> fetchProductFromDatabase(productId)); ❷
    scope.fork(() -> fetchProductFromCache(productId)); ❸
    scope.fork(() -> fetchProductFromAPI(productId)); ❹
    // join() сейчас возвращает результат первой успешной подзадачи
    return scope.join(); ❺
  } catch (InterruptedException | StructuredTaskScope.FailedException e) {
    // FailedException выбрасывается, если ВСЕ подзадачи завершились неудачно
```

https://max.ru/bookofgeek

```
    throw new RuntimeException(e); ❻
  } finally {
    Instant end = Instant.now();
    log("Total time taken: %dms%n"
        .formatted(Duration.between(start, end).toMillis()));
  }
}
```

Метод-оркестратор настраивает условия гонки.

❶ Создает область с политикой «гонка за победой» (race-to-win), которая возвращает первый успешный результат.

❷ Запускает подзадачу для базы данных с задержкой 2 с.

❸ Запускает подзадачу для кэша с задержкой 500 мс — это вероятный победитель.

❹ Запускает подзадачу для API с задержкой 3 с.

❺ Ожидает первого успешного завершения и возвращает его результат.

❻ Обрабатывает случай, когда все подзадачи завершились неудачно (выбрасывает `FailedException`).

При выполнении этого кода мы ожидаем увидеть отмену, если превышено заданное время. Вызовем данный метод из метода `main`:

```
public class RacePolicyDemo {
  // ... (реализации методов из предыдущего кода)
  void main() {
    var demo = new RacePolicyDemo();
    log("--- Running Race Scenario ---");
    log("... Three tasks will race. " +
        "Expecting to finish in ~500ms (the fastest task)...\n");
    try {
      Product winningProduct = demo.fetchProduct(123L); ❶
      log("Race finished! Winning result: " + winningProduct);
    } catch (Exception e) {
      log("Caught unexpected exception: " + e.getMessage());
      e.printStackTrace();
    }
  }
}
```

Метод `main` показывает сценарий гонки.

❶ Запускает гонку между тремя различными источниками данных и ожидает победы самого быстрого.

Для работы этого кода также потребуется следующая запись:

```
public record Product(long productId, String source) {
}
```

https://max.ru/bookofgeek

Запустим этот код:

```
java --enable-preview RacePolicyDemo.java
```

Вывод будет следующий:

```
23:03:23.917 main        : --- Running Race Scenario ---
23:03:23.917 main        : ... Three tasks will race. Expecting to finish
                           in ~500ms (the fastest task)...
23:03:23.923 VThread[#36]: -> Checking cache... (will take 500ms)
23:03:23.923 VThread[#38]: -> Calling external API... (will take 3s)
23:03:23.923 VThread[#34]: -> Querying database... (will take 2s)
23:03:24.429 VThread[#36]: <- Cache has the result!
23:03:24.431 VThread[#34]: <- Database query was canceled.
23:03:24.431 VThread[#38]: <- API call was canceled.
23:03:24.439 main        : Total time taken: 513ms
23:03:24.442 main        : Race finished! Winning result:
Product[productId=123, source=Product from Cache]
```

Анализируя вывод в консоли, мы видим идеальную демонстрацию политики «гонка за победой». Все три метода `fetch` были разветвлены и начали выполняться параллельно. Примерно через 500 мс самая быстрая задача — `fetchProductFromCache()` — завершается успешно. В тот же момент политика нашего `Joiner` считает условие выполненным и немедленно отменяет две другие выполняющиеся подзадачи, что подтверждается сообщениями `was canceled`, выведенными в поток ошибок. Затем метод `join` возвращает победивший результат из кэша, и вся операция завершается чуть более чем за 500 мс, не тратя зря время на ожидание более медленных задач.

Сбор всех результатов или завершение при первой ошибке

Мы рассмотрели политику по умолчанию, которая завершается с ошибкой при неудаче любой подзадачи. Следующая политика — `allSuccessfulOrThrow` — при возникновении ошибки ведет себя аналогично, но также предлагает более удобный способ обработки результатов при успешном выполнении.

Как и политика по умолчанию, `allSuccessfulOrThrow()` придерживается стратегии all-or-nothing («всё или ничего»): при сбое любой подзадачи она немедленно отменяет все остальные выполняющиеся подзадачи, а метод `scope.join()` выбрасывает исключение `FailedException`.

Однако при успешном выполнении ее поведение кардинально меняется. Вместо возврата `void` метод `join()` возвращает `java.util.Stream<Subtask<T>>`, который содержит ссылки на все успешно завершенные подзадачи в порядке их запуска. Это упрощает одновременную обработку результатов. Такой `Joiner` следует использовать, когда все подзадачи возвращают результаты одного типа (например, `String`, `Product` и т. д.) и требуется, чтобы все они завершились успешно.

https://max.ru/bookofgeek

Если подзадачи возвращают результаты разных типов, лучше подходит политика по умолчанию `awaitAllSuccessfulOrThrow()`, поскольку результаты каждой подзадачи будут обрабатываться индивидуально.

Допустим, нам нужно проверить список ID пользователей, сверяя их с внешним сервисом. Мы хотим выполнить эту проверку конкурентно, причем вся партия считается валидной только при успешной проверке каждого ID.

Сначала определим простой метод, имитирующий проверку пользователя. Затем создадим его версию, которая будет завершаться ошибкой для определенного ID:

```
public record ValidatedUser(long userId, String status) {
}

private ValidatedUser validateUser(long userId)
    throws InterruptedException {
  log(" -> Validating user %d...".formatted(userId));
  Thread.sleep(Duration.ofMillis(100 + new Random().nextInt(500))); ❶
  log(" <- User %d is valid.".formatted(userId));
  return new ValidatedUser(userId, "VALID");
}

private ValidatedUser validateUserWithFailure(long userId)
    throws InterruptedException {
  if (userId == 3L) {
    log(" -> Validating user %d... (will fail)".formatted(userId));
    throw new IllegalArgumentException("Invalid user ID: " + userId); ❷
  }
  return validateUser(userId);
}
```

Методы валидации имитируют реальное поведение.

❶ Добавляют случайную задержку от 100 до 600 мс для моделирования изменчивости сети.

❷ Специально завершаются ошибкой для пользователя с ID 3, чтобы продемонстрировать обработку ошибок.

Теперь напишем метод, который принимает список ID пользователей, создает подзадачу валидации для каждого и использует `allSuccessfulOrThrow()` для сбора результатов:

```
public List<ValidatedUser> validateAllUsers(List<Long> userIds)
    throws InterruptedException {
  log("Validating a batch of " + userIds.size() + " users...");
  try (var scope
           = open(Joiner.<ValidatedUser>allSuccessfulOrThrow())) { ❶
    var subtasks = userIds.stream()
       .map(id -> scope.fork(() -> validateUserWithFailure(id))) ❷
       .toList();
```

https://max.ru/bookofgeek

```
    // в случае успеха join() возвращает Stream<Subtask<ValidatedUser>>
    // или выбрасывает FailedException при первой же ошибке
    var resultStream = scope.join(); ❸
    log("...All users validated successfully. Processing stream...");
    return resultStream
        .map(Subtask::get) ❹
        .toList();
  } catch (FailedException ex) {
    log("All users validated successfully. Processing stream");
    throw new RuntimeException("Batch validation failed",
        ex.getCause()); ❺
  }
}
```

Метод пакетной валидации организует сбор нескольких результатов.

❶ Создает область с политикой `allSuccessfulOrThrow()` для сбора всех результатов.

❷ Запускает подзадачу валидации для каждого ID пользователя и собирает дескрипторы подзадач.

❸ Ожидает завершения всех валидаций и возвращает `Stream` успешных подзадач.

❹ Извлекает фактические результаты `ValidatedUser` из каждой подзадачи.

❺ Обрабатывает ошибку пакета, оборачивая исходное исключение валидации.



О var и выводе типов

Для краткости кода я использовал ключевое слово `var`. Хотя это улучшает читабельность, сокращая boilerplate-код, всегда полезно знать, какие конкретно типы используются.

В методе `validateAllUsers`:

- `var scope` выводится как `StructuredTaskScope<ValidatedUser>`;
- `var subtasks` выводится как `List<Subtask<ValidatedUser>>`;
- `var resultStream` выводится как `Stream<Subtask<ValidatedUser>>`.

Теперь соберем все вместе в полный класс `BatchValidationDemo.java`. Чтобы сохранить лаконичность кода, мы будем использовать статический импорт вложенных классов интерфейса `StructuredTaskScope`, таких как `Joiner` и `Subtask`, а также самого метода `open()`:

```
import static java.util.concurrent.StructuredTaskScope.*;
public class BatchValidationDemo {
  // ... (реализации методов из предыдущего кода)
  void main() {
    var demo = new BatchValidationDemo();
    List<Long> successfulBatch = List.of(1L, 2L, 4L, 5L);
```

https://max.ru/bookofgeek

```
    // Содержит ошибочный ID '3'
    List<Long> failingBatch = List.of(1L, 2L, 3L, 4L, 5L); ❶
    log("--- Running Success Scenario ---");
    try {
      List<ValidatedUser> results = demo.validateAllUsers(successfulBatch); ❷
      log("\nBatch validation complete. Results:");
      results.forEach(validatedUser -> log(validatedUser.toString()));
    } catch (Exception e) {
      log("Caught unexpected exception: " + e.getMessage());
    }
    log("\n=============================================\n");
    log("--- Running Failure Scenario ---");
    try {
      demo.validateAllUsers(failingBatch); ❸
    } catch (Exception e) {
      log("Caught expected exception: " + e.getMessage());
    }
  }
}
```

Метод `main()` тестирует оба сценария.

❶ Создает пакет с ID 3, который вызовет ошибочное состояние.

❷ Проверяет успешный сценарий, где все ID пользователей проходят валидацию.

❸ Проверяет сценарий с ошибкой, где одна неудачная валидация отменяет остальные.

Запустим этот код:

```
java --enable-preview BatchValidationDemo.java
```

Вывод программы будет следующим:

```
00:29:49.275 main : --- Running Success Scenario ---
00:29:49.276 main : Validating a batch of 4 users...
00:29:49.286 VThread[#39]: -> Validating user 4...
00:29:49.286 VThread[#36]: -> Validating user 2...
00:29:49.286 VThread[#34]: -> Validating user 1...
00:29:49.286 VThread[#42]: -> Validating user 5...
00:29:49.452 VThread[#39]: <- User 4 is valid.
00:29:49.480 VThread[#42]: <- User 5 is valid.
00:29:49.614 VThread[#34]: <- User 1 is valid.
00:29:49.696 VThread[#36]: <- User 2 is valid.
00:29:49.697 main : All users validated successfully. Processing stream
00:29:49.698 main :
Batch validation complete. Results:
00:29:49.701 main : ValidatedUser[userId=1, status=VALID]
00:29:49.702 main : ValidatedUser[userId=2, status=VALID] 00:29:49.702 main :
ValidatedUser[userId=4, status=VALID]
00:29:49.702 main : ValidatedUser[userId=5, status=VALID]
00:29:49.702 main :
```

https://max.ru/bookofgeek

```
==========================================
00:29:49.702 main : --- Running Failure Scenario ---
00:29:49.702 main : Validating a batch of 5 users... 00:29:49.703
VThread[#48]: -> Validating user 4...
00:29:49.703 VThread[#47]: -> Validating user 3... (will fail) 00:29:49.703
VThread[#45]: -> Validating user 1...
00:29:49.703 VThread[#46]: -> Validating user 2...
00:29:49.703 VThread[#49]: -> Validating user 5...
00:29:49.704 main : ...Validation failed for one of the users. 00:29:49.704
main : Caught expected exception: Batch validation failed
```

Просматривая вывод, мы видим два различных результата нашей политики «всё или ничего».

В сценарии успеха все четыре задачи валидации были распределены по виртуальным потокам и выполнялись конкурентно. Поток `main` ожидает до завершения последней задачи, после чего `join()` возвращает `Stream` результатов, который мы затем выводим.

В сценарии с ошибкой все пять задач распределяются, но задача для `userId=3` немедленно завершается с ошибкой. `Joiner` обнаруживает это и отменяет всю область. Вызов `join()` не возвращает `Stream`, а немедленно выбрасывает `FailedException`. Блок `catch` перехватывает это исключение и выводит финальное сообщение об ошибке. Важное наблюдение: ни одна из остальных задач не выводит сообщение `<- User X is valid`, поскольку все они были отменены до того, как завершились. Это подтверждает поведение fail-fast и доказывает, что данная политика защищает целостность пакетной операции.

Ждем всех

Мы рассмотрели политики, которые либо завершаются при первой ошибке, либо требуют успешного выполнения всех задач. У политики `awaitAll()` другой подход: она ожидает завершения всех подзадач независимо от их успешности. Она подходит для сценариев, где важны побочные эффекты или необходимо обработать все результаты, включая частичные.

В отличие от `allSuccessfulOrThrow()`, политика `awaitAll()` не отменяет выполняющиеся подзадачи при возникновении ошибки. Все запущенные задачи выполняются до завершения. Метод `scope.join()` всегда возвращает `null`, поскольку этот джоинер фокусируется на завершении задач, а не на сборе результатов.

Ключевое преимущество политики — отказоустойчивость. Когда некоторые операции могут завершиться с ошибкой, но мы хотим получить успешные результаты, `awaitAll()` гарантирует, что работа не будет выполнена впустую. Этот джоинер следует использовать, когда подзадачи дают побочные эффекты (например, выполняют логирование, сбор метрик или обработку файлов) или

https://max.ru/bookofgeek

требуется собрать максимально возможное количество данных, даже если часть источников недоступна.

Политика `awaitAll()` особенно полезна для систем мониторинга, пакетной обработки задач и серверной архитектуры Fan-In, где даже частичный успех представляет ценность.

Допустим, нам нужно отправить оповещения о критичном событии системы по нескольким каналам (электронная почта, SMS, push-уведомления). Нам требуется максимальное покрытие, при сбое одного из каналов остальные должны доставить уведомление.

Сначала определим простую запись для отслеживания результатов оповещений, а также методы, имитирующие отправку с разной надежностью:

```
public record NotificationResult(String channel, boolean success,
                                 String message) {
  public static NotificationResult success(String channel, String message) {
    return new NotificationResult(channel, true, message);
  }

  public static NotificationResult failure(String channel, String error) {
    return new NotificationResult(channel, false, error);
  }
}

// Общее состояние сбора результатов (побочные эффекты)
private final List<NotificationResult> notificationResults =
    new CopyOnWriteArrayList<>(); ❶
private final AtomicInteger successCount = new AtomicInteger(0); ❷
private final AtomicInteger failureCount = new AtomicInteger(0); ❸
```

Общее состояние для отслеживания результатов уведомлений.

❶ Потокобезопасный список для сбора результатов конкурентных попыток отправки уведомлений.

❷ Атомарный счетчик успешных доставок по всем каналам.

❸ Атомарный счетчик неудачных доставок для контроля общей надежности.

Теперь реализуем методы оповещения с разной надежностью:

```
private void sendEmailNotification(String message)
    throws InterruptedException {
  log(" -> Sending email notification...");
  Thread.sleep(Duration.ofMillis(200 + new Random().nextInt(300)));
  // Электронная почта обычно надежна (90 % успешных отправок)
  if (new Random().nextDouble() < 0.9) { ❶
    log(" <- Email sent successfully");
    notificationResults.add(
        NotificationResult.success("EMAIL", "Delivered to inbox"));
```

https://max.ru/bookofgeek

```
    successCount.incrementAndGet();
  } else {
    log(" <- Email failed: SMTP server unavailable");
    notificationResults.add(
        NotificationResult.failure("EMAIL", "SMTP server unavailable"));
    failureCount.incrementAndGet();
    throw new RuntimeException("Email delivery failed"); ❷
  }
}

private void sendSmsNotification(String message)
    throws InterruptedException {
  log(" -> Sending SMS notification...");
  Thread.sleep(Duration.ofMillis(150 + new Random().nextInt(400)));
  // SMS менее надежны (70 % успешных отправок)
  if (new Random().nextDouble() < 0.7) { ❸
    log(" <- SMS sent successfully");
    notificationResults.add(
        NotificationResult.success("SMS", "Delivered to mobile"));
    successCount.incrementAndGet();
  } else {
    log(" <- SMS failed: Carrier gateway timeout");
    notificationResults.add(
        NotificationResult.failure("SMS", "Carrier gateway timeout"));
    failureCount.incrementAndGet();
    throw new RuntimeException("SMS delivery failed");
  }
}
private void sendPushNotification(String message)
    throws InterruptedException {
  log(" -> Sending push notification...");
  Thread.sleep(Duration.ofMillis(100 + new Random().nextInt(200)));
  // Push-уведомления самые надежные (95 % успешных отправок)
  if (new Random().nextDouble() < 0.95) { ❹
    log(" <- Push notification sent successfully");
    notificationResults.add(
        NotificationResult.success("PUSH", "Delivered to device"));
    successCount.incrementAndGet();
  } else {
    log(" <- Push notification failed: Device token expired");
    notificationResults.add(
        NotificationResult.failure("PUSH", "Device token expired"));
    failureCount.incrementAndGet();
    throw new RuntimeException("Push notification delivery failed");
  }
}
```

Каждый метод уведомления имитирует реальные паттерны надежности.

❶ У электронной почты 90 % успешных доставок, что соответствует типичной надежности SMTP.

❷ Даже при сбое мы фиксируем результат перед выбрасыванием исключения.

https://max.ru/bookofgeek

❸ SMS — 70 % успешных доставок, что отражает проблемы шлюзов.

❹ Push-уведомления с 95 % успешных доставок являются самым надежным каналом.

Напишем метод, который отправляет уведомления по всем каналам конкурентно с использованием `awaitAll()`, гарантируя попытку доставки через каждый из них, независимо от отдельных сбоев:

```
public void sendCriticalAlert(String alertMessage)throws InterruptedException {
  log("Sending critical alert: " + alertMessage);

  try (var scope = open(Joiner.<Void>awaitAll())) { ❶
  // Запуск конкурентных подзадач отправки уведомлений —
  // каждая имеет побочные эффекты
  scope.fork(() -> {
    sendEmailNotification(alertMessage);
    return null; // awaitAll() игнорирует возвращаемые значения ❷
  });
  scope.fork(() -> {
    sendSmsNotification(alertMessage);
    return null;
  });
  scope.fork(() -> {
    sendPushNotification(alertMessage);
    return null;
  });
  log("Waiting for all notification attempts to complete");
  // join() всегда возвращает null для awaitAll()
  // Все задачи завершаются независимо от отдельных сбоев
  Void result = scope.join(); ❸
  log("...All notification attempts completed.");
  // Обработка побочных эффектов (собранных результатов)
  logNotificationSummary(); ❹
  } catch (InterruptedException e) {
    log("...Notification sending was interrupted");
    Thread.currentThread().interrupt();
    throw e;
  }
}
```

Метод оповещения демонстрирует образец `awaitAll()`.

❶ Создает область с политикой `awaitAll()`, которая ожидает завершения всех задач независимо от сбоев.

❷ Возвращает `null`, так как `awaitAll()` фокусируется на завершении задач, а не на сборе результатов.

❸ `join()` всегда возвращает `null`, но обеспечивает завершение всех задач.

❹ Обрабатывает побочные эффекты, собранные во время выполнения задач.

https://max.ru/bookofgeek

Добавим вспомогательный метод `logNotificationSummary()` для обработки собранных результатов:

```
private void logNotificationSummary() {
  log("\n--- Notification Summary ---");
  log("Total channels attempted: " + (successCount.get() + failureCount.get()));
  log("Successful deliveries: " + successCount.get());
  log("Failed deliveries: " + failureCount.get());
  log("\nDetailed results:");
  notificationResults.forEach(result -> {
    String status = result.success() ? "✓" : "✗";
    log(status + " " + result.channel() + ": " + result.message());
  });
  if (successCount.get() > 0) {
    log("\n    Alert successfully delivered through " +
        successCount.get() + " channel(s)");
  } else {
    log("\n⚠ Alert failed to deliver through any channel!");
  }
}
```



О var и выводе типов (продолжение)

В методе `sendCriticalAlert`:

- `var scope` выводится как `StructuredTaskScope< Void>`;
- метод `open()` импортирован статически: `import static java.util.concurrent.StructuredTaskScope.open;`.

Теперь соберем все вместе в полный класс `AwaitedAllDemo.java`. Чтобы сохранить лаконичность кода, мы будем использовать статический импорт метода `open()` и такие вложенные классы `StructuredTaskScpe`, как `Joiner`:

```
import module java.base;
import static ca.bazlur.mcj.chap4.Utils.log;
import static java.util.concurrent.StructuredTaskScope.*;
import static java.util.concurrent.StructuredTaskScope.open;

public class AwaitAllDemo {
  public record NotificationResult(String channel,
                                   boolean success,
                                   String message) {
      // фабричные методы и реализации из кода выше
  }

  private final List<NotificationResult> notificationResults =
    new CopyOnWriteArrayList<>();
  private final AtomicInteger successCount = new AtomicInteger(0);
  private final AtomicInteger failureCount = new AtomicInteger(0);
```

https://max.ru/bookofgeek

```
  // ... (реализации ВСЕХ методов из предыдущего кода)
  void main() {
    String criticalAlert =
      "URGENT: Database connection pool exhausted - " +
      "immediate attention required";
    log("Running Notification Scenario (awaitAll Policy)");
    log("This demonstrates how awaitAll() processes " +
      "ALL tasks regardless of failures\n");

    try {
      sendCriticalAlert(criticalAlert); ❶
    } catch (Exception e) {
      log("Caught exception: " + e.getMessage());
    }
    log("\n==============================\n");
    log("--- Running Second Notification Batch ---");

    try {
      sendCriticalAlert(
        "RESOLVED: Database issue fixed - " +
        "all systems operational"); ❷
    } catch (Exception e) {
      log("Caught exception: " + e.getMessage());
    }
  }
}
```

Метод `main` демонстрирует устойчивость `awaitAll()`.

❶ Отправляет первое критичное оповещение, позволяя увидеть работу разных каналов.

❷ Отправляет повторное оповещение, показывая согласованность работы паттерна с несколькими пакетами.

ОБЪЯВЛЕНИЕ ИМПОРТА МОДУЛЯ (JDK 25+)

Инструкция `import module java.base;` — это объявление импорта модуля, добавленное в JDK 25. Она импортирует по требованию все публичные классы и интерфейсы верхнего уровня из:

- пакетов, экспортируемых указанным модулем в текущий модуль;
- пакетов, экспортируемых модулями, которые транзитивно считываются при чтении указанного модуля.

Для import `module java.base` это эквивалентно 54 импортам пакетов по требованию (по одному на каждый пакет, экспортируемый `java.base`), что аналогично перечислению: `import java.io.*`, `import java.util.*`, `import java.time.*`, `import java.util.concurrent.*` и т. д. Благодаря этому не нужно перечислять множество отдельных импортов при работе с базовыми Java API и заметно сокращается объем кода.

https://max.ru/bookofgeek

Запустим код:

```
java --enable-preview AwaitedAllDemo.java
```

Вывод программы будет следующим:

```
04:59:39.852 main : Running Notification Scenario (awaitAll Policy)
04:59:39.852 main : This demonstrates how awaitAll() processes ALL tasks
                    regardless of failures

04:59:39.852 main        : Sending critical alert to all notification
                           channels...
04:59:39.853 main        : Alert message: URGENT: Database connection pool
                           exhausted - immediate attention required
04:59:39.857 main        : Waiting for all notification attempts to complete
04:59:39.857 VThread[#38]: -> Sending push notification...
04:59:39.857 VThread[#36]: -> Sending SMS notification...
04:59:39.857 VThread[#34]: -> Sending email notification...
04:59:39.970 VThread[#38]: <- Push notification sent successfully
04:59:40.088 VThread[#36]: <- SMS failed: Carrier gateway timeout
04:59:40.234 VThread[#34]: <- Email sent successfully
04:59:40.235 main        : ...All notification attempts completed.
 04:59:40.236 main       :
--- Notification Summary ---
04:59:40.237 main        : Total channels attempted: 3
04:59:40.238 main        : Successful deliveries: 2
04:59:40.239 main        : Failed deliveries: 1
04:59:40.239 main        :
Detailed results:
04:59:40.243 main        : ✓ PUSH: Delivered to device
04:59:40.244 main        : ✗ SMS: Carrier gateway timeout
04:59:40.244 main        : ✓ EMAIL: Delivered to inbox
04:59:40.244 main        :
Alert successfully delivered through 2 channel(s)
```

В выводе мы ясно видим стабильное поведение нашей политики `awaitAll()`.

В сценарии успеха все три задачи уведомлений распределяются по виртуальным потокам и выполняются конкурентно. Хотя SMS-уведомление завершается ошибкой `Carrier gateway timeout`, электронная почта и push-уведомления продолжают работать и успешно выполняются. Главный поток ожидает в `join()` до завершения всех задач, независимо от их индивидуальных результатов. Побочные эффекты (результаты уведомлений) собираются и обобщаются. Из резюме мы видим, что оповещения успешно доставлены через два канала из трех.

Главное отличие от `allSuccessfulOrThrow()` заключается в том, что неудачи одних задач не отменяют выполнения остальных. Каждый канал уведомлений получает равные возможности для доставки сообщения, что расширяет охват. Такой образ действий критически важен для систем оповещения, где частичная доставка бесконечно лучше полного ее отсутствия. Данная политика гарантирует, что временные проблемы одного канала не блокируют остальные.

https://max.ru/bookofgeek

Отказоустойчивый конкурентный сервер

Еще один очень интересный сценарий использования `awaitAll()` — создание отказоустойчивых конкурентных серверов. В таких системах необходимо обеспечить обработку и завершение каждого клиентского подключения, даже если один из обработчиков соединений столкнется с ошибкой. Политика `awaitAll()` предотвращает ситуацию, когда сбой одного клиента приводит к аварийному завершению работы сервера или преждевременной остановке обработки других активных подключений. В отличие от политики `allSuccessfulOrThrow()`, которая при сбое одного соединения отменяет все активные соединения, `awaitAll()` обеспечивает независимое выполнение каждого клиентского подключения вплоть до его завершения.

Ключевое преимущество данной политики — изоляция сбоев. Если отдельные обработчики соединений сталкиваются с ошибками (например, тайм-аутами сети, некорректными запросами, отключениями клиентов), эти сбои остаются изолированными и не распространяются на другие активные подключения. Поэтому-то `awaitAll()` идеально подходит для серверных приложений, где критически важны время бесперебойной работы и надежность.

Рассмотрим этот подход на примере отказоустойчивого эхо-сервера (echo server). Начнем с основной инфраструктуры сервера и механизма отслеживания подключений:

```
import module java.base;
import static ca.bazlur.mcj.chap4.rewrite.Utils.log;
import static java.util.concurrent.StructuredTaskScope.open;

public class ResilientServer {
  private final AtomicInteger connectionCount = new AtomicInteger(0);
  private final AtomicInteger activeConnections = new AtomicInteger(0);
}
```

Эти атомарные счетчики позволяют в условиях многопоточности безопасно отслеживать как общее число установленных соединений, так и число подключений, активных в настоящее время.

Напишем основной метод сервера, демонстрирующий, как политика `awaitAll()` управляет множеством конкурентных соединений.

```
public void serve(ServerSocket serverSocket)
       throws IOException, InterruptedException {
  log("Server starting on port: " +
    serverSocket.getLocalPort());

  try (var scope = open(StructuredTaskScope.Joiner.
       <Void>awaitAll())) {
    serverSocket.setSoTimeout(1000);
```

https://max.ru/bookofgeek

```
    while (!Thread.currentThread().isInterrupted()) {
      try {
        Socket socket = serverSocket.accept();
        int connId = connectionCount.incrementAndGet();
        activeConnections.incrementAndGet();

        log("Accepted connection #" + connId);

        // Запускаем подзадачу для обработки этого соединения
        scope.fork(() -> {
          handleConnection(socket, connId);
          return null;
        });

      } catch (SocketTimeoutException e) {
        continue; // Проверяем на прерывание
      }
    }

    log("Server stopping, waiting for " +
        "connections to finish...");
    scope.join(); // Дожидаемся завершения всех соединений

  } finally {
    if (!serverSocket.isClosed()) {
      serverSocket.close();
    }
    log("Server shutdown complete. Total connections: " +
      connectionCount.get());
  }
}
```

Обработчик соединения демонстрирует изоляцию ошибок:

```
private void handleConnection(Socket socket, int connectionId) {
  try (socket;
    var reader = new BufferedReader(
        new InputStreamReader(socket.getInputStream()));
    var writer = new PrintWriter(
        socket.getOutputStream(), true)) {

    log(" [Conn-" + connectionId + "] Started");
    writer.println("Welcome to Echo Server! " +
        "Type 'quit' to exit.");
    String line;
    while ((line = reader.readLine()) != null) {
      log(" [Conn-" + connectionId + "] Received: " + line);

      if("quit".equalsIgnoreCase(line.trim())) {
        writer.println("Goodbye!");
        break;
      }
```

https://max.ru/bookofgeek

```
        // Эхо-сервер отсылает сообщение обратно
        writer.println("Echo: " + line);
      }

      log(" [Conn-" + connectionId + "] Completed successfully");

    } catch (IOException e) {
      log(" [Conn-" + connectionId + "] Error: " +
          e.getMessage());
    } finally {
      activeConnections.decrementAndGet();
      log(" [Conn-" + connectionId + "] Finished. Active: " +
         activeConnections.get());
    }
}
```

Каждое клиентское соединение обрабатывается в собственной подзадаче. Ошибка в одном из них затрагивает только его — остальные активные соединения продолжают работу в штатном режиме. `IOException` в одном обработчике соединения не отменяет другие выполняющиеся соединения. Каждое соединение завершается в своем темпе.

Обработчик каждого соединения корректно закрывает свои ресурсы в блоке `finally`, что предотвращает их утечку даже при возникновении ошибок.

Запустим сервер и понаблюдаем за его отказоустойчивой работой:

```
java --enable-preview ResilientServer.java
```

Теперь подключимся к этому серверу с двух разных терминалов:

Терминал 1

```
telnet localhost 8080
Trying ::1...
Connected to localhost.
Escape character is '^]'.
Welcome to Echo Server! Type 'quit' to exit.
hello
Echo: hello
```

Терминал 2

```
telnet localhost 8080
Trying ::1...
Connected to localhost.
Escape character is '^]'.
Welcome to Echo Server! Type 'quit' to exit.
hello
Echo: hello
```

https://max.ru/bookofgeek

Вывод сервера:

```
06:16:05.611 main        : Server starting on port: 8080
06:16:10.631 main        : Accepted connection #1
06:16:10.641 VThread[#34]: [Conn-1] Started
06:16:23.630 VThread[#34]: [Conn-1] Received: hello
06:16:33.990 main        : Accepted connection #2
06:16:33.991 VThread[#40]: [Conn-2] Started
06:16:35.973 VThread[#40]: [Conn-2] Received: hello
06:16:37.549 VThread[#40]: [Conn-2] Received: quit
06:16:37.553 VThread[#40]: [Conn-2] Completed successfully
06:16:37.556 VThread[#40]: [Conn-2] Finished. Active: 1
06:16:41.799 VThread[#34]: [Conn-1] Received: quit
06:16:41.801 VThread[#34]: [Conn-1] Completed successfully
06:16:41.802 VThread[#34]: [Conn-1] Finished. Active: 0
```

При анализе вывода видно, что `awaitAll()` позволяет каждому подключению завершаться независимо и в своем темпе. В этом ключевое преимущество `awaitAll()` для серверных приложений: конкурентные подключения работают, не мешая друг другу, что делает сервер одновременно отказоустойчивым и эффективным.

После первого успеха. Метод `Joiner.allUntil(Predicate<Subtask> isDone)` позволяет задавать пользовательские условия останова. Создадим сервис резервного копирования, который останавливается сразу после успешного завершения операции копирования в любой локации.

Сначала определим `record` для результата и общее состояние:

```
public record BackupResult(String location, boolean success) {}
private final AtomicBoolean hasSuccess = new AtomicBoolean(false);
```

Теперь реализуем методы резервного копирования:

```
private BackupResult backupToCloud(String data) throws InterruptedException {
  log(" -> Backing up to cloud...");
  Thread.sleep(Duration.ofMillis(500));

  if(new Random().nextBoolean()) { ❶
    log(" <- Cloud backup successful");
    hasSuccess.set(true);
    return new BackupResult("Cloud", true);
  } else {
    log(" <- Cloud backup failed");
   return new BackupResult("Cloud", false);
  }
}

private BackupResult backupToUSB(String data) throws InterruptedException {
  log(" -> Backing up to USB...");
  Thread.sleep(Duration.ofMillis(300));
```

https://max.ru/bookofgeek

```
  if(new Random().nextBoolean()) { ❷
    log(" <- USB backup successful");
    hasSuccess.set(true);
    return new BackupResult("USB", true);
  } else {
    log(" <- USB backup failed");
    return new BackupResult("USB", false);
  }
}
private BackupResult backupToNetwork(String data) throws InterruptedException {
  log(" -> Backing up to network drive...");
  Thread.sleep(Duration.ofMillis(400));

  if (new Random().nextBoolean()) { ❸
    log(" <- Network backup successful");
    hasSuccess.set(true);
    return new BackupResult("Network", true);
  } else {
    log(" <- Network backup failed");
    return new BackupResult("Network", false);
  }
}
```

Каждый метод резервного копирования моделирует свой сценарий.

❶ Резервное копирование в облако имеет вероятность успеха 50 %.

❷ Резервное копирование на USB-накопитель — вероятность успеха 50 %.

❸ Сетевое резервное копирование — вероятность успеха 50 %.

Теперь реализуем оркестрацию резервного копирования с использованием `allUntil()`:

```
public void performBackup(String data) throws InterruptedException {
  log("Starting backup to multiple locations...");

  try (var scope = open(Joiner.<BackupResult>allUntil(subtask -> {
    boolean shouldStop = hasSuccess.get(); ❶
    if (shouldStop) {
      log("✓ Backup successful! Canceling other attempts...");
    }
    return shouldStop;
  }))) {

    scope.fork(() -> backupToCloud(data)); ❷
    scope.fork(() -> backupToUSB(data)); ❸
    scope.fork(() -> backupToNetwork(data)); ❹

    scope.join(); ❺

    if (hasSuccess.get()) {
      log("Backup completed successfully!");
```

https://max.ru/bookofgeek

```
    } else {
      log("All backup attempts failed!");
    }
  }
}
```

Эта оркестрация резервного копирования организует остановку при первом успехе.

❶ Предикат останавливает выполнение, как только любое резервное копирование завершается успешно.

❷ Выполняется попытка резервного копирования в облако.

❸ Выполняется попытка резервного копирования на USB-накопитель.

❹ Выполняется попытка сетевого резервного копирования.

❺ Ожидается либо первая успешная попытка, либо завершение всех попыток с ошибкой.

Полный демонстрационный класс:

```
import static java.util.concurrent.StructuredTaskScope.*;
public class BackupDemo {
  // ... (методы из предыдущего кода)

  void main() {
    try {
      performBackup("important-data.zip");
    } catch (Exception e) {
      log("Error: " + e.getMessage());
    }
  }
}
```

Обработка исключений в StructuredTaskScope

Обработка исключений в структурированной конкурентности следует четко определенным паттернам, которые зависят от текущей политики `Joiner`. `StructuredTaskScope` инициализируется с помощью `Joiner`, который управляет завершением подзадач и формирует результат для метода `join`. В одних случаях это будет возвращаемое значение, в других — исключение.

Если результатом является исключение, метод `join` выбрасывает `StructuredTaskScope.FailedException` с исходным исключением в качестве причины. Во многих реализациях `Joiner` это исключение будет из подзадачи, завершившейся с ошибкой. Например, для `allSuccessfulOrThrow()` и `awaitAllSuccessfulOrThrow()` исключение будет взято из первой подзадачи, потерпевшей неудачу.

https://max.ru/bookofgeek

Конкретные детали обработки исключений зависят от контекста использования. В некоторых случаях полезно добавить блок `catch` к `try-with-resources` для перехвата `FailedException`. В других ситуациях предпочтительнее позволить `FailedException` распространиться на более высокие уровни для централизованной обработки ошибок.

Рассмотрим различные стратегии обработки исключений на практических примерах, начиная с самого простого подхода и постепенно переходя к более сложным.

Простейший пример обработки исключений

Самый распространенный подход — обернуть код структурированной конкурентности в блок `try-catch`, который обрабатывает `FailedException`. Это позволяет проанализировать первопричину исключения и определить дальнейшие действия.

Рассмотрим сервис пользовательских данных, который получает информацию из нескольких источников. При сбое любого из них мы хотим предоставить осмысленное сообщение об ошибке, а не раскрывать внутренние детали системы:

```
import module java.base;
import static ca.bazlur.mcj.Utils.log;
import static java.util.concurrent.StructuredTaskScope.*;

public class BasicExceptionHandling {

  public String fetchUserData(String userId) {
    try (var scope = open(Joiner.<String>allSuccessfulOrThrow())) { ❶
      var profileTask = scope.fork(() -> fetchUserProfile(userId));
      var preferencesTask = scope.fork(() ->
        fetchUserPreferences(userId));

      var results = scope.join();

      // Обработка успешных результатов
      return results.map(Subtask::get)
        .collect(Collectors.joining(", "));

    } catch (FailedException e) { ❷
      log("Task failed: " + e.getCause().getMessage());
      return "Error: Unable to fetch user data";
    } catch (InterruptedException e) { ❸
      Thread.currentThread().interrupt();
      throw new RuntimeException("Operation interrupted", e);
    }
  }

  private String fetchUserProfile(String userId)
    throws InterruptedException { Thread.sleep(Duration.ofMillis(200));
```

https://max.ru/bookofgeek

```
    if ("invalid".equals(userId)) {
      throw new IllegalArgumentException("Invalid user ID");
    }
    return "Profile for " + userId;
  }

  private String fetchUserPreferences(String userId)
      throws InterruptedException {
    Thread.sleep(Duration.ofMillis(150));
    if (userId.startsWith("blocked")) {
      throw new SecurityException("User access blocked");
    }
    return "Preferences for " + userId;
  }

  void main() {
    var demo = new BasicExceptionHandling();
    demo.fetchUserData("invalid");
  }
}
```

Разберем этот код подробнее.

❶ Джоинер `allSuccessfulOrThrow()` гарантирует, что при сбое любой подзадачи вся операция завершится с `FailedException`.

❷ Для логирования используем `getCause()`, чтобы получить доступ к исходному исключению, обернутому в `FailedException`.

❸ Всегда восстанавливайте статус прерывания при обработке `InterruptedException`.

В этом примере `allSuccessfulOrThrow()` используется потому, что для формирования осмысленных данных нужен не только профиль пользователя, но и его предпочтения. Если какая-либо подзадача завершится ошибкой, вся операция должна прерваться, но при этом мы показываем пользователю понятное сообщение об ошибке, а не технические детали.

Сопоставление с образцом для сложной обработки ошибок

Когда требуется обрабатывать разные типы исключений с индивидуальными стратегиями восстановления, сопоставление с образцом (pattern matching) предлагает изящное решение. Этот подход позволяет настраивать реакцию соответственно типу сбоя.

Рассмотрим практический пример, знакомый каждому разработчику E-commerce приложений: систему обработки заказов. При оформлении заказа должны успешно выполниться несколько операций: обработка платежа, проверка наличия товара и расчет стоимости доставки. Каждая из них может завершиться различными ошибками, и мы должны обеспечить качественную обратную связь, чтобы помочь клиенту понять причину проблемы.

https://max.ru/bookofgeek

При традиционном подходе обычно перехватывается общее исключение и возвращается расплывчатое сообщение типа «что-то пошло не так». В структурированной конкурентности с помощью сопоставления с образцом мы можем предложить точные, полезные ответы, которые подскажут клиентам путь к решению проблемы.

Рассмотрим следующий фрагмент кода:

```
import module java.base;
import static java.util.concurrent.StructuredTaskScope.*;
import static java.util.concurrent.StructuredTaskScope.open;

public class OrderProcessingService {
  public record OrderResult(String orderId,
                            String status,
                            String message,
                            boolean successful) {
  }
  public OrderResult processOrder(String customerId,
                                  String productId, double amount) {
    try (var scope = open(Joiner.<String>allSuccessfulOrThrow())) { ❶
      var paymentTask = scope.fork(() ->
        processPayment(customerId, amount));
      var inventoryTask = scope.fork(() ->
        checkAndReserveInventory(productId));
      var shippingTask = scope.fork(() ->
        calculateShipping(customerId, productId));
      var results = scope.join()
        .map(Subtask::get)
        .toList();
      String orderId = generateOrderId();
      return new OrderResult(orderId, "CONFIRMED",
        "Order confirmed successfully", true);
    } catch (FailedException e) {
      Throwable cause = e.getCause(); ❷
      return handleOrderProcessingError(cause);
    } catch (InterruptedException e) {
      Thread.currentThread().interrupt();
      throw new RuntimeException("Operation interrupted", e);
    }
  }

  private static OrderResult handleOrderProcessingError(Throwable cause) {
    return switch (cause) { ❸
      case PaymentDeclinedException pde -> new OrderResult(null,
        "PAYMENT_FAILED",
         """
         Ваш платеж был отклонен. Пожалуйста, проверьте
         данные карты или попробуйте другой способ оплаты.""",
         false);
       case InsufficientInventoryException iie ->
         new OrderResult(null, "OUT_OF_STOCK",
```

https://max.ru/bookofgeek

```
            """
            Извините, данный товар временно отсутствует на складе.
            Мы уведомим вас, когда он появится в наличии.""",
            false);
        case ShippingNotAvailableException snae ->
            new OrderResult(null, "SHIPPING_UNAVAILABLE",
            """
            Сейчас мы не можем доставить заказ по вашему адресу.
            Пожалуйста, обратитесь в службу поддержки по поводу
            альтернативных вариантов.""",
            false);
        case NetworkException ne ->
            new OrderResult(null, "TEMPORARY_ERROR",
            """
            Возникли технические неполадки.
            Пожалуйста, повторите попытку через несколько минут.""", false);
        case SecurityException se ->
            new OrderResult(null, "SECURITY_CHECK_FAILED",
            """
            Требуется дополнительная проверка.
            Пожалуйста, обратитесь в службу поддержки.""", false);
        default -> ❹
            new OrderResult(null, "SYSTEM_ERROR",
            """
            Произошла системная ошибка. \
            Пожалуйста, повторите попытку или обратитесь в поддержку.""",
            false);
    };
  }
}
```

Рассмотрим ключевые моменты, делающие этот подход к обработке исключений эффективным.

❶ Все три операции должны пройти успешно для выполнения обработки заказа; сбой любой из них прерывает обработку заказа с `FailedException`.

❷ Извлекаем исходное исключение из обертки `FailedException`, чтобы точно определить тип сбоя.

❸ Через сравнение с образцом каждый тип бизнес-исключения связывается с соответствующим понятным клиенту сообщением об ошибке.

❹ Блок `default` предоставляет универсальное сообщение о системной ошибке для непредвиденных ситуаций.

Красота этого подхода заключается в использовании выражения `switch` для анализа причины сбоя. Каждая секция `case` рассматривает конкретный бизнес-сценарий с соответствующим сообщением для клиента. Обратите внимание на метод `allSuccessfulOrThrow()` — обработка заказа работает по принципу all-or-nothing («всё или ничего»): при сбое любого этапа весь заказ отклоняется, но с конкретным указанием причины ошибки.

https://max.ru/bookofgeek

Объединим и выполним отдельные операции, составляющие систему обработки заказов. Хотя это лишь плейсхолдеры методов, в реальном приложении может встречаться аналогичная функциональность:

```
private String processPayment(String customerId, double amount)
        throws PaymentDeclinedException, NetworkException,
                    SecurityException, InterruptedException {
  Thread.sleep(Duration.ofMillis(200));
  // Имитация различных сценариев оплаты
  if (amount > 5000.0) {
    throw new SecurityException("High-value transaction requires " +
      "additional verification");
  }
  if (customerId.contains("declined")) {
    throw new PaymentDeclinedException("Insufficient funds");
  }
  if (customerId.contains("network")) {
    throw new NetworkException("Payment gateway timeout");
  }
  return "Payment processed: $" + amount;
}

private String checkAndReserveInventory(String productId)
               throws InsufficientInventoryException, NetworkException,
                                    InterruptedException {
  Thread.sleep(Duration.ofMillis(150));
  if(productId.contains("outofstock")) {
    throw new InsufficientInventoryException(
      "Only 0 items available, requested 1");
  }
  if (productId.contains("network")) {
    throw new NetworkException("Inventory service unavailable");
  }
  return "Reserved inventory for " + productId;
}

private String calculateShipping(String customerId, String productId)
               throws ShippingNotAvailableException, NetworkException,
                                      InterruptedException {
  Thread.sleep(Duration.ofMillis(100));
  if (customerId.contains("remote")) {
    throw new ShippingNotAvailableException(
      "No shipping available to remote location");
  }
  if(productId.contains("hazardous")) {
    throw new ShippingNotAvailableException(
      "Cannot ship hazardous materials to this address");
  }
  if (customerId.contains("network")) {
    throw new NetworkException("Shipping service unavailable");
  }
  return "Shipping calculated: $12.99";
}
```

https://max.ru/bookofgeek

```
private String generateOrderId() {
  return "ORD-" + System.currentTimeMillis();
}
```

Каждый метод моделирует реальные сценарии, с которыми разработчики E-commerce приложений сталкиваются ежедневно. Обработка платежей может завершиться ошибкой из-за недостатка средств или блокировки по соображениям безопасности. Проверка наличия товара позволяет выявить его отсутствие на складе. При расчете стоимости доставки могут обнаружиться географические или регуляторные ограничения.

Мы определяем свои типы исключений, которые представляют эти конкретные бизнес-сценарии:

```
public static class PaymentDeclinedException extends Exception {
  public PaymentDeclinedException(String message) {
    super(message);
  }
}

public static class InsufficientInventoryException extends Exception {
  public InsufficientInventoryException(String message) {
    super(message);
  }
}

public static class ShippingNotAvailableException extends Exception {
  public ShippingNotAvailableException(String message) {
    super(message);
  }
}

public static class NetworkException extends Exception {
  public NetworkException(String message) {
    super(message);
  }
}
```



Рекомендуется создавать предметно-ориентированные типы исключений, отражающие бизнес-сценарии, а не технические сбои. При таком подходе код проще поддерживать, а обработка ошибок становится более осмысленной.

Теперь протестируем это в методе `main()`:

```
void main() {
  var demo = new OrderProcessingService();
  var result1 = demo.processOrder("user123", "outofstock", 3500);
  System.out.println("Out of stock scenario: " + result1);
  var result2 = demo.processOrder("declined_user", "laptop", 1200);
  System.out.println("Payment declined scenario: " + result2);
```

https://max.ru/bookofgeek

```
  var result3 = demo.processOrder("remote_customer", "book", 25);
  System.out.println("Shipping unavailable scenario: " + result3);
  var result4 = demo.processOrder("user456", "laptop", 1200);
  System.out.println("Successful order: " + result4);
}
```

При запуске примера мы увидим, что различные сценарии генерируют конкретные, понятные пользователю сообщения об ошибках. `FailedException` предоставляет доступ к первопричине, а сопоставление с образцом позволяет связать каждое бизнес-исключение с соответствующим сообщением для клиента.

Стратегическое распространение исключений

Иногда лучше всего передавать исключения на более высокие уровни, где их можно обработать с учетом расширенного контекста. Такой подход особенно полезен при разработке многоуровневых приложений, где каждый слой выполняет различные функции.

Рассмотрим пример критически важного сервиса обработки данных, где любой сбой должен активировать оповещения и процедуры отката во всей системе:

```
import module java.base;
import static java.util.concurrent.StructuredTaskScope.open;

public class ExceptionPropagationExample {
  // Разрешаем распространение исключений для централизованной обработки
  public List<String> fetchCriticalData(List<String> sources)
         throws StructuredTaskScope.FailedException,
         InterruptedException { ❶

    try (var scope = open(StructuredTaskScope.Joiner.
      <String>allSuccessfulOrThrow())) {

      var tasks = sources.stream()
        .map(source -> scope.fork(()
        -> fetchFromSource(source)))
        .toList();

      // При сбое любого источника разрешаем распространение
      // FailedException
      var results = scope.join();

      return results.map(StructuredTaskScope.Subtask::get)
        .toList();
    }
    // Блок catch отсутствует — разрешаем распространение FailedException ❷
  }
  // Метод верхнего уровня с централизованной обработкой исключений
  public void processDataWithCentralizedHandling() {
    try {
      var sources = List.of("source1", "source2");
```

https://max.ru/bookofgeek

```
      var data = fetchCriticalData(sources);
      log("Successfully fetched data: " + data);
    } catch (StructuredTaskScope.FailedException e) { ❸
      // Централизованное логирование и обработка ошибок
      log("Critical data fetch failed: " +
        e.getCause().getMessage());

      // Может включать оповещения, процедуры отката и т. д.
      handleCriticalSystemFailure(e);

    } catch (InterruptedException e) {
      Thread.currentThread().interrupt();
      log("Operation was interrupted");
    }
  }

  private String fetchFromSource(String source)
    throws InterruptedException {
      // Метод-заглушка
      return "Data from " + source;
    }
  }

  private void handleCriticalSystemFailure(FailedException e) {
    Throwable cause = e.getCause();
    // Методы-заглушки, можно добавить собственные реализации
  }
}
```

Вот как работает стратегия распространения исключений в данном примере.

❶ Сигнатура метода явно объявляет, что он может выбрасывать как `FailedException`, так и `InterruptedException`, что дает сигнал вызывающим методам: эти исключения необходимо обрабатывать.

❷ Намеренное отсутствие блока `catch` позволяет исключениям распространяться вверх по стеку вызовов до места, где их можно обработать с более широким контекстом.

❸ Метод более высокого уровня предоставляет централизованную обработку исключений, которая может включать комплексные ответные меры на ошибки (например, логирование, оповещение и процедуры отката).

Разрешая распространение `FailedException` из `fetchCriticalData()`, мы позволяем вызывающему методу реализовать комплексную обработку ошибок, которая может включать логирование, оповещение и активацию систем отката.

Рекомендация: используйте распространение исключений, если методы нижнего уровня не обладают достаточным контекстом для адекватной обработки ошибок. Позвольте исключениям подниматься до компонентов, которые могут принимать обоснованные решения о стратегиях восстановления.

https://max.ru/bookofgeek



При распространении проверяемых исключений убедитесь, что промежуточные слои корректно объявляют выбрасываемые исключения в сигнатурах своих методов, чтобы поддерживать безопасность на этапе компиляции.

Обработка исключений внутри подзадач

Если определенные исключения должны давать результат по умолчанию, а не сбой всей операции, как правило, лучше обрабатывать их непосредственно в самой подзадаче. Особенно хорошо это работает с опциональными источниками данных, поскольку в таком случае частичный сбой не препятствует успешному выполнению всей операции.

Представьте себе дашборд-сервис, который собирает данные из многих опциональных источников. Если некоторые из них недоступны, мы все равно будем выводить данные, которые удалось получить.

```
import module java.base;
import static ca.bazlur.mcj.chap4.Utils.log;
import static java.util.concurrent.StructuredTaskScope.*;

public class SubtaskExceptionHandling {
  public record ServiceResponse(String service, String data,
                                boolean successful) {
  }
  public List<ServiceResponse> gatherOptionalData(List<String> services)
         throws InterruptedException {
    try (var scope
        = open(Joiner.<ServiceResponse>allSuccessfulOrThrow())) { ❶
      var tasks = services.stream()
        .map(service -> scope.fork(() ->
        fetchWithDefaults(service))) ❷
        .toList();

      var results = scope.join();
        return results.map(Subtask::get)
        .toList();
    } catch (FailedException e) { ❸
      // Это должно происходить редко, так как мы обрабатываем
      // исключения в подзадачах
      log("Unexpected failure: " + e.getCause().getMessage());
      throw new RuntimeException("System error", e);
    }
  }

  // Обрабатываем исключения внутри подзадачи,
  // чтобы предоставлять значения по умолчанию
  private ServiceResponse fetchWithDefaults(String service) { ❹
    try {
      String data = fetchServiceData(service);
      return new ServiceResponse(service, data, true);
```

https://max.ru/bookofgeek

```
    } catch (IOException e) {
      log("Network error for " + service + ": " + e.getMessage());
      return new ServiceResponse(service, "Default data", false);
    } catch (TimeoutException e) {
      log("Timeout for " + service + ": " + e.getMessage());
      return new ServiceResponse(service, "Cached data", false);
    } catch (Exception e) {
      log("Unexpected error for " + service + ": " +
        e.getMessage());
      return new ServiceResponse(service, "Error", false);
    }
  }

  private String fetchServiceData(String service)
          throws IOException, TimeoutException, InterruptedException {
    Thread.sleep(Duration.ofMillis(100));
    return "Data from " + service;
  }
}
```

Рассмотрим, как данная стратегия обработки исключений обеспечивает постепенную деградацию (graceful degradation).

❶ При использовании `allSuccessfulOrThrow()` с подзадачами, которые обрабатывают собственные исключения, `FailedException` будет выбрасываться редко.

❷ Каждый сервис получает данные в отдельной подзадаче, которая обрабатывает исключения внутри и прибегает к резервным ответам.

❸ Блок `catch` обрабатывает непредвиденные сбои, поскольку ошибки отдельных сервисов обрабатываются внутри подзадач.

❹ Обработка исключений в подзадачах преобразует сбои в успешные ответы с данными по умолчанию, предотвращая каскад ошибок при отказах отдельных сервисов.

При обработке исключений внутри `fetchWithDefaults()` сетевые сбои или тайм-ауты отдельных сервисов не помешают загрузке данных всего дашборда. Каждая попытка обращения к сервису либо возвращает реальные данные, либо в случае ошибки предоставляются данные по умолчанию или из кэша.

Рекомендация: если требуется реализовать graceful degradation, обрабатывайте исключения внутри подзадач. Этот способ отлично подходит для опциональных операций, в которых допустим частичный успех.



Когда подзадачи обрабатывают собственные исключения, внешний для `FailedException` блок `catch` будет выполняться редко. Рекомендуется добавить логирование на случай его срабатывания, поскольку так можно выявить непредвиденную системную проблему.

https://max.ru/bookofgeek

Обработка исключений разными джоинерами

От того, какой джоинер вы выбрали, во многом зависит обработка исключений. Сравним поведение `allSuccessfulOrThrow()` и `awaitAll()` при возникновении ошибок.

Джоинер `allSuccessfulOrThrow()` реализует стратегию fail-fast, при которой любая единичная ошибка приводит к немедленному завершению всей операции:

```
private void demonstrateAllSuccessfulOrThrow() {
  try (var scope = open(StructuredTaskScope.Joiner.
       <String>allSuccessfulOrThrow())) { ❶
    scope.fork(() -> successfulTask("Task1"));
    scope.fork(() -> failingTask("Task2")); ❷
    scope.fork(() -> successfulTask("Task3"));
    var results = scope.join(); ❸
    log("All tasks completed successfully");
  } catch (StructuredTaskScope.FailedException e) { ❹
    log("Failed due to: " + e.getCause().getMessage());
    log("Remaining tasks were canceled");
  } catch (InterruptedException e) {
    Thread.currentThread().interrupt();
  }
}
```

Рассмотрим основные характеристики подхода `allSuccessfulOrThrow()`.

❶ Создаем область, где все задачи должны завершиться успешно, иначе вся операция завершится ошибкой.

❷ При возникновении исключения в этой задаче активируется стратегия fail-fast.

❸ Вызов `join()` выбросит `FailedException` из-за неудачи `Task2`, при этом `Task3` будет отменена.

❹ Обработчик исключений перехватывает ошибку и логирует факт отмены оставшихся задач.

Джоинер `awaitAll()` использует иной подход, позволяя всем задачам завершиться независимо от отдельных сбоев:

```
private void demonstrateAwaitAll() {
  try (var scope = open(StructuredTaskScope.Joiner.
       <Void>awaitAll())) { ❶
    scope.fork(() -> successfulTask("Task1"));
    scope.fork(() -> failingTask("Task2")); ❷
    scope.fork(() -> successfulTask("Task3"));
    scope.join(); ❸
    log("All tasks were allowed to complete");
  } catch (InterruptedException e) {
    Thread.currentThread().interrupt();
  }
}
```

https://max.ru/bookofgeek

Вот основные характеристики подхода `awaitAll()`.

❶ Джоинер `awaitAll()` ожидает завершения всех задач, независимо от возникновения индивидуальных сбоев.

❷ `Task2` по-прежнему будет выбрасывать исключение, но это не помешает выполнению остальных задач.

❸ Метод `join()` никогда не выбрасывает `FailedException` при использовании `awaitAll()`, он ожидает окончания выполнения всех задач, даже если некоторые завершаются с ошибкой.

Джоинер `allSuccessfulOrThrow()` реализует стратегию fail-fast: если `Task2` терпит неудачу, он немедленно отменяет оставшиеся задачи и выбрасывает `FailedException`. В отличие от него, `awaitAll()` позволяет всем задачам завершиться, независимо от индивидуальных сбоев. Поэтому, если требуется собрать как успешные результаты, так и ошибки, исключения нужно обрабатывать внутри каждой подзадачи.

Общие исключения

Передача `null` в качестве аргумента методу класса приведет к выбросу `NullPointerException`, если нет иных указаний. Такое единообразное поведение помогает выявлять ошибки программирования на ранних этапах и делает API более предсказуемым.

Помимо проверок на `null`, API выбрасывает для различных ошибочных ситуаций соответствующие им исключения, что упрощает обработку проблем.

Рассмотрим следующий пример:

```
import java.util.concurrent.Callable;
import java.util.concurrent.StructuredTaskScope;
import static ca.bazlur.mcj.chap4.Utils.log;
import static java.util.concurrent.StructuredTaskScope.open;

public class ExceptionBehaviorDemo {
  public void demonstrateCommonExceptions() {
    try {
      log("Testing null joiner...");
      try (var scope = open(null)) { ❶
        log("This should not be reached");
      }
    } catch (NullPointerException e) {
      log("NullPointerException caught for null joiner");
    }

    try {
      Callable<? extends String> nullCallable = null; ❷
      try (var scope = open(StructuredTaskScope.Joiner.
          <String>allSuccessfulOrThrow())) {
        scope.fork(nullCallable); ❸
      }
```

https://max.ru/bookofgeek

```
    } catch (NullPointerException e) {
      log("NullPointerException caught for null callable");
    }

    try {
      var scope = open(StructuredTaskScope.Joiner.
        <String>allSuccessfulOrThrow());
      scope.close(); ❹
      scope.fork(() -> "This should fail"); ❺
    } catch (IllegalStateException e) {
      log("IllegalStateException caught for closed scope operation");
    }
  }
  void main() {
    var demo = new ExceptionBehaviorDemo();
    demo.demonstrateCommonExceptions();
  }
}
```

Рассмотрим сценарий каждого исключения.

❶ При попытке открыть область с джоинером со значением `null` немедленно выбрасывается `NullPointerException`, что предотвращает создание области с недопустимой конфигурацией.

❷ Создаем Callable-переменную со значением `null`, чтобы продемонстрировать поведение области при передаче `null`-задач.

❸ При вызове `fork()` с `nullCallable` выбрасывается `NullPointerException`. Это гарантирует, что для выполнения могут быть отправлены только валидные задачи.

❹ Закрываем область вручную перед завершением блока `try-with-resources`.

❺ При попытке запустить задачу в закрытой области выбрасывается `IllegalStateException`, предотвращая операции с недействительными экземплярами областей.

Запустим приведенную выше программу:

```
java --enable-preview ExceptionBehaviorDemo.java
```

Вывод будет следующим:

```
23:14:21.314 main    : NullPointerException caught for null joiner
23:14:21.317 main    : NullPointerException caught for null callable
23:14:21.317 main    : IllegalStateException caught for closed scope operation
```

Конфигурация

Хотя стандартная конфигурация интерфейса `StructuredTaskScope` подходит для многих сценариев, в реальных приложениях часто требуется более тонкий контроль над выполнением конкурентных задач. Конфигурирование позволяет

https://max.ru/bookofgeek

кастомизировать создание потоков, добавлять мониторинг и реализовывать политики тайм-аутов в соответствии с требованиями приложения.

Основы конфигурации

При создании `StructuredTaskScope` через простые методы `open()` или `open(Joiner)` мы получаем разумную конфигурацию по умолчанию:

- `ThreadFactory` создает безымянные виртуальные потоки;
- имя для мониторинга не задано (анонимная область);
- тайм-аут отсутствует (задачи выполняются до завершения или отмены).

```
// Используем конфигурацию по умолчанию
try (var scope = open(Joiner.allSuccessfulOrThrow())) { ❶
  // Виртуальные потоки создаются со стандартными настройками
  scope.fork(() -> doWork());
  scope.join();
}
```

❶ Данная конфигурация по умолчанию создает анонимные виртуальные потоки без ограничения времени выполнения.

Однако в продакшен-приложениях обычно требуется управление с расширенными возможностями. Configuration API предоставляет перегруженный метод `open()`, который принимает функцию конфигурации:

```
static <T, R> StructuredTaskScope<T, R>
       open(Joiner<? super T, ? extends R> joiner,
       Function<Configuration, Configuration> configFunction) {
  return StructuredTaskScopeImpl.open(joiner, configFunction);
}
```

`Configuration` является sealed-интерфейсом и предоставляет три метода конфигурации, каждый из которых возвращает новый объект `Configuration` с указанными модификациями:

```
sealed interface Configuration {
  Configuration withThreadFactory(ThreadFactory threadFactory);
  Configuration withName(String name);
  Configuration withTimeout(Duration timeout);
}
```

Конфигурация следует неизменяемому паттерну Строитель — каждый метод возвращает новый объект `Configuration`, а не изменяет существующий. Это позволяет объединять несколько вызовов конфигурации в цепочку в стиле fluent.

Основная идея заключается в том, что функция конфигурации преобразует конфигурацию, используемую по умолчанию, применяя одну или несколько модификаций.

https://max.ru/bookofgeek

Вот базовый шаблон:

```
ThreadFactory factory = Thread.ofVirtual()
  .name("user-processor-", 0)
  .factory();
try (var scope = open(Joiner.allSuccessfulOrThrow(), cf -> cf
  .withThreadFactory(factory)
  .withTimeout(Duration.ofSeconds(30))
  .withName("my-scope"))) {
  // Продолжайте со своим кодом
}
```

Именованные потоки

Для улучшения отладки и мониторинга чаще всего применяется создание именованных потоков. Именование потоков значительно облегчает идентификацию выполняемых задач в дампах потоков и профилировщиках.

Рассмотрим следующие фрагменты кода:

```
import module java.base;
import static java.util.concurrent.StructuredTaskScope.*;

public class NamedThreadExample {
  public void processUserRequests(List<String> userIds) {
    ThreadFactory factory = Thread.ofVirtual()
      .name("user-processor-", 0)
      .factory();
    try (var scope = open(Joiner.<String>allSuccessfulOrThrow(),
           cf -> cf.withThreadFactory(factory))) {
      var tasks = userIds.stream()
        .map(userId -> scope.fork(() -> processUser(userId)))
        .toList();
      var results = scope.join()
        .map(Subtask::get)
        .toList();
      System.out.println("Processed users: " + results);
    } catch (FailedException | InterruptedException e) {
      System.out.println("Processing failed: " + e.getMessage());
    }
  }

  private String processUser(String userId)
      throws InterruptedException {
    System.out.println("Processing user " + userId +
      " on thread: " + Thread.currentThread().getName());
    Thread.sleep(Duration.ofMillis(100));
    return "User " + userId + " processed";
  }
```

https://max.ru/bookofgeek

```
  void main() {
    processUserRequests(List.of("user1", "user2", "user3"));
  }
}
```

После запуска этого кода мы увидим:

```
Processing user user2 on thread: user-processor-1
Processing user user3 on thread: user-processor-2
Processing user user1 on thread: user-processor-0
```

Настройка тайм-аута

Тайм-ауты критически важны для создания отказоустойчивых приложений. Они не допускают бесконечного зависания операций и позволяют приложению быстро завершаться с ошибкой, если сервисы недоступны.

Рассмотрим следующий фрагмент кода:

```
import module java.base;
import static java.util.concurrent.StructuredTaskScope.*;

public class TimeoutExample {
  public List<String> fetchDataWithTimeout(List<String> sources)
         throws TimeoutException,
         FailedException,
         InterruptedException {
    Duration timeout = Duration.ofSeconds(5); ❶
    try (var scope = open(Joiner.<String>allSuccessfulOrThrow(),
         cf -> cf.withTimeout(timeout))) { ❷
      var tasks = sources.stream()
        .map(source -> scope.fork(() -> fetchFromSource(source)))
        .toList();
      // Если тайм-аут истекает до завершения join(),
      // выбрасывается TimeoutException
      return scope.join() ❸
        .map(Subtask::get)
        .toList();
    }
  }

  private String fetchFromSource(String source) throws InterruptedException {
    int delay = switch (source) {
      case "fast_source" -> 1000;
      case "slow_source" -> 3000;
      case "very_slow_source" -> 8000; ❹
      default -> 2000;
    };
    Thread.sleep(Duration.ofMillis(delay));
    return "Data from " + source;
  }
```

https://max.ru/bookofgeek

```
  void main() {
    var sources = List.of("fast_source", "slow_source", "very_slow_source");
    try {
      var results = fetchDataWithTimeout(sources)
      System.out.println("Success: " + results);
    } catch (TimeoutException e) { ❺
      System.out.println("Operation timed out after 5 seconds");
      System.out.println("Some sources were too slow to respond");
    } catch (FailedException e) {
      System.out.println("Task failed: " + e.getCause().getMessage());
    } catch (InterruptedException e) {
      Thread.currentThread().interrupt();
      System.out.println("Operation interrupted");
    }
  }
}
```

Рассмотрим, как работает механизм тайм-аута в данном примере.

❶ Устанавливаем пятисекундный тайм-аут для всей операции.

❷ Настраиваем область с этим тайм-аутом — отсчет начинается при ее открытии.

❸ Если какая-либо подзадача не завершится в течение 5 с, метод `join()` выбросит `TimeoutException`.

❹ Этот источник данных требует 8 с, что превышает пятисекундный тайм-аут и вызовет его срабатывание.

❺ Перехватываем и обрабатываем исключение тайм-аута с понятными пользователю сообщениями об ошибке.

При выполнении этого кода вывод будет примерно таким:

```
Operation timed out after 5 seconds
Some sources were too slow to respond
```



Тайм-аут начинает отсчет с момента открытия области, а не при вызове `join()`, то есть охватывает весь жизненный цикл области.

Таким образом, тайм-ауты обеспечивают поведение fail-fast при превышении времени операций, предотвращая бесконечное зависание приложения.

Комбинирование опций конфигурации

Самое мощное преимущество конфигурации — это возможность комбинировать различные варианты для точной настройки среды выполнения приложения.

https://max.ru/bookofgeek

Рассмотрим следующий пример кода:

```
import module java.base;
import static java.util.concurrent.StructuredTaskScope.*;

public class ComprehensiveConfigurationExample {
  public void main() {
    ThreadFactory factory = Thread.ofVirtual() ❶
      .name("api-client-", 0) ❷
      .factory(); ❸

    Duration timeout = Duration.ofSeconds(15); ❹

    try (var scope = open(Joiner.<String>allSuccessfulOrThrow(),
      cf -> cf.withThreadFactory(factory) ❺
        .withTimeout(timeout) ❻
        .withName("api-integration-scope"))) { ❼

      System.out.println("Starting comprehensive API integration");
      System.out.println("Timeout: " + timeout.getSeconds() + " seconds");

      // Запускаем несколько конкурентных вызовов API
      var userTask = scope.fork(this::callUserAPI);
      var profileTask = scope.fork(this::callProfileAPI);
      var preferencesTask = scope.fork(this::callPreferencesAPI);

      var results = scope.join()
        .map(Subtask::get)
        .toList();

      System.out.println("All API calls completed: " + results);

    } catch (StructuredTaskScope.TimeoutException e) {
      System.out.println("API integration timed out - some services too slow");

    } catch (StructuredTaskScope.FailedException e) {
      System.out.println("API integration failed: " + e.getCause().getMessage());

    } catch (InterruptedException e) {
      Thread.currentThread().interrupt();
      System.out.println("API integration interrupted");
    }
  }

  private String callUserAPI() throws InterruptedException {
    Thread.sleep(Duration.ofSeconds(3));
    IO.println(Thread.currentThread().getName()
      + " : " + "User data retrieved");
    return "User data";
  }
```

https://max.ru/bookofgeek

```
  private String callProfileAPI() throws InterruptedException {
    Thread.sleep(Duration.ofSeconds(2));
    IO.println(Thread.currentThread().getName()
      + " : " + "Profile data retrieved");
    return "Profile data";
  }

  private String callPreferencesAPI() throws InterruptedException {
    Thread.sleep(Duration.ofSeconds(4));
    IO.println(Thread.currentThread().getName()
      + " : " + "Preferences data retrieved");
    return "Preferences data";
  }
}
```

Рассмотрим эту конфигурацию пошагово.

❶ Создаем строитель (builder) виртуальных потоков — легковесных и идеально подходящих для операций, связанных с вводом-выводом, таких как вызовы API.

❷ Настраиваем шаблон именования потоков вида `api-client-0`, `api-client-1`, `api-client-2` и т. д., что упрощает их идентификацию в инструментах отладки.

❸ Преобразуем сконфигурированный строитель потоков в `ThreadFactory`, которую область может использовать для создания потоков.

❹ Устанавливаем 15-секундный тайм-аут, чтобы обеспечить быстрое завершение (fail-fast) операции в случае, если не ответит любой из API.

❺ Конфигурируем область, чтобы использовать нашу кастомную `ThreadFactory` для создания рабочих потоков.

❻ Применяем тайм-аут к области, запуская отсчет при ее открытии.

❼ Присваиваем области описательное имя для облегчения мониторинга и управления, упрощая идентификацию в логах и инструментах мониторинга.

Кастомные джоинеры

Хотя встроенные джоинеры, такие как `allSuccessfulOrThrow()` и `awaitAll()`, — надежная основа для координации задач и управления результатами, структурированная конкурентность на них не останавливается. Она позволяет разработчикам создавать собственные джоинеры под конкретные потребности приложения. Такая гибкость открывает совершенно новые возможности для оркестрации конкурентных рабочих процессов с точным контролем над подзадачами и сбором результатов.

Собственные джоинеры можно создать, реализовав интерфейс `StructuredTaskScope.Joiner<T, R>`. Он предоставляет методы для обработки событий

https://max.ru/bookofgeek

завершения подзадач, а также определяет, когда область должна завершиться и какой результат должна вернуть.

Каждый джоинер реализует интерфейс `StructuredTaskScope.Joiner<T, R>`, где `T` — тип значений, возвращаемых подзадачами, а `R` — тип итогового результата. Интерфейс определяет три ключевых метода.

`onFork(Subtask<? extends T> subtask)`
: Вызывается каждый раз при ветвлении подзадачи, перед созданием потока для ее выполнения.

`onComplete(Subtask<? extends T> subtask)`
: Вызывается при завершении каждой подзадачи, позволяет обработать результат и принять решение: продолжить выполнение или завершить его досрочно.

`result()`
: Вызывается, когда область готова вернуть итоговый результат.

Далее мы рассмотрим практические примеры, демонстрирующие возможности и гибкость кастомных джоинеров.

Сбор всех результатов и исключений

Рассмотрим сценарий, в котором нужно собрать результаты всех подзадач — как успешных, так и неудачных, — а также все возникающие исключения. Такой подход полезен при пакетной обработке, когда требуется максимально полная информация об успешных и провальных операциях:

```
import module java.base;

public class CollectingJoiner<T>
      implements StructuredTaskScope.Joiner<T, CollectingJoiner.Result<T>> {
  private final Queue<T> results = new ConcurrentLinkedQueue<>();
  private final Queue<Throwable> exceptions = new ConcurrentLinkedQueue<>();

  @Override
  public Result<T> result() {
    return new Result<>(
      results.stream().toList(),
      exceptions.stream().toList()
    );
  }

  @Override
  public boolean onComplete(StructuredTaskScope.Subtask<? extends T> subtask) {
    switch (subtask.state()) {
      case SUCCESS -> results.add(subtask.get()); ❶
      case FAILED -> exceptions.add(subtask.exception()); ❷
```

https://max.ru/bookofgeek

```
      case UNAVAILABLE -> { ❸
        // Задача была отменена, обрабатываем как сбой
        exceptions.add(new RuntimeException("Task was canceled"));
      }
    }
    return false; ❹
  }

  public record Result<T>(List<T> successes, List<Throwable> failures) {
    public boolean hasFailures() {
      return !failures.isEmpty();
    }
    public int totalTasks() {
      return successes.size() + failures.size();
    }
  }
}
```

Рассмотрим, как этот джоинер обрабатывает различные варианты выполнения подзадач.

❶ При успешном завершении подзадачи мы добавляем ее результат в очередь результатов для последующего извлечения.

❷ При неудаче подзадачи и выбрасывании исключения мы берем его для анализа вместо немедленного прерывания всей операции.

❸ Задачи в состоянии `UNAVAILABLE` были отменены до завершения. Мы рассматриваем отмену как ошибку и создаем описательное исключение.

❹ Возврат `false` позволяет оставшимся подзадачам продолжать выполнение до их естественного завершения.

Теперь используем этот джоинер для создания сервиса, который получает заголовки из различных онлайн-источников, стремясь извлечь максимум данных даже при недоступности части из них.

```
import module java.base;
import static ca.bazlur.mcj.chap4.Utils.log;
import static java.util.concurrent.StructuredTaskScope.open;

public class NewsAggregator {
  public CollectingJoiner.Result<String> fetchAllHeadlines()
      throws InterruptedException {
    var newsSources = List.of(
      "TechCrunch",
      "InfoWorld",
      "InfoQ",
      "FailingSource"
    );

    try (var scope = open(new CollectingJoiner<String>())) {
      for (String source : newsSources) {
```

https://max.ru/bookofgeek

```
        scope.fork(() -> fetchHeadlines(source)); ❶
      }
      var result = scope.join(); ❷
      log("Successfully fetched from " + result.successes().size() +
        " sources");
      log("Failed to fetch from " + result.failures().size() + " sources");
      return result; ❸
    }
  }

  private String fetchHeadlines(String source) throws InterruptedException {
    log("Fetching headlines from " + source);
    Thread.sleep(Duration.ofMillis(200 + new Random().nextInt(300)));
    if (source.equals("FailingSource")) { ❹
      throw new RuntimeException("Network timeout for " + source);
    }
    return "Headlines from " + source + ": Breaking news, Tech updates";
  }

  void main() {
    var aggregator = new NewsAggregator();
    try {
      var result = scope.join();
      log("\n=== Results ===");
      result.successes() ❺
        .forEach(headline -> log(headline));
      if (result.hasFailures()) { ❻
        log("\n=== Failures ===");
        result.failures()
          .forEach(error ->
          log(error.getMessage()));
      }
    } catch (InterruptedException e) {
      Thread.currentThread().interrupt();
      log("Operation interrupted");
    }
  }
}
```

Пример показывает, для чего собирать как успешные результаты, так и неудачи.

❶ Для каждого источника новостей запускается подзадача, что позволяет им выполняться параллельно.

❷ `CollectingJoiner` ожидает завершения всех подзадач, собирая как полученные заголовки, так и возникшие ошибки.

❸ Возвращаем результат, содержащий как полученные данные, так и информацию о неудачных попытках.

❹ Один из источников мы намеренно запрограммировали на ошибку, чтобы показать, как джоинер обрабатывает смешанные результаты.

https://max.ru/bookofgeek

❺ Выводятся все успешно полученные заголовки. Пользователи получают максимум доступной информации.

❻ Получаем отчет об ошибках, не позволяя им блокировать отображение успешных результатов.

Запустим пример и получим следующее:

```
00:53:44.291 VThread[#34]: Fetching headlines from TechCrunch
00:53:44.291 VThread[#42]: Fetching headlines from FailingSource
00:53:44.291 VThread[#39]: Fetching headlines from InfoQ
00:53:44.291 VThread[#36]: Fetching headlines from InfoWorld
00:53:44.773 main        : Successfully fetched from 3 sources
00:53:44.773 main        : Failed to fetch from 1 sources
00:53:44.773 main        :
=== Results ===
01:43:17.431 main        : Headlines from InfoWorld: Breaking news, Tech updates
01:43:17.431 main        : Headlines from TechCrunch: Breaking news, Tech updates
01:43:17.431 main        : Headlines from InfoQ: Breaking news, Tech updates
01:43:17.431 main        :
=== Failures ===
01:43:17.431 main        : Network timeout for FailingSource
```

Завершение по кворуму

Реализуем джоинер, взяв за образец распределенные системы: он завершает работу при достижении кворума успешно выполненных подзадач. Этот подход полезен в сценариях, где перед продолжением работы требуется получить определенное количество подтверждений:

```
import java.util.concurrent.StructuredTaskScope;
import java.util.concurrent.atomic.AtomicInteger;

public class QuorumJoiner<T> implements StructuredTaskScope.Joiner<T, Boolean> {
  private final int requiredSuccesses;
  private final AtomicInteger successCount = new AtomicInteger(0); ❶
  private final AtomicInteger totalCount = new AtomicInteger(0); ❷
  private volatile boolean quorumReached = false; ❸

  public QuorumJoiner(int requiredSuccesses) {
    this.requiredSuccesses = requiredSuccesses;
  }

  @Override
  public Boolean result() {
    return quorumReached; ❹
  }

  @Override
  public boolean onFork(StructuredTaskScope.Subtask<? extends T> subtask) {
    totalCount.incrementAndGet(); ❺
    return false; //  Разрешаем продолжение всех задач ❻
  }
```

https://max.ru/bookofgeek

```
  @Override
  public boolean onComplete(StructuredTaskScope.Subtask<? extends T> subtask) {
    if (subtask.state() == StructuredTaskScope.Subtask.State.SUCCESS) {
      int currentSuccess = successCount.incrementAndGet(); ❼
      if (currentSuccess >= requiredSuccesses) { ❽
        quorumReached = true;
        return true; ❾
      }
    }
    return false; ❿
  }

  public int getSuccessCount() {
    return successCount.get();
}

  public int getTotalCount() {
    return totalCount.get();
  }
}
```

Рассмотрим, как кворумный джоинер управляет консенсусом.

❶ Чтобы обеспечить потокобезопасность, отслеживает количество успешно завершенных подзадач с использованием атомарного счетчика.

❷ Ведет учет количества запущенных подзадач, что полезно для расчета процента завершенных и оставшихся задач.

❸ Чтобы обеспечить видимость между потоками, использует volatile-переменную типа boolean, указывающую, достигнут ли необходимый кворум.

❹ Возвращает информацию о том, был ли достигнут кворум к моменту завершения области.

❺ Увеличивает общий счетчик на единицу при каждой запущенной подзадаче, предоставляя информацию об общем количестве задач.

❻ Возвращает `false`, чтобы позволить создать все подзадачи, — мы не будем препятствовать созданию задач на этапе fork.

❼ При успешном завершении подзадачи атомарно увеличивает на единицу счетчик успехов и фиксирует новое значение.

❽ Проверяет, достигнуто ли достаточное для кворума число успешно выполненных подзадач.

❾ Как только достаточное количество будет достигнуто, устанавливает флаг и возвращает `true` для отмены оставшихся задач и завершения области.

❿ Если кворум еще не достигнут, возвращает `false`, позволяя другим задачам продолжать выполнение.

https://max.ru/bookofgeek

Пример использования кворумного джоинера для операции записи в распределенную базу данных:

```
import java.time.Duration;
import java.util.List;
import java.util.Random;
import static ca.bazlur.mcj.chap4.Utils.log;
import static java.util.concurrent.StructuredTaskScope.open;

public class DistributedDatabase {
  private final List<String> nodes = List.of(
    "node-1", "node-2", "node-3", "node-4", "node-5" ❶
  );

  private final int quorumSize = 3; // Нужно 3 из 5 узлов ❷

  public boolean writeData(String key, String value)
      throws InterruptedException {
    log("Writing data to distributed database:\n " + key + "=" + value);
    try (var scope = open(new QuorumJoiner<Boolean>(quorumSize))) { ❸
      for (String node : nodes) {
        scope.fork(() -> writeToNode(node, key, value)); ❹
      }
      boolean success = scope.join(); ❺
      log("Write operation " + (success ? "succeeded" : "failed"));
      log("Required quorum: " + quorumSize + " nodes");
      return success;
    }
  }

  private Boolean writeToNode(String node, String key, String value)
      throws InterruptedException {
    log("Writing to " + node);
    Thread.sleep(Duration.ofMillis(100 + new Random().nextInt(200))); ❻
    // Имитация сбоев случайных узлов
    if (new Random().nextDouble() < 0.2) { ❼
      log(node + " write failed");
      throw new RuntimeException("Write failed on " + node);
    }
    log(node + " write succeeded");
    return true;
  }

  void main() {
    var database = new DistributedDatabase();
    try {
      boolean result = database.writeData("user:123", "John Doe"); ❽
      log("\nFinal result: " + (result ? "SUCCESS" : "FAILURE"));
    } catch (InterruptedException e) {
      Thread.currentThread().interrupt();
      log("Database operation interrupted");
    }
  }
}
```

https://max.ru/bookofgeek

Рассмотрим, как это работает.

❶ Определяем кластер из пяти узлов базы данных. В реальной системе это были бы разные физические серверы или контейнеры.

❷ Устанавливаем кворум размером три из пяти узлов. Это обеспечивает отказоустойчивость, сохраняя доступность, — система продолжит действовать, если откажут один или два узла.

❸ Создаем `StructuredTaskScope` с пользовательским `QuorumJoiner`, который завершается при достижении требуемого количества успешных операций, а не ожидает завершения всех задач.

❹ Для каждой операции записи в узел выделяется отдельный виртуальный поток. Все записи выполняются конкурентно, что доводит параллелизм до максимума и снижает общую задержку.

❺ Блокируем выполнение, когда кворум достигнут или обнаружено, что он невозможен. `QuorumJoiner` возвращает `true`, если успешно сделано достаточно записей, и `false`, когда из-за слишком большого числа ошибок кворума нельзя достичь.

❻ Имитируем реалистичные сетевые задержки со случайными значениями от 100 до 300 мс, что соответствует типичному времени записи в базу данных по сети.

❼ Вводим вероятность сбоя 20 %, чтобы смоделировать реальные проблемы распределенных систем: разделение сети, отказы дисков или аварии узлов.

❽ Высокоуровневый API абстрагирует сложность распределенной координации, предоставляя вызывающему коду простой синхронный интерфейс.

При запуске этой программы мы получим следующий вывод:

```
21:11:24.267 main        : Writing data to distributed database:
  user:123=John Doe
01:18:38.141 VThread[#34]: Writing to node-1
01:18:38.142 VThread[#36]: Writing to node-2
01:18:38.141 VThread[#44]: Writing to node-5
01:18:38.142 VThread[#42]: Writing to node-4
01:18:38.142 VThread[#39]: Writing to node-3
01:18:38.252 VThread[#42]: node-4 write succeeded
01:18:38.285 VThread[#34]: node-1 write succeeded
01:18:38.364 VThread[#39]: node-3 write failed
01:18:38.376 VThread[#36]: node-2 write succeeded
01:18:38.378 main        : Write operation succeeded
01:18:38.378 main        : Required quorum: 3 nodes
01:18:38.379 main        :
Final result: SUCCESS
```

https://max.ru/bookofgeek

Адаптивное завершение

Иногда логика завершения должна адаптироваться к наблюдаемым результатам. Создадим джоинер, который завершит работу досрочно, если будет замечено слишком много ошибок:

```
import module java.base;

public class AdaptiveJoiner<T>
    implements StructuredTaskScope.Joiner<T, CollectingJoiner.Result<T>> {
  private final int minSampleSize;
  private final double maxFailureRate;
  private final Queue<T> successes = new ConcurrentLinkedQueue<>();
  private final Queue<Throwable> failures = new ConcurrentLinkedQueue<>();

  public AdaptiveJoiner(double maxFailureRate, int minSampleSize) {
    this.maxFailureRate = maxFailureRate;
    this.minSampleSize = minSampleSize;
  }

  @Override
  public CollectingJoiner.Result<T> result() {
    return new CollectingJoiner.Result<>(
      successes.stream().toList(),
      failures.stream().toList()
    );
  }

  @Override
  public boolean onComplete(StructuredTaskScope.Subtask<? extends T> subtask) {
    switch (subtask.state()) {
      case SUCCESS -> successes.add(subtask.get()); ❶
      case FAILED -> failures.add(subtask.exception()); ❶
      case UNAVAILABLE -> failures.add(
        new RuntimeException("Task canceled"));
    }
    int total = successes.size() + failures.size();
    // Проверяем частоту отказов только после достижения
    // указанного минимального размера выборки
    if (total >= minSampleSize) { ❷
      double failureRate = (double) failures.size() / total;
      return failureRate > maxFailureRate; ❸
    }
    return false; // Продолжаем обработку ❹
  }
}
```

Выясним, как работает код.

❶ Собирает как успешные результаты, так и ошибки по мере их появления.

❷ Оценивает процент ошибок только после накопления такого числа точек данных, которого достаточно для принятия осмысленного решения.

https://max.ru/bookofgeek

❸ Если процент ошибок превышает заданный порог, завершает работу досрочно, чтобы не тратить зря ресурсы на явно неудачную операцию.

❹ Если процент ошибок приемлемый или данных пока недостаточно, продолжает обработку.

Теперь рассмотрим пример использования:

```
import module java.base;
import static ca.bazlur.mcj.chap4.rewrite.Utils.log;
import static java.util.concurrent.StructuredTaskScope.open;

public class WebCrawlerWithCircuitBreaker {
  private final List<String> urls = List.of(
    "https://api.service1.com/data", ❶
    "https://api.service2.com/data",
    "https://api.service3.com/data",
    "https://api.service4.com/data",
    "https://api.service5.com/data"
  );
  private final Random random = new Random();
  private double systemFailureRate = 0.1; // Начальный уровень ошибок 10 %

  public CollectingJoiner.Result<String> crawlWithCircuitBreaker()
      throws InterruptedException {
    log("Starting web crawl with circuit breaker protection");
    log("Max failure rate: 30%, Min sample size: 5");
    // Создаем адаптивный джоинер, который останавливается, если после
    // пяти выборок доля ошибок выше 30 %
    try (var scope = open(new AdaptiveJoiner<String>(0.30, 5))) { ❷
      for (String url : urls) {
          scope.fork(() -> fetchUrl(url)); ❸
      }
      var result = scope.join(); ❹
      log("\n=== FINAL RESULTS ===");
      log("Successful fetches: " + result.successes().size());
      log("Failed fetches: " + result.failures().size());
      log("Total processed: " + (result.successes().size()
         + result.failures().size()));
      log("Remaining URLs (not processed): " +
        (urls.size() - result.successes().size() - result.failures()
        .size()));
      return result;
    }
  }

  private String fetchUrl(String url) throws InterruptedException {
    log("Fetching: " + url);
    // Имитация сетевой задержки
    Thread.sleep(Duration.ofMillis(200 + random.nextInt(300))); ❺
    // Имитация деградации системы с течением времени
    if(random.nextDouble() < systemFailureRate) { ❻
      systemFailureRate += 0.05; // Число ошибок со временем нарастает
```

https://max.ru/bookofgeek

```
      throw new RuntimeException("Network timeout for " + url);
    }
    log("✓ Successfully fetched: " + url);
    return "Data from " + url;
  }

  public void demonstrateScenarios() throws InterruptedException {
    log("=== SCENARIO 1: Normal Operation ===");
    systemFailureRate = 0.1; // Низкий уровень ошибок
    crawlWithCircuitBreaker();
    Thread.sleep(1000);
    log("\n=== SCENARIO 2: System Under Stress ===");
    systemFailureRate = 0.4; // Высокий начальный уровень ошибок
    crawlWithCircuitBreaker();
    Thread.sleep(1000);
    log("\n=== SCENARIO 3: Cascading Failures ===");
    systemFailureRate = 0.6; // Очень высокий уровень ошибок
    crawlWithCircuitBreaker();
  }

  void main() {
    var crawler = new WebCrawlerWithCircuitBreaker();
    try {
      crawler.demonstrateScenarios();
    } catch (InterruptedException e) {
      Thread.currentThread().interrupt();
      log("Crawling interrupted");
    }
  }
}
```

Что происходит в приведенном выше коде?

❶ Определяем список эндпоинтов API для обхода. В реальной системе это могут быть тысячи URL из очереди веб-краулера или сервиса обнаружения API.

❷ Конфигурируем адаптивный джоинер с параметрами устойчивости к сбоям. Максимальный уровень сбоев 30 % при минимуме в пять выборок обеспечивает баланс между отзывчивостью и стабильностью.

❸ Создаем конкурентные HTTP-запросы для каждого URL. Все запросы выполняются параллельно с использованием виртуальных потоков, что увеличивает пропускную способность, сохраняя при этом стабильность системы.

❹ Блокировка до завершения или до активации Circuit Breaker. При слишком большом числе сбоев адаптивный джоинер завершит работу досрочно, экономя время и ресурсы.

❺ Моделируем реалистичные сетевые условия с переменной задержкой. У реальных HTTP-запросов время отклика непредсказуемо из-за состояния сети и нагрузки на сервер.

https://max.ru/bookofgeek

❻ Моделируем деградацию системы, где сбои накапливаются со временем. Мы имитируем реальные сценарии, такие как каскадные сбои, утечки памяти или проблемы инфраструктуры, которые усугубляются при высокой нагрузке.

При выполнении нашего кода мы получим следующий вывод:

```
01:55:50.225 main : === SCENARIO 1: Normal Operation ===
01:55:50.226 main : Starting web crawl with circuit breaker protection
01:55:50.226 main : Max failure rate: 30%, Min sample size: 5 01:55:50.269
VThread[#42]: Fetching: https://api.service4.com/data
01:55:50.270 VThread[#45]: Fetching: https://api.service5.com/data
... [3 more fetch operations] ...
01:55:50.473 VThread[#39]: ✓ Successfully fetched: https://api.service3.com/
data
... [3 more successful fetches] ...
01:55:50.753 main : === FINAL RESULTS ===
01:55:50.753 main : Successful fetches: 4, Failed fetches: 1

01:55:51.759 main : === SCENARIO 2: System Under Stress ===
... [initialization and fetch operations] ...
01:55:52.182 main : Successful fetches: 3, Failed fetches: 2

01:55:53.188 main : === SCENARIO 3: Cascading Failures ===
... [initialization and fetch operations] ...
01:55:53.653 main : Successful fetches: 0, Failed fetches: 5
=== FINAL RESULTS ===
01:55:53.653 main : Successful fetches: 0
01:55:53.653 main : Failed fetches: 5
01:55:53.653 main : Total processed: 5
01:55:53.653 main : Remaining URLs (not processed): 0
```

Джоинер, ограничивающий скорость

В некоторых случаях необходимо контролировать скорость выполнения подзадач, чтобы избежать перегрузки внешних сервисов. Создадим джоинер, ограничивающий скорость с помощью `Semaphore`:

```
import module java.base;

public class RateLimitedJoiner<T>
    implements StructuredTaskScope.Joiner<T, List<T>> {
  private final Semaphore semaphore; ❶
  private final Queue<T> results = new ConcurrentLinkedQueue<>();
  private final Queue<Throwable> failures = new ConcurrentLinkedQueue<>();

  public RateLimitedJoiner(int maxConcurrentTasks) {
    this.semaphore = new Semaphore(maxConcurrentTasks); ❷
  }

  @Override
  public boolean onFork(StructuredTaskScope.Subtask<? extends T> subtask) {
```

https://max.ru/bookofgeek

```
    try {
      semaphore.acquire(); ❸
    } catch (InterruptedException e) {
      Thread.currentThread().interrupt();
    }
    return false; ❹
  }

  @Override
  public boolean onComplete(StructuredTaskScope.Subtask<? extends T> subtask) {
    switch (subtask.state()) {
      case SUCCESS -> results.add(subtask.get()); ❺
      case FAILED -> failures.add(subtask.exception()); ❻
      case UNAVAILABLE -> failures.add(new RuntimeException("Task canceled"));
    }
    semaphore.release(); ❼
    return false; ❽
  }

  @Override
  public List<T> result() {
    return results.stream().toList();
  }
}
```

Посмотрим, как работает ограничение скорости с помощью семафора.

❶ Используем `Semaphore` для контроля числа разрешенных конкурентно выполняющихся задач.

❷ Создаем `Semaphore` с максимальным числом разрешенных конкурентных задач.

❸ Получаем разрешение перед запуском задачи. Скорость выполнения задач контролируется так: если разрешения недоступны, вызов блокируется до тех пор, пока хотя бы одно не освободится.

❹ Возвращается `false`, чтобы позволить создание подзадачи и начать ее выполнение (теперь, когда есть разрешение).

❺ Собираем результаты при успешном завершении задач.

❻ Собираем ошибки, чтобы обеспечить полную видимость их причин.

❼ При завершении задачи (неважно, успешном или неудачном) всегда освобождается разрешение семафора. Оно становится доступным для других ожидающих задач.

❽ Позволяем всем задачам завершиться, а не останавливаем процесс раньше времени.

https://max.ru/bookofgeek

Продемонстрируем работу ограничивающего скорость джоинера на примере пакетного вызова API:

```
import module java.base;
import static ca.bazlur.mcj.chap4.Utils.log;
import static java.util.concurrent.StructuredTaskScope.open;

public class RateLimitedAPIService {
  public List<String> fetchDataWithRateLimit() throws InterruptedException {
    var endpoints = List.of(
        "api/users", "api/orders", "api/products", "api/analytics",
        "api/reports", "api/logs"
    );
    // Разрешается максимум три конкурентных вызова API
    var rateLimitedJoiner = new RateLimitedJoiner<String>(3);
    try (var scope = open(rateLimitedJoiner)) {
      for (String endpoint : endpoints) {
        scope.fork(() -> callAPI(endpoint)); ❶
      }
      var results = scope.join(); ❷
      log("Completed " + results.size() +
          " API calls with rate limiting");
      return results;
    }
  }

  private String callAPI(String endpoint) throws InterruptedException {
    log("Starting API call: " + endpoint );
    Thread.sleep(Duration.ofMillis(500 + new Random().nextInt(1000)));
    log("Completed API call: " + endpoint);
    return "Response from " + endpoint;
  }

  void main() {
    var service = new RateLimitedAPIService();
    try {
      var results = service.fetchDataWithRateLimit();
      log("All results: " + results);
    } catch (InterruptedException e) {
      Thread.currentThread().interrupt();
      log("Service interrupted");
    }
  }
}
```

Рассмотрим, как работает данный пример.

❶ Каждый вызов `fork()` должен получить разрешение семафора на начало выполнения задачи — это гарантия того, что конкурентно будут выполняться только три задачи.

https://max.ru/bookofgeek

❷ Ожидается завершение всех вызовов API, при этом семафор автоматически управляет конкурентностью.

При запуске этого примера с шестью эндпоинтами, но с ограничением в три конкурентные задачи мы увидим следующий вывод:

```
02:28:43.648 VThread[#38]: Starting API call: api/products
02:28:43.648 VThread[#36]: Starting API call: api/orders
02:28:43.648 VThread[#34]: Starting API call: api/users
02:28:44.266 VThread[#34]: Completed API call: api/users
02:28:44.268 VThread[#44]: Starting API call: api/analytics
02:28:44.499 VThread[#38]: Completed API call: api/products
02:28:44.501 VThread[#46]: Starting API call: api/reports
02:28:45.021 VThread[#36]: Completed API call: api/orders
02:28:45.022 VThread[#47]: Starting API call: api/logs
02:28:45.381 VThread[#44]: Completed API call: api/analytics
02:28:45.849 VThread[#47]: Completed API call: api/logs
02:28:45.974 VThread[#46]: Completed API call: api/reports
02:28:45.975 main        : Completed 6 API calls with rate limiting
02:28:45.975 main        : All results: [Response from api/users, Response
from api/products, Response from api/orders, Response from api/analytics,
Response from api/logs, Response from api/reports]
```

Условные джоинеры

Метод `onFork` позволяет реализовать условия выполнения в зависимости от состояния подзадач. Примером может служить универсальный джоинер, который принимает некое условие для прекращения запуска новых задач.

Реализуем эту идею:

```
import module java.base;

public class ConditionalJoiner<T>
    implements StructuredTaskScope.Joiner<T, List<T>> {
  private final Supplier<Boolean> shouldContinue; ❶
  private final Queue<T> results = new ConcurrentLinkedQueue<>();
  private final Queue<Throwable> failures = new ConcurrentLinkedQueue<>();

  public ConditionalJoiner(Supplier<Boolean> shouldContinue) {
    this.shouldContinue = shouldContinue; ❷
  }

  @Override
  public boolean onFork(StructuredTaskScope.Subtask<? extends T> subtask) {
    if (!shouldContinue.get()) { ❸
      System.out.println("Condition failed, stopping new tasks");
      return true; // Отменить область, чтобы предотвратить новую работу ❹
    }
    return false; // Условие выполнено, разрешить задачу ❺
  }

  @Override
  public boolean onComplete(StructuredTaskScope.Subtask<? extends T> subtask) {
```

https://max.ru/bookofgeek

```
    switch (subtask.state()) {
      case SUCCESS -> results.add(subtask.get()); ❻
      case FAILED -> failures.add(subtask.exception()); ❼
      case UNAVAILABLE -> failures.add(new RuntimeException("Task canceled"));
    }
    return false; // Продолжить обработку существующих задач ❽
  }

  @Override
  public List<T> result() {
    return results.stream().toList();
  }

  public List<Throwable> getFailures() {
    return results.stream().toList();
  }
}
```

Рассмотрим, как этот джоинер обрабатывает различные результаты подзадач.

❶ Принимает любое условие, определяющее продолжение обработки, в виде `Supplier<Boolean>`.

❷ Сохраняет поставщика условия для оценки при разбиении задач на подзадачи.

❸ Проверяет условие каждый раз при запуске новой задачи.

❹ Если условие не выполняется, останавливает область, предотвращая запуск новых задач.

❺ При выполнении условия позволяет задачам запускаться в штатном режиме.

❻ Собирает успешные результаты как обычно.

❼ Отслеживает сбои для последующего анализа.

❽ Позволяет завершиться уже запущенным задачам.

Теперь применим это к простому примеру:

```
package ca.bazlur.modern.concurrency.c04;

import java.time.Duration; import java.util.List;
import java.util.Random;
import java.util.concurrent.StructuredTaskScope;
import java.util.concurrent.atomic.AtomicInteger;
import static java.util.concurrent.StructuredTaskScope.open;

public class SystemHealthCheckDemo {

  void main() throws InterruptedException {
    var results = new SystemHealthCheckDemo()
        .processWithSystemHealthCheck();
    results.forEach(System.out::println);
  }
```

https://max.ru/bookofgeek

```
  public List<String> processWithSystemHealthCheck()
      throws InterruptedException {
    var healthChecker = new SystemHealthChecker();
    var joiner = new ConditionalJoiner<String>(healthChecker::isSystemHealthy);

    try (var scope = open(joiner)) {
      for (int i = 0; i < 10; i++) {
        int id = i;
        scope.fork(() -> processTask("health-task-" + id));
      }
      return scope.join();
    }
  }

  private String processTask(String taskName)
      throws InterruptedException {
    Thread.sleep(Duration.ofMillis(200
        + new Random().nextInt(300)));

    if (taskName.contains("error-task")
        && new Random().nextDouble() < 0.4)
      throw new RuntimeException("Task failed: " + taskName);

    return "Completed: " + taskName;
  }
}
```

Данный код проверяет состояние внешней системы и останавливает выполнение, если ее зависимости становятся недоступными.

Простой класс `SystemHealthChecker`:

```
private static class SystemHealthChecker {
  private final AtomicInteger checkCount
      = new AtomicInteger(0);

  public boolean isSystemHealthy() {
    checkCount.incrementAndGet();
    try {
      Thread.sleep(100);
    } catch (InterruptedException e) {
      throw new RuntimeException(e);
    }
    return checkCount.get() < 7;
  }
}
```

Кастомные джоинеры — это одна из самых мощных возможностей структурированной конкурентности в Java. Они превращают `StructuredTaskScope` из простого инструмента параллельного выполнения в сложную платформу координации, способную адаптироваться практически к любой задаче конкурентного программирования.

https://max.ru/bookofgeek

Красота джоинера заключается в его простоте и гибкости. Всего три метода — `onFork`, `onComplete` и `result` — позволяют реализовать сколь угодно сложную логику координации, сохраняя при этом гарантии безопасности и четкую структуру.

Примеры, которые мы рассмотрели, — лишь начало. Настоящая сила проявляется, когда вы проектируете джоинеры, отражающие уникальные паттерны координирования в нужной вам предметной области, создавая повторно используемые компоненты, которые делают конкурентное программирование не только более безопасным, но и по-настоящему изящным.

Согласованность памяти

`StructuredTaskScope` помогает обеспечить согласованность памяти и видимость данных, совместно используемых потоками.

Действия в потоке-владельце `StructuredTaskScope` перед созданием подзадачи выполняются раньше *(happens-before)* любых действий самой подзадачи, которые, в свою очередь, предшествуют получению ее результата (https://oreil.ly/QRQkV). Отношение *happens-before* гарантирует, что все изменения данных, сделанные в потоке-владельце до вызова `fork()`, будут видны подзадаче, а любые изменения, сделанные подзадачей, будут видны при получении ее результата.

СВОЙСТВА СОГЛАСОВАННОСТИ ПАМЯТИ

Гарантии согласованности памяти в `StructuredTaskScope` основаны на фундаментальных отношениях happens-before, определенных в спецификации языка Java (глава 17, https://oreil.ly/TUOvw). Эти правила обеспечивают предсказуемое и безопасное выполнение потоками операций с памятью.

В Java существуют встроенные отношения happens-before, обеспечивающие безопасность конкурентного программирования.

- Операции в рамках одного потока выполняются в порядке программы.
- Синхронизированные блоки задают порядок между потоками через операции с мониторами.
- Ключевое слово `volatile` гарантирует видимость изменений переменной без блокировок.
- Создание и завершение потоков устанавливают четкие границы памяти.

Пакет `java.util.concurrent` расширяет эти базовые правила, предоставляя гарантии более высокого уровня. Такие операции, как отправка задач в исполнители (`executors`), добавление элементов в конкурентные коллекции и получение результатов `Future`, устанавливают отношения happens-before, обеспечивающие потокобезопасность.

https://max.ru/bookofgeek

Рассмотрим реалистичный пример. Допустим, требуется разработать сервис конфигурации, который будет проверять настройки в нескольких системах одновременно:

```
import module java.base;
import java.util.concurrent.ConcurrentHashMap;
import java.util.concurrent.atomic.AtomicInteger;
import static ca.bazlur.mcj.chap4.Utils.log;
import static java.util.concurrent.StructuredTaskScope.open;

public class MemoryConsistencyDemo {
  private String configuration = "default";
  private final Map<String, String> cache = new ConcurrentHashMap<>();

  public void demonstrateMemoryConsistency() throws InterruptedException {
    // Поток-владелец настраивает общее состояние
    // перед созданием подзадач
    configuration = "production-config"; ❶
    cache.put("database-url", "prod.example.com"); ❶
    cache.put("api-key", "secret-key-123"); ❶
    log("Owner thread prepared: " + configuration);
    try (var scope = open(StructuredTaskScope.Joiner.
         <String>allSuccessfulOrThrow())) {
      // Создаем подзадачи, читающие данные потока-владельца
    var configTask = scope.fork(() -> { ❷
      log("Subtask sees: " + configuration); ❸
      return "Config: " + configuration;
    });
    var cacheTask = scope.fork(() -> { ❷
      String url = cache.get("database-url"); ❸
      log("Subtask found URL: " + url); return "Connected to: " + url;
    });
    var results = scope.join() ❹
        .map(StructuredTaskScope.Subtask::get)
        .toList();
    log("Owner received: " + results); ❺
    }
  }
}
```

Проследим, что здесь происходит.

❶ Перед ветвлением задачи на подзадачи поток-владелец подготавливает общее состояние: обновляет строку конфигурации и заполняет кэш необходимыми параметрами подключения.

❷ При вызове `fork()` происходят важные внутренние события. Фреймворк структурированной конкурентности устанавливает отношение happens-before, гарантируя видимость всех подготовительных изменений (шагов 1–3) для подзадач.

❸ Здесь все и происходит. Без дополнительной синхронизации с нашей стороны обе подзадачи уверенно видят обновленную конфигурацию и содержимое

https://max.ru/bookofgeek

кэша. Конфигурационная переменная содержит `production-config` вместо `default`, а кэш — все добавленные записи.

❹ Операция `join()` создает очередное перенаправление happens-before — на этот раз от подзадач обратно к потоку-владельцу.

❺ Поток-владелец получает все результаты от подзадач, причем они гарантированно актуальны и согласованы.

Гарантия happens-before действует в обе стороны: изменения общего состояния, внесенные подзадачами, становятся видимыми для потока-владельца после завершения операции `join`.

Рассмотрим следующий пример:

```
public void demonstrateSubtaskUpdates() throws InterruptedException {
  var results = new ConcurrentHashMap<String, String>(); ❶
  var counter = new AtomicInteger(0); ❶
  log("Initial count: " + counter.get());

  try (var scope = open(StructuredTaskScope.Joiner.
      <String>allSuccessfulOrThrow())) {
    var worker1 = scope.fork(() -> { ❷
      results.put("task1", "completed"); ❸
      int count = counter.incrementAndGet(); ❹
      log("Worker1 incremented to: " + count);
      return "Worker1 done";
    });
    var worker2 = scope.fork(() -> { ❷
      results.put("task2", "completed"); ❸
      int count = counter.incrementAndGet(); ❹
      log("Worker2 incremented to: " + count);
      return "Worker2 done";
    });
    scope.join(); ❺
    // Поток-владелец видит все обновления подзадач
    log("Final count: " + counter.get()); ❻
    log("Final results: " + results); ❻
  }
}
```

Пример показывает, как несколько подзадач могут безопасно обновлять общее состояние.

❶ Сначала мы создаем потокобезопасные контейнеры: `ConcurrentHashMap` для сбора результатов и `AtomicInteger` для подсчета операций.

❷ Запускаем два рабочих потока, которые будут выполняться одновременно. Они могут работать в любом порядке, но оба безопасно обновляют общие контейнеры.

❸ Каждый рабочий поток добавляет запись к результату в структуре `map`. Поскольку мы используем `ConcurrentHashMap`, эти операции остаются потокобезопасными даже при конкурентном выполнении.

https://max.ru/bookofgeek

❹ Атомарный счетчик гарантирует, что инкременты из разных потоков не будут мешать друг другу, предотвращая классическую проблему потерянного обновления (lost update).

❺ Операция `join()` обеспечивает видимость всех изменений рабочих потоков для потока-владельца.

❻ После завершения `join()` поток-владелец видит конечное состояние как счетчика, так и `map` с результатами, со всеми обновлениями от рабочих потоков.

Как видим, `StructuredTaskScope` использует проверенные механизмы для обеспечения своих гарантий. При создании подзадачи, как и при запуске потока, создается направление happens-before, делающее все предыдущие действия потока-владельца видимыми для подзадачи. Соединение подзадач (`scope.join()`), аналогично вызову `Thread.join()`, устанавливает отношение happens-before, которое делает все действия подзадачи видимыми для потока-владельца.

Конкурентные коллекции из `java.util.concurrent` расширяют эти гарантии, обеспечивая безопасную видимость операций (таких как добавление элементов в `ConcurrentHashMap` или инкремент `AtomicInteger`) между потоками.

Автоматическая согласованность памяти — одно из ключевых преимуществ структурированной конкурентности. Мы можем спокойно сосредоточиться на бизнес-логике конкурентных операций, не беспокоясь о деталях согласованности памяти. Операции `fork` и `join` предоставляют четкие точки синхронизации, благодаря которым обеспечивается уверенность в установке всех необходимых барьеров памяти. Это не означает, что можно совершенно не уделять внимания потокобезопасности. Когда несколько подзадач изменяют общее состояние, по-прежнему необходимо использовать соответствующие потокобезопасные коллекции и атомарные операции.

Вложенные области

Мы узнали, как создавать собственные джоинеры и управлять конкурентными задачами в рамках одной области, но возможности структурированной конкурентности этим не ограничиваются. Для создания сложных многоуровневых иерархий конкурентных задач можно вкладывать экземпляры `StructuredTaskScope`, подобно тому как блоки кода вкладываются в методы.

Вложенные области особенно полезны для управления сложными рабочими процессами, задачи которых естественным образом разбиваются на несколько уровней подзадач. Каждая вложенная область является самодостаточной, инкапсулируя жизненный цикл своих подзадач и обработку ошибок.

Когда подзадача создает собственный вложенный экземпляр `StructuredTaskScope`, устанавливается отношение «родитель — потомок» и формируется древовидная структура. Родительская область становится ответственной за

https://max.ru/bookofgeek

жизненный цикл дочерней, включая завершение работы и выполнение ее подзадач.

Эта иерархическая структура (рис. 4.3) распространяет ошибки и отмены вверх по иерархии. Если подзадача во вложенной области завершается с ошибкой, она сначала обрабатывается непосредственной родительской областью, а затем передается дальше по цепочке, пока не будет обработана или не достигнет корневой области.

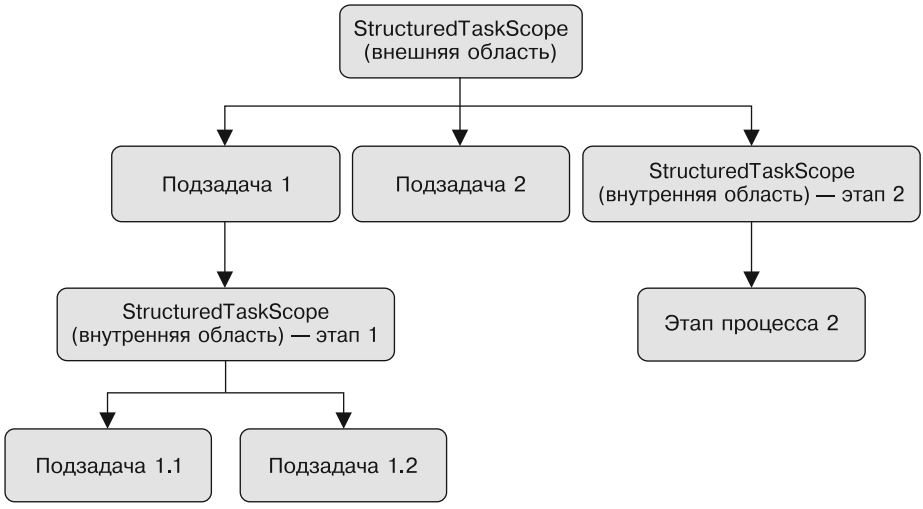


Рис. 4.3. Иерархическая структура задач в структурированной конкурентности

Рассмотрим процессор документов, который собирает содержимое из нескольких источников, а затем выполняет анализ:

```
public class DocumentProcessor {

  public DocumentReport processDocument(String documentId)
                              throws InterruptedException {

    try (var gatheringScope = open(StructuredTaskScope.Joiner.
              <String>allSuccessfulOrThrow())) { ❶
        var headerTask = gatheringScope.fork(() ->
                         fetchHeader(documentId)); ❷
        var bodyTask = gatheringScope.fork(() -> fetchBody(documentId)); ❷
        var metadataTask = gatheringScope.fork(() ->
          fetchMetadata(documentId)); ❷

          gatheringScope.join(); ❸

          return analyzeContent(headerTask.get(),
                                bodyTask.get(),
                                metadataTask.get()); ❹
```

https://max.ru/bookofgeek

```
    } catch (StructuredTaskScope.FailedException e) {
        throw new RuntimeException("Failed to gather document content", e);
    }
  }
}
```

В приведенном выше коде происходит следующее.

❶ Создается область верхнего уровня для конкурентного сбора всех компонентов документа.

❷ Запускаются конкурентные задачи для получения заголовка, тела и метаданных из разных источников.

❸ Ожидается завершение сбора всего контента перед переходом к анализу.

❹ Переход к фазе анализа происходит только при наличии всего необходимого контента.

Теперь исследуем вложенную область анализа:

```
private DocumentReport analyzeContent(String header,
                                      String body,
                                      String metadata)
    throws InterruptedException {

  try (var analysisScope = open(StructuredTaskScope.Joiner.
      allSuccessfulOrThrow())) { ❶
    var wordCountTask = analysisScope.fork(() ->
        countWords(body)); ❷
    var sentimentTask = analysisScope.fork(() ->
        analyzeSentiment(body)); ❷
    var summaryTask = analysisScope.fork(() ->
        generateSummary(header, body, metadata)); ❷
    analysisScope.join(); ❸
    return new DocumentReport(
        wordCountTask.get(),
        sentimentTask.get(),
        summaryTask.get()
    );
  } catch (StructuredTaskScope.FailedException e) {
    throw new RuntimeException("Failed to analyze document content", e);
  }
}
```

Приведенный выше код выполняет следующее.

❶ Создает область видимости второго уровня для конкурентного сбора всех компонентов документа.

❷ Запускает конкурентные задачи анализа: подсчет слов, анализ тональности и генерацию сводки.

❸ Ждет завершения анализа и формирует итоговый отчет.

https://max.ru/bookofgeek

Формируется четкая иерархическая структура:

```
Обработка документов (Уровень 1)
├── Получение заголовка (Уровень 2)
├── Получение тела документа (Уровень 2)
├── Получение метаданных (Уровень 2)
└── Анализ контента (Уровень 2)
├── Подсчет слов (Уровень 3)
├── Анализ тональности (Уровень 3)
└── Генерация сводки (Уровень 3)
```

Далее представлены вспомогательные методы, завершающие наш пример:

```
private String fetchHeader(String documentId)
    throws InterruptedException {
  Thread.sleep(Duration.ofMillis(100));
  return "Header for document " + documentId;
}

private String fetchBody(String documentId)
    throws InterruptedException {
  Thread.sleep(Duration.ofMillis(200));
  return "This is the main content of document " + documentId +
      " with multiple sentences and important information.";
}

private String fetchMetadata(String documentId)
    throws InterruptedException {
    Thread.sleep(Duration.ofMillis(150));
    return "Created: 2024-01-01, Author: John Doe";
}

private Integer countWords(String content)
    throws InterruptedException {
  Thread.sleep(Duration.ofMillis(100));
  return content.split("\\s+").length;
}

private String analyzeSentiment(String content)
  throws InterruptedException {
  Thread.sleep(Duration.ofMillis(200));
  return content.toLowerCase().contains("important") ? "Positive" : "Neutral";
}

private String generateSummary(String header, String body, String metadata)
    throws InterruptedException {
  Thread.sleep(Duration.ofMillis(150));
  return header + ": " + body.substring(0, Math.min(50, body.length())) + "...";
}

public record DocumentReport(int wordCount,
                             String sentiment, String summary) {
  @Override
  public String toString() {
```

https://max.ru/bookofgeek

```
    return String.format("Document Report:\n Words: %d\n " +
            "Sentiment: %s\n Summary: %s",
        wordCount, sentiment, summary);
  }
}
```

Теперь воспользуемся следующим методом main и выполним приведенный выше код:

```
void main() {
  var processor = new DocumentProcessor();
  try {
    var report = processor.processDocument("DOC-123");
    System.out.println(report);
  } catch (InterruptedException e) {
    Thread.currentThread().interrupt();
    System.out.println("Processing interrupted");
  }
}
```

После выполнения кода получим вывод:

```
Document Report:
  Words: 14
  Sentiment: Positive
  Summary: Header for document DOC-123: This is the main content of document
  DOC-123 with ...
```

Такой подход демонстрирует изящество структурированной конкурентности. Каждая область управляет собственным жизненным циклом и ресурсами, создавая четкую иерархию, в которой ошибки распространяются естественным образом, ресурсы освобождаются автоматически, а отмена задач корректно спускается от родительских областей к дочерним.

Наблюдаемость

Итак, мы выяснили, что структурированная конкурентность устанавливает четкую иерархическую связь между задачами и подзадачами. Она отражает вложенную структуру кода, значительно упрощая понимание и анализ потока конкурентного выполнения.

Древовидная структура крайне полезна при отладке. Традиционные дампы потоков часто представляют собой хаотичное нагромождение потоков без явных взаимосвязей. В отличие от них, структурированные дампы обеспечивают иерархическую схему конкурентной системы, упрощая выявление проблем.

Сгенерировать структурированные дампы потоков в формате JSON можно с помощью инструмента `jcmd`:

```
jcmd <pid> Thread.dump_to_file -format=json <output_file>
```

https://max.ru/bookofgeek

Такие дампы содержат ценную информацию, включая иерархию задач и состояние подзадач.

Рассмотрим пример с нашим процессором документов:

```
import module java.base;
import static java.util.concurrent.StructuredTaskScope.open;

public class DocumentProcessor {
  private final ThreadFactory threadFactory = Thread.ofVirtual()
      .name("doc-proc", 1)
      .factory();

public DocumentReport processDocument(String documentId)
        throws InterruptedException {
        try (var gatheringScope = open(
            StructuredTaskScope.Joiner.<String>allSuccessfulOrThrow(),
                conf -> conf.withThreadFactory(threadFactory)
                            .withName("doc-gathering-scope"))) { ❶

            var headerTask = gatheringScope.fork(() ->
                fetchHeader(documentId)); ❷
            var bodyTask = gatheringScope.fork(() ->
                fetchBody(documentId)); ❷
            var metadataTask = gatheringScope.fork(() ->
                fetchMetadata(documentId)); ❷

            gatheringScope.join(); ❸

            return analyzeContent(headerTask.get(),
                   bodyTask.get(), metadataTask.get());
        } catch (StructuredTaskScope.FailedException e) {
            throw new RuntimeException("Failed to gather document content", e);
        }
  }

  private String fetchHeader(String documentId) throws InterruptedException {
    Thread.sleep(Duration.ofSeconds(10)); ❹
    return "Header for document " + documentId;
  }

  private String fetchBody(String documentId) throws InterruptedException {
    Thread.sleep(Duration.ofSeconds(10));
    return "This is the main content of document " + documentId;
  }

  private String fetchMetadata(String documentId) throws InterruptedException {
    Thread.sleep(Duration.ofSeconds(10));
    return "Created: 2024-01-01, Author: John Doe";
  }
}
```

https://max.ru/bookofgeek

Мы немного изменили программу для демонстрации дампа потоков.

❶ Дали области описательное имя и использовали кастомную `ThreadFactory` с именованными потоками для упрощения идентификации в дампах.

❷ Запустили параллельные задачи для получения разных частей документа.

❸ Ожидаем завершения всех операций сбора.

❹ Сымитировали медленные операции, которые будут четко видны в дампе потоков.

Запустим этот код и сгенерируем дамп потоков во время выполнения операций, в результате получим прекрасно структурированное представление:

```
{
  "threadDump": { "processId": "10259",
    "time": "2025-06-10T00:49:22.146091Z",
    "runtimeVersion": "25-ea+25-3096",
    "threadContainers": [
      {
        "container": "<root>",
        "parent": null,
        "owner": null,
        "threads": [
        {
          "tid": "3",
          "name": "main",
          "stack": [
            "java.base/jdk.internal.misc.Unsafe.park(Native Method)",
            "java.base/java.util.concurrent.locks.LockSupport.park(LockSupport.
            java:369)",
            "java.base/jdk.internal.misc.ThreadFlock.awaitAll(ThreadFlock.java:
            305)",
            "java.base/java.util.concurrent.StructuredTaskScopeImpl.join(
            Struct uredTaskScopeImpl.java:243)",
            "ca.bazlur.mcj.chap4.DocumentProcessor.processDocument(DocumentProc
            essor.java:23)"
          ]
        }
      ],
      "threadCount": "8"
    },
    {
      "container": "doc-gathering-scope/jdk.internal.misc.ThreadFlock$ThreadCon
      tainerImpl@78fd3572",
      "parent": "<root>",
      "owner": "3", "threads": [
      {
        "tid": "36",
        "name": "doc-proc2",
        "stack": [
          "java.base/java.lang.Thread.sleep(Thread.java:574)",
          "ca.bazlur.mcj.chap4.DocumentProcessor.fetchBody(DocumentProcessor.
          java:69)",
```

https://max.ru/bookofgeek

```
          "ca.bazlur.mcj.chap4.DocumentProcessor.lambda$processDocument$2(Doc
          umentProcessor.java:20)"
        ]
      },
      {
        "tid": "38",
        "name": "doc-proc3",
        "stack": [
          "java.base/java.lang.Thread.sleep(Thread.java:574)",
          "ca.bazlur.mcj.chap4.DocumentProcessor.fetchMetadata(DocumentProces
          sor.java:76)",
          "ca.bazlur.mcj.chap4.DocumentProcessor.lambda$processDocument$3(Doc
          umentProcessor.java:21)"
        ]
      },
      {
        "tid": "26",
        "name": "doc-proc1",
        "stack": [
          "java.base/java.lang.Thread.sleep(Thread.java:574)",
          "ca.bazlur.mcj.chap4.DocumentProcessor.fetchHeader(DocumentProcesso
          r.java:63)",
          "ca.bazlur.mcj.chap4.DocumentProcessor.lambda$processDocument$1(Doc
          umentProcessor.java:19)"
          ]
        }
      ],
      "threadCount": "3"
    }
  ]
 }
}
```

Анализ дампа потоков выявляет несколько важных моментов.

- Мы четко видим именованную область `doc-gathering-scope` как отдельный контейнер с родительским элементом <root> и владельцем — потоком 3 (главным потоком).
- Все три подзадачи (`doc-proc1`, `doc-proc2`, `doc-proc3`) явно сгруппированы в рамках нашей области для сбора документа: легко понять, какие потоки к какой логической операции относятся.
- В стек-трейсе каждого потока точно указана выполняемая им операция — `fetch Header`, `fetchBody` и `fetchMetadata`, — что делает отладку интуитивно понятной.
- Поле `owner`: `3` показывает, что главный поток (`tid 3`) владеет этой областью, устанавливая явную родительско-дочернюю связь.

Современные инструменты могут анализировать структурированные дампы потоков для получения дополнительной информации. Например, инструмент Threadly (https://oreil.ly/oTZEH) способен выполнять парсинг JSON-дампов

https://max.ru/bookofgeek

потоков и визуализировать древовидную структуру конкурентности. Это упрощает понимание иерархических связей и отладку сложных конкурентных приложений.

Хотя утилита `jcmd` предоставляет удобный способ генерации структурированных дампов потоков, в некоторых случаях требуется более гибкое управление или автоматизация процесса. Здесь удобнее применять интерфейс `HotSpotDiagnosticMXBean`. С его помощью можно программно инициировать дампы потоков в конкретных условиях, например при возникновении исключения в `StructuredTaskScope`.

Пример того, как его можно использовать:

```
public void processWithErrorHandling(String documentId) {
  try (var scope = open(StructuredTaskScope.Joiner.
          <String>allSuccessfulOrThrow(),
      cf -> cf.withName("error-prone-scope"))) {
    scope.fork(() -> {
      if (new Random().nextBoolean()) { ❶
        throw new RuntimeException("Simulated failure");
      }
      return fetchHeader(documentId);
    });
    scope.join();
  } catch (StructuredTaskScope.FailedException e) {
    HotSpotDiagnosticMXBean bean = ManagementFactory
        .getPlatformMXBean(HotSpotDiagnosticMXBean.class); ❷
    try {
      Path path = Path.of("./structured-concurrency-error.json");
      bean.dumpThreads(path.toAbsolutePath().toString(),
        HotSpotDiagnosticMXBean.ThreadDumpFormat.JSON); ❸
      System.out.println("Thread dump captured: " + path);
    } catch (IOException ex) {
      throw new RuntimeException("Failed to generate thread dump", ex);
    }
    throw new RuntimeException("Processing failed", e);
  } catch (InterruptedException e) {
    Thread.currentThread().interrupt();
    throw new RuntimeException("Processing interrupted", e);
  }
}
```

В данном коде мы выполнили несколько действий.

❶ Намеренно внесли случайные сбои, чтобы активировать обработку ошибок и генерацию дампа потоков.

❷ Получили ссылку на `HotSpotDiagnosticMXBean` для программной генерации дампа потоков.

❸ При возникновении ошибки автоматически сгенерировали дамп потоков в формате JSON для последующего анализа.

https://max.ru/bookofgeek

Можно уверенно сделать вывод, что структурированная конкурентность не только упрощает конкурентное программирование, но и улучшает отладку и поддержку кода, обеспечивая более четкое представление о выполнении задач и их взаимосвязях.

В заключение

Появление структурированной конкурентности — значительный шаг вперед в конкурентном программировании на Java. Привязка жизненного цикла подзадач к родительским задачам обеспечивает новый уровень надежности, возможности сопровождения и наблюдаемости в конкурентных приложениях.

В этой главе вы увидели, как структурированная конкурентность решает фундаментальные проблемы конкурентного программирования. Четкое управление ресурсами обеспечивает автоматическую очистку подзадач, предотвращая утечки и появление «зомби-потоков», что характерно для традиционного подхода. Иерархическая обработка ошибок позволяет сбоям естественно распространяться по иерархии задач, благодаря чему обработка ошибок становится предсказуемой и полной. Улучшенные возможности отладки: благодаря структурированным дампам потоков с ясной видимостью связей между задачами поиск ошибок превращается из гадания в системный анализ. Кастомные джоинеры поддерживают сложные стратегии координации, сохраняя при этом безопасность структурированной конкурентности. Пожалуй, наиболее важное — вложенная структура отражает бизнес-логику, поэтому конкурентный код читается и поддерживается так же легко, как последовательный.

Часто возникает вопрос: совместима ли структурированная конкурентность как с виртуальными, так и с платформенными потоками? Да, она работает с обоими типами, позволяя разработчикам выбирать подходящий вариант. Например, `StructuredTaskScope` можно сконфигурировать для платформенных потоков через кастомную `ThreadFactory`:

```
public void demonstratePlatformThreads() throws InterruptedException {
  ThreadFactory platformFactory = Thread.ofPlatform()
    .name("platform-worker-", 0)
    .factory();

  try (var scope = open(Joiner.allSuccessfulOrThrow(),
       cf -> cf.withThreadFactory(platformFactory))) {

    scope.fork(() -> performTask("Task 1"));
    scope.fork(() -> performTask("Task 2"));

    scope.join();
  }
}
```

https://max.ru/bookofgeek

Однако это возвращает нас к изначальной проблеме: ограниченности платформенных потоков. Необходимо учитывать потенциальные ограничения масштабируемости и ресурсов, связанные с платформенными потоками в сценариях с высокой конкурентностью. Виртуальные потоки являются предпочтительным выбором для большинства приложений со структурированной конкурентностью. Они предлагают преимущества масштабируемости, делая эту модель программирования по-настоящему мощной.

Независимо от того, управляете ли вы входящими серверными запросами, координируете вызовы микросервисов или работаете со сложными многоэтапными процессами, структурированная конкурентность предоставляет мощный и интуитивно понятный каркас для работы со сложностями конкурентного программирования. Приняв эту парадигму, вы сможете писать не только более эффективный и надежный код, но и более простой для понимания, что облегчает работу с ним.

https://max.ru/bookofgeek

ГЛАВА 5

Scoped values

> Тот, кто не расставляет все по своим местам,
> совершает несправедливость.
>
> *Али ибн Абу Талиб*
> *(да будет доволен им Аллах)*

В этой главе мы рассмотрим `ScopedValue` — важное дополнение Java, окончательно реализованное в JDK 25. Оно предлагает структурированный способ привязывать значения к определенной области, сохраняя их доступность и согласованность с контекстом. Традиционные переменные типа `ThreadLocal` довольно громоздкие и порой вызывают утечки памяти. В отличие от них, `ScopedValue` предлагает для многопоточных приложений более понятный и эффективный подход. В этой главе мы разберемся, почему `ScopedValue` — лучшая альтернатива для `ThreadLocal`, изучим его API и рассмотрим практические примеры использования.

В чем проблема передачи контекста

Нередко возникают ситуации, когда необходимо обмениваться данными между различными частями кода, а использовать для этого параметры методов невозможно. Это особенно характерно для приложений, зависящих от какого-либо фреймворка. Поясню данную ситуацию на примере.

Представим себе систему планирования заданий, где пользовательский код регистрирует задачи, а их выполнение берет на себя фреймворк. При запуске задания фреймворк создает объект `JobContext`, содержащий метаданные: имя задания, приоритет и ограничения, влияющие на планирование. Этот контекст необходим для работы фреймворка, но к пользовательскому коду обычно не имеет отношения.

https://max.ru/bookofgeek

Следующий пример демонстрирует типичный фреймворк для планирования заданий, который страдает от так называемой «проблемы передачи параметров»:

```
import java.time.Instant;
import java.util.HashMap;
import java.util.Map;

interface Job {
  void execute(JobContext context);
}

enum Priority { LOW, MEDIUM, HIGH }

public record JobContext(String jobName, Priority priority,
                         Map<String, Object> metadata) {
  public JobContext(String jobName, Priority priority) {
    this(jobName, priority, new HashMap<>());
    metadata.put("jobName", jobName);
    metadata.put("priority", priority);
    metadata.put("creationTime", Instant.now());
  }

  public Object getMetadataValue(String key) {
    return metadata.get(key);
  }
}
```

Основная логика планирования фреймворка создает контекст заданий и управляет им:

```
// Код фреймворка
public class JobScheduler {
  public void schedule(Job job, String jobName, Priority priority) { ❶
    JobContext context = new JobContext(jobName, priority);
    runJob(job, context);
  }

  private void runJob(Job job, JobContext context) { ❷
    // Фреймворк вызывает пользовательский код, передавая контекст
    job.execute(context);
  }

  public Object getJobMetadata(String key, JobContext context) { ❸
    if (context == null) {
      return null;
    }
    return context.getMetadataValue(key);
  }
}
```

Для использования этого фреймворка мы реализуем интерфейс `Job` и будем работать с его объектом `context`:

https://max.ru/bookofgeek

```
public class UserJob implements Job {
  private final JobScheduler jobScheduler;

  public UserJob(JobScheduler jobScheduler) {
    this.jobScheduler = jobScheduler;
  }

  @Override
    public void execute(JobContext context) { ❹
      System.out.println("User job is running!");

      // Пользовательский код обращается к фреймворку,
      // чтобы получить метаданные
      Object creationTime
          = jobScheduler.getJobMetadata("creationTime", context); ❺
      System.out.println("Job creation time: " + creationTime);
      // Вспомогательные методы также требуют параметр context
      processJobData(context); ❻
    }

    private void processJobData(JobContext context) { ❼
      // Хотя этот метод может не использовать контекст напрямую,
      // ему нужен параметр для передачи другим методам фреймворка
      Object priority = jobScheduler.getJobMetadata("priority", context);
      System.out.println("Processing job with priority: " + priority);
    }
}
```

❶ Фреймворк создает `JobContext`, который содержит метаданные, необходимые на протяжении всего жизненного цикла задания.

❷ Этот `context` приходится передавать в пользовательский код, хотя пользователей не интересует внутренняя работа фреймворка.

❸ Методам фреймворка требуется параметр контекста для доступа к метаданным, что заставляет передавать его через всю цепочку вызовов.

❹ Пользовательские реализации вынуждены принимать параметр `JobContext` в свой метод `execute()`.

❺ Когда пользовательскому коду нужны сервисы фреймворка, он должен передавать `context` обратно во фреймворк.

❻ Вспомогательные методы в пользовательском коде «засоряются» параметрами фреймворка.

❼ Каждый метод в цепочке вызовов требует параметр `context`, даже если не использует его напрямую.

Хотя такой подход работает корректно, он создает ряд архитектурных проблем, которые усугубляются по мере масштабирования приложений.

https://max.ru/bookofgeek

Загрязнение параметрами

`JobContext` — это в первую очередь внутренний механизм фреймворка, который разработчикам, использующим этот фреймворк, знать необязательно. Но поскольку фреймворк должен управлять своим внутренним контекстом вдоль всей цепочки вызовов — от `schedule()` до пользовательского кода в `execute()`, а затем обратно во фреймворк в `getJobMetadata()`, — пользовательский код вынужден передавать параметры `JobContext`.

Каждый метод, участвующий в выполнении задания, должен объявлять этот параметр, даже если сам метод не использует контекст. Это загромождает сигнатуры методов специфичными для фреймворка деталями, которые не имеют отношения к бизнес-логике приложения.

Уязвимость интерфейсов

Что же происходит при развитии фреймворка? Если впоследствии потребуется расширить `JobContext` дополнительными данными (например, добавить новое поле `job category`, контекст распределенной трассировки или ссылку на логгер), то может потребоваться модифицировать каждую сигнатуру метода в пользовательском коде, который передает контекст. Сам класс `JobContext` поддерживает обратную совместимость, однако его сквозная передача через пользовательский код приводит к тому, что любые изменения требований фреймворка распространяются по всей кодовой базе.

Стыковка и тестируемость

Пользовательский код оказывается тесно связанным с деталями реализации фреймворка. Тестирование отдельных методов усложняется, поскольку требуется всегда предоставлять актуальный `JobContext`, даже при тестировании не имеющей отношения к фреймворкам бизнес-логики.

Такая стыковка также затрудняет переход с одного фреймворка на другой и отделение бизнес-логики для повторного использования в ином контексте.

Использование ThreadLocal

Для решения этой проблемы код фреймворка можно спроектировать с использованием `ThreadLocal` — инструмента, который часто применяется в таком коде.

Перепишем предыдущий код с применением `ThreadLocal`:

```
public class JobScheduler {
  private static final ThreadLocal<JobContext> jobContextHolder =
    new ThreadLocal<>(); ❶
```

https://max.ru/bookofgeek

```
  public void schedule(Job job, String jobName, Priority priority) {
    JobContext context = new JobContext(jobName, priority);
    try {
      jobContextHolder.set(context); ❷
      runJob(job);
    } finally {
      jobContextHolder.remove(); ❸
    }
  }

  private void runJob(Job job) { ❹
    job.execute();
  }

  public Object getJobMetadata(String key) {
    JobContext context = jobContextHolder.get(); ❺
    return (context != null) ? context.getMetadataValue(key) : null;
  }
}
```

❶ Создаем переменную `ThreadLocal`, чтобы хранить `JobContext` для каждого потока.

❷ Устанавливаем `context` в текущем потоке перед выполнением задания.

❸ Всегда удаляем `context` в блоке `finally`, чтобы не допустить утечки памяти.

❹ Методу `runJob` больше не нужны параметры контекста.

❺ По мере необходимости методы фреймворка могут получать контекст `ThreadLocal`.

Теперь пользовательский код стал значительно чище, отпала необходимость обрабатывать контекст фреймворка:

```
public class UserJob implements Job {
  private final JobScheduler jobScheduler;

  public UserJob(JobScheduler jobScheduler) {
    this.jobScheduler = jobScheduler;
  }

  @Override
    public void execute() { ❶
      System.out.println("User job is running!");
      // Параметр контекста не нужен - фреймворк обрабатывает его внутренне
      Object creationTime = jobScheduler.getJobMetadata("creationTime"); ❷
      System.out.println("Job creation time: " + creationTime);
      // Вспомогательные методы теперь без параметра контекста
      processJobData();
    }
```

https://max.ru/bookofgeek

```
    private void processJobData() { ❸
      // Чистая сигнатура метода - без параметров фреймворка
      Object priority = jobScheduler.getJobMetadata("priority");
      System.out.println("Processing job with priority: " + priority);
    }
}
```

❶ Мы полностью освободили пользовательский код от забот о контексте фреймворка.

❷ Сервисы фреймворка доступны без параметров контекста.

❸ Сигнатуры вспомогательных методов сфокусированы только на бизнес-логике.

Данный подход эффективно решает проблему передачи параметров. Пользовательскому коду больше не требуются присущие фреймворку параметры, что позволяет сосредоточиться на бизнес-логике. Внутреннее управление контекстом фреймворка теперь полностью скрыто от пользовательского кода.

Шаблон настолько эффективен, что практически все современные фреймворки в той или иной форме используют `ThreadLocal` для управления контекстом. Например, Spring Framework активно применяет `ThreadLocal` для работы с контекстами безопасности, транзакций и запросов.

Ограничения переменных ThreadLocal

Несмотря на многочисленные преимущества, переменные `ThreadLocal` имеют ряд конструктивных недостатков. Рассмотрим их подробнее.

Во-первых, при использовании `ThreadLocal` изменяемость ничем не ограничена. Любой код, имеющий доступ к методу `get()` переменной `ThreadLocal`, может также вызывать `set()` и в любой момент внести изменения. Это затрудняет отслеживание времени и места модификации данных.

Рассмотрим следующий код:

```
public class MutableLoggingContext {
  // ThreadLocal для хранения текущего уровня логирования
  private static final ThreadLocal<String> LOG_LEVEL = new ThreadLocal<>(); ❶

  public static void setLogLevel(String level) {
    LOG_LEVEL.set(level); ❷
  }

  public static String getLogLevel() {
    return LOG_LEVEL.get();
  }
```

https://max.ru/bookofgeek

```
  public static void log(String message) {
    System.out.println("[" + getLogLevel() + "] " + message);
}

  public static void main(String[] args) throws InterruptedException {
    setLogLevel("INFO");
    log("Starting process..."); ❸
    Thread thread = new Thread(() -> {
      setLogLevel("DEBUG"); ❹
      log("Thread-specific debug mode enabled");
    });
    thread.start();
    Thread.sleep(100); // Даем время на выполнение другого потока
    log("Main thread still at INFO level"); ❺
  }
}
```

❶ Любой код имеет доступ к данному статическому `ThreadLocal`.

❷ Уровень логирования можно изменить из любой части кодовой базы.

❸ Главный поток устанавливает свой уровень логирования в `INFO`.

❹ Дочерний поток независимо устанавливает свой уровень логирования в `DEBUG`.

❺ Уровень логирования главного потока остается `INFO` (изоляция на уровне потоков).

Нетрудно заметить, что уровень логирования может устанавливаться практически в любом месте кода, из-за чего трудно понять, где именно он был задан.

Во-вторых, у переменной неограниченный срок жизни. После установки значения в `ThreadLocal` оно сохраняется на протяжении всего времени жизни потока, пока не будет явно удалено. Для обычных потоков это не проблема, но в пуле, где одни и те же потоки используются неоднократно, возможна утечка данных из одной задачи в другую, если забыли вызвать `remove()`. Появляется вероятность утечки памяти или даже уязвимости безопасности, если непреднамеренно сохранятся конфиденциальные данные.

Рассмотрим пример:

```
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;

public class ThreadLocalLeakExample {
  private static final ThreadLocal<String> currentUser = new ThreadLocal<>();

  public static void main(String[] args) throws InterruptedException {
    try (ExecutorService executor = Executors.newFixedThreadPool(1)) { ❶
```

https://max.ru/bookofgeek

```
    // Первая задача устанавливает текущего пользователя
    executor.submit(() -> {
      currentUser.set("Alice"); ❷
      System.out.println("Task 1: currentUser = " + currentUser.get());
      // Забыли вызвать currentUser.remove()! ❸
    });
    Thread.sleep(100); // Ожидаем завершения задачи 1
    // Вторая задача использует тот же поток
    executor.submit(() -> {
      System.out.println("Task 2: Leaked value = " + currentUser.get()); ❹
        currentUser.set("Bob");
        System.out.println("Task 2: currentUser = " + currentUser.get());
        currentUser.remove();
        // Очистили перед завершением задачи
      });
    }
  }
}
```

В этом примере мы видим следующее.

❶ Поскольку пул состоит из одного потока, оба задания выполняются в одном и том же потоке.

❷ Первая задача устанавливает значение переменной.

❸ Поскольку очистки не будет, значение сохранится.

❹ Вторая задача получает «утекшее» из первого задания значение.

Если ссылки на объекты остаются дольше, чем требуется, может произойти утечка памяти. Кроме того, появляются уязвимости, связанные с утечкой конфиденциальных данных (например, токенов аутентификации) или персональной информации пользователей при передаче между запросами, не имеющими к ним отношения. Наконец, это может привести к некорректному поведению системы, когда задачи выполняются с устаревшим или ошибочным контекстом.

При использовании `InheritableThreadLocal` дочерние потоки автоматически наследуют значения от родительского потока. Это удобно, но может оказаться слишком затратным, если создается большое количество дочерних потоков.

Взглянем на следующий пример:

```
public class InheritanceOverheadExample {
  private static final InheritableThreadLocal<byte[]> LARGE_DATA =
    new InheritableThreadLocal<>(); ❶

  public static void main(String[] args) {
    // Родительский поток устанавливает большой объект
    LARGE_DATA.set(new byte[10_000_000]); // 10 Мбайт ❷
    // Создаем множество дочерних потоков
    for (int i = 0; i < 100; i++) {
      new Thread(() -> {
        // Каждый дочерний поток получает ссылку на данные родителя ❸
```

https://max.ru/bookofgeek

```
        byte[] inherited = LARGE_DATA.get();
        System.out.println("Child has access to " +
          inherited.length + " bytes");
      }).start();
    }
  }
}
```

Рассмотрим проблемы наследования.

❶ `InheritableThreadLocal` автоматически копирует значения в дочерние потоки.

❷ Большие объекты остаются доступными для всех дочерних потоков.

❸ Данные сохраняются, даже если дочерние потоки их не используют.

На этом примере отчетливо видны недостатки `InheritableThreadLocal` при работе с большими объектами. Хотя дочерние потоки явно не устанавливают и не изменяют локальную переменную, они автоматически наследуют собственную отдельную копию данных родительского потока, то есть объем используемой памяти удваивается.

Выяснив проблему переменной `ThreadLocal`, поищем решение.

Облегченная модель совместного использования данных

С появлением виртуальных потоков (JEP 444, https://oreil.ly/nXHrR) ограничения `ThreadLocal` стали заметнее. В отличие от традиционных платформенных потоков, где каждому потоку выделяются собственные ресурсы операционной системы, тысячи или даже миллионы легковесных виртуальных потоков может обслуживать один поток ОС. Технически каждый виртуальный поток способен хранить собственные данные `ThreadLocal`, однако очень быстро перерасход памяти становится неприемлемым. Представьте миллион виртуальных потоков, каждый из которых содержит свой фрагмент состояния, — затраты будут ошеломляющими. Очевидно, что нам требуется более эффективный подход.

Поскольку виртуальные потоки существуют недолго, проблема долгоживущих переменных `ThreadLocal` менее критична — сборщик мусора удалит их; однако расход памяти из-за множества копий все равно остается высоким. В идеале нужен новый механизм, который позволит хранить наследуемые данные для каждого потока без создания большого количества копий. Если данные неизменяемые, это еще лучше: дочерние потоки могут ссылаться на одну общую версию без дополнительного дублирования. Необходимо также ограничить время жизни таких данных: как только использующий их метод завершит работу, все прикрепленные переменные `ThreadLocal` должны потерять релевантность. В конце концов, они предназначены для временного, а не постоянного хранения в памяти состояния выполняемой задачи.

Всем этим условиям удовлетворяет новый API `ScopedValue` (https://oreil.ly/Bb5W2). Познакомимся с ним.

https://max.ru/bookofgeek

Основные компоненты ScopedValue

`ScopedValue` выступает в роли неявного параметра метода, позволяя передавать данные через последовательность вызовов методов без явного объявления каждого из них в сигнатуре. Это делает код чище и удобнее в сопровождении, особенно при работе с глубоко вложенными вызовами или колбэк-структурами.

Вот три основные характеристики.

Неизменяемость (immutability)

После привязки `ScopedValue` к значению внутри определенной области его нельзя изменить, что гарантирует предсказуемое поведение на протяжении всего времени жизни.

Привязка к текущему потоку (thread-scoped binding)

Привязки ограничены текущим потоком, что предотвращает нежелательный обмен данными между потоками и укрепляет потокобезопасность.

Ограниченное время жизни (bounded lifetime)

Привязка `ScopedValue` ограничена временем выполнения конкретного блока кода, после чего автоматически снимается, благодаря чему улучшается управление ресурсами и снижается риск утечек памяти.



`ScopedValue`, с которым мы знакомимся в этой главе, стал доступен в JDK 25. Если вы работаете в более старой версии JDK, на этапах компиляции и выполнения используйте флаг `--enable-preview`. Например, для компиляции и запуска кода в JDK 24 предназначены следующие команды.

- Для javac: `javac --release 24 --enable-preview Main.java`
- Для java: `java --enable-preview Main`
- Через лаунчер исходного кода: `java --enable-preview Main.java`
- С помощью JShell: `jshell --enable-preview`

Ожидается, что эта функциональность будет интегрирована в будущие релизы JDK без необходимости как-то модифицировать код.

Чтобы использовать значение с ограниченной областью действия — scoped value, сначала объявляем его как статическое финальное (static final) поле. Объявляем `ScopedValue` тем же способом, что и `ThreadLocal`:

```
private static final ScopedValue<String> NAME = ScopedValue.newInstance();
```

Мы не используем оператор `new` для создания экземпляра, а применяем фабричный метод, поскольку конструкторы класса намеренно сделаны приватными.

https://max.ru/bookofgeek

Затем мы можем привязать значение к scoped value и выполнить код внутри данной области. Для этого используются методы `where()` (https://oreil.ly/2lCFt) и `run()` (https://oreil.ly/ x73-z):

```
ScopedValue.where(NAME, "duke").run(() -> doSomething());
```

Здесь метод `where()` привязывает scoped value `NAME` к текущему пользователю. Метод `run()` выполняет предоставленный блок кода (в данном случае лямбда-выражение) в области действия привязанного значения. Код внутри метода `run()` получает доступ к scoped value с помощью метода `get()`. В нашем случае метод `doSomething()` получит доступ к этому значению, вызвав метод `get()`:

```
NAME.get()
```

Чтобы заменить класс `JobScheduler` на `ScopedValue`, сделаем следующее:

```
private static final ScopedValue<JobContext> CONTEXT
                     = ScopedValue.newInstance(); ❶

  public void schedule(Job job, String jobName, Priority priority) {
    JobContext context = new JobContext(jobName, priority); ❷
    ScopedValue.where(CONTEXT, context)
        .run(() -> runJob(job)); ❸
  }

  private void runJob(Job job) {
    job.execute(); ❹
  }

  public static JobContext getContext() { ❺
    return CONTEXT.get();
  }

  public static Object getJobMetadata(String key) {
    JobContext context = CONTEXT.get(); ❻
    if (context != null) {
      return context.getMetadataValue(key);
    }
    return null;
  }
}
```

Разберем ключевые моменты этой реализации.

❶ Создаем экземпляр `ScopedValue` с помощью фабричного метода.

❷ Строим объект `context` для привязки.

❸ Привязываем `context` и выполняем задание в рамках области.

❹ Задание выполняется с доступом к этому ограниченному областью контексту.

https://max.ru/bookofgeek

❺ Предоставляем статический доступ к текущему контексту (более чистое решение по сравнению с исходным).

❻ Безопасно получаем метаданные с проверкой на `null`.

Принцип работы вы поняли, теперь рассмотрим детали.

В предыдущих примерах для выполнения кода внутри области использовался метод `run()`, однако `ScopedValue` также предоставляет метод `call()`, который позволяет вернуть значение из выполняемого блока. Различие очевидно: `run()` выполняет `Runnable` (возвращает void), а `call()` выполняет `Callable` (возвращает значение).

Рассмотрим следующий пример кода:

```
private static final ScopedValue<Double> DISCOUNT_RATE
                                    = ScopedValue.newInstance();

  public double calculatePrice(double basePrice) {
    // Используем call(), чтобы вернуть из области подсчитанную цену
    return ScopedValue.where(DISCOUNT_RATE, 0.20) // 20% discount
        .call(() -> basePrice * (1 - DISCOUNT_RATE.get()));
  }

  void main() {
    PricingService service = new PricingService();
    double finalPrice = service.calculatePrice(100.0);
    System.out.println("Final price: $" + finalPrice);
    // Вывод: Final price: $80.00
  }
}
```

В этом примере нам нужно вернуть подсчитанную цену, поэтому используем `call()`. Этот метод особенно полезен для вычислений, преобразований и любых операций, где требуется получить результат, сохраняя при этом доступ к контексту области выполнения.

Запуск ScopedValue

`ScopedValue` имеет ограниченное время действия. Прежде чем использовать переменную, сначала нужно ее определить. Рассмотрим следующий пример:

```
public static void main(String[] args) {

  ScopedValue<String> NAME = ScopedValue.newInstance();

  Runnable task = () -> {
    if (NAME.isBound()) {
      System.out.println("Name is bound: " + NAME.get());
    } else {
```

https://max.ru/bookofgeek

```
      System.out.println("Name is not bound");
    }
  };

  task.run();
}
```

При запуске этого кода будет выведено `Name is not bound`, так как значение еще не установлено. Для привязки значения и выполнения кода в соответствующей области мы используем методы `where()` и `run()`:

```
public static void main(String[] args) {
  ScopedValue<String> NAME = ScopedValue.newInstance();

  Runnable task = () -> {
    if (NAME.isBound()) {
        System.out.println("Name is bound: " + NAME.get());
    } else {
        System.out.println("Name is not bound");
    }
  };
  ScopedValue.where(NAME, "Bazlur") ❶
             .run(task); ❷
}
```

Такой подход:

❶ привязывает значение `Bazlur` к scoped value `NAME`;

❷ выполняет задачу в области действия этой привязки.

Этот код будет выполняться в контексте главного потока, так как его запускает метод `main`. В результате мы получим следующий вывод:

```
Name is bound: Bazlur
```

Возникает вопрос: можно ли после того, как мы установили `ScopedValue` с помощью `where().run()`, выполнить задачу отдельно и получить значение?

Сейчас проверим:

```
public static void main(String[] args) {
  ScopedValue<String> NAME = ScopedValue.newInstance();
  Runnable task = () -> {
    if (NAME.isBound()) {
      System.out.println("Name is bound: " + NAME.get());
    } else {
      System.out.println("Name is not bound");
    }
  };
  // Выполнение внутри области
  ScopedValue.where(NAME, "Bazlur").run(task); ❶
  // Попытка выполнения вне области
  task.run(); ❷
}
```

https://max.ru/bookofgeek

Мы видим следующее.

❶ В первый раз выполнение происходит в контексте, привязанном к scoped value.

❷ Во второй — выполнение происходит вне этой области, поэтому значение больше не привязано.

Вывод будет такой:

```
Name is bound: Bazlur
Name is not bound
```

`ScopedValue` остается привязанным только в пределах динамической области вызова метода `run()`. После завершения метода привязка автоматически удаляется, что обеспечивает четкие границы области действия и предотвращает утечку значений между несвязанными секциями кода.



Область (scope) значения определяет, где оно существует и где к нему можно получить доступ. В Java под областью часто подразумевается лексическая область видимости (lexical scope), которая задается блоками `{}`. Переменные в ней доступны только в пределах объявленных границ. Однако `ScopedValue` работает с динамической областью (dynamic scope), определяемой потоком выполнения кода (execution flow).

В динамической области значение доступно во время выполнения определенных методов, а также методов, вызываемых ими прямо или косвенно. Например, если метод `a` вызывает `b`, а `b` вызывает `c`, то область распространяется на `c`, но заканчивается, когда `c` завершается. `ScopedValue` привязывает значение к потоку выполнения, делая его доступным только в пределах области действия метода `run`.

Именно точное ограничение области и времени действия делает `ScopedValue` более чистой и безопасной альтернативой для `ThreadLocal`, особенно при передаче контекстных данных.

Возникает еще один вопрос: можно ли выполнить задачу не в главном потоке, а в другом?

Рассмотрим следующий код:

```
public static void main(String[] args) throws InterruptedException {
  ScopedValue<String> NAME = ScopedValue.newInstance();

  Runnable task = () -> {
    if (NAME.isBound()) {
      System.out.println("Name is bound: " + NAME.get());
    } else {
      System.out.println("Name is not bound");
    }
  };
```

https://max.ru/bookofgeek

```
  Thread thread = Thread.ofPlatform().unstarted(task); ❶
  ScopedValue.where(NAME, "Bazlur")
             .run(thread::start); ❷

  thread.join();
}
```

❶ Создаем пока не запущенный платформенный поток с нашей задачей.

❷ Привязываем scoped value и запускаем поток внутри этой области.

После выполнения этого кода получаем следующий вывод:

```
Name is not bound
```

Дело в том, что scoped value не наследуется автоматически вновь созданными потоками. Хотя `thread::start` выполняется в области, где привязан `NAME`, актуальная задача выполняется в отдельном потоке, который не наследует привязку scoped value. Так себя ведут как платформенные, так и виртуальные потоки.

Теперь вместо создания `ScopedValue` в главном потоке перенесем его во вновь созданный поток:

```
public static void main(String[] args) throws InterruptedException {
  ScopedValue<String> NAME = ScopedValue.newInstance();

  Runnable task = () -> {
    if (NAME.isBound()) {
      System.out.println("Name is bound: " + NAME.get());
    } else {
      System.out.println("Name is not bound");
    }
  };

  Thread thread = Thread.ofVirtual().start(() -> { ❶
    ScopedValue.where(NAME, "Bazlur")
               .run(task); ❷
  });

  thread.join();
}
```

Теперь код работает как положено.

❶ Создает и запускает виртуальный поток.

❷ Связывает scoped value и выполняет задачу в контексте нового потока.

`ScopedValue` предоставляет fluent API, позволяющий связывать несколько `ScopedValue` в цепочку.

https://max.ru/bookofgeek

Рассмотрим следующий пример:

```
public class MultiScopedExample {
  private static final ScopedValue<String> USER_ID
                                        = ScopedValue.newInstance();
  private static final ScopedValue<String> SESSION_ID
                                        = ScopedValue.newInstance();

  public static void main(String[] args) {
    ScopedValue.where(USER_ID, "user123") ❶
        .where(SESSION_ID, "session456") ❷
        .run(() -> performTask()); ❸
  }

  public static void performTask() {
    String userId = USER_ID.get();
    String sessionId = SESSION_ID.get();
    System.out.println("Performing task for user: " + userId +
        " in session: " + sessionId);
    logAction();
  }

  public static void logAction() {
    String userId = USER_ID.get(); ❹
    String sessionId = SESSION_ID.get();
    System.out.println("Logging action for user: " + userId +
        " in session: " + sessionId);
  }
}
```

Рассмотрим цепочечный подход.

❶ Связываем первое scoped value.

❷ Добавляем следующую привязку в цепочку.

❸ Выполняем код с обоими привязанными значениями.

❹ Обеспечиваем доступность обоих значений во всем стеке вызовов.

`ScopedValue` имеет два удобных метода: `orElse` (https://oreil.ly/BkL2X) и `orElseThrow` (https://oreil.ly/pOT8D). Они применяются, если в `ScopedValue` нет установленного значения и требуется использовать значение по умолчанию или необходимо выбросить исключение.

Рассмотрим следующий пример:

```
public class ScopedValueDefaultsExample {
  private static final ScopedValue<String> USER_NAME
      = ScopedValue.newInstance();

  public static void main(String[] args) {
    // Используем orElse для значений по умолчанию
    String userNameUnbound = USER_NAME.orElse("Guest"); ❶
```

https://max.ru/bookofgeek

```
    System.out.println("No binding -> user name defaults to: "
        + userNameUnbound);
    // Используем orElseThrow для валидации
    try {
      USER_NAME.orElseThrow(() ->
          new IllegalStateException("No user name bound yet!")); ❷
    } catch (IllegalStateException e) {
      System.out.println("Caught exception: " + e.getMessage());
    }
    // Внутри привязанной области
    ScopedValue.where(USER_NAME, "Bazlur").run(() -> {
      String boundUserName = USER_NAME.orElse("Guest"); ❸
      System.out.println("Within binding -> user name is: " + boundUserName);
      // Исключение не будет выброшено, так как значение привязано
      String validatedName = USER_NAME.orElseThrow(()
          -> new IllegalStateException("No user name bound yet!")); ❹
      System.out.println("Validated name: " + validatedName);
    });
  }
}
```

Эти методы делают следующее.

❶ Обеспечивают безопасный доступ со значением по умолчанию при отсутствии привязки.

❷ Позволяют выполнять валидацию с выбрасыванием кастомного исключения при отсутствии привязки.

❸ Возвращают привязанное значение (игнорируя значение по умолчанию).

❹ Могут использоваться без аргументов, когда вы уверены в наличии значения.

Такой дизайн API гарантирует, что код может корректно обрабатывать как привязанное, так и непривязанное значение, благодаря чему scoped values — надежное решение для различных сценариев использования.

Перепривязка ScopedValue во вложенных областях

Одна из мощных возможностей `ScopedValue` — способность перепривязать *(rebind)* значения во вложенных областях. Перепривязка позволяет временно назначить новое значение тому же `ScopedValue` внутри конкретной подобласти. После завершения подобласти исходное значение автоматически восстанавливается, что обеспечивает четкое и предсказуемое управление контекстом.

Эта возможность особенно полезна в таких сценариях, как управление доступом на основе ролей, где роль пользователя может временно изменяться в ходе определенной операции. Она также позволяет реализовывать

https://max.ru/bookofgeek

контекстно-зависимые конфигурации (например, настройки для конкретного потока выполнения могут быть изменены без влияния на основной контекст). Кроме того, перепривязка поддерживает переопределение на уровне задачи, предоставляя временные данные для конкретной задачи или действия.

Рассмотрим очередной пример:

```
public class ScopedValueRebindingExample {
  private static final ScopedValue<String> USER_ROLE = ScopedValue.newInstance();
  public static void main(String[] args) {
    // Привязываем начальное значение во внешней области
    ScopedValue.where(USER_ROLE, "Admin").run(() -> { ❶
      System.out.println("Outer scope: User role is " + USER_ROLE.get());
      performTask();
      // Перепривязываем во вложенной области
      ScopedValue.where(USER_ROLE, "Guest").run(() -> { ❷
        System.out.println("Inner scope: User role is " + USER_ROLE.get());
        performTask();
      }); ❸
      // Исходное значение автоматически восстанавливается
      System.out.println("Back to outer scope: User role is " + USER_ROLE.get());
      performTask();
    });
  }
  public static void performTask() {
    System.out.println(" Performing task as: " + USER_ROLE.get()); ❹
  }
}
```

Разберем ключевые моменты этого примера.

❶ Устанавливаем привязку исходного значения `Admin` во внешней области.

❷ Создаем вложенную область, которая временно перепривязывается к значению `Guest`.

❸ При выходе из этой области исходное значение Admin автоматически восстанавливается.

❹ Один и тот же метод получает разные значения в зависимости от текущей области.

Запустив код, мы получим следующий вывод:

```
Outer scope: User role is Admin
  Performing task as: Admin
Inner scope: User role is Guest
  Performing task as: Guest
Back to outer scope: User role is Admin
  Performing task as: Admin
```

https://max.ru/bookofgeek

ScopedValue и структурированная конкурентность

`ScopedValue` прекрасно вписывается в работу со структурированной конкурентностью. Как мы видели в предыдущем примере, `ScopedValue` не наследуется дочерними потоками по умолчанию, но при использовании внутри `StructuredTaskScope` привязки `ScopedValue` автоматически наследуются всеми дочерними потоками, созданными в этой области. Такой механизм обеспечивает эффективный обмен данными между родительским и дочерними потоками без явной передачи данных через аргументы. Это возможно благодаря четко определенным границам структурированной конкурентности. При выходе из `StructuredTaskScope` все созданные в этой области потоки прерываются и затем удаляются сборщиком мусора, что исключает проблемы с утечкой памяти.

Рассмотрим пример:

```
import java.util.concurrent.StructuredTaskScope;

public class ScopedValueStructuredConcurrencyExample {
  private static final ScopedValue<String>
          USERNAME = ScopedValue.newInstance();

  public static void main(String[] args) {
    ScopedValue.where(USERNAME, "Bazlur").run(() -> {
      doSomething();
    });
  }

  public static void doSomething() {
    try (var scope = StructuredTaskScope.open()) {
      StructuredTaskScope.Subtask<String> task1 = scope.fork(()
                -> USERNAME.get() + " from task 1");
      StructuredTaskScope.Subtask<String> task2 = scope.fork(()
                -> USERNAME.get() + " from task 2");
      scope.join();
      String result1 = task1.get();
      String result2 = task2.get();
      System.out.println(result1);
      System.out.println(result2);
    } catch (InterruptedException e) {
      throw new RuntimeException(e);
    }
  }
}
```

В этом примере со scoped value `USERNAME` связана строка `Bazlur` в главном потоке. Когда мы вызываем метод `doSomething()`, он создает `StructuredTaskScope` и запускает два дочерних потока. Они наследуют привязку `USERNAME` от родительского потока и могут получить к ней доступ с помощью `USERNAME.get()`.

https://max.ru/bookofgeek

Вопросы производительности

`ScopedValue` в целом демонстрирует лучшую производительность по сравнению с `ThreadLocal`, особенно при работе с виртуальными потоками. Это преимущество обуславливается несколькими факторами. Во-первых, снижается расход ресурсов: переменные `ThreadLocal` значительно увеличивают потребление памяти из-за того, что для каждого виртуального потока создается собственная копия. `ScopedValue`, наоборот, позволяет всем потокам совместно использовать данные в заданной области, минимизируя использование памяти и повышая производительность. Во-вторых, `ScopedValue` оптимально работает с виртуальными потоками и структурированной конкурентностью. Облегченная природа виртуальных потоков способствует эффективному совместному использованию данных без узких мест, свойственных традиционным переменным `ThreadLocal`. Например, веб-сервер создает тысячи виртуальных потоков для обработки конкурентных запросов. Каждый запрос может требовать контекстных данных (сведений об аутентификации пользователя или другое). Используя `ScopedValue` для совместного доступа к этим данным в пределах области действия каждого запроса, мы значительно сокращаем потребление памяти и повышаем общую пропускную способность по сравнению с использованием `ThreadLocal`.

Удобство использования и проектирование API

Вдобавок к высокой производительности `ScopedValue` может похвастаться лучшим, чем у `ThreadLocal`, удобством в использовании. Во-первых, `ScopedValue` обеспечивает неизменяемость: после того, как значение привязано к `ScopedValue` внутри области, его нельзя изменить. Это упрощает анализ кода и снижает риск ошибок, вызванных непреднамеренными изменениями. В конкурентной среде неизменяемость особенно полезна, так как предотвращает состояние гонки и несогласованность данных, которые часто происходят при попытке нескольких потоков изменить общие переменные.

Во-вторых, `ScopedValue` явно задает жизненный цикл общих данных с помощью своего API. Метод `run()` четко обозначает область, где значение доступно. Код с явно указанным жизненным циклом легче читать, к тому же проще понять движение потока данных в программе. В отличие от `ThreadLocal`, обеспечиваются более четкие границы, в которых значения допустимы.

В-третьих, `ScopedValue` API более лаконичный и интуитивно понятный. Методы `where()` и `run()` предоставляют структурированный способ установки общих данных и доступа к ним в пределах определенной области. По сравнению с зачастую многословными и менее понятными методами `ThreadLocal` подход `ScopedValue` воспринимается как более естественный и удобный в работе.

К тому же `ScopedValue` действует как объект-возможность (capability object), который обеспечивает точный контроль над доступом к общим данным. Чтобы

https://max.ru/bookofgeek

повысить уровень безопасности и инкапсуляции, вы можете разрешить доступ только авторизованным компонентам, объявив объект `ScopedValue` с модификатором доступа `private`.

Наконец, `ScopedValue` обрабатывает значения `null` более явно, чем `ThreadLocal`. В `ThreadLocal` вызов `get()` вернет `null` как при явной установке значения в `null`, так и когда оно вовсе не задавалось. Это ведет к ошибкам, которые трудно обнаружить. `ScopedValue` действует иначе. Если значение не привязано, при вызове `get()` выбрасывается `NoSuchElementException`, поэтому ошибки при установке значения перехватываются на ранней стадии, а код становится более надежным и предсказуемым.

Миграция на ScopedValue

Очень похоже, что scoped values окажутся полезными и предпочтительными для многих сценариев, где сегодня используются переменные `ThreadLocal`. Помимо исполнения роли скрытых параметров методов, scoped values можно успешно применить и в других случаях.

Во-первых, иногда требуется обнаруживать рекурсию, если фреймворк не поддерживает многократное использование компонентов или глубину рекурсии нужно каким-то образом ограничить. Scoped value предоставляет способ выявления и обработки таких ситуаций.

Его можно настроить обычным способом через `ScopedValue.run()`, а затем в глубине стека вызовов вызвать `ScopedValue.isBound()`, чтобы проверить, есть ли привязка к текущему потоку. Более трудоемкий вариант — использование scoped values для моделирования счетчика рекурсии путем неоднократной перепривязки.

Рассмотрим систему обработки документов, работающую с вложенными шаблонами. Если нет ограничения рекурсии, злоумышленник может создать шаблон с циклической ссылкой, что приведет к аварийному отказу приложения. Рассмотрим следующий код:

```
public class TemplateProcessor {
  private static final ScopedValue<Integer> RECURSION_DEPTH
                                  = ScopedValue.newInstance(); ❶

  private static final int MAX_NESTING_LEVEL = 50;

  public String processTemplate(String template) {
    if (!RECURSION_DEPTH.isBound()) { ❷
      return ScopedValue.where(RECURSION_DEPTH, 0) ❸
                  .call(() -> processTemplateInternal(template));
    } else {
      return processTemplateInternal(template);
    }
  }
}
```

https://max.ru/bookofgeek

❶ Создаем `ScopedValue` для отслеживания глубины рекурсии при вызове методов.

❷ Используем `isBound()` в глубине стека вызовов, чтобы проверить, есть ли привязка к текущему потоку.

❸ Настраиваем как обычно, с помощью `ScopedValue.run()` (обратите внимание: `where()` и `call()` относятся к современному API).

Основная логика обработки выполняет парсинг шаблона и управляет рекурсией. Этот метод вызывается в ограниченной области контекста (scoped context) и имеет доступ к текущей глубине рекурсии:

```
private String processTemplateInternal(String template) {
  int currentDepth = RECURSION_DEPTH.get(); ❹

  if(currentDepth >= MAX_NESTING_LEVEL) { ❺
    throw new TemplateProcessingException(
      "Template nesting too deep: " + currentDepth + " levels");
  }

  StringBuilder result = new StringBuilder();

  // Упрощенная логика обработки шаблона
  int includeStart = template.indexOf("{{include:");
  if (includeStart >= 0) {
    int includeEnd = template.indexOf("}}", includeStart);
    String includePath = template.substring(includeStart + 10, includeEnd);

    String nestedContent = ScopedValue
      .where(RECURSION_DEPTH, currentDepth + 1) ❻
      .call(() -> processTemplateInternal(loadTemplate(includePath)));

    result.append(template, 0, includeStart);
    result.append(nestedContent);
    result.append(template.substring(includeEnd + 2));
  } else {
    result.append(template);
  }

  return result.toString(); ❼
}
```

❹ Получаем текущую глубину рекурсии в пределах области.

❺ Реализуем проверку безопасности, чтобы не допустить чрезмерной вложенности при работе в нереентерабельных фреймворках.

❻ Scoped value моделирует счетчик рекурсии путем повторной привязки с инкрементированными значениями.

❼ При возврате из метода предыдущее значение глубины автоматически восстанавливается.

https://max.ru/bookofgeek

Для завершения реализации нам потребуются вспомогательные методы для загрузки шаблонов и обработки ошибок. Метод `loadTemplate()` имитирует чтение шаблонов из файловой системы, включая такие, которые создают глубокую вложенность для тестирования:

```
private String loadTemplate(String path) {
  return switch (path) {
    case "header.tpl" -> "<!-- HEADER START -->";
    case "footer.tpl" -> "<!-- FOOTER END -->";
    default -> {
      if (path.startsWith("level")) {
        int level = Integer.parseInt(path.replaceAll("[^0-9]", ""));
        if (level < 10) {
          yield "Level " + level
            + " {{include:level" + (level + 1) + ".tpl}}";
        }
      }
      yield "<!-- Template content from " + path + " -->";
    }
  };
}
```

Проверим, как будет работать все вместе. Следующий метод `main` демонстрирует различные сценарии использования: от простых до глубоко вложенных шаблонов, которые активируют нашу защиту от рекурсии:

```
void main() {
  TemplateProcessor processor = new TemplateProcessor();

  // Пример 1: простой шаблон без вложенности
  String simpleTemplate = "Hello, this is a simple template!";
  System.out.println("Simple template result:");
  System.out.println(processor.processTemplate(simpleTemplate));
  System.out.println();

  // Пример 2: шаблон с вложенными включениями
  String nestedTemplate = "Header: {{include:header.tpl}} " +
                          "Content goes here {{include:footer.tpl}}";
  System.out.println("Nested template result:");
  System.out.println(processor.processTemplate(nestedTemplate));
  System.out.println();

  // Пример 3: отслеживание глубины рекурсии
  String deeplyNested = "Level 0 {{include:level1.tpl}}";
  System.out.println("Processing deeply nested template...");
  try {
    String result = processor.processTemplate(deeplyNested);
    System.out.println("Result: " + result);
  } catch (TemplateProcessingException e) {
    System.err.println("Error: " + e.getMessage());
  }
}
```

https://max.ru/bookofgeek

После выполнения кода мы получим следующий результат:

```
Simple template result:
Hello, this is a simple template!

Nested template result:
Header: <!-- HEADER START --> Content goes here <!-- FOOTER END -->

Processing deeply nested template...
Result: Level 0 Level 1 Level 2 Level 3 Level 4 Level 5 Level 6 Level 7 Level
8
Level 9 <!-- Template content from level10.tpl -->
```

Во-вторых, обнаружение рекурсии также полезно в случае плоских транзакций: любая транзакция, начатая во время выполнения другой, становится частью самой внешней транзакции.

Рассмотрим упрощенный пример:

```
public class FlattenedTransactionExample {
  private static final ScopedValue<Transaction> CURRENT_TRANSACTION =
    ScopedValue.newInstance();

  public static void main(String[] args) {
    performBusinessOperation();
  }
  private static void performBusinessOperation() {
    // Старт внешней транзакции
    Transaction outerTx = new Transaction("OUTER_TX");
    ScopedValue.where(CURRENT_TRANSACTION, outerTx).run(() -> { ❶
      System.out.println("Starting: " + outerTx.name());
      // Вложенная операция, которая может инициировать собственную транзакцию
      performNestedOperation();
      System.out.println("Committing: " + outerTx.name());
    });
  }

  private static void performNestedOperation() {
    if (CURRENT_TRANSACTION.isBound()) { ❷
      // Присоединение существующей транзакции
      Transaction currentTx = CURRENT_TRANSACTION.get();
      System.out.println("Joining existing transaction: " + currentTx.name());
      performDatabaseOperation();
    } else {
      // Начать новую транзакцию, если она еще не существует
      Transaction newTx = new Transaction("NESTED_TX");
      ScopedValue.where(CURRENT_TRANSACTION, newTx).run(() -> { ❸
        System.out.println("Starting new transaction: " + newTx.name());
        performDatabaseOperation();
      });
    }
  }
  private static void performDatabaseOperation() {
    Transaction tx = CURRENT_TRANSACTION.get();
```

https://max.ru/bookofgeek

```
    System.out.println(" Executing in transaction: " + tx.name()); ❹
  }

  record Transaction(String name) {
  }
}
```

Данный шаблон делает следующее.

❶ Создает самую внешнюю транзакцию.

❷ Определяет, активна ли уже какая-либо транзакция.

❸ Создает новую транзакцию только при отсутствии активной.

❹ Гарантирует, что все операции выполняются в едином контексте транзакций.

В предыдущем коде мы использовали `ScopedValue`, чтобы управлять вложенными транзакциями. Сначала запускается внешняя транзакция и привязывается к `ScopedValue`. При попытке создать вложенную транзакцию код проверяет наличие активной — и если она уже есть, присоединяется к ней. Это гарантирует, что все операции, включая вложенные, будут частью одной общей транзакции, что упрощает управление и обеспечивает согласованность данных.

Запустив код, мы получим следующий вывод:

```
Starting: OUTER_TX
  Joining existing transaction: OUTER_TX
    Executing in transaction: OUTER_TX
Committing: OUTER_TX
```

Наш следующий пример касается графики, где часто требуется сделать контекст рисования общим для разных частей программы. Благодаря повторному вхождению (reentrancy) и автоматическому освобождению ресурсов scoped values лучше подходят для этой цели, чем переменные `ThreadLocal`.

Рассмотрим типичный графический редактор, в котором такие операции рисования, как изменение цвета, толщины линии и преобразования, используют общее состояние. Традиционный подход с переменными `ThreadLocal` чреват утечками контекста, к тому же требует освобождать ресурсы вручную. `ScopedValue` предлагает более чистую и безопасную альтернативу.

Посмотрим, как scoped values могут управлять контекстом рисования в иерархии компонентов:

```
import java.awt.Color;

public class SimpleGraphicsExample {
  // Контекст рисования как ScopedValues
  private static final ScopedValue<Color> DRAW_COLOR
                            = ScopedValue.newInstance(); ❶
  private static final ScopedValue<Integer> LINE_WIDTH
                            = ScopedValue.newInstance();
```

https://max.ru/bookofgeek

```
  // Моделирование методов рисования
  static void drawLine(String from, String to) {
      Color color = DRAW_COLOR.isBound() ? DRAW_COLOR.get() : Color.BLACK; ❷
      int width = LINE_WIDTH.isBound() ? LINE_WIDTH.get() : 1;

      System.out.printf("Drawing line from %s to %s [Color: %s, Width: %d]\n",
          from, to, color.toString(), width);
}

  static void drawRectangle(String name) {
      Color color = DRAW_COLOR.isBound() ? DRAW_COLOR.get() : Color.BLACK;
      int width = LINE_WIDTH.isBound() ? LINE_WIDTH.get() : 1;

      System.out.printf("Drawing rectangle '%s' [Color: %s, Width: %d]\n",
          name, color.toString(), width);
  }
```

❶ Свойства контекста рисования объявляются как `ScopedValues`, обеспечивая потокобезопасное управление контекстом.

❷ Проверка `isBound()`предоставит резервный вариант, если контекст не установлен.

Элементы иерархии UI часто требуют собственных стилей отрисовки, наследуя при этом свойства от родительских элементов:

```
  //  Компонент, отрисовывающий себя и свои дочерние элементы
  static void drawButton(String label) {
    System.out.println("\n--- Drawing Button: " + label + " ---");

    // Используем для кнопки синий цвет и толстую границу
    ScopedValue.where(DRAW_COLOR, Color.BLUE) ❸
               .where(LINE_WIDTH, 3)
               .run(() -> {
      drawRectangle("button-background");

        // Для текста внутри кнопки используем другой цвет
        ScopedValue.where(DRAW_COLOR, Color.WHITE) ❹
                   .where(LINE_WIDTH, 1)
                   .run(() -> {
          System.out.println("Drawing text: " + label);
        });

        // Граница автоматически снова использует синий цвет
        drawRectangle("button-border"); ❺
      });
  }
```

❸ Для кнопки задаем контекст рисования с синим цветом и толщиной линии 3 пикселя.

❹ При отрисовке цвет текста временно переопределяется на белый, демонстрируя безопасную вложенность.

https://max.ru/bookofgeek

❺ Когда область создания текста заканчивается, контекст кнопки автоматически восстанавливается до синего цвета.

Контейнеры могут иметь контекст рисования, который влияет на все их дочерние элементы:

```
  // Панель, содержащая несколько компонентов
  static void drawPanel() {
    System.out.println("\n--- Drawing Panel ---");

    // Панель использует серый цвет
    ScopedValue.where(DRAW_COLOR, Color.GRAY)
               .where(LINE_WIDTH, 2)
               .run(() -> {
      drawRectangle("panel-background");

      // Рисуем дочерние компоненты, каждый со своим стилем
      drawButton("OK"); ❻
      drawButton("Cancel");

      // Автоматически возвращаемся к серому цвету панели
      drawLine("divider-start", "divider-end"); ❼
    });
  }
```

❻ Каждая кнопка сохраняет собственный графический контекст, не зависящий от серого цвета панели.

❼ После отрисовки дочерних элементов контекст панели остается неизменным.

Теперь, в завершение примера, рассмотрим наш метод `main`:

```
  void main() {
    System.out.println("=== Graphics Context Example ===\n");

    // Устанавливаем контекст рисования по умолчанию
    ScopedValue.where(DRAW_COLOR, Color.BLACK) ❽
               .where(LINE_WIDTH, 1)
               .run(() -> {
      // Рисуем линию черным цветом по умолчанию
      drawLine("A", "B");

      // Рисуем панель (у которой свои цвета)
      drawPanel();

      // После рисования панели автоматический возврат к черному цвету
      System.out.println("\n--- Back to main context ---");
      drawLine("C", "D"); ❾
    });

    // За пределами области контекст недоступен
    System.out.println("\n--- Outside any context ---");
    drawLine("E", "F"); // Будет использовано по умолчанию ❿
  }
}
```

https://max.ru/bookofgeek

❽ Устанавливаем контекст рисования по умолчанию для всего приложения.

❾ Когда рисование панели завершается, основной контекст восстанавливается без ручного вмешательства.

❿ Операции вне любого контекста возвращаются к разумным умолчаниям.

При запуске этого примера выводится следующий результат, ясно показывающий изменения контекста:

```
=== Graphics Context Example ===
Drawing line from A to B [Color: java.awt.Color[r=0,g=0,b=0], Width: 1]
--- Drawing Panel ---
Drawing rectangle 'panel-background' [Color: java.awt.
Color[r=128,g=128,b=128],
Width: 2]
--- Drawing Button: OK ---
Drawing rectangle 'button-background' [Color: java.awt.Color[r=0,g=0,b=255],
Width: 3]
Drawing text: OK
Drawing rectangle 'button-border' [Color: java.awt.Color[r=0,g=0,b=255],
Width: 3]
--- Drawing Button: Cancel ---
Drawing rectangle 'button-background' [Color: java.awt.Color[r=0,g=0,b=255],
Width: 3]
Drawing text: Cancel
Drawing rectangle 'button-border' [Color: java.awt.Color[r=0,g=0,b=255],
Width: 3]
Drawing line from divider-start to divider-end [Color: java.awt.Color
[r=128,g=128,b=128], Width: 2]
--- Back to main context ---
Drawing line from C to D [Color: java.awt.Color[r=0,g=0,b=0], Width: 1]
--- Outside any context ---
Drawing line from E to F [Color: java.awt.Color[r=0,g=0,b=0], Width: 1]
```

Мы оценили преимущества `ScopedValue`, теперь рассмотрим несколько факторов, которые следует учесть при подготовке к миграции. Во-первых, пока API находится в предварительной стадии, было бы полезно изучить его возможности, прежде чем он станет доступен в JDK без флагов preview. Такая подготовка упрощает внедрение после официального релиза. Кроме того, важно перейти на последнюю версию JDK, поскольку `ScopedValue` доступен только в ней. Чтобы обновление прошло без проблем, сначала проанализируйте возможные ограничения.

При использовании `ScopedValue` важно убедиться, что данные, которые вы собираетесь сделать общими, являются неизменяемыми, — это обязательное требование. Если для вашего сценария требуются изменяемые данные, `ScopedValue` не подойдет. Не менее важно явно определять область общих данных с помощью методов `where()` и `run()`, чтобы данные оставались доступными только внутри нужного контекста и не вызывали непредвиденных побочных эффектов.

https://max.ru/bookofgeek

Следует также не забывать, что `ScopedValue` и `ThreadLocal` по-разному обрабатывают значение `null`. При доступе к непривязанным значениям `ScopedValue` выбрасывает `NoSuchElementException`, тогда как `ThreadLocal` возвращает `null`. Явное управление ошибками помогает раньше выявлять проблемы, но требует повышенного внимания при миграции.

Наконец, после внедрения `ScopedValue` необходимо тщательно протестировать приложение. Это позволит убедиться в том, что конкурентность и согласованность данных сохранились, а также устранить возможные непредвиденные проблемы.

В заключение

`ScopedValue` предлагает более чистую, безопасную и эффективную альтернативу `ThreadLocal` для совместного использования контекста по всей цепочке вызовов. Благодаря неизменяемости и привязке к четко определенной динамической области scoped values лишены многих недостатков `ThreadLocal`: неподконтрольной изменяемости, неограниченного времени жизни и перерасхода памяти на хранение данных (memory overhead). Легковесная природа scoped values особенно хорошо подходит для работы с виртуальными потоками и структурированной конкурентностью, помогая снизить потребление ресурсов при создании большого числа конкурентных задач. Явный синтаксис методов `where()` и `run()` позволяет легко понять, когда и где действует заданное значение, благодаря чему код становится проще читать и поддерживать.

Начиная с JDK 25 `ScopedValue` стал стабильным API, готовым к использованию в продакшене. Это позволяет рекомендовать его для передачи контекста в современных Java-приложениях. Теперь можно уверенно переходить с существующего кода на основе `ThreadLocal` на scoped values, не беспокоясь по поводу изменений API.

https://max.ru/bookofgeek

ГЛАВА 6

Актуальность реактивного программирования на Java в контексте виртуальных потоков

> Я считаю, что Loom положит конец реактивному программированию... Реактивное программирование было переходной технологией.
>
> *Брайан Гетц (Brian Goetz)*

Я не буду комментировать это утверждение, по крайней мере пока, но в этой главе мы рассмотрим альтернативу виртуальным потокам — реактивное программирование на Java, которому отдают предпочтение многие разработчики.

В предыдущих главах мы говорили о виртуальных потоках: как они работают, какие преимущества дают и как помогают сделать конкурентность в Java более доступной. Мы пришли к выводу, что это замечательная инновация в области многопоточности. Они позволяют разработчикам создавать высококонкурентные приложения, используя привычную императивную модель программирования, и при этом эффективно обрабатывать блокирующие операции.

Однако виртуальные потоки — не единственное решение для эффективной обработки конкурентности и блокирующего ввода-вывода. До их появления разработчики для создания масштабируемых неблокирующих приложений использовали реактивное программирование. Этот подход, который часто ассоциируется с такими библиотеками и фреймворками, как Project Reactor, RxJava, Eclipse Vert.x и др., основан на иной парадигме: событийно-ориентированной, функциональной и изначально асинхронной.

В этой главе мы рассмотрим реактивное программирование на Java, изучим его модель выполнения, стиль программирования, удобство использования, сложность освоения, а также преимущества и проблемы.

https://max.ru/bookofgeek

Основы реактивного программирования в Java

Реактивное программирование (reactive programming) — это декларативная парадигма, которая фокусируется на асинхронных потоках данных (asynchronous data streams) и автоматическом распространении изменений.

РЕАКТИВНЫЕ СИСТЕМЫ: КРАТКИЙ ОБЗОР

В 2013 году появился Reactive Manifesto (https://oreil.ly/Cb14t), который дал определение реактивным системам и стандартизировал понятие «реактивный».

Reactive Manifesto («Манифест реактивных систем») описывает четыре ключевых принципа.

Отзывчивость (responsiveness)

Реактивные системы обеспечивают стабильно быстрое время отклика, что повышает уверенность пользователей и упрощает обработку ошибок.

Устойчивость (resilience)

Такие системы сохраняют устойчивость к отказам даже при сбоях благодаря изоляции проблем и делегированию восстановления, а также используют репликации для обеспечения высокой доступности.

Эластичность (elasticity)

Для поддержания производительности реактивные системы динамически адаптируются к изменяющейся нагрузке, эффективно масштабируя ресурсы в обе стороны (увеличение при росте нагрузки и уменьшение при ее снижении).

Ориентированность на сообщения (message-driven)

Системы используют асинхронный обмен сообщениями для обеспечения слабой связанности, изоляции и эффективного использования ресурсов, что обеспечивает устойчивость и эластичность.

Технически все сводится к трем ключевым аспектам: неблокирующей обработке, событийно-ориентированной архитектуре и асинхронности.

Вместо того чтобы сосредоточиться на пошаговом выполнении инструкций, разработчики определяют взаимосвязи между потоками данных и способы преобразования данных. Такие потоки могут представлять любые изменяющиеся во времени сущности: пользовательский ввод, показания сенсоров или данные из внешних систем. При изменении значения в потоке реактивный фреймворк автоматически распространяет это изменение по цепочке, обновляя все зависимые компоненты. Это позволяет эффективно обрабатывать асинхронные события и упрощает разработку ориентированных на них сложных приложений.

https://max.ru/bookofgeek

Имеется возможность создавать системы, которые проще поддерживать и масштабировать, описывая, *какие* преобразования и реакции должны происходить, а не то, *как* их выполнять. Популярные библиотеки и фреймворки реактивного программирования (например, RxJava, Reactor и Akka Streams) предоставляют инструменты для создания и управления этими потоками данных.

В экосистеме Java реактивное программирование обычно ассоциируется с *неблокирующими асинхронными операциями* (*non-blocking, asynchronous operations*) и событийно-ориентированной архитектурой (*event-driven architectures*).

Разберем эти концепции подробнее, одну за другой.

Блокирующий и неблокирующий ввод-вывод

В предыдущих главах мы много узнали о блокирующих операциях. Коротко говоря, при выполнении операции ввода-вывода вызывающий поток ожидает ее завершения, простаивая в заблокированном состоянии до получения данных. Именно это называется блокирующей операцией.

Альтернативный подход — неблокирующий ввод-вывод. В этой модели потоки не блокируются операциями ввода-вывода. Пока они обрабатываются в фоновом режиме, поток может выполнять другие задачи.

Как это работает? В неблокирующей системе обработкой ввода-вывода занимается ОС, а по завершении операции уведомляет приложение, обычно через вызов колбэк-функции. Таким образом, поток, инициировавший запрос, не простаивает — он просто получает сигнал, когда данные готовы.

Чтобы добавить идее конкретики, рассмотрим пример. Представьте себе HTTP-сервер, который должен выполнять конвейерную обработку запросов. Это критичная особенность HTTP/1.1, позволяющая клиентам отправлять несколько запросов без ожидания ответов. Такой сервер должен:

- принимать соединения от множества клиентов;
- обрабатывать HTTP-запросы разной сложности (быстрые и медленные операции);
- поддерживать состояние соединения для keep-alive сессий;
- корректно выполнять конвейерную обработку запросов;
- масштабироваться для обработки сотен или тысяч конкурентных соединений.

Для него этого достаточно. Реализуем такой сервер в двух вариантах — блокирующем и неблокирующем, чтобы наглядно увидеть разницу между подходами.

https://max.ru/bookofgeek

Начнем с однопоточного блокирующего сервера:

```
public class BlockingHttpServer {
  private static final int PORT = 8080;
  private static final AtomicInteger requestCounter = new AtomicInteger(0);

  public static void main(String[] args) throws IOException {
    System.out.println("Blocking HTTP Server starting on port " + PORT);
    System.out.println("Features: Single-threaded, Request pipelining");
    try (ServerSocket serverSocket = new ServerSocket(PORT)) {
      serverSocket.setReuseAddress(true); ❶
      while (true) {
        Socket clientSocket = serverSocket.accept(); ❷
        handleConnection(clientSocket);
      }
    }
  }
  private static void handleConnection(Socket socket) {
    System.out.println("New connection from: " +
                       socket.getRemoteSocketAddress());
    try (socket;
        BufferedReader in = new BufferedReader(
            new InputStreamReader(socket.getInputStream()));
        PrintWriter out = new PrintWriter(
            socket.getOutputStream(), true)) {
      socket.setSoTimeout(5000); ❸
      boolean keepAlive = true;
      while (keepAlive) {
        HttpRequest request = parseRequest(in); ❹
        if (request == null) {
          break; // Соединение закрыто
        }
        int requestId = requestCounter.incrementAndGet();
        System.out.println("Request #" + requestId + ": " +
                           request.method + " " + request.path);
        // Проверка на keep-alive
        keepAlive = "keep-alive".equalsIgnoreCase(
            request.getHeader("Connection"));
        // Обработка запроса. Может занять немало времени! ❺
        processRequest(request);
        // Отправка ответа
        sendResponse(out, request, requestId, keepAlive);
      }
    } catch (SocketTimeoutException e) {
      System.out.println("Connection timeout");
    } catch (IOException e) {
      System.err.println("Connection error: " + e.getMessage());
    }
  }
}
```

https://max.ru/bookofgeek

Рассмотрим ключевые фрагменты этого кода.

❶ Мы разрешаем повторное использование адреса, чтобы во время разработки избежать ошибок `Address already in use` («Адрес уже используется»).

❷ Метод `accept()` блокирует выполнение до подключения клиента. Здесь наш единственный поток застревает в ожидании новых соединений.

❸ Мы устанавливаем тайм-аут сокета 5 с для обнаружения «мертвых» соединений и предотвращения бесконечного зависания сервера.

❹ Метод `parseRequest()` блокирует выполнение до получения полного HTTP-запроса. Пока сервер ожидает запрос от одного клиента, он не может принимать новые соединения.

❺ Это критическое ограничение: если `processRequest()` выполняется долго (например, при медленном запросе к базе данных), блокируется весь сервер. Другие клиенты не могут подключиться или обслуживаться.

Серверу требуются вспомогательные методы для обработки обмена данными по протоколу HTTP:

```
private static void sendResponse(PrintWriter out,
                                 HttpRequest request,
                                 int requestId,
                                 boolean keepAlive) {
  // Отправка HTTP-ответа
  out.println("HTTP/1.1 200 OK");
  out.println("Content-Type: text/plain");
  out.println("Connection: " + (keepAlive ? "keep-alive" : "close"));
  String body = String.format(
      "Request #%d processed\nPath: %s\nTime: %s\n",
      requestId, request.path, Instant.now());
  out.println("Content-Length: " + body.length());
  out.println(); // Пустая строка между заголовками и телом
  out.print(body);
  out.flush(); ❻
}
```

❻ Гарантируем немедленную доставку HTTP-ответа, принудительно отправляя все буферизированные данные клиенту, что предотвращает задержки при передаче ответа.

Мы имитируем разную длительность обработки, чтобы продемонстрировать, как выполняется блокирование:

```
static void processRequest(HttpRequest request)
    throws IOException {
  try {
    if (request.path.startsWith("/slow")) {
      Thread.sleep(Duration.of(30, ChronoUnit.SECONDS)); ❼
      System.out.println(" Slow request processed");
    } else if (request.path.startsWith("/medium")) {
      Thread.sleep(Duration.of(500, ChronoUnit.MILLIS));
```

https://max.ru/bookofgeek

```
      System.out.println(" Medium request processed");
    } else {
      Thread.sleep(Duration.of(100, ChronoUnit.MILLIS));
        System.out.println(" Fast request processed");
      }
    } catch (InterruptedException e) {
      Thread.currentThread().interrupt();
    }
}
```

❼ Медленный запрос блокирует весь сервер на 30 с. В это время новые соединения не принимаются, а другие запросы не обрабатываются. Это добавлено для имитации медленного вызова API.

Наш простой класс `HttpRequest` содержит основные элементы, необходимые для обработки:

```
static class HttpRequest {
  String method;
  String path;
  String version;
  Map<String, String> headers = new HashMap<>();
  String remainingBuffer;  // Для неблокирующего сервера
  String getHeader(String name) {
    return headers.get(name.toLowerCase());
  }
}
```

Вспомогательный метод для парсинга запроса:

```
static HttpRequest parseRequest(BufferedReader in)
    throws IOException {
  // Чтение строки запроса
  String requestLine = in.readLine(); // Блокирующий I/O
  if (requestLine == null || requestLine.isEmpty()) {
    return null;
  }
  String[] parts = requestLine.split(" ");
  if (parts.length != 3) {
    throw new IOException("Invalid request line");
  }
  HttpRequest request = new HttpRequest();
  request.method = parts[0];
  request.path = parts[1];
  request.version = parts[2];
  // Чтение заголовков
  String headerLine;
  while ((headerLine = in.readLine()) != null) { // Блокирующий I/O
    if (headerLine.isEmpty()) {
      break; // Конец заголовков
    }
    int colonPos = headerLine.indexOf(':');
    if (colonPos > 0) {
      String name = headerLine.substring(0, colonPos).trim();
```

https://max.ru/bookofgeek

```
            String value = headerLine.substring(colonPos + 1).trim();
            request.headers.put(name.toLowerCase(), value);
        }
      }
    return request;
}
```

В данной реализации мы создаем `ServerSocket` (https://oreil.ly/pqjiz), который слушает порт 8080. `Socket` (https://oreil.ly/uEj8h) представляет собой единый эндпоинт двустороннего канала связи между программами в сети, в то время как `ServerSocket` непрерывно слушает входящие соединения и соответствующим образом обрабатывает клиентские запросы.

В нашем случае это работает так.

1. Сервер ожидает подключения клиента.
2. После установки соединения сервер передает его методу `handleConnection()`, который обеспечивает взаимодействие.
3. Этот метод обрабатывает HTTP-запросы, поддерживая соединения keep-alive и конвейерную обработку запросов. Данная реализация HTTP-обработки демонстрирует влияние блокирующего I/O на производительность сервера.

Однако здесь имеется подвох. При обработке клиента сервером вызывающий поток блокируется до отключения клиента. Если клиент прекращает отправлять данные, но не разрывает соединение, сервер зависает в ожидании ввода. Поскольку сервер однопоточный, другие клиенты не могут подключиться до окончания текущей сессии.

Для тестирования сервера и наблюдения за его блокирующим поведением воспользуемся curl. Проведем несколько экспериментов:

Терминал 1. Запуск сервера

```
$ java BlockingHttpServer.java
Single-threaded HTTP Server with Pipelining starting on port 8080
Features: Request pipelining, Keep-alive connections
```

Сервер запущен и ожидает подключений. Отправим простой запрос:

Терминал 2. Отправлен один быстрый запрос

```
$ curl -v http://localhost:8080/fast
* Host localhost:8080 was resolved.
* IPv6: ::1
* IPv4: 127.0.0.1
*   Trying [::1]:8080...
* Connected to localhost (::1) port 8080
> GET /fast HTTP/1.1
> Host: localhost:8080
```

https://max.ru/bookofgeek

```
> User-Agent: curl/8.7.1
> Accept: */*
>
* Request completely sent off
< HTTP/1.1 200 OK
< Content-Type: text/plain
< Connection: close
< Content-Length: 67
<
Request #1 processed Path: /fast
Time: 2025-06-14T09:27:02.235713Z
* Closing connection
```

Быстрый запрос выполняется мгновенно. Сервер обрабатывает его и закрывает соединение. Теперь продемонстрируем проблему блокировки:

Терминал 3. Медленный запрос

```
$ curl -v http://localhost:8080/slow
* Host localhost:8080 was resolved.
* IPv6: ::1
* IPv4: 127.0.0.1
*   Trying [::1]:8080...
* Connected to localhost (::1) port 8080
> GET /slow HTTP/1.1
> Host: localhost:8080
> User-Agent: curl/8.7.1
> Accept: */*
>
* Request completely sent off
< HTTP/1.1 200 OK
< Content-Type: text/plain
< Connection: close
< Content-Length: 67
<
Request #2 processed Path: /slow
Time: 2025-06-14T09:28:16.218432Z
* Closing connection
```

Пока медленный запрос обрабатывается, попробуем соединиться с другим клиентом:

Терминал 4. Попытка быстрого запроса во время обработки медленного

```
$ curl -v --max-time 5 http://localhost:8080/fast
* Host localhost:8080 was resolved.
* IPv6: ::1
* IPv4: 127.0.0.1
*   Trying [::1]:8080...
* Connected to localhost (::1) port 8080
> GET /fast HTTP/1.1
> Host: localhost:8080
```

https://max.ru/bookofgeek

```
> User-Agent: curl/8.7.1
> Accept: */*
>
* Request completely sent off
* Operation timed out after 5006 milliseconds with 0 bytes received
* Closing connection
curl: (28) Operation timed out after 5006 milliseconds with 0 bytes received
```

Обратите внимание: второй клиент успешно устанавливает TCP-соединение (оно обрабатывается ОС), но сервер не принимает его, так как все еще заблокирован обработкой медленного запроса. Это фундаментальное ограничение однопоточного дизайна.

Самый показательный тест — использовать функции тайминга утилиты curl для измерения эффекта блокировки:

```
# Отправить три конвейерных запроса: быстрый, медленный, быстрый
$ time curl -s -Z http://localhost:8080/fast \
            http://localhost:8080/slow \
            http://localhost:8080/fast
Request #11 processed Path: /fast
Time: 2025-06-14T09:34:49.986347Z
Request #12 processed Path: /slow
Time: 2025-06-14T09:35:19.989656Z
Request #13 processed Path: /fast
Time: 2025-06-14T09:35:20.094539Z
curl -s -Z http://localhost:8080/fast http://localhost:8080/slow 0.01s user
0.01s system 0% cpu 47.811 total
```

Хотя два из трех запросов выполняются быстро, общее время превышает 30 с, так как они должны обрабатываться последовательно. Медленный запрос, отправленный вторым, блокирует всю систему.

При попытке подключиться к серверу с другого клиента во время активной сессии первого клиента видно, что соединение блокируется. Сервер может обрабатывать только одного клиента за раз. Дело в однопоточной реализации — сервер ждет отключения текущего клиента перед тем, как принять новое соединение.

Простое решение этой проблемы — использование многопоточности. Создание нового потока для каждого клиента позволяет обрабатывать несколько соединений конкурентно. Модифицируем наш код, добавив поддержку многопоточности:

```
public class MultithreadedHttpServer {
  private static final int PORT = 8080;
  private static final AtomicInteger requestCounter = new AtomicInteger(0);
  private static final int CONNECTION_THREADS = 10;
```

https://max.ru/bookofgeek

```
  public static void main(String[] args) throws IOException {
    System.out.println("Multi-threaded HTTP Server starting on port " + PORT);
    System.out.println("Features: Concurrent connections, Request pipelining");
    System.out.println("Connection pool size: " + CONNECTION_THREADS);

    try (ServerSocket serverSocket = new ServerSocket(PORT);
        ExecutorService connectionExecutor =
            Executors.newFixedThreadPool(CONNECTION_THREADS)) { ❶
      serverSocket.setReuseAddress(true);
      while (true) {
        Socket clientSocket = serverSocket.accept(); ❷
        System.out.println("New connection from: " +
                           clientSocket.getRemoteSocketAddress());
      // Обработка каждого соединения в отдельном потоке
      connectionExecutor.submit(() -> handleConnection(clientSocket)); ❸
      }
    }
  }

  private static void handleConnection(Socket socket) {
    String clientAddr = socket.getRemoteSocketAddress().toString();
    System.out.println("Thread " + Thread.currentThread().getName() +
                       " handling connection from: " + clientAddr);
    // Та же логика обработки запросов, что и в BlockingHttpServer
    // но выполняемая в отдельном потоке ❹
    try (socket;
        BufferedReader in = new BufferedReader(
            new InputStreamReader(socket.getInputStream()));
        PrintWriter out = new PrintWriter(
            socket.getOutputStream(), true)) {
        socket.setSoTimeout(5000);
        boolean keepAlive = true;
        while (keepAlive) {
          HttpRequest request = parseRequest(in);
          if (request == null) break;
          int requestId = requestCounter.incrementAndGet();
          System.out.println("Thread " + Thread.currentThread().getName() +
                             " - Request #" + requestId + ": " +
                             request.method + " " + request.path);
          keepAlive = "keep-alive".equalsIgnoreCase(
            request.getHeader("Connection"));
        processRequest(request);
        sendResponse(out, request, requestId, keepAlive);
      }
    } catch (Exception e) {
      System.err.println("Connection error from " + clientAddr +
                         ": " + e.getMessage());
    }
  }
}
```

https://max.ru/bookofgeek

Основные улучшения в многопоточной версии.

❶ Создаем фиксированный пул из десяти потоков для обработки соединений.

❷ Главный поток может немедленно вернуться к приему новых соединений.

❸ Каждое соединение выполняется в своем потоке, что создает условия для конкурентной обработки.

❹ Отдельные соединения по-прежнему обрабатывают запросы последовательно (сохраняется корректная конвейерная обработка), но теперь несколько соединений могут обслуживаться одновременно.

В многопоточной реализации мы создали фиксированный пул из десяти потоков, что позволяет одновременно обслуживать до десяти клиентов. Хотя можно было увеличить количество потоков для обработки большего числа клиентов, до появления виртуальных потоков этот подход имел внутренние ограничения. Платформенные потоки очень ресурсоемкие. В главе 2 мы говорили о том, что увеличение их числа сверх определенного предела не дает преимуществ.

С виртуальными потоками масштабировать стало так же просто, как изменить одну строку кода:

```
try (ServerSocket serverSocket = new ServerSocket(PORT);
  ExecutorService executor = Executors.newVirtualThreadPerTaskExecutor();
) {}
```

Виртуальные потоки позволяют серверу с минимальными затратами обрабатывать огромное количество конкурентных соединений. Однако сейчас мы сосредоточимся на альтернативных вариантах, которые появились до виртуальных потоков.

Одним из эффективных решений стал NIO (New Input/Output), обеспечивающий высокую пропускную способность при минимальном количестве потоков.

Java NIO, появившийся в JDK 1.4, — это неблокирующий readiness-based API, который заменяет потоковый java.io на буфер-ориентированные каналы (`SocketChannel`, `FileChannel` и др.) и цикл событий `Selector`. В отличие от блокировки потока до завершения операции, неблокирующие `read()` и `write()` возвращаются немедленно, часто без передачи данных. Канал регистрируется в объекте селектора. Селектор внутри `select()` ожидает уведомления от операционной системы о готовности одного или нескольких каналов к вводу/выводу. В дальнейшем поток обрабатывает только эти каналы, что позволяет несколькими потоками обслуживать тысячи соединений. В сочетании с прямыми или memory-mapped-буферами, исключающими передачу копируемых данных между пространством пользователя и ядра, NIO обеспечивает высокую пропускную способность и масштабируемость без излишнего расхода ресурсов, присущего модели thread-per-socket («один поток на сокет»).

https://max.ru/bookofgeek

СРАВНЕНИЕ JAVA I/O И NIO

В Java есть два подхода к управлению вводом-выводом (I/O): традиционный java.io и современный java.nio (New I/O). Их ключевое различие — в модели работы. Java I/O использует потоки и является в целом блокирующим: потоки должны ожидать завершения операций. Неблокирующий Java NIO основан на буферах, что обеспечивает повышенную масштабируемость для работы с большими объемами передаваемых данных и высокопроизводительными приложениями.

Пример чтения файла с помощью Java I/O:

```
try (BufferedReader reader
     = new BufferedReader(
  new FileReader("input.txt"))) {
    String line;
    while ((line = reader.readLine()) != null) {
      System.out.println(line);
    }
  } catch (IOException e) {
    e.printStackTrace();
  }
```

`BufferedReader` (https://oreil.ly/kMKl7) построчно читает текст из потока ввода, что удобно для последовательного чтения файлов.

Сравним с FileChannel из Java NIO:

```
try (FileChannel channel = FileChannel.open(
     Path.of("input.txt"),
     StandardOpenOption.READ)) {
  ByteBuffer buffer = ByteBuffer.allocate(1024);
  while (channel.read(buffer) > 0) {
    buffer.flip();
    while (buffer.hasRemaining()) {
      System.out.print((char) buffer.get());
    }
    buffer.clear();
  } catch (IOException e) {
    e.printStackTrace();
  }
```

Здесь используется буферизация: данные сначала загружаются в `ByteBuffer`, обрабатываются в памяти и только затем отправляются на дальнейшую обработку.

Вместо `ServerSocket` в NIO используется `ServerSocketChannel` (https://oreil.ly/-nprM). Аналогично `Socket` заменяется на `SocketChannel` (https://oreil.ly/OsyaE). Вместо `InputStream` (https://oreil.ly/VG-g_) и `OutputStream` (https://oreil.ly/YfUM5) NIO применяет буферы (https://oreil.ly/WdXHC). Буфер служит временным хранилищем для фиксированного объема данных перед их отправкой по назначению.

https://max.ru/bookofgeek

Такое изменение в архитектуре обеспечивает бóльшую гибкость и эффективность при обработке операций ввода-вывода.



Буферы в Java NIO повышают производительность, поскольку уменьшается прямое взаимодействие с ОС и данные передаются эффективнее. В отличие от традиционного ввода-вывода, где данные читаются и записываются побайтно, буферы поддерживают пакетную обработку, тем самым сокращая количество системных вызовов. Это особенно ценно для высокопроизводительных приложений, таких как сетевые серверы, а также при работе с файлами.

Одним из ключевых средств повышения эффективности являются механизмы zero-copy, где `ByteBuffer` (https://oreil.ly/zr3wz) может работать с отображаемыми в память файлами (memory-mapped files) или с прямыми буферами (памятью вне кучи). Это устраняет лишнее копирование данных между пространством пользователя и ядра, благодаря чему передача больших файлов значительно ускоряется. Кроме того, неблокирующий ввод-вывод с использованием буферов позволяет создавать масштабируемые событийно-ориентированные приложения, где один поток эффективно управляет множеством соединений.

Еще одним ключевым компонентом в NIO является `Selector` (https://oreil.ly/QjWUJ), который играет важную роль в управлении несколькими каналами (например, `ServerSocketChannel` и `SocketChannel`) с помощью одного потока. Его можно сравнить с интеллектуальным диспетчером трафика, который отслеживает различные события ввода-вывода и направляет операции соответствующим образом. Например, когда новое клиентское соединение готово быть принятым, селектор сигнализирует о событии `OP_ACCEPT` (https://oreil.ly/zDSSz). Когда данные доступны для чтения из `SocketChannel`, инициируется событие `OP_READ` (https://oreil.ly/pyHhL). Когда канал готов принять исходящие данные, срабатывает событие `OP_WRITE` (https://oreil.ly/L1SPh), а в клиентском режиме успешная попытка соединения приводит к событию `OP_CONNECT` (https://oreil.ly/1NnVA).

Реализация селекторов зависит от операционной системы. Например, в macOS используется `KQueueSelectorImpl` (https://oreil.ly/XmP4Y), который задействует системный вызов `queue` (https://oreil.ly/xI-UN). Более универсальная реализация `PollSelectorImpl` (https://oreil.ly/N72Qw) основана на системном вызове `poll`. Конкретный механизм варьируется в зависимости от ОС, но концепция остается неизменной: эффективное управление множеством каналов с минимальными затратами ресурсов на потоки.

Исследуем код, чтобы посмотреть, как эти компоненты работают вместе:

```
public class NonBlockingHttpServer {
  private static final int PORT = 8080;
  private static final AtomicInteger requestCounter = new AtomicInteger(0);
```

https://max.ru/bookofgeek

```
  private final ConcurrentLinkedQueue<PendingUpdate> pendingUpdates
                                    = new ConcurrentLinkedQueue<>();

  public static void main(String[] args) {
    System.out.println("Starting non-blocking NIO server...");
    System.out.println("Features: Single-threaded event loop, " +
                       "Non-blocking I/O, High concurrency");
    try {
      new NonBlockingHttpServer().start();
    } catch (IOException e) {
      System.err.println("Server failed to start: " + e.getMessage());
    }
  }
  private void start() throws IOException {
    try (Selector selector = Selector.open();
    ServerSocketChannel serverChannel = ServerSocketChannel.open()) {
      serverChannel.bind(new InetSocketAddress(PORT));
      serverChannel.configureBlocking(false); ❶
      serverChannel.register(selector, SelectionKey.OP_ACCEPT);
      System.out.println("Non-blocking HTTP server on port " + PORT);
      // Однопоточный цикл событий ❷
      while (true) {
        // Обработка ожидающих обновлений из асинхронных потоков
        processPendingUpdates(); ❸

        selector.select(100); ❹
        Set<SelectionKey> selectedKeys = selector.selectedKeys();
        Iterator<SelectionKey> iterator = selectedKeys.iterator();
        while (iterator.hasNext()) {
          SelectionKey key = iterator.next();
          iterator.remove();
          try {
            if (key.isAcceptable()) {
              handleAccept(key, selector); ❺
            } else if (key.isReadable()) {
              handleRead(key); ❻
            } else if (key.isWritable()) {
              handleWrite(key); ❼
            }
          } catch (IOException e) {
            System.err.println("Error handling key: "
                               + e.getMessage());
            key.cancel();
            if (key.channel() != null) {
              key.channel().close();
            }
          }
        }
        // Обработка всех ожидающих запросов
        processAllPendingRequests(selector);
      }
    }
  }
}
```

https://max.ru/bookofgeek

Ключевые улучшения в неблокирующей версии.

❶ Серверный канал сконфигурирован как неблокирующий, что необходимо для работы NIO.

❷ Все события ввода-вывода обрабатываются одним потоком через цикл событий.

❸ Перед обработкой событий выполняются обновления селекторов, поставленных в очередь асинхронными потоками. Этот шаг предотвращает состояния гонки, которые могут привести к падению сервера.

❹ Чтобы избежать бесконечного блокирования, `selector` использует тайм-аут (100 мс). Это гарантирует оперативную обработку обновлений, ожидаемых из асинхронных потоков, даже при отсутствии I/O-событий.

❺ Новые соединения принимаются без блокирования потока.

❻ Чтение данных неблокирующее; чтение части данных обрабатывается постепенно.

❼ Запись данных неблокирующая; запись части данных ставится в очередь для последующего завершения.

В приведенном выше коде мы начинаем с создания `Selector`, используя `Selector.open()`. Вместо традиционного конструктора здесь предпочтительнее использовать статический фабричный метод. Этот `Selector` становится ядром нашего сервера, позволяя эффективно управлять множеством клиентских подключений с помощью единственного потока.

Затем мы создаем `ServerSocketChannel` и привязываем его к конкретному адресу. Необходимо сконфигурировать канал в неблокирующем режиме, используя `serverSocketChannel.configureBlocking(false)`. Без этого сервер будет блокироваться при каждой операции и сведет на нет преимущества NIO.

После настройки `ServerSocketChannel` мы регистрируем его в `Selector`, указывая, что нас интересуют события `accept`. В результате сервер будет получать уведомления при попытке нового подключения клиента.

Здесь мы также создали все обработчики. Эти классы, отвечающие за обработку каждого события, мы просто вынесли в отдельные компоненты.

В основном цикле сначала вызываем `processPendingUpdates()`, решающий ключевую проблему синхронизации. Поскольку объекты `SelectionKey` не являются потокобезопасными, их нельзя модифицировать напрямую из асинхронных потоков. Поэтому для взаимодействия между потоками используем потокобезопасную очередь:

```
private void processPendingUpdates() {
  PendingUpdate update;
  while ((update = pendingUpdates.poll()) != null) {
```

https://max.ru/bookofgeek

```
    try {
      if (update.key.isValid()) {
        update.key.interestOps(update.key.interestOps()
        | SelectionKey.OP_WRITE);
      }
    } catch (Exception e) {
      System.err.println("Error updating key: " + e.getMessage());
    }
  }
}
static class PendingUpdate {
  final SelectionKey key;

  PendingUpdate(SelectionKey key) {
    this.key = key;
  }
}
```

После обработки обновлений вызывается метод `selector.select(100)`, который блокирует выполнение на срок до 100 мс. Тайм-аут очень важен: без него селектор мог бы блокироваться бесконечно, препятствуя своевременной обработке обновлений из асинхронных потоков. Как только обнаружено событие или истекает тайм-аут, `select()` завершает работу, позволяя серверу обработать события или ожидающие обновления.

При обнаружении события из селектора извлекаются ключи выбора (selection keys). Каждый ключ соответствует конкретному событию для зарегистрированного канала. Поскольку таких событий может возникнуть несколько, сервер обрабатывает их последовательно в цикле. После обработки ключ удаляется из набора, чтобы избежать повторной обработки.

Когда `key.isAcceptable()` возвращает `true`, это означает, что новый клиент пытается подключиться. В этот момент вызывается метод `handleAccept(key, selector)` для обработки события подключения.

Рассмотрим код обработчика принятия соединения:

```
private void handleAccept(SelectionKey key, Selector selector)
    throws IOException {
  ServerSocketChannel serverChannel = (ServerSocketChannel) key.channel();
  SocketChannel clientChannel = serverChannel.accept(); ❶

  if (clientChannel != null) {
    clientChannel.configureBlocking(false);
    clientChannel.register(selector, SelectionKey.OP_READ,
                           new ClientState()); ❷

    System.out.println("Accepted connection from: " +
                       clientChannel.getRemoteAddress());
  }
}
```

https://max.ru/bookofgeek

```
static class ClientState {
  ByteBuffer readBuffer = ByteBuffer.allocate(8192);
  StringBuilder requestBuilder = new StringBuilder();
  ConcurrentLinkedQueue<String> responseQueue =
                                new ConcurrentLinkedQueue<>(); ❸
  Queue<HttpRequest> pendingRequests = new LinkedList<>();
  boolean keepAlive = true;
  boolean isProcessing = false; ❹
}
```

Обработчик событий принятия соединения управляет новыми клиентскими подключениями.

❶ Вызов `accept()` неблокирующий и может вернуть `null`, если нет готовых подключений.

❷ Каждый клиент получает объект `ClientState` для отслеживания состояния подключения и частичных данных.

❸ Очередь ответов использует `ConcurrentLinkedQueue` вместо обычного `LinkedList`. Это важно, поскольку ответы будут добавляться из асинхронных потоков обработки и нам нужны потокобезопасные операции без явной синхронизации.

❹ Флаг `isProcessing` предотвращает дублирование обработки запросов от одного и того же клиента, избегая состояний гонки при быстрой последовательности событий.

В методе обработки мы сначала извлекаем `ServerSocketChannel`. В данном случае мы уверены, что `key.channel()` вернет `ServerSocketChannel`, поэтому можем безопасно выполнить приведение типа без риска получить `ClassCastException`. Соединение принимается с помощью `serverChannel.accept()`, и, как обычно, мы конфигурируем его в неблокирующем режиме.

После установки соединения мы создаем объект `ClientState` для клиента и регистрируем канал клиента в `Selector` для операций чтения, используя `SelectionKey.OP_READ`. Для удобства управления объект `ClientState` прикрепляется к `SelectionKey` в качестве вложения, что позволит извлечь его при необходимости.

Класс `ClientState` играет ключевую роль в управлении состоянием каждого подключенного клиента. Он содержит `SocketChannel` клиента, буферы для чтения и записи данных (`readBuffer` и `responseQueue`), а также поддерживает `StringBuilder` (`requestBuilder`) для накопления входящих данных HTTP-запроса. Такое структурированное управление состоянием для каждого клиента необходимо при работе со множеством клиентов — каждое соединение изолировано и не влияет на остальные.

https://max.ru/bookofgeek

Когда `key.isReadable()` возвращает true, это означает, что клиент отправил данные. В этом случае мы вызываем метод `handleRead(key)`.

Рассмотрим обработчик чтения:

```
private void handleRead(SelectionKey key) throws IOException {
  SocketChannel channel = (SocketChannel) key.channel();
  ClientState state = (ClientState) key.attachment();

  int bytesRead = channel.read(state.readBuffer); ❶
  if (bytesRead == -1) {
    // Клиент отключился
    channel.close();
    return;
  }
  if (bytesRead > 0) {
    state.readBuffer.flip();
    byte[] data = new byte[state.readBuffer.remaining()];
    state.readBuffer.get(data);
    state.requestBuilder.append(new String(data, StandardCharsets.UTF_8));
    state.readBuffer.clear();

    // Обработка завершенных запросов
    processCompleteRequests(state, key); ❷
  }
}

private void processCompleteRequests(ClientState state, SelectionKey key) {
  String buffer = state.requestBuilder.toString();
  while (true) {
    HttpRequest request = parseHttpRequest(buffer); ❸
    if (request == null) {
      break; // Не найдены завершенные запросы
    }
    state.pendingRequests.offer(request);
    buffer = request.remainingBuffer;
    //  Проверка keep-alive на основе версии HTTP и заголовков
    String connection = request.getHeader("Connection");
    if (request.version.equals("HTTP/1.0")) {
      // HTTP/1.0 по умолчанию закрывает соединение, если не указано keep-alive
      state.keepAlive = "keep-alive".equalsIgnoreCase(connection);
    } else {
      // HTTP/1.1 по умолчанию поддерживает соединение, если не указано close
      state.keepAlive = !"close".equalsIgnoreCase(connection);
    }
  }
  state.requestBuilder = new StringBuilder(buffer);
  if (!state.responseQueue.isEmpty()) {
    key.interestOps(key.interestOps() | SelectionKey.OP_WRITE); ❹
  }
}
```

https://max.ru/bookofgeek

Обработчик чтения работает с входными данными, учитывая несколько важных особенностей.

❶ Чтение является неблокирующим и может возвращать часть данных.

❷ Полные HTTP-запросы обрабатываются сразу по мере поступления, а частичные — постепенно.

❸ Логика keep-alive теперь корректно работает с протоколами HTTP/1.0 и HTTP/1.1. HTTP/1.0 по умолчанию закрывает соединения, если клиент явно не запросил keep-alive, тогда как HTTP/1.1 по умолчанию поддерживает соединения активными, если клиент не запросил их закрытие. Это различие критично для совместимости с такими инструментами, как Apache Bench.

❹ Интерес к событиям записи регистрируется только при наличии данных для отправки.

Обработчик читает данные из канала клиента и помещает их в `readBuffer`. После получения байты декодируются в символы с использованием UTF-8 и добавляются в `requestBuilder`. Система обрабатывает HTTP-запросы, отыскивая стандартный терминатор запроса (`"\r\n\r\n"`). При обнаружении полного запроса сервер выполняет его парсинг и обрабатывает асинхронно, не блокируя цикл обработки событий.

Когда `key.isWritable()` — `true`, это значит, что канал клиента готов к приему данных. В этот момент управление берет метод `handleWrite(key)`, который обеспечивает эффективную отправку всех ожидающих ответов из очереди обратно клиенту.

Рассмотрим код:

```
private void handleWrite(SelectionKey key) throws IOException {
  SocketChannel channel = (SocketChannel) key.channel();
  ClientState state = (ClientState) key.attachment();

  while (!state.responseQueue.isEmpty()) {
    String response = state.responseQueue.peek();
    ByteBuffer buffer
      = ByteBuffer.wrap(response.getBytes(StandardCharsets.UTF_8));

    int written = channel.write(buffer);
    if (buffer.hasRemaining()) {
      // Буфер сокета заполнен, повторить попытку позже
      break;
    }

    // Ответ полностью записан, удаляем из очереди
    state.responseQueue.poll();
  }

  if (state.responseQueue.isEmpty()) {
    // Нет данных для записи, прекращаем отслеживать события записи
    key.interestOps(key.interestOps() & ~SelectionKey.OP_WRITE);
```

https://max.ru/bookofgeek

```
    if (!state.keepAlive && state.pendingRequests.isEmpty()) {
      // Закрываем соединение, если не указано keep-alive и нет ожидающих запросов
      System.out.println("Closing connection: "
                         + channel.getRemoteAddress());
      channel.close();
      key.cancel();
    }
  }
}
```

Обработчик перебирает `responseQueue` клиента, записывая каждый HTTP-ответ из очереди в канал клиента с использованием неблокирующей записи. Если буфер сокета заполняется, процесс записи приостанавливается до следующего события `OP_WRITE`. После успешной записи всех ответов и опустошения `responseQueue` сервер удаляет `OP_WRITE` интерес из `Selector` для данного клиента, предотвращая ненужные события записи. Применение потокобезопасной `ConcurrentLinkedQueue` гарантирует отсутствие состояния гонки при добавлении ответов из асинхронных потоков.

После обработки всех выбранных ключей мы переходим к любым ожидающим запросам, для которых был выполнен парсинг:

```
private void processAllPendingRequests(Selector selector) {
  for (SelectionKey key : selector.keys()) {
    if (!key.isValid() || key.attachment() == null)
      continue;
      ClientState state = (ClientState) key.attachment();

      if (state.isProcessing || state.pendingRequests.isEmpty())
        continue;

      state.isProcessing = true; ❶
      while (!state.pendingRequests.isEmpty()) {
        HttpRequest request = state.pendingRequests.poll();
        int requestId = requestCounter.incrementAndGet();
        System.out.println("Request #" + requestId + ": "
               + request.method + " " + request.path);
        // Имитация асинхронной обработки
        CompletableFuture.runAsync(() -> { ❷
          String response;
          try {
            // Обработка запроса (имитация работы)
            if (request.path.startsWith("/slow")) {
              Thread.sleep(2000); // Имитация медленной операции
              System.out.println(" Slow request processed");
            } else {
              System.out.println(" Fast request processed");
            }
            response = buildHttpResponse(request, requestId, state.keepAlive);
          } catch (Exception e) {
            System.err.println("Error processing request #" + requestId
              + ": " + e.getMessage());
```

https://max.ru/bookofgeek

```
                response = buildErrorResponse(request, requestId, e);
            }
            state.responseQueue.offer(response); ❸

            // Поставить обновление в очередь для потока селектора
            pendingUpdates.offer(new PendingUpdate(key)); ❹
            selector.wakeup();
        });
    }

    state.isProcessing = false;
  }
}
```

Преимущества асинхронной обработки теперь очевидны.

❶ Флаг обработки гарантирует, что мы не начнем обрабатывать запросы одного клиента несколько раз при быстрой последовательности событий.

❷ Медленные операции выполняются асинхронно без блокировки цикла событий.

❸ Ответы безопасно добавляются в конкурентную очередь из асинхронного потока.

❹ Вместо прямой модификации `SelectionKey` из асинхронного потока (что вызвало бы состояние гонки) мы ставим обновление в очередь для обработки главным потоком селектора. Это критически важный шаблон для потокобезопасности в NIO-приложениях.



Есть распространенное заблуждение, которое возникает из-за формулировки в документации Java: Selection keys are safe for use by multiple concurrent threads («Ключи выбора безопасны для использования несколькими конкурентными потоками»). Из-за этого многие могут решить, что допустимо изменять `interestOps` из любого потока. Однако данное утверждение означает лишь то, что чтение свойств ключа не приведет к повреждению памяти, но не гарантирует атомарность или безопасность операций, подобных `interestOps()`, при конкурентном изменении.

Изменение `interestOps` у `SelectionKey` во время выполнения `selector.select()` может вызвать:

- состояние гонки, при котором изменения `interestOps` теряются;
- пропуск событий ввода-вывода;
- ошибки `CancelledKeyException`;
- непредсказуемое поведение, которое часто проявляется только под высокой нагрузкой.

Всегда изменяйте `interestOps` у `SelectionKey` из того же потока, в котором работает цикл селектора. Именно поэтому мы используем шаблон очереди `pendingUpdates` — он гарантирует, что все модификации выполняются безопасно в потоке селектора.

https://max.ru/bookofgeek

Далее мы рассмотрим вспомогательные методы, завершающие реализацию неблокирующего HTTP-сервера.

Метод `buildHttpResponse` формирует корректно отформатированный HTTP-ответ с заголовками и телом:

```
private String buildHttpResponse(HttpRequest request,
                                 int requestId, boolean keepAlive) {
  StringBuilder response = new StringBuilder();
  // Соответствие версии HTTP запроса
  response.append(request.version).append(" 200 OK\r\n");
  // Заголовки
  response.append("Content-Type: text/plain\r\n");
  response.append("Server: NonBlockingHttpServer/1.0\r\n");
  response.append("Date: ").append(Instant.now()).append("\r\n");
  // Корректная обработка заголовка Connection
  if(request.version.equals("HTTP/1.0") && keepAlive) {
    response.append("Connection: keep-alive\r\n");
    return null;
  } else if (request.version.equals("HTTP/1.1") && !keepAlive) {
    response.append("Connection: close\r\n");
  }
  // Для HTTP/1.1 с keep-alive заголовок Connection не требуется
  // (это значение по умолчанию)
  String body = String.format(
      "Request #%d processed\nPath: %s\nMethod: %s\nTime: %s\nThread: %s\n",
      requestId,
      request.path,
      request.method,
      Instant.now(),
      Thread.currentThread().getName());
  response.append("Content-Length: ").append(body.length()).append("\r\n");
  response.append("\r\n"); // Empty line between headers and body
  response.append(body);
  return response.toString();
}
```

Обратите внимание: ответ отражает версию HTTP клиента вместо постоянного возврата HTTP/1.1. Это обеспечивает совместимость с клиентами, использующими HTTP/1.0, такими как Apache Bench. Заголовок `Connection` также корректно обрабатывается в зависимости от версии протокола и статуса keep-alive.

Основу обработки запросов составляет парсинг необработанных HTTP-данных. Метод `parseHttpRequest` извлекает HTTP-запросы из потока байтов, постепенно обрабатывая частичные данные:

```
private HttpRequest parseHttpRequest(String buffer) {
  int requestEndIndex = buffer.indexOf("\r\n\r\n");
  if (requestEndIndex == -1) {
    requestEndIndex = buffer.indexOf("\n\n");
```

https://max.ru/bookofgeek

```
    if (requestEndIndex == -1) {
      return null; // Еще нет завершенных запросов
    }
  }

  String requestText = buffer.substring(0, requestEndIndex);
  String[] lines = requestText.split("\r\n|\n");
  if (lines.length == 0) return null;

  // Парсинг строки запроса
  String[] requestLineParts = lines[0].split(" ");
  if (requestLineParts.length != 3) return null;
  HttpRequest request = new HttpRequest();
  request.method = requestLineParts[0];
  request.path = requestLineParts[1];
  request.version = requestLineParts[2];

  for (int i = 1; i < lines.length; i++) {
    String line = lines[i];
    int colonPos = line.indexOf(':');
    if (colonPos > 0) {
      String name = line.substring(0, colonPos).trim();
      String value = line.substring(colonPos + 1).trim();
      request.headers.put(name.toLowerCase(), value);
    }
  }

  // Сохранение остатка буфера для следующего запроса
  request.remainingBuffer = buffer.substring(
    requestEndIndex
      + (buffer.substring(requestEndIndex)
              .startsWith("\r\n\r\n") ? 4 : 2));
  return request;
}
```

При сбое обработки запроса формируется специальный ответ. Метод `buildErrorResponse` создает HTTP-ответ, соответствующий ошибке 500:

```
private String buildErrorResponse(HttpRequest request,
                                  int requestId, Exception error) {
  StringBuilder response = new StringBuilder();
  // Сопоставление HTTP-версии запроса
  response.append(request.version).append(" 500 Internal Server Error\r\n");

  // Заголовки
  response.append("Content-Type: text/plain\r\n");
  response.append("Server: NonBlockingHttpServer/1.0\r\n");
  response.append("Date: ").append(Instant.now()).append("\r\n");
  // Всегда закрывать соединение при ошибке
  response.append("Connection: close\r\n");

  // Тело
  String body = String.format(
    "Request #%d failed\nPath: %s\nError: %s\nTime: %s",
```

https://max.ru/bookofgeek

```
    requestId,
    request.path,
    error.getMessage() != null
        ? error.getMessage()
        : error.getClass().getSimpleName(),
    Instant.now());

  response.append("Content-Length: ")
          .append(body.length()).append("\r\n")
          .append("\r\n") // Пустая строка между заголовками и телом
          .append(body);

  return response.toString();
}
```

Класс `HttpRequest` остается таким же, как в предыдущем примере, поэтому не будем повторять его здесь. Он содержит HTTP-метод, путь, версию, карту заголовков и оставшийся буфер для обработки частичных запросов в неблокирующем сервере.

Если теперь запустить программу на Java, то даже при использовании одного потока она сможет обрабатывать несколько соединений одновременно. В этом сила NIO: он эффективно управляет множеством клиентов без создания отдельного потока для каждого соединения.

Для тестирования масштабируемости создадим простое мультиклиентное приложение, которое может отправлять тысячи запросов соединения к этому серверу, заставляя обрабатывать большой объем одновременных соединений и наблюдая его поведение под нагрузкой:

```
import java.io.BufferedReader;
import java.io.IOException;
import java.io.InputStreamReader;
import java.io.PrintWriter;
import java.net.Socket;
import java.util.ArrayList;
import java.util.List;

public class PipeliningLoadTest {
  public static void main(String[] args) throws Exception {
    int numConnections = 10;
    int requestsPerConnection = 100;
    long startTime = System.currentTimeMillis();
    List<Thread> threads = new ArrayList<>();
    for (int i = 0; i < numConnections; i++) {
      Thread t = Thread.ofVirtual().start(() -> { ❶
        try {
          testPipelining(requestsPerConnection);
        } catch (Exception e) {
          e.printStackTrace();
        }
      });
```

https://max.ru/bookofgeek

```
      threads.add(t);
    }
    for (Thread t : threads) {
      t.join();
    }
    long elapsed = System.currentTimeMillis() - startTime;
    System.out.println("Total time: " + elapsed + "ms");
    System.out.println("Requests per second: " +
         (numConnections * requestsPerConnection * 1000.0 / elapsed));
  }
  private static void testPipelining(int numRequests)
      throws Exception {
    Socket socket = new Socket("localhost", 8080);
    PrintWriter out = new PrintWriter(
        socket.getOutputStream(), true);
    BufferedReader in = new BufferedReader(
        new InputStreamReader(socket.getInputStream()));
    for (int i = 0; i < numRequests; i++) {
      String path = (i % 10 == 0) ? "/slow" : "/fast"; ❷
      out.println("GET " + path + " HTTP/1.1");
      out.println("Host: localhost");
      out.println("Connection: " +
          (i == numRequests - 1 ? "close" : "keep-alive")); ❸
      out.println();
    }
    for (int i = 0; i < numRequests; i++) {
      readResponse(in); ❹
    }
    socket.close();
  }

  static void readResponse(BufferedReader in)
      throws IOException {
    // Чтение строки статуса и заголовков
    String line;
    int contentLength = 0;
    while ((line = in.readLine()) != null) {
      System.out.println(line);
      if(line.startsWith("Content-Length: ")) {
        contentLength = Integer.parseInt(
            line.substring("Content-Length: ".length()));
      }
      if(line.isEmpty()) {
        break; // Конец заголовков
      }
    }
    // Чтение тела
    char[] body = new char[contentLength];
    in.read(body);
    System.out.println(new String(body));
  }
}
```

https://max.ru/bookofgeek

Нагрузочное тестирование показывает несколько важных моментов конвейерной обработки HTTP и обработки конкурентных соединений.

❶ Мы использовали виртуальные потоки, создающие десять конкурентных соединений. Каждый поток обрабатывает одно соединение со 100 конвейерными запросами.

❷ Каждый десятый запрос мы замедлили (задержка 2 с), чтобы имитировать реалистичную рабочую нагрузку.

❸ Последний запрос в каждом соединении использует заголовок `"Connection: close"` для корректного завершения keep-alive-сессии.

❹ После того как все запросы отправлены, ответы читаются строго по порядку — это фундаментальный принцип конвейерной обработки HTTP/1.1.

Нагрузочный тест запускает десять клиентов на виртуальных потоках, где каждый устанавливает одно постоянное соединение с сервером и передает через него 100 конвейерных HTTP/1.1 запросов (всего 1000 запросов). Каждый десятый запрос направляется по медленному пути `/slow`, с двухсекундной паузой для имитации ресурсоемкой задачи, остальные девять — по пути `/fast` и возвращают ответ немедленно. После отправки всех запросов клиенты последовательно читают ответы, сохраняя семантику конвейера. Когда все потоки останавливаются, выводится общее время процесса и рассчитывается пропускная способность в запросах в секунду. Поскольку активно только десять сокетов ОС, а селектор сервера обрабатывает события готовности в одном потоке, тест оценивает эффективность конкурентности соединений и конвейерной обработки, а не количество сокетов. Это наглядно демонстрирует способность NIO поддерживать высокую пропускную способность даже при значительном различии задержек отдельных запросов.

Событийно-ориентированная архитектура

Хотя наша простая программа, спроектированная на глазок, хорошо масштабируется и выполняет свою задачу, реальные приложения гораздо сложнее. Они требуют гораздо большего, чем просто несколько предположений о реализации клиента. Как структурировать наш код, чтобы он поддерживал более крупные и динамичные системы?

К счастью, многие фреймворки построены на основе событийно-ориентированного программирования (event-driven programming), которое упрощает эффективную обработку конкурентности и масштабируемости. Отличный пример такого подхода — фреймворки Netty (https://netty.io) и Vert.x (https://vertx.io).

Перепишем нашу программу с использованием Vert.x:

```
import io.vertx.core.AbstractVerticle;
import io.vertx.core.Promise;
import io.vertx.core.Vertx;
```

https://max.ru/bookofgeek

```
import io.vertx.core.json.JsonObject;
import io.vertx.ext.web.Router;
import io.vertx.ext.web.RoutingContext;
import java.util.concurrent.atomic.AtomicLong;

public class VertxHttpServer extends AbstractVerticle {
  private static final int PORT = 8080;
  private final AtomicLong requestCounter = new AtomicLong(0); ❶
  private long startTime;

  @Override
  public void start(Promise<Void> startPromise) {
    startTime = System.currentTimeMillis();
    Router router = Router.router(vertx); ❷
    router.get("/fast").handler(this::handleFastRequest);
    router.get("/slow").handler(this::handleSlowRequest);
    router.get("/stats").handler(this::handleStats);
    vertx.createHttpServer()
        .requestHandler(router)
        .listen(PORT, "localhost"); ❸
  }

  private void handleFastRequest(RoutingContext ctx) {
    long requestId = requestCounter.incrementAndGet();
    ctx.response()
      .putHeader("content-type", "text/plain")
      .end("Request #" + requestId + ": Fast request processed"); ❹
  }

  private void handleSlowRequest(RoutingContext ctx) {
    long requestId = requestCounter.incrementAndGet();
    vertx.setTimer(2000, id -> { ❺
      ctx.response()
          .putHeader("content-type", "text/plain")
          .end("Request #" + requestId + ": Slow request processed");
    });
  }

  private void handleStats(RoutingContext ctx) {
    long uptimeMillis = System.currentTimeMillis() - startTime;
    JsonObject stats = new JsonObject()
            .put("totalRequests", requestCounter.get())
            .put("uptimeMillis", uptimeMillis)
            .put("currentThread", Thread.currentThread().getName())
            .put("isEventLoopThread", Vertx.currentContext()
                                .isEventLoopContext()); ❻
    ctx.response()
        .putHeader("content-type", "application/json")
        .end(stats.encodePrettily());
  }
```

https://max.ru/bookofgeek

```
  public static void main(String[] args) {
    Vertx vertx = Vertx.vertx();
    vertx.deployVerticle(new VertxHttpServer()); ❼
  }
}
```

Наша реализация с применением Vert.x демонстрирует ключевые принципы событийно-ориентированной архитектуры.

❶ Для потокобезопасного подсчета запросов используем `AtomicLong`, так как несколько потоков цикла событий могут конкурентно его инкрементировать.

❷ Маршрутизацию HTTP-запросов осуществляем декларативно через `Router`: каждый маршрут сопоставляется с методом-обработчиком.

❸ Сервер запускается асинхронно в цикле событий, не блокируя главный поток.

❹ Быстрые запросы выполняются немедленно в потоке цикла событий, демонстрируя неблокирующий ввод-вывод.

❺ Важное отличие: `vertx.setTimer()` планирует ответ через 2 с *без* блокировки потока цикла событий — это основа поддержки высокой конкурентности.

❻ Эндпоинт `stats` указывает, какой именно поток обрабатывает запрос, что доказывает: все запросы выполняются в потоках цикла событий.

❼ Развертывание `verticle` запускает событийно-ориентированный сервер с Vert.x, автоматически управляющий циклами событий.

Для запуска этого Vert.x-сервера нужно добавить следующие зависимости в Maven-проект. Создайте Maven-проект, добавьте зависимости и запустите его:

```
<dependency>
    <groupId>io.vertx</groupId>
    <artifactId>vertx-core</artifactId>
    <version>5.0.0</version>
</dependency>
<dependency>
    <groupId>io.vertx</groupId>
    <artifactId>vertx-web</artifactId>
    <version>5.0.0</version>
</dependency>
```

Это дает нам HTTP-эндпоинты для тестирования через curl:

```
curl -X GET http://localhost:8080/fast
curl -X GET http://localhost:8080/slow
curl -X GET http://localhost:8080/stats
```

https://max.ru/bookofgeek

Не будем углубляться в детали реализации Vert.x, сосредоточимся на концептуальном уровне. По сути, данный фреймворк построен на паттерне многоядерного реактора (multicore reactor pattern) и использует модель конкурентности, основанную на цикле событий. В нем применяется легковесный набор потоков цикла событий, которые непрерывно прослушивают события ввода-вывода через неблокирующие операции NIO. При возникновении события (например, входящего запроса) в рамках цикла событий выполняется короткий неблокирующий колбэк, что обеспечивает отзывчивость.

Однако не все задачи можно обрабатывать в цикле событий. Некоторые операции, такие как запросы к базе данных или доступ к файловой системе, по своей природе являются или блокирующими, или длительными. Vert.x выгружает их в отдельный пул рабочих потоков, чтобы не замедлять цикл событий. Это позволяет ему оставаться свободным для обработки новых запросов без ненужных задержек.

Одно из ключевых преимуществ Vert.x — шина событий (event bus, https://oreil.ly/9Fs7I), которая позволяет компонентам приложения взаимодействовать асинхронно. Таким образом сохраняется слабая связанность системы и при этом обеспечивается согласованная работа всех частей приложения (рис. 6.1).

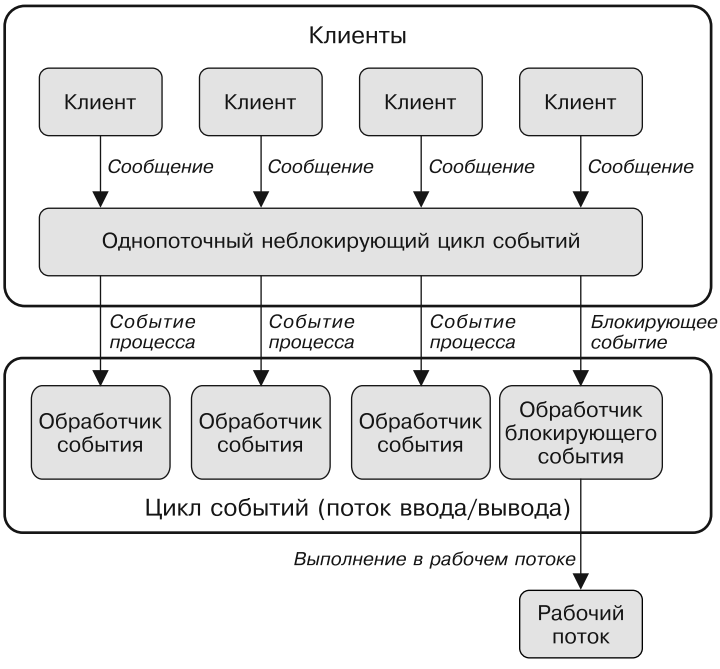


Рис. 6.1. Обзор цикла событий событийно-ориентированной архитектуры в Vert.x

https://max.ru/bookofgeek

На рис. 6.1 показана работа фреймворка Vert.x, где центральным элементом выступает цикл событий. Фреймворк отвечает за управление подключениями клиентов, обработку исходящих запросов и формирование ответов — все операции выполняются неблокирующим способом. Как правило, работающий поверх этой схемы реактивный фреймворк предоставляет высокоуровневый API для упрощения сетевых операций. Далее мы рассмотрим это более подробно. Вместо того чтобы работать напрямую с необработанным, неблокирующим вводом-выводом, разработчики используют связанные с обменом данными абстракции: HTTP-запросы и ответы или сообщения Kafka, которые значительно упрощают процесс разработки.

Код нашего приложения находится на верхнем уровне. Он не взаимодействует с циклом событий напрямую, а работает через обработчики событий. Например, в предыдущем примере мы создали обработчики `handleFastRequest` и `handleSlowRequest` для обработки определенных HTTP-маршрутов.

Пока все выглядит отлично. Однако здесь есть опасный момент: обработчики событий в вашем приложении выполняются в потоке цикла событий — по сути, это поток ввода-вывода. Если код блокирует его, другие конкурентные события не будут обрабатываться. В результате резко падают отзывчивость и конкурентность — это катастрофа для высокореактивной системы.

Обратите внимание, что обработчик медленных запросов в нашем примере с Vert.x использует `vertx.setTimer()` вместо `Thread.sleep()`. Это принципиально, поскольку таймер планирует выполнение колбэка через 2 с без блокировки потока цикла событий. Поток сразу освобождается для обработки других запросов во время ожидания.

Здесь имеется простое решение: код должен быть неблокирующим. Ни при каких обстоятельствах нельзя блокировать потоки ввода-вывода. Если это произойдет, весь цикл обработки событий остановится, что обрушит отзывчивость приложения.

Конечно, есть и обходные решения. Можно вынести блокирующие операции в отдельный пул рабочих потоков, что частично реализовано в самом Vert.x. Однако это не должно становиться стандартной практикой. Более того, чрезмерное использование рабочих потоков противоречит самой идее реактивной системы. Почему? Потому что каждое переключение выполнения с потока ввода-вывода на рабочий поток и обратно приводит к переключению контекста. Затраты ресурсов возрастают, увеличивается время отклика и сходят на нет те преимущества в эффективности, ради которых все и затевалось.

Как же добиться, чтобы код приложения оставался неблокирующим? Легко сказать: «пишите неблокирующий код», но на практике это гораздо сложнее. Не всегда очевидно, как написать по-настоящему неблокирующий код. Требуется изменить привычный подход к программированию.

https://max.ru/bookofgeek

Здесь-то и могут помочь реактивные фреймворки. Они предоставляют асинхронные API, специально разработанные, чтобы избежать блокирующих операций и при этом эффективно обрабатывать сложные рабочие процессы. Вместо того чтобы ждать, пока завершится задача, мы регистрируем колбэк-функции, используем промисы (promises) или применяем реактивные потоки для оркестрации выполнения программы без блокировки цикла событий.

Выясним, что же представляют собой асинхронные API и как они работают.

Асинхронные API

Наверняка многие знакомы с синхронными API, но напомню вкратце эту концепцию. Рассмотрим простой метод Java:

```
public class AiService {
  public String chat(String message) {
    return "Echo: " + message.toUpperCase();
  }
}
```

Этот метод принимает строку, обрабатывает ее и возвращает ответ. Вызов выглядит так:

```
String result = aiService.chat("What is the meaning of life?");
```

Здесь вызывающий поток выполняет метод `chat` и блокируется до получения ответа. Это классический пример *синхронного* API — вызывающая сторона вынуждена ожидать.

Теперь преобразуем API в асинхронный: метод будет возвращать управление немедленно, а когда результат будет готов, колбэк-функция его обработает. Модифицированная версия:

```
public void chat(String message, Consumer<String> consumer) {
  Thread.startVirtualThread(() -> {
    try {
      String response = "Echo: " + message.toUpperCase();
      consumer.accept(response);
    } catch (Exception e) {
      consumer.accept("Error during chat: " + e.getMessage());
    }
  });
}
```

Эта версия метода теперь принимает два параметра.

❶ Само сообщение.

❷ Колбэк-функцию (типа `Consumer<String>`), которая вызывается, когда ответ будет готов.

https://max.ru/bookofgeek

Теперь посмотрим, как можно использовать этот асинхронный API:

```
aiService.chat("Hello, how are you?", response -> {
  System.out.println("Response 1: " + response);
});
aiService.chat("What is your name?", response -> {
  System.out.println("Response 2: " + response);
});
```

Что здесь происходит? Первый вызов `chat` выполняется, но, в отличие от синхронного подхода, главный поток *не* ждет результата. Вместо этого он немедленно переходит к выполнению второго вызова `chat`. Тем временем ответы будут обработаны асинхронно, как только станут доступны. В этом суть асинхронного программирования — освобождение главного потока для выполнения других задач во время ожидания результатов.

Основной проблемой колбэков считается плохая интеграция. Что произойдет, например, если мы попробуем передать результат из первого колбэка в другой метод? Сейчас проверим:

```
aiService.chat("What is the meaning of life?", response -> {
  aiService.chat(response, response2 -> {
    aiService.chat(response2, response3 -> {
        aiService.chat(response3, response4 -> {
            System.out.println(response4);
        });
      });
  });
});
```

Как видно из примера, при использовании колбэков для асинхронного выполнения код быстро может стать неуправляемым. Чем больше колбэков добавить, тем сложнее будет читать и поддерживать код. Это явление часто называют «адом колбэков» (callback hell).

Каков же выход из этой ситуации?

Java предлагает более чистое решение — `CompletableFuture`. Этот класс позволяет писать асинхронный код так, что он остается читабельным, структурированным и композируемым (composable). Изменим метод `chat`, чтобы он возвращал `CompletableFuture<String>`:

```
public CompletableFuture<String> chat(String message) {
  return CompletableFuture.supplyAsync(() -> {
    try {
      return "Echo: " + message.toUpperCase();
    } catch (Exception e) {
      return "Error during chat: " + e.getMessage();
    }
  });
}
```

https://max.ru/bookofgeek

Теперь, не используя колбэки, мы можем объединять несколько асинхронных вызовов в более структурированную цепочку:

```
aiService.chat("What is the meaning of life?")
       .thenCompose(aiService::chat)
       .thenCompose(aiService::chat)
       .thenCompose(aiService::chat)
       .thenAccept(System.out::println);
```

Так гораздо проще создавать композицию асинхронных методов по сравнению с вложением колбэков. Хотя мы пока не рассматривали обработку ошибок, скажу заранее, что `CompletableFuture` предоставляет API с богатым функционалом для грамотного управления ошибками.

Значит ли это, что мы решили проблему блокировки потоков ввода-вывода? Что ж, `CompletableFuture` отлично справляется с обработкой единичных асинхронных результатов, но мы пока не видели, как он работает с потоками данных. `CompletableFuture` прекрасно подходит для методов, возвращающих одиночное значение, но что происходит, когда вместо разового ответа нужно обрабатывать последовательности результатов — непрерывный поток данных?

Здесь все гораздо интереснее. Вот где потенциал реактивных фреймворков раскрывается в полной мере.

Реактивное программирование в Java

Реактивное программирование в Java — это парадигма, ориентированная на асинхронные, неблокирующие и событийно-управляемые приложения. Его ключевая концепция — *реактивный поток (reactive stream)*, стандарт, определяемый спецификацией Reactive Streams (https://oreil.ly/HhQ9M), для управления асинхронными потоками данных с поддержкой *обратного давления (backpressure)*.

Сначала разберем, что такое реактивный поток, а затем обсудим механизм обратного давления.

Понятие реактивных потоков

В реактивном программировании код организован вокруг потоков, образуя цепочки преобразований (пайплайны). В этой парадигме любые данные представляются как поток событий, который передается от продюсеров (producer) к потребителям (consumer) через серию преобразований.

Событиями может быть что угодно: клики мышкой, показания датчиков, входящие сообщения и многое другое. Они передаются от источника выше по потоку (upstream source) к подписчику ниже по потоку (downstream Subscriber), проходя через операторы, которые преобразуют или фильтруют данные (рис. 6.2).

https://max.ru/bookofgeek

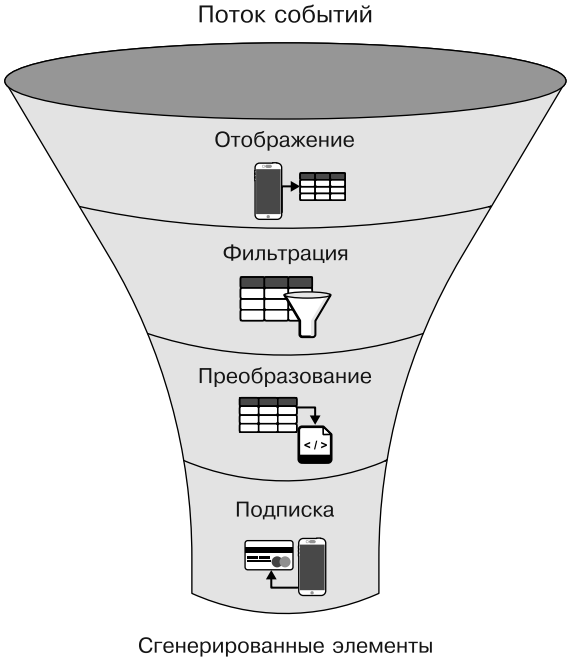


Рис. 6.2. Поток данных в реактивном программировании

Каждый оператор наблюдает за своим upstream-источником и создает новый поток. Однако потоки по умолчанию ленивые, они не начинают обработку, пока не появится подписчик. Поскольку мы не знаем, *когда* придет событие (асинхронность), мы настраиваем «наблюдателей» (observers), которые будут реагировать на его появление.

В реактивном программировании мы работаем с четырьмя базовыми компонентами.

Издатель (Publisher)

Источник, который асинхронно выдает элементы данных. Его можно представить как продюсера данных, способного выпускать от нуля до множества элементов с течением времени.

Подписчик (Subscriber)

Потребитель, который получает и обрабатывает выданные элементы. Подписчики выражают интерес в получении данных и определяют способ их обработки.

Подписка (Subscription)

Связующее звено между издателем и подписчиком, управляющее потоком данных и обеспечивающее контроль обратного давления.

https://max.ru/bookofgeek

Процессор (Processor)
: Компонент, который одновременно выступает как подписчик и издатель, преобразуя данные при их прохождении через поток.

Реактивный поток может генерировать три типа сигналов.

Элементы данных (Data items)
: Фактические значения, передаваемые через поток.

Сигнал ошибки (Error signal)
: Сигнал, указывающий на возникновение неустранимой ошибки, которая завершает поток.

Сигнал завершения (Completion signal)
: Успешное окончание потока, когда элементов для выпуска больше нет.

Существует несколько библиотек для реактивного программирования на Java. Наиболее известные из них следующие.

Project Reactor (https://oreil.ly/rKyko)
: Предоставляет такие основные типы:
- `Flux<T>` (https://oreil.ly/5Xb9z) — последовательность из 0...*N* элементов;
- `Mono<T>` (https://oreil.ly/2fKjm) — 0 или 1 элемент.

RxJava (https://oreil.ly/hrj2a)
: Реактивное расширение для Java. Предоставляет следующие типы:
- `Observable<T>` (https://oreil.ly/HY8sm) — последовательность из 0...*N* элементов;
- `Single<T>` (https://oreil.ly/UlIwi) — ровно 1 элемент;
- `Maybe<T>` (https://oreil.ly/Or-Vx) — 0 или 1 элемент.

Рассмотрим реактивное программирование на примерах с применением Project Reactor:

```
public class ReactiveExample {
  public static void main(String[] args) {
    // Создаем Flux, генерирующий целые числа от 1 до 5
    Flux<Integer> numbers = Flux.just(1, 2, 3, 4, 5); ❶
    //  Обрабатываем поток: фильтруем четные числа и преобразуем в строки
    numbers
       .filter(n -> n % 2 == 0) ❷ // Оставляем четные числа
       .map(n -> "Value: " + n) ❸ // Преобразуем в строки
       .subscribe(
         System.out::println, ❹ // onNext: выводим каждое значение
         error -> System.err.println("Error: " + error), ❺ // onError
         () -> System.out.println("Done!") ❻ // onComplete
       );
  }
}
```

https://max.ru/bookofgeek

Данный код представляет собой базовый пайплайн реактивного потока.

❶ Создается холодный поток (cold stream) с пятью целочисленными значениями. Он не будет испускать значения до подписки подписчика.

❷ Оператор `filter` создает новый поток, содержащий только элементы, соответствующие условию отбора (в данном случае — четные числа).

❸ Оператор `map` отображает каждый элемент, конвертируя целые числа в форматированные строки.

❹ Метод `subscribe` запускает выполнение пайплайна и определяет, как обрабатывать каждое сгенерированное значение.

❺ Если любой оператор в пайплайне выбрасывает исключение, выполняется колбэк обработки ошибок.

❻ Когда поток завершает все эмиссии, выполняется колбэк завершения.

Результат выполнения:

```
Value: 2
Value: 4
Done!
```

Получив базовое представление о реактивном программировании, применим его на практике: создадим систему мониторинга цен на криптовалюту.

Представьте себе систему, которая в реальном времени отслеживает цены криптовалюты на нескольких биржах. Этот сценарий отлично демонстрирует преимущества реактивного программирования, включая обработку множества асинхронных источников данных, преобразование потоков данных и реакцию на определенные условия.

Сначала определим модель данных:

```
public record PriceData(String exchange, String symbol, double price,
                        Instant timestamp) {}

public record PriceAlert(String symbol, String message, AlertType type) {}

enum AlertType {THRESHOLD_CROSSED, RAPID_CHANGE, ANOMALY}
```

Теперь создадим простую реактивную систему мониторинга цен:

```
import reactor.core.publisher.Flux;
import java.time.Duration;
import java.time.Instant;

public class SimplePriceMonitor {
  public static void main(String[] args) throws InterruptedException {
    // Создаем поток обновлений цен ❶
    Flux<PriceData> priceStream = Flux.interval(Duration.ofSeconds(1))
    .map(i -> new PriceData(
         "Binance",
```

https://max.ru/bookofgeek

```
            "BTC/USD",
            50000 + (Math.random() - 0.5) * 1000, ❷
            Instant.now()
        ));

        // Обрабатываем поток ❸
        priceStream
            .filter(price -> price.price() > 50200) ❹
            .map(price -> String.format("BTC price $%.2f exceeds threshold!",
                                        price.price())) ❺

            .subscribe(
              alert -> System.out.println(alert), ❻
              error -> System.err.println("Error: " + error),
              () -> System.out.println("Monitoring complete")
            );

        // Не допускаем завершения главного потока
        Thread.sleep(10000);
    }
}
```

Разберемся, что происходит в этом коде.

❶ Создается холодный поток, генерирующий значение каждую секунду. Оператор `interval` использует демон-поток, поэтому необходимо поддерживать работу главного потока. Поток начинает эмиссию данных только после того, как подписчик подпишется на него.

❷ Генерируются смоделированные данные о ценах со случайными колебаниями вокруг базовой цены 50 000 долл.

❸ Запускается пайплайн потоковой обработки данных. Каждый оператор создает новый поток, наблюдающий за предыдущим.

❹ Поток фильтруется, остаются только цены выше 50 200 долл., что подтверждает избирательную обработку.

❺ Данные о цене преобразуются в оповещения, показывая, как меняется форма данных в потоке.

❻ Выполняется подписка на поток, активируя начало обработки во всем пайплайне.

В реальных приложениях требуется более сложная обработка потоков. Создадим комплексную систему мониторинга цен, работающую с несколькими биржами и символами криптовалют:

```
public class CryptoPriceMonitor {
  private static final List<String> EXCHANGES =
    List.of("Binance", "Coinbase", "Kraken");
  private static final List<String> SYMBOLS =
    List.of("BTC/USD", "ETH/USD", "SOL/USD");
```

https://max.ru/bookofgeek

```
// Представления Sink для рассылки оповещений нескольким подписчикам ❶
private static final Sinks.Many<PriceAlert> alertSink =
  Sinks.many().multicast().onBackpressureBuffer();

public static void main(String[] args) throws InterruptedException {
  // Создаем объединенный поток из нескольких бирж ❷
  Flux<PriceData> priceStream = Flux.merge(
    EXCHANGES.stream()
             .map(CryptoPriceMonitor::createExchangeFeed)
             .toList()
    );

    // Группируем цены по символам криптовалют для параллельной обработки ❸
    priceStream
      .groupBy(PriceData::symbol)
      .subscribe(symbolFlux -> {
        String symbol = symbolFlux.key();

        // Вычисляем скользящее среднее за 5 с ❹
        symbolFlux
          .window(Duration.ofSeconds(5))
          .flatMap(window -> calculateMovingAverage(window, symbol))
          .subscribe(avg -> System.out.printf(
            " %s Moving Avg: $%.2f%n", symbol, avg));

        // Обнаруживаем резкие изменения цен ❺
        symbolFlux
          .buffer(2, 1)
          .filter(buffer -> buffer.size() == 2)
          .map(buffer -> detectRapidChange(buffer.get(0), buffer.get(1)))
          .filter(Optional::isPresent)
          .map(Optional::get)
          .subscribe(alertSink::tryEmitNext);
      });

      // Подписываемся на оповещения ❻
      alertSink.asFlux()
        .subscribe(alert -> System.out.printf(" [%s] %s: %s%n",
                   alert.type(), alert.symbol(), alert.message()));

      Thread.sleep(30000);
}

private static Flux<PriceData> createExchangeFeed(String exchange) {
  return Flux.interval(Duration.ofMillis(
    100 + (int) (Math.random() * 400))) ❼
    .map(i -> {
      String symbol = SYMBOLS.get(
        (int) (Math.random() * SYMBOLS.size()));
        double basePrice = getBasePrice(symbol);
        double variation = (Math.random() - 0.5) * 0.01;
        double price = basePrice * (1 + variation);
```

https://max.ru/bookofgeek

```
        return new PriceData(exchange, symbol, price, Instant.now());
      })
      .doOnNext(price -> System.out.printf(
        "%s [%s]: $%.2f%n", ❽
        price.exchange(), price.symbol(), price.price()));
}
```

Этот более сложный пример демонстрирует несколько ключевых реактивных паттернов.

❶ Представления Sink перекидывают мостик между императивным и реактивным кодами, позволяя вручную отправлять элементы в поток.

❷ `merge` объединяет несколько потоков, чередуя элементы по мере их поступления.

❸ `groupBy` разделяет поток на подпотоки по ключу, позволяя параллельно обрабатывать разные символы.

❹ `window` разбивает данные на чанки на основе времени, идеально подходящие для вычисления метрик в скользящем временном окне.

❺ `buffer` собирает элементы в списки с помощью скользящих окон, созданных параметром `stride`.

❻ Горячие потоки (создаваемые Sinks) позволяют нескольким подписчикам получать те же события.

❼ Неравномерные интервалы имитируют нерегулярное поступление данных (как в реальности) с разных бирж.

❽ Побочные эффекты через `doOnNext` позволяют выполнять логирование, не влияя на поток данных.

Существуют вспомогательные методы, которые дополняют этот пример.

Метод `calculateMovingAverage` вычисляет скользящее среднее цен во временном окне, что важно для выявления ценовых трендов и сглаживания краткосрочных колебаний:

```
private static Mono<Double> calculateMovingAverage(Flux<PriceData> window,
                                                   String symbol) {

  return window
    .map(PriceData::price)
    .reduce(new double[]{0, 0}, (acc, price) -> {
      acc[0] += price; // sum
      acc[1] += 1; // count
      return acc;
    })
    .map(acc -> acc[1] > 0 ? acc[0] / acc[1] : 0.0)
    .filter(avg -> avg > 0);
}
```

https://max.ru/bookofgeek

Хотя скользящие средние помогают выявлять тренды, также необходимо обнаруживать резкие колебания цен. Метод `detectRapidChange` сравнивает последовательные пункты курса ценных бумаг, чтобы оповестить о значительных изменениях:

```
private static Optional<PriceAlert> detectRapidChange(PriceData prev,
                                                      PriceData current) {

  if(!prev.symbol().equals(current.symbol())) {
    return Optional.empty();
  }

  double changePercent = Math.abs((current.price() - prev.price())
    / prev.price()) * 100;
  if(changePercent > 0.5) { // порог изменения 0,5 %
    return Optional.of(new PriceAlert(
      current.symbol(),
      String.format("Rapid %.2f%% change: $%.2f → $%.2f",
          changePercent, prev.price(), current.price()),
      AlertType.RAPID_CHANGE
    ));
  }
  return Optional.empty();
}
```

Метод `getBasePrice` возвращает базовые значения для поддерживаемых пар валют:

```
private static double getBasePrice(String symbol) {
  return switch (symbol) {
    case "BTC/USD" -> 50000;
    case "ETH/USD" -> 3000;
    case "SOL/USD" -> 100;
    default -> 1000;
  };
}
```

Это всего лишь краткое введение в реактивные потоки. В книге я хотел дать общее представление о сути реактивного программирования, не вдаваясь в подробности. Для более глубокого изучения темы рекомендую обратиться к специализированной литературе.



Реактивные потоки (Reactive Streams) не следует путать с Java Streams.

- Java Streams API (`java.util.stream.Stream`, https://oreil.ly/7tgYY) предназначен для синхронной обработки коллекций и обработки данных в оперативной памяти (in-memory).
- Реактивные потоки обеспечивают асинхронную, событийно-ориентированную и неблокирующую обработку данных с поддержкой backpressure (обратного давления).

https://max.ru/bookofgeek

Backpressure

В реактивном программировании backpressure — это ключевой механизм, позволяющий потребителю (он же подписчик) сигнализировать upstream-издателю о невозможности обработки данных. По сути, потребитель может сообщить: «Помедленнее! Я не успеваю так быстро обрабатывать данные».

Рассмотрим различные стратегии обработки backpressure на примере высокочастотного трейдинга:

```
public class BackpressureDemo {
  public static void main(String[] args) throws InterruptedException {
    // Симулируем сверхвысокочастотную ленту цен ❶
    Flux<PriceData> extremeFeed = Flux.interval(Duration.ofNanos(100_000))
    .map(i -> new PriceData(
      "HFT-Exchange", "BTC/USD",
      50000 + ThreadLocalRandom.current().nextDouble(-100, 100),
      Instant.now()
    ))
    .share(); // Горячий поток для совместного использования подписчиками

    // Стратегия 1. Sampling — принимает периодические снапшоты ❷
    System.out.println("SAMPLING Strategy:");
    extremeFeed
      .sample(Duration.ofMillis(100))
      .take(10)
      .subscribe(price -> System.out.printf(
        "[SAMPLED] Price: $%.2f at %s%n",
        price.price(), price.timestamp()));

    Thread.sleep(1500);

    // Стратегия 2. Drop — сброс при переполнении ❸
    System.out.println("\nDROP Strategy:");
    AtomicInteger dropped = new AtomicInteger(0);
    extremeFeed
      .onBackpressureDrop(price -> {
        if (dropped.incrementAndGet() % 1000 == 0) {
          System.out.printf("[DROPPED] %d updates dropped%n",
                            dropped.get());
        }
      })
      .publishOn(Schedulers.boundedElastic()) ❹
      .take(Duration.ofSeconds(1))
      .subscribe(price -> {
        simulateWork(10); // Имитация медленной обработки
        System.out.printf("[PROCESSED] Price: $%.2f%n", price.price());
    });

    Thread.sleep(1500);
```

https://max.ru/bookofgeek

```
    // Стратегия 3. Latest — хранит только последнее значение ❺
    System.out.println("\nLATEST Strategy:");
    extremeFeed
      .onBackpressureLatest()
      .publishOn(Schedulers.boundedElastic())
      .subscribe(price -> {
        simulateWork(100); // Очень медленная обработка
        System.out.printf("[LATEST] Price: $%.2f%n", price.price());
      });

    Thread.sleep(2000);
  }

  private static void simulateWork(int millis) {
    try {
      Thread.sleep(millis);
    } catch (InterruptedException e) {
      Thread.currentThread().interrupt();
      throw new RuntimeException(e);
    }
  }
}
```

Рассмотрим каждую стратегию обработки backpressure на примере торговых данных.

❶ Создаем поток, генерирующий 10 000 элементов в секунду, имитируя высокочастотные торговые данные.

❷ Стратегия Sampling делает периодические снапшоты — это оптимально, когда требуются регулярные обновления, но обработка каждого элемента невозможна.

❸ `Drop` отбрасывает элементы, если потребитель не успевает их обрабатывать. Это подходит для случаев, когда допустима потеря части данных.

❹ `publishOn` переносит обработку в другой пул потоков, разделяя этапы производства и потребления данных.

❺ `Latest` сохраняет только самый последний необработанный элемент — подходит для сценариев, где важно только текущее состояние.

Reactor предлагает различные стратегии для управления backpressure.

`onBackpressureBuffer()`

Хранит все элементы в буфере до тех пор, пока потребитель их не получит. Используется, когда потеря данных недопустима, но имеется достаточный объем памяти.

`onBackpressureBuffer(maxSize)`

Хранит элементы в буфере с ограничением по размеру и завершается с ошибкой при его превышении. Обеспечивает защиту от исчерпания памяти.

https://max.ru/bookofgeek

`onBackpressureDrop()`

Отбрасывает избыточные элементы без сообщения об этом. Подходит для обработки данных в реальном времени, где важнее последние значения, а не полнота данных.

`onBackpressureLatest()`

Сохраняет только самый недавний элемент. Идеален для обновлений состояния, где промежуточные значения устаревают.

`onBackpressureError()`

Завершается с ошибкой при возникновении backpressure. Применяется, когда backpressure указывает на конструктивный недостаток системы.

Этот пример демонстрирует функционирование реактивных потоков в реальном сценарии, объединяя концепции издателей, операторов, подписчиков и backpressure.

Теперь выясним, как будет выглядеть наша система мониторинга цен, если использовать виртуальные потоки.

Сравним оба подхода на примере нашей системы мониторинга цен:

```
package ca.bazlur.mcj.chap6.virtualthreads;
import ca.bazlur.mcj.chap6.reactive.AlertType;
import ca.bazlur.mcj.chap6.reactive.PriceAlert;
import ca.bazlur.mcj.chap6.reactive.PriceData;
import java.time.Instant;
import java.util.*;
import java.util.concurrent.*;
import java.util.concurrent.atomic.AtomicReference;

public class PriceMonitorWithVirtualThreads {
  private static final List<String> SYMBOLS =
    List.of("BTC/USD", "ETH/USD", "SOL/USD");
  private static final List<String> EXCHANGES =
    List.of("Binance", "Coinbase", "Kraken");
  private static final Map<String, AtomicReference<PriceData>> latestPrices =
    new ConcurrentHashMap<>();
  private static final BlockingQueue<PriceAlert> alertQueue =
    new LinkedBlockingQueue<>();

  public static void main(String[] args) throws InterruptedException {
    try (var executor = Executors.newVirtualThreadPerTaskExecutor()) { ❶
      // Начало потока котировок с каждой биржи
      for (String exchange : EXCHANGES) {
        executor.submit(() -> generatePriceFeed(exchange));
      }
      // Запуск процессоров
      executor.submit(PriceMonitorWithVirtualThreads::processAlerts);
      executor.submit(PriceMonitorWithVirtualThreads::monitorThresholds);
      Thread.sleep(30000);
    }
  }
```

https://max.ru/bookofgeek

```
private static void generatePriceFeed(String exchange) {
  var random = ThreadLocalRandom.current();
  while (!Thread.currentThread().isInterrupted()) {
    try {
      Thread.sleep(100 + random.nextInt(400)); ❷
      String symbol = SYMBOLS.get(random.nextInt(SYMBOLS.size()));
      double basePrice = getBasePrice(symbol);
      double variation = (random.nextDouble() - 0.5) * 0.01;
      double price = basePrice * (1 + variation);
      PriceData priceData = new PriceData(
        exchange,
        symbol,
        price,
        Instant.now()
      );

      System.out.printf(" %s [%s]: $%.2f%n", exchange, symbol, price);
      // Обработать эту цену в новом виртуальном потоке
      Thread.startVirtualThread(() -> processPrice(priceData)); ❸
    } catch (InterruptedException e) {
      Thread.currentThread().interrupt();
      break;
    }
  }
}

private static void processPrice(PriceData currentPrice) {
  PriceData previousPrice = latestPrices
      .computeIfAbsent(currentPrice.symbol(), k -> new AtomicReference<>())
      .getAndSet(currentPrice); ❹
  if (previousPrice != null) {
    detectRapidChange(previousPrice, currentPrice);
  }
  calculateSimpleMovingAverage(currentPrice.symbol(), currentPrice);
}

private static void detectRapidChange(PriceData prev, PriceData current) {
  if (!prev.symbol().equals(current.symbol())) return;
  double changePercent = Math.abs((current.price() - prev.price())
                                  / prev.price()) * 100;
  if (changePercent > 0.5) { // Порог изменения 0,5 %
    PriceAlert alert = new PriceAlert(
        current.symbol(),
        String.format("Rapid %.2f%% change: $%.2f → $%.2f",
            changePercent, prev.price(), current.price()),
        AlertType.RAPID_CHANGE
    );
    alertQueue.offer(alert); ❺
  }
}

private static void monitorThresholds() {
  Map<String, Double> thresholds = Map.of(
      "BTC/USD", 51000.0,
```

https://max.ru/bookofgeek

```
      "ETH/USD", 3100.0,
      "SOL/USD", 105.0
  );
  while (!Thread.currentThread().isInterrupted()) {
    try {
      Thread.sleep(2000); // мониторинг раз в 2 с
      thresholds.forEach((symbol, threshold) -> {
        AtomicReference<PriceData> ref = latestPrices.get(symbol);
        if (ref != null) {
          PriceData latest = ref.get();
          if(latest != null && latest.price() > threshold) {
            alertQueue.offer(new PriceAlert(
                symbol,
                String.format("Price $%.2f exceeded threshold $%.2f",
                    latest.price(), threshold),
                AlertType.THRESHOLD_CROSSED
            ));
          }
        }
      });

    } catch (InterruptedException e) {
      Thread.currentThread().interrupt();
      break;
    }
  }
}

private static final Map<String, LinkedList<Double>> priceWindows =
  new ConcurrentHashMap<>();

private static void calculateSimpleMovingAverage(String symbol,
  PriceData price) {
  var window = priceWindows.computeIfAbsent(symbol, k ->
    new LinkedList<>());
  synchronized (window) { // исключаем множественную обработку одного окна
    window.add(price.price());
    if (window.size() > 10) {
      window.removeFirst();
    }
    if(window.size() >= 5) {
      double avg = window.stream()
          .mapToDouble(Double::doubleValue)
          .average()
          .orElse(0.0);
      System.out.printf("%s Moving Avg: $%.2f%n", symbol, avg);
    }
  }
}

static void monitorThresholds() {
  while (!Thread.currentThread().isInterrupted()) {
  try {
    PriceAlert alert = alertQueue.poll(100, TimeUnit.MILLISECONDS);
```

https://max.ru/bookofgeek

```
      if (alert != null) {
        System.out.printf(" [%s] %s: %s%n",
            alert.type(), alert.symbol(), alert.message());
      }
    } catch (InterruptedException e) {
      Thread.currentThread().interrupt();
      break;
    }
  }
}

  private static double getBasePrice(String symbol) {
    return switch (symbol) {
      case "BTC/USD" -> 50000;
      case "ETH/USD" -> 3000;
      case "SOL/USD" -> 100;
      default -> 1000;
    };
  }
}
```

Вот основные отличия подхода с использованием виртуальных потоков.

❶ Виртуальные потоки создаются неявно через исполнитель (executor), что делает этот процесс легким и ресурсосберегающим.

❷ Явные циклы с вызовами sleep заменяют реактивные интервалы, требуя ручного управления временем.

❸ Каждое обновление цены порождает новый виртуальный поток для обработки, достигая конкурентности благодаря созданию потоков, а не операторов потоков.

❹ Разделяемое изменяемое состояние с атомарными ссылками требует внимательной синхронизации, в отличие от неизменяемых преобразований при реактивном подходе.

❺ Для разделяемых структур данных необходима ручная синхронизация, тогда как реактивные потоки выполняют эту задачу через операторы.

Сравнение и компромиссы

При рассмотрении обеих реализаций становятся очевидны их фундаментальные различия. Реактивная версия использует декларативный пайплайн, где данные проходят через цепочку преобразований. Например, `window(Duration.ofSeconds(5))` создает чанки данных на основе времени, `onBackpressureDrop()` грамотно обрабатывает переполнение, а `groupBy()` организует параллельную обработку без явного управления потоками.

В отличие от реактивной, версия с виртуальными потоками применяет императивный код с ручным управлением состоянием: поддерживает `LinkedList` с синхронизированными блоками для скользящих средних, используя `BlockingQueue` для backpressure, и явно создает потоки с помощью `Thread.startVirtualThread()`.

https://max.ru/bookofgeek

Если реактивный код читается как описание преобразования данных, то код с виртуальными потоками воспринимается как последовательность инструкций. Такой стиль более знаком разработчикам, которые привыкли к традиционному программированию на Java, но требует особенно внимательного подхода к синхронизации и управлению состоянием.

Преимущества и недостатки реактивного программирования

После того как мы получили общее представление о работе реактивных систем, использующих неблокирующий ввод-вывод, событийно-ориентированном подходе и потоковой обработке данных, рассмотрим преимущества и сложности, связанные с этой парадигмой.

Преимущества

Самое главное: реактивное программирование основано на неблокирующих операциях и асинхронном выполнении. Это позволяет приложениям обрабатывать множество конкурентных запросов, потребляя минимум ресурсов системы. Как следствие, для задач, связанных с вводом-выводом, значительно повышаются масштабируемость и производительность.

Кроме того, реактивное программирование предлагает мощный набор инструментов для написания неблокирующего асинхронного кода. Оно поддерживает функциональный и декларативный стиль, который многие разработчики считают более интуитивно понятным и выразительным по сравнению с традиционным императивным подходом. При грамотном использовании реактивный код становится проще читать и поддерживать, поскольку он структурированно организует потоки данных и обработку событий.

К тому же реактивное программирование легко интегрируется с современными облачными приложениями и потому идеально подходит для микросервисной архитектуры. Оно обеспечивает эффективный поток данных и взаимодействие между распределенными компонентами, уменьшая число узких мест и повышая отзывчивость системы.

Недостатки

Несмотря на многочисленные преимущества, реактивное программирование связано с определенными сложностями.

Во-первых, это принципиально иная парадигма программирования. Разработчикам, привыкшим к императивному стилю, может быть сложно переучиваться. Для адаптации к реактивным шаблонам, таким как композиция потоков и управление backpressure, требуется изменение образа мышления, а также освоение новых концепций и техник.

https://max.ru/bookofgeek

Отладка реактивного кода тоже чрезвычайно сложна. Асинхронная природа выполнения в сочетании с событийно-управляемым потоком данных затрудняет отслеживание ошибок и понимание путей выполнения. Логический поток, который мы видим в коде, полностью отличается от фактического выполнения, что делает сложной как мысленную отладку, так и работу в IDE.

Рассмотрим следующий пример реактивного кода, в котором выбрасывается исключение:

```
import reactor.core.publisher.Flux;

public class ReactiveErrorExample {
  public static void main(String[] args) {
      Flux.just(1, 2, 3, 0, 5)
    .map(number -> 10 / number)
    .subscribe(
      System.out::println,
      Throwable::printStackTrace,
      () -> System.out.println("Done!")
    );
  }
}
```

Выполнение этого кода приводит к исключению:

```
java.lang.ArithmeticException: / by zero
  at.ReactiveErrorExample.lambda$main$0(ReactiveErrorExample.java:8)
  at reactor.core.publisher.FluxMapFuseable$MapFuseableSubscriber
    .onNext(FluxMapFuseable.java:113)
  at reactor.core.publisher.FluxArray$ArraySubscription
    .fastPath(FluxArray.java:171)
  at reactor.core.publisher.FluxArray$ArraySubscription
    .request(FluxArray.java:96)
  at reactor.core.publisher.FluxMapFuseable$MapFuseableSubscriber
    .request(FluxMapFuseable.java:171)
  at reactor.core.publisher.LambdaSubscriber
    .onSubscribe(LambdaSubscriber.java:119)
  at reactor.core.publisher.FluxMapFuseable$MapFuseableSubscriber
    .onSubscribe(FluxMapFuseable.java:96)
  at reactor.core.publisher.FluxArray.subscribe(FluxArray.java:53)
  at reactor.core.publisher.FluxArray.subscribe(FluxArray.java:59)
  at reactor.core.publisher.Flux.subscribe(Flux.java:8836)
  at reactor.core.publisher.Flux.subscribeWith(Flux.java:8957)
  at reactor.core.publisher.Flux.subscribe(Flux.java:8801)
  at reactor.core.publisher.Flux.subscribe(Flux.java:8725)
  at ReactiveErrorExample.main(ReactiveErrorExample.java:9)
```

Хотя первая строка корректно указывает на `ReactiveErrorExample.java:8` как на источник проблемы, остальная часть стек-трейса состоит из внутренних вызовов библиотеки Reactor. Дамп потока выглядит как полная бессмыслица.

https://max.ru/bookofgeek

Это усложняет отладку по сравнению с традиционным синхронным кодом, где стек-трейс обычно гораздо понятнее.

Еще одна проблема — сложность сопровождения. Хотя реактивный код может быть изящным и лаконичным, он в то же время может оказаться сложнее для понимания и модификации. Декларативная природа реактивного программирования в сочетании с богатым набором операторов часто затрудняет предсказание поведения кода. При внесении изменений из-за этого могут возникать непреднамеренные побочные эффекты.

Наконец, хотя реактивное программирование отлично справляется с задачами, связанными с вводом-выводом, оно не всегда эффективно для нагрузок, зависящих от производительности CPU. Традиционные подходы с использованием многопоточности дают более интуитивно понятное и управляемое решение для определенных типов приложений, но это уже выходит за рамки нашего обсуждения.

В заключение

К каким выводам мы пришли, познакомившись с традиционными моделями конкурентности и их общей направленностью на обработку операций ввода-вывода?

Поделюсь личным мнением. Я приобретал опыт в течение многих лет и, хотя имею определенные пристрастия, все же приветствую применение различных подходов для разных сценариев. У каждой технологии свои достоинства и недостатки. Виртуальные потоки предлагают простоту и множество преимуществ, однако в них отсутствует встроенный механизм реализации backpressure, который реактивные фреймворки предоставляют изначально. Безусловно, можно самостоятельно реализовать эти механизмы, но тут ответственность полностью ложится на разработчика.

При этом обе парадигмы стремятся к общей цели: эффективное управление операциями, связанными с вводом-выводом. Если бы мне пришлось выбирать между ними, я бы выбрал виртуальные потоки. Реактивное программирование сохраняет свою актуальность, особенно в сценариях, требующих детального контроля backpressure, сложного преобразования данных или событийно-ориентированных микросервисов.

Виртуальные потоки блестяще себя зарекомендовали в высококонкурентных приложениях, ограниченных вводом-выводом. Они особенно полезны при обработке большого числа конкурентных запросов пользователей или работе с существующими синхронными системами. С виртуальными потоками мы можем бесшовно заменить традиционные потоки ОС, не переписывая код под неблокирующие API.

https://max.ru/bookofgeek

Я полагаю, что в перспективе реактивное программирование и виртуальные потоки будут сосуществовать. Со временем следует ожидать их сближения. Например, библиотеки реактивных потоков могут начать использовать виртуальные потоки, что позволит писать более простой и эффективный реактивный код. Аналогично принципы структурированной конкурентности способны повлиять на модели реактивного программирования, делая их более надежными и поддерживаемыми.

Первые признаки такой конвергенции уже видны. Vert.x 4.5 (https://oreil.ly/Lv3GW) начал экспериментировать с виртуальными потоками. Надеюсь, что в будущем эти парадигмы будут гармонично дополнять друг друга.

В долгосрочной перспективе виртуальные потоки могут стать стандартом для многих приложений благодаря своей простоте и совместимости с существующим синхронным кодом. Возможность писать конкурентные программы, используя привычные императивные техники программирования, — это существенное преимущество: снижается порог входа и уменьшается когнитивная нагрузка на разработчиков.

Время покажет, но одно можно сказать наверняка: конкурентность в Java развивается, и нам, как разработчикам, необходимо развиваться вместе с ней.

https://max.ru/bookofgeek

ГЛАВА 7

Современные фреймворки, использующие виртуальные потоки

> Лучший способ предсказать будущее — изобрести его.
>
> *Алан Кэй (Alan Kay)*

С появлением виртуальных потоков в JDK 21 экосистема Java претерпела значительные изменения в подходах к обработке конкурентности. Многие фреймворки начали активно использовать виртуальные потоки для повышения производительности, масштабируемости и ресурсоэффективности.

В этой главе мы рассмотрим, как ведущие фреймворки Spring Boot, Quarkus и Jakarta EE интегрируют виртуальные потоки. Я не буду углубляться в их внутреннее устройство, так как предполагаю, что заинтересованные читатели предпочтут изучить каждый фреймворк самостоятельно.

Spring Boot

Spring Boot (https://oreil.ly/JE9kr) — один из фреймворков для создания корпоративных приложений в экосистеме Java. Де-факто является стандартом.

Традиционно веб-приложения на Spring Boot использовали модель *thread-per-request*, в которой каждый входящий запрос клиента обрабатывается отдельным, выделенным для него платформенным потоком. Такой подход хорошо работает при умеренной нагрузке, но при большом количестве конкурентных запросов ввода-вывода возникают проблемы с масштабируемостью, поскольку все запросы по-прежнему обслуживаются платформенными потоками.

Чтобы как-то справиться с этой проблемой, в Spring Boot добавили поддержку асинхронного программирования через аннотации, например `@Async` (https://oreil.ly/_7x7K), и абстракции, такие как `TaskExecutor` (https://oreil.ly/Hkgxi). Однако

https://max.ru/bookofgeek

и этот подход зависел от лимитированных пулов платформенных потоков, что видно из следующего примера:

```
@Configuration
public class ThreadPoolConfig {
  @Bean
  public AsyncTaskExecutor applicationTaskExecutor() {

    ThreadPoolTaskExecutor executor = new ThreadPoolTaskExecutor();
    executor.setCorePoolSize(10);
    executor.setMaxPoolSize(100);
    executor.initialize();
    return executor;
  }
}
```

Здесь каждый запрос или задача потребляет поток из ограниченного пула, что при высокой нагрузке может привести к узким местам производительности.

В ноябре 2023 года, с выходом Spring Boot 3.2 (основанного на Spring Framework 6.1), была добавлена официальная поддержка Java 21, в которой виртуальные потоки стали стандартной функцией. Хотя базовой версией JDK остается Java 17, в рамках фреймворка Java 21 теперь рассматривается как основная среда выполнения. Важным аспектом этой поддержки является возможность включения виртуальных потоков через простое свойство конфигурации (configuration property):

```
# application.properties
spring.threads.virtual.enabled=true
```

Или в формате YAML:

```
# application.yml
spring:
  threads:
    virtual:
      enabled: true
```

Установка этого свойства конфигурации активирует виртуальные потоки во всем приложении. Spring Boot автоматически конфигурирует `AsyncTaskExecutor` (https://oreil.ly/8F5jm) на основе виртуальных потоков (через `SimpleAsyncTaskExecutor`, https://oreil.ly/h95RG), если не задан бин, который определяет настраиваемый исполнитель. Эта конфигурация бесшовно применяется для нескольких функций, связанных с конкурентностью, таких как:

- асинхронное выполнение задач (`@EnableAsync`, https://oreil.ly/ASk3i);
- асинхронная обработка возвращаемых значений `Callable` (https://oreil.ly/k5KXf) в Spring;
- асинхронная обработка запросов в Spring Web MVC;
- поддержка периодических блокирующих операций в Spring WebFlux.

https://max.ru/bookofgeek

Рассмотрим следующий пример:

```
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController
  public class GreetingsController {
    private static final Logger LOGGER
            = LoggerFactory.getLogger(GreetingsController.class.getName());
    @GetMapping("/hello")
    public String hello() {
      LOGGER.info("Received request for /hello");
      LOGGER.info("Running on {}", Thread.currentThread());
      return "Hello from Spring Boot";
    }
}
```

Если виртуальные потоки активированы и мы запустим Spring Boot-приложение, а затем отправим curl-запрос к эндпоинту, то в консоли увидим следующий лог:

```
06:11:51.355 [tomcat-handler-0] INFO  c.b.m.c.c.GreetingsController - Received
request for /hello
06:11:51.356 [tomcat-handler-0] INFO  c.b.m.c.c.GreetingsController - Running
on VirtualThread[#63,tomcat-handler-0]/runnable@ForkJoinPool-1-worker-1
```

Рассмотрим метод контроллера для обработки асинхронных запросов:

```
import org.slf4j.Logger;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;
import java.util.concurrent.Callable;

@RestController
  public class AsyncController {
    private static final Logger LOGGER
            = LoggerFactory.getLogger(AsyncController.class.getName());
    @GetMapping("/async-call")
    public Callable<String> handleAsyncRequest() {
      return () -> {
        Thread.sleep(500); // Моделирует операцию, ограниченную вводом-выводом
        LOGGER.info("Running on {}", Thread.currentThread());
        return "Hello from Virtual Thread!";
      };
    }
  }
```

При включенных виртуальных потоках вы увидите примерно такой вывод:

```
06:55:01.784 [task-1] INFO c.b.m.c.controller.AsyncController - Running on
VirtualThread[#74,task-1]/runnable@ForkJoinPool-1-worker-1
```

https://max.ru/bookofgeek

Рассмотрим еще один практический сценарий с запланированными задачами:

```
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.scheduling.annotation.EnableScheduling;
import org.springframework.scheduling.annotation.Scheduled;
import org.springframework.stereotype.Component;

@EnableScheduling
  @Component
    public class ScheduledTasks {
      private static final Logger LOGGER
              = LoggerFactory.getLogger(ScheduledTasks.class.getName());
      @Scheduled(fixedRate = 1000)
      public void scheduledTask() {
        LOGGER.info("Scheduled task running on:  {}", Thread.currentThread());
      }
    }
```

При включенных виртуальных потоках логи запланированных задач будут примерно такими:

```
07:01:51.082 [scheduling-36] INFO  c.b.m.c.controller.ScheduledTasks -
Scheduled task running on:  VirtualThread[#105,scheduling-36]/
runnable@ForkJoinPool-1-worke r-5
```



Spring Boot автоматически конфигурирует исполнители и планировщики для выполнения задач. При активированных виртуальных потоках он автоматически использует `SimpleAsyncTaskExecutor` (https://oreil.ly/ODEKW) — реализацию интерфейса `applicationTaskExecutor` и `SimpleAsyncTaskScheduler` (https://oreil.ly/nOpwy) — реализацию интерфейса `taskScheduler`. Без виртуальных потоков по умолчанию применяются настраиваемые `ThreadPoolTaskExecutor` (https://oreil.ly/rs8sF) и `ThreadPoolTaskScheduler` (https://oreil.ly/w769a). Эти стандартные настройки можно переопределить или создать собственные исполнители и планировщики с помощью классов из Spring Boot, реализующих классы-строители. Такая бесшовная конфигурация помогает упростить управление конкурентностью и оптимизировать производительность приложений.

Ручное конфигурирование

Если используется версия Spring Boot ниже 3.2 или требуется усилить контроль над конфигурацией исполнителя, его можно явно определить как virtual-thread-backed executor.

Рассмотрим, как такая кастомизация (`applicationTaskExecutor`) работает на практике:

```
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.core.task.AsyncTaskExecutor;
```

https://max.ru/bookofgeek

```
import org.springframework.core.task.support.TaskExecutorAdapter;
import java.util.concurrent.Executors;

@Configuration
  public class VirtualThreadConfig {
    @Bean
    public AsyncTaskExecutor applicationTaskExecutor() {
      return new TaskExecutorAdapter(Executors
        .newVirtualThreadPerTaskExecutor());
    }
  }
```

С такой конфигурацией все методы с аннотацией `@Async` будут выполняться на виртуальных потоках вместо стандартных платформенных, как показано ниже:

```
import org.springframework.scheduling.annotation.Async;
import org.springframework.stereotype.Component;
import java.util.concurrent.CompletableFuture;

@Component
  public class RemoteApiService {
    @Async
    public CompletableFuture<String> fetchDataFromRemoteApi() {
      try {
        Thread.sleep(1000); // Имитация операции, связанной с вводом-выводом
      } catch (InterruptedException e) {
        Thread.currentThread().interrupt();
      }
      return CompletableFuture
        .completedFuture("Data fetched using virtual thread");
    }
  }
```

В модели асинхронного программирования фреймворка Spring методы, помеченные аннотацией `@Async`, используют сконфигурированный `applicationTaskExecutor`. Если заменить стандартный исполнитель на virtual-thread-based executor, то методы с аннотацией `@Async` бесшовно запустятся на легковесных виртуальных потоках.



При ручной настройке исполнителя виртуальных потоков в Spring Boot важно понимать разницу между `TaskExecutorAdapter` и `Executors.newVirtualThreadPerTaskExecutor()`. Хотя они оба поддерживают виртуальные потоки, `TaskExecutorAdapter` обеспечивает лучшую интеграцию с механизмом управления жизненным циклом бинов в Spring. При выборе подхода следует учитывать возможности управления Spring (например, инициализацию и завершение работы).

Помимо этого, можно использовать виртуальные потоки для встроенного веб-сервера, чтобы повысить масштабируемость и эффективность использования

https://max.ru/bookofgeek

ресурсов. Например, если встроенным веб-сервером выступает Apache Tomcat, его обработчик протокола можно настроить для работы с виртуальными потоками, определив бин `TomcatProtocolHandlerCustomizer` (https://oreil.ly/mDDIk):

```
import org.springframework.boot.web.embedded.tomcat
       .TomcatProtocolHandlerCustomizer;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import java.util.concurrent.Executors;

@Configuration
  public class TomcatConfig {
    @Bean
    public TomcatProtocolHandlerCustomizer<?> virtualThreadExecutor() {
      return protocolHandler
        -> protocolHandler.setExecutor(
          Executors.newVirtualThreadPerTaskExecutor());
      }
  }
```

Quarkus

Quarkus (https://quarkus.io) — это Java-фреймворк, разработанный для повышения продуктивности разработчиков, оптимизации времени запуска приложений и более эффективного использования ресурсов. Он идеально подходит для создания микросервисов и бессерверных приложений. В Quarkus, построенном на основе Vert.x (https://vertx.io) с применением парадигмы реактивного программирования, целенаправленно и явно интегрировали поддержку виртуальных потоков, что повышает удобство его использования для конкурентных приложений.

В отличие от глобального подхода в Spring Boot, Quarkus предлагает точечный способ использования виртуальных потоков через специальные аннотации. Разработчики могут выборочно включать виртуальные потоки для конкретных эндпоинтов или сервисов, помечая методы аннотацией `@RunOnVirtualThread` (https://oreil.ly/p2aCw). Она указывает, что Quarkus должен выполнять аннотированный метод на виртуальном потоке вместо традиционного платформенного. При этом фреймворк берет на себя управление жизненным циклом этих виртуальных потоков, включая их создание, выполнение и завершение.

Quarkus интегрирует виртуальные потоки в основном через свои реактивные расширения, сочетая реактивное программирование с возможностями виртуальных потоков. Такой подход позволяет Quarkus-приложениям оптимально обрабатывать конкурентные задачи. Это делает их особенно подходящими для микросервисов и бессерверных сред благодаря быстрому запуску и эффективному использованию ресурсов.

https://max.ru/bookofgeek



Quarkus опирается на Vert.x как на базовый реактивный движок. Модель конкурентности Vert.x, основанная на цикле событий, резко отличается от традиционной модели thread-per-request («один поток на запрос»). Аннотация `@RunOnVirtualThread` в Quarkus позволяет разработчикам выгружать определенные задачи из цикла событий Vert.x в виртуальные потоки, предотвращая влияние блокирующих операций на отзывчивость цикла событий. Сочетание реактивного программирования и виртуальных потоков повышает способность Quarkus эффективно обрабатывать конкурентные задачи.

Вот практический пример использования виртуальных потоков в Quarkus:

```
package ca.bazlur;
import io.smallrye.common.annotation.RunOnVirtualThread;
import jakarta.inject.Inject;
import jakarta.ws.rs.GET;
import jakarta.ws.rs.Path;
import jakarta.ws.rs.Produces;
import jakarta.ws.rs.core.MediaType;
import org.eclipse.microprofile.rest.client.inject.RestClient;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;

@Path("/greetings")
  public class VirtualThreadApp {
    private static final Logger logger
            = LoggerFactory.getLogger(VirtualThreadApp.class);
    @Inject
    @RestClient
    RemoteService remoteService;
    @GET
    @RunOnVirtualThread
    @Produces(MediaType.TEXT_PLAIN)
    public String process() {
      logger.info("Received greetings request");
      var response = remoteService.greetings();
      logger.info("Received response: {}", response);
      logger.info("Running on {}", Thread.currentThread());
      return response.toUpperCase();
    }
  }
```

Метод `process()` с аннотацией `@RunOnVirtualThread` будет выполняться в виртуальном потоке.

При запуске кода в консоли появится следующий лог:

```
2025-03-16 00:12:31,453 INFO [ca.baz.VirtualThreadApp] (quarkus-virtual-
thread-1) Received greetings request
2025-03-16 00:12:31,458 INFO [ca.baz.VirtualThreadApp] (quarkus-virtual-
thread-1) Received response: Hey!
2025-03-16 00:12:31,458 INFO [ca.baz.VirtualThreadApp] (quarkus-virtual-
thread-1) Running on VirtualThread[#219,quarkus-virtual-thread-1]/
runnable@ForkJoinPool-1-worker-2
```

https://max.ru/bookofgeek

Данный лог подтверждает, что метод `process` действительно выполняется в виртуальных потоках.

Для работы этого удаленного сервиса мы создали следующий интерфейс:

```
import jakarta.ws.rs.GET;
import jakarta.ws.rs.Path;
import jakarta.ws.rs.Produces;
import jakarta.ws.rs.core.MediaType;
import org.eclipse.microprofile.rest.client.inject.RegisterRestClient;

@Path("/remote")
@RegisterRestClient
  public interface RemoteService {
    @Path("greetings")
    @GET
    @Produces(MediaType.TEXT_PLAIN)
    String greetings();
  }
```

Для регистрации необходимо добавить следующий элемент в файл `application.properties`:

```
quarkus.rest-client."ca.bazlur.RemoteService".url=http://localhost:8081
```

Для внешних сервисов мы создали простой эндпоинт в REST API, возвращающий случайные сообщения с приветствиями. Его легко реализовать, используя встроенные в Java возможности HTTP-сервера. JDK предоставляет такие классы, как `HttpServer` (https://oreil.ly/fd9lJ) и `HttpHandler` (https://oreil.ly/wphhE), которые позволяют быстро настраивать легковесные HTTP-сервисы без дополнительных фреймворков или зависимостей:

```
import com.sun.net.httpserver.HttpServer;
import com.sun.net.httpserver.HttpHandler;
import com.sun.net.httpserver.HttpExchange;
import java.io.IOException;
import java.io.OutputStream;
import java.net.InetSocketAddress;
import java.util.Random;
import java.util.concurrent.Executors;

public class SimpleHttpServer {
  private static final String[] GREETINGS = {
    "Hello, world!",
    "Hi there!",
    "Greetings!",
    "Good day!",
    "Hey!",
    "Howdy!",
    "Hola!",
    "Bonjour!",
    "Ciao!"
  };
```

https://max.ru/bookofgeek

```
  public static void main(String[] args) throws IOException {
    int port = 8081;
    HttpServer server = HttpServer.create(new InetSocketAddress(port), 0);
    server.createContext("/remote/greetings", new GreetingHandler());
    server.setExecutor(Executors.newVirtualThreadPerTaskExecutor());
    System.out.println("Server started on port " + port);
    server.start();
  }
  static class GreetingHandler implements HttpHandler {
    @Override
    public void handle(HttpExchange exchange) throws IOException {
      if ("GET".equalsIgnoreCase(exchange.getRequestMethod())) {
        String response = getRandomGreeting();
        exchange.sendResponseHeaders(200, response.length());
        try (OutputStream os = exchange.getResponseBody()) {
          os.write(response.getBytes());
        }
        } else {
          exchange.sendResponseHeaders(405, -1); // Недопустимый метод
        }
      }
      private String getRandomGreeting() {
        Random random = new Random();
        return GREETINGS[random.nextInt(GREETINGS.length)];
      }
    }
  }
```



Можно запустить этот сервер напрямую, используя функцию запуска исходных файлов, доступную начиная с Java 11. Она позволяет выполнять однофайловые Java-программы без предварительной компиляции. Просто откройте терминал и выполните команду:

```
java SimpleHttpServer.java
```

Эта команда компилирует и запускает файл за один шаг, устраняя необходимость ручного использования `javac`. Очень полезная возможность для быстрого прототипирования или запуска простых автономных Java-приложений.

Можно даже интегрировать возможности реактивного программирования Quarkus с виртуальными потоками. Рассмотрим следующий код:

```
import jakarta.inject.Inject;
import jakarta.ws.rs.GET;
import jakarta.ws.rs.Path;
import jakarta.ws.rs.Produces;
import jakarta.ws.rs.core.MediaType;
import org.eclipse.microprofile.rest.client.inject.RestClient;
import io.smallrye.common.annotation.RunOnVirtualThread;
```

https://max.ru/bookofgeek

```
import io.smallrye.mutiny.Uni;
import java.time.Duration;
import jakarta.enterprise.context.ApplicationScoped;

@Path("/reactive")
  public class ReactiveResource {
    @Inject
    HelloService helloService;
    @GET
    @Path("/hello")
    @RunOnVirtualThread
    @Produces(MediaType.TEXT_PLAIN)
    public String hello() {
      return helloService.getHello()
                         .await()
                         .atMost(Duration.ofSeconds(5));
    }
  }
@ApplicationScoped
  class HelloService {
    @Inject
    @RestClient
    ExternalService externalService;
    Uni<String> getHello() {
      return externalService.hello();
    }
  }
  @Path("/reactive")
  @RegisterRestClient
  interface ExternalService {
    @GET
    @Path("/hello")
    Uni<String> hello();
  }
```

Метод `hello()` в `ReactiveResource` вызывает внешний сервис через реактивный REST-клиент (`ExternalService`). Метод `getHello()`возвращает `Uni<String>`, который представляет отложенное вычисление. В методе `hello()` используется `uni.await().atMost(Duration.ofSeconds(5))` для блокировки виртуального потока до получения результата или истечения тайм-аута.

Jakarta EE

Jakarta Concurrency (https://oreil.ly/JbYoV) — это стандартная спецификация, позволяющая приложениям использовать конкурентность, сохраняя преимущества работы внутри среды выполнения Jakarta EE (https://jakarta.ee). В версии Jakarta Concurrency 3.1 (https://oreil.ly/ffFz_) появилась поддержка виртуальных потоков, что позволяет приложениям задействовать легковесную модель многопоточности Java 21.

https://max.ru/bookofgeek

Виртуальные потоки можно активировать в Jakarta Concurrency 3.1, указав параметр `virtual = true` в следующих аннотациях:

- `@ManagedExecutorDefinition` (https://oreil.ly/-ip4E);
- `@ManagedScheduledExecutorDefinition` (https://oreil.ly/-TVMe);
- `@ManagedThreadFactory` (https://oreil.ly/5nQS-).

При работе с Java 21 используются виртуальные потоки. Однако если приложение запускается на Java 17, который их не поддерживает, Jakarta EE runtimes автоматически переключается на использование платформенных потоков.

В следующем примере показано, как конфигурировать и использовать виртуальные потоки в Jakarta Concurrency 3.1:

```
import jakarta.enterprise.concurrent.ManagedExecutorDefinition;
import jakarta.enterprise.concurrent.ManagedExecutorService;
import jakarta.inject.Inject;
import jakarta.ws.rs.GET;
import jakarta.ws.rs.Path;
import jakarta.ws.rs.Produces;
import jakarta.ws.rs.core.MediaType;

import java.util.concurrent.ExecutionException;

@ManagedExecutorDefinition(name = "java:module/concurrent/virtual-executor",
  qualifiers = WithVirtualThreads.class,
  virtual = true)
@Path("/virtualThreads")
  public class VirtualThreadExampleService {
    @Inject
    @WithVirtualThreads
    ManagedExecutorService virtualManagedExecutor;
    @Inject
    GreetingService greetingService;
    @GET
    @Produces(MediaType.TEXT_PLAIN)
    public String virtualThreads()
    throws InterruptedException, ExecutionException {
      return virtualManagedExecutor.submit(() -> {
        System.out.println("Received request on virtual thread: "
               + Thread.currentThread());
        return greetingService.getRandomGreeting();
      }).get();
    }
  }
```

При запуске предыдущего кода в среде выполнения Open Liberty будет доступен эндпоинт:

```
GET http://localhost:9080/api/virtualThreads
```

https://max.ru/bookofgeek

Вы видите часть кода, которая используется для получения случайных приветствий; она будет выполняться в виртуальных потоках:

```
[INFO] Received request on virtual thread: VirtualThread[#115,application
[jakarta ee-vthrads]/module[jakartaee-vthrads.war]/managedExecutorService
[java:module/conc urrent/virtual-executor]/concurrencyPolicy:1]/
runnable@ForkJoinPool-1-worker-1
```

Если требуется использовать виртуальные потоки через `ManagedScheduledExecutorService` для задач с периодичностью или с задержкой, сделаем следующее:

```
import jakarta.annotation.Resource;
import jakarta.enterprise.context.ApplicationScoped;
import jakarta.enterprise.concurrent.ManagedScheduledExecutorService;
import jakarta.enterprise.concurrent.ManagedScheduledExecutorDefinition;
import java.util.concurrent.TimeUnit;

@ApplicationScoped
@ManagedScheduledExecutorDefinition(
  name = "java:module/concurrent/virtual-scheduler",
    virtual = true // Активируем виртуальные потоки для планируемых задач
  )
  public class VirtualThreadSchedulerExample {
    @Resource(lookup = "java:module/concurrent/virtual-scheduler")
    private ManagedScheduledExecutorService scheduledExecutor;
    public void scheduleTask() {
      scheduledExecutor.schedule(() -> {
        System.out.println("Scheduled task running in virtual thread: "
              + Thread.currentThread());
      }, 5, TimeUnit.SECONDS); // Задержка выполнения на 5 с
    }
  }
```



Хотя Jakarta Concurrency 3.1 предоставляет спецификации для работы с виртуальными потоками, фактическое поведение может различаться в зависимости от реализации Jakarta EE runtime. Необходимо учитывать возможные проблемы совместимости и изучать документацию конкретной реализации (например, Open Liberty, Payara, WildFly), чтобы получить сведения о реализации определенных деталей и рекомендации. Например, некоторые функции и конфигурации могут действовать по-разному или вовсе отсутствовать в каких-то реализациях Jakarta EE.

На тот момент, когда я закончил писать эту книгу, Open Liberty 24.0.0.6-beta стала первой средой выполнения Jakarta EE с полной поддержкой Jakarta Concurrency 3.1 и виртуальных потоков. Однако по мере приближения финального релиза Jakarta EE 11 ожидается, что в других реализациях — Payara, WildFly, TomEE и GlassFish — также добавят поддержку виртуальных потоков.

https://max.ru/bookofgeek

В заключение

Помимо основных фреймворков Jakarta EE, Spring и Quarkus, в ряде других фреймворков также стали поддерживаться виртуальные потоки для повышения эффективности конкурентности в современных Java-приложениях.

Один из таких фреймворков — Helidon Níma (https://helidon.io/nima). Это микросервисный фреймворк, изначально спроектированный для нативного использования виртуальных потоков. В отличие от традиционных фреймворков, куда поддержка виртуальных потоков была добавлена постфактум, Helidon Níma построен вокруг них, что делает его легковесным и высокопроизводительным решением для микросервисов.

Фреймворк Micronaut (https://micronaut.io) также автоматически задействует виртуальные потоки, если приложение работает на версии JDK с их поддержкой (Java 21 или новее). Иначе говоря, разработчики, использующие Micronaut, без дополнительного конфигурирования могут получить преимущества от повышения производительности благодаря использованию виртуальных потоков.

По мере развития Java виртуальные потоки стали ключевой функциональностью для создания масштабируемых, производительных и удобных для разработчиков приложений. Какой бы фреймворк ни выбрали разработчики — Jakarta EE 11, Quarkus, Spring Boot 3, Helidon Níma или Micronaut, — они получают мощные инструменты для создания высококонкурентных приложений, сохраняя при этом простой императивный стиль кода.

Теперь, когда виртуальные потоки стали неотъемлемой частью экосистемы Java, конкурентность будущего обещает быть более эффективной, масштабируемой и удобной для разработчиков, чем когда-либо.

https://max.ru/bookofgeek

ГЛАВА 8

Заключение и основные выводы

Простота — высшая степень утонченности.

Леонардо да Винчи (Leonardo da Vinci)

Мы совершили путешествие по миру конкурентности Java: прошли от базового понимания классических потоков до инновационных виртуальных потоков, структурированной конкурентности и других технологий. Однако наш путь был не только техническим, но и философским. Java продолжает развиваться, руководствуясь принципами эффективности, масштабируемости и простоты, чтобы отвечать постоянно меняющимся требованиям современных вычислений.

Одной из главных тем нашего обсуждения стали виртуальные потоки — полноценная смена парадигмы в конкурентности Java. Исторически создание множества потоков было сложной задачей из-за высоких расходов ресурсов и ограничений операционной системы. Виртуальные потоки решают эту проблему, на порядки сокращая потребление ресурсов и фактически позволяя использовать миллионы таких потоков. С развитием вычислительного оборудования и распространением облачных технологий виртуальные потоки станут основным инструментом для создания масштабируемых приложений на Java. Их простота и удобство позволяют разработчикам писать более понятный и поддерживаемый асинхронный код, значительно повышая продуктивность и качество программного обеспечения.

Конечно, несмотря на простоту добавления виртуальных потоков, не стоит сразу повсеместно их внедрять, особенно если приложения вашей организации построены на устаревших технологиях. Процесс требует планирования и постепенной реализации. Рекомендую начать с миграции отдельных модулей или сервисов в существующих приложениях. Постепенное развертывание позволяет наблюдать, учиться и совершенствовать методы, не замедляя работу команды или системы. Внедрение структурированной конкурентности дополнительно упростит сложные рабочие процессы, делая конкурентный код более безопасным и читабельным. Таким образом, ваш код станет более управляемым

https://max.ru/bookofgeek

и менее подверженным ошибкам. Контролируйте использование ресурсов на начальных этапах развертывания — раннее выявление узких мест помогает избежать наиболее распространенных ошибок. И помните: инвестиции в обучение команды разработчиков виртуальным потокам, структурированной конкурентности и scoped values формируют культуру лучших практик и непрерывного улучшения.

Особую осторожность следует проявлять с проблемой пиннинга — ситуациями, когда виртуальные потоки непроизвольно привязываются к определенным платформенным потокам на продолжительное время, что существенно снижает их эффективность. Хотя JDK 24 решает проблемы пиннинга, связанные с `synchronized`-блоками, важно учитывать, что не все производственные системы мгновенно перейдут с предыдущих версий (например, JDK 21) на JDK 24. Еще одна потенциальная ловушка — некорректное управление переменными `ThreadLocal`, которое может вызывать трудноуловимые ошибки и утечки памяти в контексте виртуальных потоков. Критически важно понимать различия между виртуальными и платформенными потоками, особенно при блокирующих I/O-операциях, поскольку традиционные блокирующие вызовы будут работать не так, как ожидается. Классические инструменты мониторинга также могут оказаться недостаточными для диагностики проблем, специфичных для виртуальных потоков. Следует внедрять более совершенные методы мониторинга и использовать современные инструменты, такие как Java Flight Recorder.

Несмотря на сложности, разработчики, успешно внедрившие виртуальные потоки, получают ощутимые преимущества. Наблюдаются значительное улучшение масштабируемости приложений, снижение задержек и более эффективное использование ресурсов. Многочисленные статьи и отчеты фиксируют улучшенную отзывчивость, упрощенное управление конкурентностью и сокращение циклов разработки, что подтверждается опытом предприятий, стартапов и облачных провайдеров.

С учетом этих улучшений необходимо выбирать подходящую модель конкурентности, исходя из конкретных требований приложения. Виртуальные потоки особенно эффективны для высоконагруженных I/O-зависимых приложений, таких как веб-серверы, подсистемы обработки данных или сетевые сервисы, где важны масштабируемость и удобство работы без накладных расходов на переключение контекста, характерных для традиционных потоков. Платформенные потоки остаются востребованными, особенно для CPU-зависимых вычислений, взаимодействия с нативными методами или устаревшего кода, где неизбежно активное закрепление потоков. Реактивное программирование имеет преимущества в сценариях, требующих явных асинхронных шаблонов, эффективного управления backpressure или событийно-ориентированной архитектуры. Понимание этих различий позволяет принимать взвешенные решения, соответствующие потребностям проекта.

https://max.ru/bookofgeek

В заключение дам простой совет: оставайтесь любознательными, смело экспериментируйте и не прекращайте учиться. Модель конкурентности в Java лучше всего развивается благодаря обратной связи сообщества, практическому опыту и постоянному совершенствованию. Активно осваивайте новые технологии, делитесь наработками и присоединяйтесь к Java-сообществу. Регулярно отслеживайте новости на OpenJDK (https://openjdk.org), чтобы быть в курсе инноваций, улучшений и работ по решению оставшихся проблем конкурентности. Вносите свой вклад в формирование будущего конкурентности в Java. Непрерывное обучение поможет оставаться мотивированным и вовлеченным в постоянно развивающуюся Java-экосистему.

Благодарю за совместное путешествие. Пусть ваши потоки остаются легковесными, приложения — бесконечно масштабируемыми, а код — чистым, лаконичным и изящным.

https://max.ru/bookofgeek